Manifester Onboarding Book

What Manifester is, how it works, and how to make your first change safely

Generated from docs/onboarding (README + Chapters 1–12)

Introduction
- How to use this book
- Contents
- Five ideas in every chapter
- After the book

Why Manifester keeps brokers, data, and metadata apart
- Kafka cost grows because one machine does three jobs
- Three binaries run the split
- One deterministic step lets the same machine run in production and in simulation
- A produce earns an acknowledgement only after the PUT and the commit
- WarpStream shares the broad split and differs in the parts that matter
- One ordered log gives every broker the same offsets
- What Manifester leaves out limits the claims you can make
- The current evidence proves less than the headline numbers suggest
- Glossary
- Reading order for your first week
- Key takeaways
- Checkpoint questions
- Try it
- Further reading

Chapter 2. The repository has one deterministic centre and six binaries in `cmd/`
- 2.1 The tree lists the top level and the subpackages you open first
- 2.2 The deterministic packages sit at the centre and the edge packages hold the nondeterminism
- 2.3 The two executors live in `sim/` and `sys/`
- 2.4 Each of the six binaries in `cmd/` has the same shape
- 2.5 `build/` and `results/` hold outputs, and two checked-in files come from generators
- 2.6 `docs/` holds the decision log, the lessons, the design notes, and the runbooks
- 2.7 The table maps a task to the package, the file, or the command that holds the work
- Key takeaways
- Checkpoint questions
- Try it
- Further reading

Chapter 3. The deterministic core takes events and returns actions
- 3.1 The core holds the contracts and leaves the work to other packages
- 3.2 The Step contract lets one machine run under two drivers
- 3.3 Plain data and closed sets keep every input visible
- 3.4 Time arrives on the event so that replay controls time
- 3.5 Env names each source of nondeterminism that replay must control
- 3.6 The event and action tables list the whole contract
- 3.7 One Step in the broker shows the split in code
- 3.8 A machine returns facts because a log line is an effect
- 3.9 The bans on clocks, goroutines, and map order protect replay
- 3.10 Wire work and control plane work stay inside the same contract
- 3.11 One policy picks the next event so that Go cannot pick the order
- 3.12 One Produce request shows each decision as a step
- 3.13 The invariants state what replay must preserve
- 3.14 Mutants test whether an invariant catches a wrong machine
- Key takeaways
- Checkpoint
- Try it
- Further reading

Chapter 4: the broker acks a produce only after S3 stores the segment and the control plane commits the segment
4.1 The package map puts the produce, seal, commit, and fetch code in separate files
4.2 The open segment gathers batches before the broker seals one object
4.3 A seal rule ends the wait for more batches
4.4 The broker acks only after S3 stores the segment and the control plane commits the segment
4.5 Fetch reverses the produce order through the control plane and then S3
4.6 The broker stores compressed batches as bytes the broker never reads
4.7 Metadata names the answering broker as the leader for each partition
4.8 Backpressure holds the bytes that the ack rule keeps in the broker
4.9 The Config table lists each knob and the default
4.10 Slots keep responses in request order on each connection
Key takeaways
Checkpoint
Try it
Further reading
Chapter 5. The Kafka wire format and the segment format
5.1 Each file in `kwire` and `format` owns one task
5.2 `kwire.Framer` reassembles a TCP stream into frames
5.3 `kwire.Codec` decodes requests and encodes responses
5.4 `kwire/supported.go` serves five API keys
5.5 Decoded data aliases the input frame
5.6 `kwire/batch.go` uses record batch format version 2 only
5.7 Fetch patches `baseOffset` and `partitionLeaderEpoch` outside the CRC range
5.8 `format/format.go` puts the index before the payloads and has no footer
5.9 `format.SegmentKey` names each segment with 32 lowercase hex digits
5.10 One batch moves from Produce to Fetch through one segment
Key takeaways
Checkpoint
Try it
Further reading
Chapter 6: the replica assigns the offset when the replica applies the log
1. The `cp/` packages split the old store from the log store
2. `cmd/manicp` serves the log store in production and the old store for rollback
3. Both stores keep the same `cp.Store` contract
4. Three rules explain both stores
5. The old store computes offsets first and defends the computed offsets with one transaction
6. The prototype proved the log design before the log store used the design in production
7. The log store records commands first and assigns offsets at apply time
8. The broker retries with one segment key across the old store and the log store
9. The diagrams show commit and failover in slot order
Key takeaways
Checkpoint
Try it
Further reading
Chapter 7. One goroutine owns the broker state in production
7.1 The `sys/` layer holds every real input and output
7.2 One loop goroutine owns the machine in `sys.Run`
7.3 The executor moves real input and output off the loop goroutine
7.4 Control completions run before new data in the two-queue inbox
7.5 Real adapters sit under the `sys` interfaces
7.6 Metrics are facts, so sim and prod report the same durations
7.7 Graceful shutdown gives the machine 10 s to emit `Exit`

7.8 `cmd/manibroker` wires the runtime, then starts `sys.Run`
7.9 Two drivers run one machine, and the split tells you where a bug lives
Key takeaways
Checkpoint
Try it
Further reading
One seed replays a whole cluster
The contract in `loop.md` makes one history per seed
The kernel in `sim/` holds no Kafka, S3, or TCP code
The queue order `(At, Tie, Seq)` decides what the seed means
One counted stream makes each draw part of the history
The kernel loop takes one step at a time and charges for the step
A crash throws the node away and the stores rebuild the node
Faults draw in a fixed order, so one fault cannot move another fault
The engines supply the world, and the kernel supplies the order
Echo teaches the shape, and Kafka tests the real broker
The trace hash turns one changed step into one failed seed
Determinism is worth the price for this project
You run campaigns first, and you replay one seed after a failure
Key takeaways
Checkpoint
Try it
Further reading
Chapter 9. Verification, veritas specs, conformance, and quality gates
9.1 Four parts check the implementation against the spec
9.2 The veritas specs state the required behavior
9.3 `veritastest` runs the implementation and the spec in lockstep
9.4 C1, C2, C3, and C4 compare four pairs
9.5 Each test layer detects a different class of bug
9.6 Integration tests run real brokers and the real client (`it/`)
9.7 The determinism gate rejects nondeterministic code and unaudited dependencies
9.8 The CRAP gate rejects complex functions that lack coverage (`cmd/crapscore`)
9.9 Each check must fail on a mutant
9.10 Each bug has a test that detects the bug first
9.11 Commands
Key takeaways
Checkpoint
Try it
Further reading
Chapter 10: Deployment, benchmarking, and observability on AWS
1. Local deployment runs test backends only
2. AWS topology in `deploy/aws/terraform/` puts each part on its own hosts
3. Three load tools run in three places and measure three things
4. OMB needs four patches to run this broker
5. A benchmark run has seven steps in order
6. The results directory starts with `summary.md`
7. Past runs report short stages, and most runs have one sample
8. Observability puts `run_id` on each metric series and each log line
9. Cost ends with destroy, and failure depends on the store
Key takeaways
Checkpoint
Try it

Further reading
The design changed its mind in public, and two verbs tell you what still holds
1. A later decision has more authority than the earlier text it names
2. A row in `docs/decisions.md` holds only where no later row limits it
3. Measurements forced the large turns, and the dates show the order
4. `docs/lessons.md` gives the shortest reason for each odd contract
5. An S-id names one claim in the audit, and the audit froze on 2026-10-19
6. Design A superseded design L, and the three debate docs keep distinct jobs
7. The current design docs describe stages 2-4 as built
8. Toys3 supplied the evidence, and the later proposals mostly stayed unbuilt
9. `docs/TODO.md` alone lists work that the team accepted but did not build
10. Results supersede the plan that produced them, and the legal caution blocks the teardown as a source
11. Superseded decisions stay in the repo, and you must not follow the superseded text
12. The trust table tells you when to read each doc and how far to trust each doc
Key takeaways
Checkpoint
Try it
Further reading
Chapter 12. Newcomer playbook. Your first week, first PR, and where to change what
Check the prerequisites
Plan the first week
Run the build gate
Run the broker locally
Run a simulator campaign
Run the integration tests
Replay a failing seed
Read a trace
Fix a detcheck failure
Fix a CRAP failure
Fix a conformance failure
Follow a change recipe
Check a deterministic package
Format the code
Add a decision row
Pick a first issue
Pair with a reviewer before you touch a dangerous area
Ship the first PR
Complete the graduation checklist
Use the glossary
Key takeaways
Checkpoint
Further reading
Try it (a half-day hands-on lab)

Introduction

This book describes the repository. The chapters describe what `manifester` is, how `manifester` works, and why the code has its current form. The chapters also give the steps for your first change. The chapters use the code and docs on the `toys3` branch. Each chapter cites the files the chapter describes.

Manifester is a Kafka-compatible broker with no local disk. Record batches live in S3 segment objects. A separate DynamoDB-backed control plane assigns offsets. The broker is a single-threaded deterministic `Step(Event) → []Action` state machine. The broker runs unchanged in production and in a seeded simulator.

How to use this book

The list below describes use of this book.

- Read the chapters in order on your first read. Each chapter uses facts from the chapters before it. Each chapter ends with key takeaways, checkpoint questions, a Try it exercise, and further reading. The checkpoint answers are hidden. Answer the questions before you read the answers.
- Use the book as a reference after your first read. Chapter 2 has an “If you want X, go to Y” table. Chapter 12 has “Change X then touch Y and test Z” recipes.
- If a chapter and the code disagree, follow the code and `docs/decisions.md`. Fix the chapter in the same PR.

Choose your reading path

The table below lists reading paths by the time you have.

Time you have	Read	You will be able to
1 hour	Ch 1, the map and binaries sections of Ch 2, and the cookbook section of Ch 12	Explain the system, find anything, build and run it locally
1 day	Ch 1-4, Ch 7, Ch 12	Trace a Produce from TCP to ack, run the local stack, read broker code
1 week	All 12 chapters and the Try-it labs	Own a change end to end, across core, control plane, simulation test, and all quality gates

---

Contents

The table below lists the 12 chapters in order.

#	Chapter	Lines	Core question the chapter answers
1	The Big Picture	313	What is this, why does it exist, what is it *not*?
2	Map of the Repository	336	Where does everything live, and which binary does what?
3	The Deterministic Core	358	What are Events and Actions, and what may the core never do?
4	Broker Data Plane	416	How does Produce get sealed, stored, committed, acked, and Fetched back?
5	Wire and Storage Formats	538	What bytes go on the wire and into S3?
6	Control Plane	397	Who assigns offsets, and how does failover work without double-assigning?
7	Production Runtime	312	How do Actions become real goroutines, S3 calls, and TCP writes?
8	Deterministic Simulation	286	How does one seed replay an entire cluster, crashes included?
9	Verification and Quality Gates	263	How do you *know* it is correct, with specs C1-C4, detcheck, and CRAP?
10	Deployment, Benchmarking, Observability	265	How does it run on AWS, and how fast is it?
11	Decisions, History, and Roadmap	305	How did we get here, what was superseded, what is next?
12	Newcomer Playbook	558	What do you do in your first week, and how do you land a first PR?

The 12 chapters total 4,347 lines.

Five ideas in every chapter

The list below names five ideas that appear in every chapter.

- Logic does no I/O. `core/` and `broker/` only transform `Step(Event) → []Action`. Executors in `sys/prod` and `sim/` do the I/O. See Ch 3, 7, and 8.
- Ack only after the data is durable and committed. The order is S3 PUT, control plane commit, and ack. Do not ack before the commit. See Ch 4 and 6.
- The control plane owns offsets. Brokers are stateless and interchangeable. The newest committed “run” defines a partition’s end. See Ch 6.

- One seed produces the same history on each run. Simulation uses one random number generator, one queue, and no wall clock. Each failure replays with the same history. See Ch 8.
- Specs define correctness. `veritas/` holds executable specs C1-C4. Mutants must fail. `detcheck` and the CRAP gate check the rest. See Ch 9.

After the book

The list below names the next steps after you finish the book.

- Skim `../decisions.md`. Read at least decisions I, 21, 35, 41-45, and 48-53. Chapter 11 explains the decisions in the list.
- Run the half-day lab at the end of Chapter 12.
- Pick a “safe first area” from Chapter 12 and ship a small change.

Why Manifester keeps brokers, data, and metadata apart

You read this chapter before you read code. You learn why Manifester exists. You learn why the broker, the object store, and the control plane hold separate jobs. You also learn what the project leaves out. Later chapters explain the broker, the control plane, and the simulator in detail.

The view in this chapter is plain. Manifester trades produce latency for lower storage cost and simpler broker replacement. The deterministic core pays for that trade. The core lets the same code run in production and in a simulator that repeats a failure from one seed.

Every factual claim in this chapter traces to a file in this repository. Paths such as `core/doc.go` use the repository root as the base. The log in `docs/decisions.md` numbers each entry. The words decision 41 name that row.

Manifester is a Kafka-compatible broker written in Go. The module is `manifester` in `go.mod`. The broker keeps no local disk for log data. The broker PUTs producer record batches into segment objects. S3 or another object store holds the segment objects. A separate control plane service owns all metadata. The control plane records which segments exist. The control plane also records the offsets that the control plane assigned to each batch inside a segment.

Brokers hold no durable state. One broker can replace another broker. Any broker can accept a produce for any partition. The control plane assigns offsets only when the control plane commits a segment. The broker acknowledges a producer only after that commit is durable.

The broker is a single-threaded deterministic state machine. The machine has the signature `Step(Event) -> []Action`. The same logic runs in production against real S3 and DynamoDB. The same logic also runs in the seeded single-goroutine simulator in `sim/`. The simulator injects crashes, lost responses, and slow storage. The simulator must reproduce any failure from the seed alone.

Kafka cost grows because one machine does three jobs

A conventional Kafka cluster puts three jobs on the same machines. The machines serve requests. The machines store log data on local disks. The machines copy that data between brokers and zones. You scale partitions by scaling brokers. You retain data by retaining disks. A broker failure starts a new copy of the data. Section 5 compares this design with WarpStream from public sources.

Manifester splits the three jobs in the list below.

- Data lives in object storage. The broker PUTs batches once to S3 as segment objects. Object storage gives durability. Brokers never copy log data. Decision 5 says “No local disk in v1.”
- Metadata lives in a separate control plane. The control plane is a distinct HTTP service in `cp/`. The binary is `cmd/manicp`. The control plane assigns offsets. The control plane records where every batch landed. Brokers never see the control plane storage directly. See decision 3 and `core/controlplane.go`.

- Brokers hold no durable state. Any broker can serve any partition. The design has no partition leadership, broker epoch, or fencing. See decisions 29 and 44. A dead broker loses only memory. The object store or the control plane holds everything durable.

The trade is latency for cost and simplicity. A produce waits for an object store PUT and a metadata commit. The broker can acknowledge the produce only after the PUT and the commit finish. `loop.md` gives this example.

```
ProduceReq ──► Step ──► [S3Put(seg-42)]
S3PutDone  ──► Step ──► [MetaCommit(p0: 100..199)]
CommitDone ──► Step ──► [Reply(conn7, offset=100)]     ← ack happens here, never earlier
```

That order is the durability contract. The broker acknowledges no offset before the object store holds the data. The broker acknowledges no offset before the control plane applies the commit. Simulator invariants check this rule. `loop.md` item 13 states the rule as “no offset acked unless it’s durable”.

Three binaries run the split

The diagram below shows the parts and the order of the calls between the parts.

```
flowchart TB
    subgraph clients["Kafka clients"]
        P["Producers<br/>(idempotence disabled)"]
        C["Consumers<br/>(static assign today)"]
    end
    subgraph brokers["Brokers — cmd/manibroker (stateless)"]
        B1["manibroker 1<br/>broker/ Step(Event)"]
        B2["manibroker 2<br/>broker/ Step(Event)"]
        BN["manibroker N<br/>broker/ Step(Event)"]
    end
    S3[("Object store (S3)<br/>seg/&lt;random id&gt; segments")]
    CP["Control plane — cmd/manicp<br/>one active appender + hot standby"]
    DDB[("DynamoDB<br/>LOG/&lt;slot&gt; log + R# index rows")]
    PEB[("Pebble per CP node<br/>applier: ends, runs, keys")]
    P -->|"Produce (kwire/)"| B1
    P --> B2
    C -->|"Fetch / ListOffsets / Metadata"| B2
    C --> BN
    B1 -->|"1. ObjPut segment"| S3
    B2 -->|"1. ObjPut segment"| S3
    BN -->|"1. ObjPut segment"| S3
    B1 -->|"2. CPCall CommitSegment<br/>3. ack after CPDone"| CP
    B2 -->|"CPCall CommitSegment / Lookup"| CP
    BN -->|"CPCall CommitSegment / Lookup"| CP
    B2 -->|"Fetch: ranged ObjGet per batch"| S3
```

```
    CP -->|"append if-absent PutItem"| DDB
    CP -->|"standby tails log, applies in order"| DDB
    CP -->|"apply via cp/applier"| PEB
    CP -.->|"R# rows ~every 10 s (decision 50)"| DDB
```

The table below is book apparatus. You use the table as reference. The argument in this chapter does not depend on you memorising the table.

Binary	Source	Role
`manibroker`	`cmd/manibroker`	Kafka broker. It handles the wire protocol, buffering, sealing, PUT, commit, and ack.
`manicp`	`cmd/manicp`	Control plane service. It appends commands to the log, applies them, assigns offsets, and answers `Lookup` and `DescribeTopics`.
`manisim`	`cmd/manisim`	Simulator runner. It runs the broker with fake S3, a fake control plane, and fake clients. It uses one goroutine per seed.

The list below names the package that owns each part. You meet the packages in this order in the book.

- `core/` has plain-data Events and Actions, the `StateMachine` contract, and the `Env` interface. The package does no input or output.
- `broker/` has the broker state machine.
- `kwire/` has Kafka framing and codec code built on franz-go `kmsg`. See decision A.
- `format/` has the segment byte layout and keys. See decision 22.
- `cp/` has the control plane handler and stores. The package also has `cp/cpclient`, `cp/logcp`, and `cp/applier`.
- `sys/` has the production runtime. The package has TCP, goroutine executors, real S3 and DynamoDB adapters, and metrics.
- `sim/` has the simulator kernel and the fault-injecting engines.
- `veritas/` has executable specs. The project uses the specs as independent oracles. See decision 21.

One deterministic step lets the same machine run in production and in simulation

One rule shapes the shape of the code. The rule follows.

> All stateful logic is a single-threaded function `Step(Event) []Action`. The function does no input or output. The function reads no clock. The function draws no randomness except through an injected `Env`. Everything else exists to feed events in and carry actions out.

`core/doc.go` states the rules for deterministic packages. The rules cover `core/` and `sim/`. Continuous integration enforces the rules. The packages must not import `time`, `math/rand`, `crypto/rand`, `os`, `net`,

`sync`, or any software development kit. The packages must not use goroutines, channels, or `select`. Map iteration must never decide behaviour.

Events and actions are plain data. `core/machine.go` defines the contract. `StateMachine` calls `Step` the only place state changes. The contract says `Step` never blocks. The contract says `Step` does no input or output. `Machine` adds `StateHash()`. The contract requires hashing. `Factory` builds a fresh machine on each boot. The boot case includes a boot after a crash.

`core/event.go` and `core/action.go` define the event names and the action names. Events include lifecycle events such as `Started` and `Stop`. Events include network input such as `BytesReceived` and `ConnClosed`. Events include timers such as `TimerFired`. Events include a completion for every input or output action. The completions include `ObjPutDone`, `ObjGetDone`, `CPDone`, and `SendDone`. Each completion has a matching correlation `ReqID`.

Actions include `ObjPut`, `ObjGet`, `CPCall`, `Send`, `SetTimer`, and `PauseReads`. Actions also include `Trip` and `Exit`. `Trip` means fail-stop. You use `Trip` when the machine can no longer trust memory. `Exit` means graceful shutdown.

Input and output always stay asynchronous. An action goes out. A `...Done` event comes back. Nothing inside `Step` waits. See `loop.md` item 6.

Timers work the same way. `SetTimer` goes out. `TimerFired` comes back. A generation number matches the timer request and the timer event. A stale timer therefore causes no harm. The segment flush timer in section 4 works this way.

Production runs `Step` on one goroutine and carries out actions on many goroutines

`sys/` runs the machine in production. `cmd/manibroker` wires the machine and the runtime together. Reader goroutines push `BytesReceived` into a bounded data inbox. Executor goroutines run S3 PUTs and control plane HTTP calls. Executor goroutines push completions into a strict-priority control inbox. See decisions G and H.

Exactly one goroutine dequeues events. The goroutine takes control events first. The goroutine uses the shared pure function `core.InboxPolicy`. The goroutine then calls `Step`. Hundreds of PUTs may run at once. Only `Step` runs serially. `Step` mutates only memory. `loop.md` puts the time for `Step` at microseconds. The same note puts the production cost of the design at a plain function call. The note measures that cost in nanoseconds. The note contrasts that cost with S3 calls that take milliseconds.

Simulation runs the same machine on one goroutine and one virtual clock

`sim/` and `cmd/manisim` run the machine in simulation. One goroutine owns everything. The goroutine owns a global queue. The queue orders events by `(simTime, seededRandomTie, seq)`. See decision 18. The goroutine owns one seeded random number generator. The goroutine owns a virtual clock. The clock jumps to the next event.

The fake engines inject faults. The engines inject random latency, 503s, and timeouts. The engines can report a PUT as lost after the PUT succeeds. The engines can duplicate completions. The engines can reorder completions. The engines can crash and restart the broker at any event. See `loop.md` item 12.

A crash discards broker memory. The broker then rebuilds from the object store and the metadata store. That rebuild works because brokers hold no durable state. Section 1 describes that choice.

The simulation uses no real goroutines, sockets, or wall clock. The same seed gives the exact same history. The simulator checks invariants after every step. The simulator hashes state into a running trace hash. Continuous integration runs each seed twice. The second run catches leaked nondeterminism. See `loop.md` items 13-15.

A failure gives you one command. The command has the form `go run ./cmd/manisim -scenario kafka-log -seed <n>`. The determinism rules stay strict for one reason. Production code and simulated code are the same code.

A produce earns an acknowledgement only after the PUT and the commit

The four steps below form one sequence. Each step waits for the step before the step.

1. Buffer and seal. `broker/` collects batches from many clients and partitions into one open segment. The broker seals the segment at 16 MiB. The broker also seals the segment 200 ms after the first batch enters the segment. Later batches do not extend that fixed timer. The segment holds at most 4096 partitions and 256 batches. See `broker/config.go` and decisions 35 and 52. The broker stores batches verbatim. The verbatim bytes include compression. The broker validates headers and CRCs. The broker decompresses nothing. See decision 45.
2. PUT to object storage. The segment becomes one object at `seg/<random id>`. The id has 128 random bits in hex. The broker draws the id through `Env.NewID()`. Keys therefore stay deterministic in simulation. See decision 43. The `format/` layout has a header, a per-batch index, and the opaque batch bytes.
3. Commit. The broker issues `CPCall{CommitSegment{SegmentKey, Batches}}`. See `core/controlplane.go`. The request has no offsets. The control plane assigns each batch a `Base`. The control plane sets `Base` to the current end of the partition. The control plane advances in request order. Offsets stay contiguous per partition. The control plane decides offsets at commit time. The control plane does not decide offsets at upload time. A commit stays idempotent per segment key. An identical retry returns the original ranges. The same key with a different request returns `ErrConflict`.
4. Acknowledge. The broker replies only after the applier applies the slot for the commit. Section 6 describes the applier. Commit retries run to a 120 s deadline. The deadline has the name `CommitDeadline`. See decision 47. A control plane restart or failover therefore stalls acknowledgements. The restart or failover does not fail the acknowledgements.

Fetch follows a similar path. `Lookup` asks the control plane which runs hold offsets from a given point. The broker then runs ranged GET requests. The broker patches each batch `baseOffset` and leader epoch. Both fields sit outside the batch CRC. The CRC therefore stays valid. See decision 30 and `broker/fetch.go`.

WarpStream shares the broad split and differs in the parts that matter

Manifester and WarpStream share broad design choices. Manifester is a separate clean-room project. One document gives the only approved comparison. The document is `docs/warpstream-architecture-comparison.md`. That document calls itself a “desk study from PUBLIC sources only” in section 0 of the document. The sources are published docs, the blog, and marketing pages.

No one studied the vendor container image for that document. The project treats reverse engineering as presumptively prohibited. A separate teardown exists outside the committed tree. Section 0 of `docs/roadmap-from-warpstream-teardown.md` recommends legal advice before anyone publishes material derived from the teardown. Section 0 also recommends that you keep the teardown out of the repository. You do not cite the teardown.

`docs/warpstream-architecture-comparison.md` verifies several points from public sources. The document marks the verified points [V]. The WarpStream agent is “a single stateless binary”. The agent has no local disks. The agent has no cross-AZ replication. One file holds many topics and partitions.

WarpStream writes buffer for a default of 250 ms or 8 MiB. A file has a maximum of 4 MiB. The agent then PUTs the file. The agent commits metadata to a hosted journaled metadata store. The store has the name “Virtual Cluster”. The agent acknowledges only after that commit. The store assigns order and offsets at commit. Reads use a zone-aware distributed file cache. The cache loads data roughly once per zone.

Manifester uses the same broad design. Brokers hold no durable state. Segments hold batches from many partitions. The broker PUTs, then commits, then acknowledges. The control plane assigns offsets at commit.

The mechanisms differ in most other respects. Manifester hosts the control plane for Manifester. Manifester uses a DynamoDB command log with a Pebble applier. Section 6 describes that log and that applier. WarpStream hosts the metadata store for WarpStream.

Manifester drops idempotent producers. WarpStream supports idempotent producers. Manifester lacks compaction. Manifester lacks the zone-aware cache. Manifester lacks backend consumer groups.

You read the side-by-side table in section 3 of the comparison document before you claim parity. You read the ranked takeaways in section 4 of the comparison document too. The document warns you about the cost figures in the document. The document calls the cost figures hypotheses until someone measures the cost figures from a bill.

One ordered log gives every broker the same offsets

The production control plane uses a log, slots, and an applier. See decisions 48-53, `docs/log-cp-service.md`, and `cp/logcp/doc.go`.

The list below explains why the log has the shape that the log has.

- Brokers send every `CommitSegment` to one active control plane node. That node is the appender. Other control plane nodes are hot standbys. The standbys follow the log. All brokers use the same

appender. You spread commits across nodes and you get back the contention of the earlier design. See decision 48.
- The appender writes the raw command into the next slot. The command has no offsets. One slot is one DynamoDB item. The item has the name `LOG/<slot>`. The appender writes the item only if the item is absent. The slot acts as the fence. Two candidate appenders race on the same slot. One appender wins. The other appender retries later. Safety needs no lease. See decision 51.
- Every replica applies slots in strict order through `cp/applier`. See decision 49. The applier is an embedded Pebble key-value store. The applier keeps partition ends. The applier keeps one packed run per partition and segment pair. The applier keeps per-key idempotency records. Offsets exist only after apply. The control plane answers a commit only after the applier of the control plane applies that slot.
- The idempotency window is a deterministic function of slot number. `WindowSlots` is 131,072. See decision 53. Every replica therefore forgets the same keys at the same log point. Slots carry a version. A replica meets a newer format and stops. The replica serves 503. The replica does not skip the slot. Upgrades take two deploys.
- A background indexer writes the per-partition `R#` DynamoDB index rows after the ack. The indexer runs about every 10 s. See decision 50. `Lookup` answers from applier memory. The project accepts work to move the index itself to S3. The project does not schedule that work. See `docs/TODO.md`.

An older control plane remains in the tree. The older control plane used a DynamoDB transaction store first. The older control plane then used a single sequencer with group commit. See decisions 39, 42, and 46, and `cp/doc.go`. The log store is the benchmark default.

What Manifester leaves out limits the claims you can make

The list below names the absent parts. Each item names the file or the decision that records the absence.

- Consumer groups are absent. The broker serves exactly five Kafka APIs. The APIs are Produce, Fetch, ListOffsets, Metadata, and ApiVersions. Produce covers v3-9. Fetch covers v4-12. See `kwire/supported.go`. The broker serves no FindCoordinator, JoinGroup, OffsetCommit, or transactions. Benchmarks consume with static `assign()`. Consumer groups are roadmap item 4 in `docs/roadmap-from-warpstream-teardown.md` section 2. The estimate is 35-45 engineer-days. No one has built consumer groups yet.
- Idempotent producers are absent by decision. Decision 41 removed idempotent producers. Delivery is at-least-once. Idempotent batches get `INVALID_REQUEST`. Clients must disable idempotence. Clients use franz-go `kgo.DisableIdempotentWrite()` or `enable.idempotence=false`. Retries can duplicate records. Retries can also reorder records within a partition. That reorder case needs `max.in.flight.requests.per.connection > 1`. `docs/TODO.md` says idempotent producers are “explicitly not planned”. A retried segment commit cannot duplicate records. Section 4 explains why a retried segment commit cannot duplicate records.
- The consumer read path is minimal. Fetch answers immediately. Fetch ignores `MinBytes` and `MaxWait`. Fetch therefore has no long-polling. See decision 29. Fetch runs one ranged GET per batch. Fetch does no coalescing. Fetch has no cache. Fetch costs one control plane `Lookup` per partition. See `broker/fetch.go`. Audit arithmetic measured one idle consumer at about 20.6k Lookups per second locally. Audit arithmetic predicted about 6.5k GET per second at 1 GiB per second of reads. The read path work is roadmap item 1. The work covers long-poll, a Lookup

cache, coalesced GETs, and a chunk cache. `docs/TODO.md` calls the work “needed before any consume stage”. You treat every produce number in this book as produce-only until that work lands.

- Compaction, retention, and garbage collection are absent. The broker never merges segments. The broker never deletes segment objects. Orphans from failed commits therefore accumulate. See roadmap item 2. The audit notes a present hazard. A bucket lifecycle rule could delete live `seg/` objects today. You treat that rule as a hazard to know. You do not copy that rule.
- Metadata names only the answering broker. See decision 29. Client failover therefore depends on the seed list. Client failover takes about 5 s with franz-go defaults. `docs/TODO.md` accepts work to list every live broker through a heartbeat lease. No one has done that work. The project declined per-broker partition ownership. That choice keeps brokers interchangeable.
- TLS, SASL, and ACLs are absent. Roadmap section 2 calls TLS, SASL, and ACLs “required before a serious OSS announcement”. You do not expose these binaries beyond a trusted network.
- The control plane has a single appender by design. Failover costs an ack gap. Run w4 measured the gap at about 8 s at 500k messages per second. See section 8.

The current evidence proves less than the headline numbers suggest

The repository states the caveats plainly. You repeat the caveats. You do not soften the caveats.

Run w4 is the main measurement. See `docs/w4-log-store-results.md`. The run bears the dates 2026-10-20 and 2026-10-21. The run used single availability zone ap-south-2a. Six `m6in.4xlarge` brokers used the log-backed control plane. The brokers held about 998k messages per second for a 4-minute stage. The rate equals about 975 MiB per second across 288 partitions. The stage used 64 producers, 1 KiB lz4, and produce only. Publish p99 was 756 ms. The run had zero errors.

The control plane used little capacity during run w4. Append p99 was 8 ms. The control plane used about 0.2 of 4 cores. The control plane ran at 20-32 slots per second. The team killed the active appender at 500k messages per second. The kill gave an about 8 s ack gap. The kill gave zero producer errors.

This path replaced an older commit path. The older path failed at about 240-275k messages per second in w2 and w3. The w4 document summarises those runs.

The w4 document also states the caveats. The stages were short. Each stage ran 1 to 4 minutes. Each stage had no warm-up. Each stage had one sample. The document calls the stages indicative. The document denies that the stages give a steady-state characterisation. No run measured more than 288 partitions. The run used one availability zone. The run therefore supports no cross-AZ cost claim. The run produced no consumer latency number. The run produced no end-to-end latency number.

Cost per GiB came from unit prices. Cost per GiB did not come from a bill. Local numbers in `docs/benchmarks.md` predate decisions 41-44. The local numbers say so. You never quote a figure without the stage length and the sample count.

The simulator gives separate evidence. The `kafka-log` campaign ran 2,000 seeds with 0 failing. The campaign ran 50.7M steps. The campaign re-checked determinism every tenth seed. Each deliberate mutant failed within 6 seeds. See `docs/sim-log-engine.md`. That result covers logic under injected faults.

That result does not cover steady-state performance. That result does not cover multi-AZ performance. That result does not cover consumer performance.

The project itself is early. The project is a pre-release single-author research codebase. The project has fewer than two hundred commits. `docs/roadmap-from-warpstream-teardown.md` section 3 gives a bus factor of 1. The repository had no root `LICENSE` or `README.md` at the time of that roadmap. The roadmap marks both files as BLOCKERs for open-sourcing. The module is still the non-importable `module manifester` in `go.mod`, with go 1.24.

Determinism continuous integration lives at `build/ci/determinism.workflow.yml`. Determinism continuous integration does not live in `.github/workflows`. You read the open source checklist in the roadmap before you assume any release process exists.

Glossary

The glossary below is book apparatus. You use the glossary to check a name. The glossary does not add new claims.

- Broker. One `manibroker` process. The broker runs the stateless Kafka-protocol state machine in `broker/` through `Step`. The broker buffers produces. The broker seals segments. The broker PUTs segments. The broker commits through the control plane. The broker serves Fetch from object storage. Any broker can serve any partition. See decisions 29 and 44. Brokers hold no durable state.
- CP. CP names the control plane. The control plane is the separate `manicp` HTTP service in `cp/`. The control plane owns metadata. The metadata includes topic partition counts and per-partition end offsets. The control plane records which segment holds which offset range. The control plane alone assigns offsets. See decisions 3 and 42. The control plane assigns offsets through `CommitSegment`, `Lookup`, and `DescribeTopics` in `core/controlplane.go`.
- Segment. One immutable object at `seg/<random id>`. The segment holds verbatim batches for many partitions. The segment holds up to 4096 partitions by default. The broker seals the segment at 16 MiB or after a 200 ms window. See `broker/config.go` and `format/`. A segment is the unit of PUT, commit, and idempotent retry.
- Run. The contiguous slice of one partition log from one segment commit. The run records `[Base, Base+Count)` mapped to byte ranges inside one segment. See `core.BatchRange`. The control plane stores one packed item per partition and segment pair. See decision 39. A partition log is a sequence of runs. The end offset of a partition log is the newest run `Base+Count`.
- Slot. One numbered position in the control plane DynamoDB command log. The item is `LOG/<slot>`, written if-absent. The item holds one or more raw commands and no offsets. Replicas apply slots in strict order. The slot fences the appender against a standby. See decisions 48, 51, and 53. Trimmed slots become tombstones.
- Applier. The deterministic state machine in `cp/applier`. Each control plane replica runs the applier over the log. Applying a slot assigns offsets, extends runs, and records idempotency keys in the embedded Pebble store of the applier. The control plane acknowledges a commit only after the applier applies the slot for the commit. Replicas apply the same slots and reach the same state. See decision 49. `veritas/` is the independent oracle.

Reading order for your first week

You do not need to read the repository from top to bottom. You read the repository in data flow order. You keep `docs/decisions.md` open beside the code.

The five steps below form one sequence. You do the steps in order.

1. Start with `loop.md` and `core/`. Read `doc.go`, `machine.go`, `event.go`, `action.go`, and `controlplane.go`. The files run to a few hundred lines together. The files define every noun used elsewhere.
2. Follow one produce through `broker/`. Trace how bytes from a connection become batches. Trace how the open segment seals. Find where the code emits `ObjPut`, `CPCall`, and the final `Send`. The package has no sockets, clock reads, or goroutines.
3. Read the control plane from the contract outward. Start with `core/controlplane.go`. Then read `cp/logcp/doc.go` and `cp/applier/doc.go`. Then read `docs/log-cp-service.md` for slots, checkpoints, and failover.
4. Read the production runtime last in this pass. The runtime lives in `sys/`, `cmd/manibroker`, and `cmd/manicp`. Runtime files should contain wiring and adapters. You raise a question in review if a runtime file contains business logic.
5. Run the simulator. See the “Try it” section below. Then read one invariant in `sim/`. Look at how the trace hash, the oracle, and the mutants fit together. The simulator is the main correctness argument for the project.

You search `docs/decisions.md` first when a behaviour surprises you. You search code second. The 200 ms fixed timer has a decision behind the timer. The missing high-watermark item has a decision behind the absence. The absence of fencing has a decision behind the absence. Each decision records a measured reason.

You trust the code when a document and the code disagree. You flag the document. Prose lags behind code at this project pace.

Key takeaways

The list below sums up the chapter. You quote the caveats with the numbers.

- Manifester splits the jobs that a Kafka broker normally combines. Brokers serve requests and hold no durable state. S3 or another object store holds the data. A separate control plane holds metadata and assigns offsets. See decisions 3, 5, 29, and 44.
- The durability contract is PUT, then commit, then ack. The broker never acknowledges earlier. The control plane decides offsets at commit and apply time. Any broker can therefore write any partition. See `loop.md` and `core/controlplane.go`.
- All stateful logic is a single-threaded deterministic `Step(Event) -> []Action` machine. See `core/doc.go` and `core/machine.go`. Production and simulation are two executors for the same code. The executors differ only in how the executors carry out actions.
- Large areas are absent by design. The absent areas include consumer groups and idempotent producers. The absent areas include a full consumer read path with long-poll, caching, and

coalescing. The absent areas include compaction, retention, garbage collection, TLS, SASL, and ACLs. See section 7 and `docs/TODO.md`.

- The strongest evidence today is narrow. Run w4 was short, single-sample, and produce-only at about 1M messages per second. The simulator campaign ran 2,000 clean seeds. You quote both results with the caveats. You do not present the results as steady-state or symmetric claims. See `docs/w4-log-store-results.md` and `docs/sim-log-engine.md`.

Checkpoint questions

You answer the five questions below from memory. You open the answers only after you answer.

1. A producer sends a batch and the broker crashes after the S3 PUT succeeds but before the CP commit is applied. What does the producer observe, and why is no data lost or double-acked?

Answer: The producer is never acked. The broker emits the ack only in the Step that handles the commit completion, after the applier applies the slot. See section 4 and `loop.md`. The segment bytes are durable under their `seg/<random id>` key. Without a commit, the segment is an orphan. Readers cannot see it because Lookup returns only committed runs. A re-produce can duplicate records under at-least-once delivery. See decision 41. A retried commit under the same key is idempotent. It returns the original ranges instead of assigning new offsets.

1. Why can any broker accept a produce for any partition, and what makes the resulting offsets coherent?

Answer: Brokers never choose offsets. A broker PUTs a segment and asks the CP to commit the segment. The applier assigns each batch `Base` = the partition's end at apply time. The applier assigns offsets contiguously and in log order. See decisions 42 and 48 and `core/controlplane.go`. All commits pass through one ordered log. Concurrent brokers therefore cannot race into overlapping offsets. The design needs no partition leadership or fencing. See decisions 29 and 44.

1. What is forbidden inside `core/` and `sim/`, and what breaks if you quietly call `time.Now()` in the broker?

Answer: `core/doc.go` bans `time`, `math/rand`, `crypto/rand`, `os`, `net`, `sync`, and any SDK. It also bans goroutines, channels, `select`, and behaviour decided by map iteration. A wall-clock read makes Step depend on state the simulator cannot reproduce. The same seed would diverge between its two CI runs. The trace-hash determinism check in decision 16 would fail. A failing seed would no longer reproduce. Time must enter as event fields such as `At` and `Wall` in `core/event.go`. It can also enter through timers such as `SetTimer` and `TimerFired`.

1. Name three things a newcomer might assume Manifester has, which it does not, and cite where each absence is recorded.

Answer: Any three of the following will do. Consumer groups are absent. Five APIs are listed in `kwire/supported.go`, and consumer groups are roadmap item 4. Idempotent producers are absent. See decision 41 and "explicitly not planned" in `docs/TODO.md`. A long-polling or cached read path is absent. Fetch ignores `MinBytes` and `MaxWait` under decision 29 in `broker/fetch.go`, and the read path is roadmap item 1. Compaction, retention, and garbage collection are absent. See section 7 and roadmap items 2 and 5.

TLS, SASL, and ACLs are absent. See roadmap section 2. Metadata listing all live brokers is absent. `docs/TODO.md` records it as accepted but not done.

1. A colleague says, “Manifester does 1 GiB per second, cheaper than WarpStream.” What two corrections do you make before that sentence leaves the team?

Answer: First, w4 measured about 975 MiB per second for produce only. The stage ran 4 minutes with one sample, 288 partitions, a single AZ, and no consumers. The stage does not prove steady-state or symmetric 1 GiB per second. The stage does not compare to a symmetric benchmark either. See “Gaps and caveats” in `docs/w4-log-store-results.md`. Second, the cost advantage came from unit prices. No bill measured the cost advantage. The cost advantage also excludes reads. The comparison document calls the cost advantage a hypothesis and warns against claiming parity. See sections 3-4 of `docs/warpstream-architecture-comparison.md` and section 4a of `docs/roadmap-from-warpstream-teardown.md`.

Try it

Run these commands from the repository root. All three are read-only. The campaign is CPU-only and needs no AWS, Docker, or credentials.

```
# 1. Read the contract the whole design hangs on (two short files).
cat core/doc.go core/machine.go

# 2. Run a small deterministic campaign against the log-backed CP.
#    Expect a summary line ending in "0 failing"; any failure prints
#    a one-seed repro command. (Full 2,000-seed runs take longer.)
go run ./cmd/manisim -scenario kafka-log -seeds 20

# 3. Skim the decision index before your first code review.
grep -E '^\| (4[1-9]|5[0-3]) ' docs/decisions.md
```

Further reading

The list below names the sources for this chapter. You read the sources before you cite a figure from this chapter.

- `loop.md` is the design note this chapter summarises. It covers the event loop, production and simulation, and the 15 instructions.
- `core/doc.go` and `docs/decisions.md` give the determinism rules and the numbered decisions cited in this chapter. Decisions 29 and 35 and decisions 41-53 matter most here.
- `docs/warpstream-architecture-comparison.md` is the WarpStream comparison from public sources only. Read section 0 and section 6 before citing any vendor figure.
- `docs/log-cp-service.md` and `cp/logcp/doc.go` describe the log-backed control plane in full. They cover slots, the applier, and standby.
- `docs/w4-log-store-results.md` is the current best performance evidence. It includes “Gaps and caveats”.

- `docs/TODO.md` and `docs/roadmap-from-warpstream-teardown.md` list accepted work that is not scheduled. They also give the ranked roadmap behind section 7.

Chapter 2. The repository has one deterministic centre and six binaries in `cmd/`

This page lists the top-level directories, the dependency rules, the two executors, and the six binaries. Section 2.7 maps a task to the package, file, or command that holds the work. The table in section 2.7 is the lookup that the rest of the book uses.

The Go module is a single module, `manifester` (`go.mod`, Go 1.24). Every import path starts with `manifester/…`. There is no top-level `internal/` directory. Convention, package documentation, and the determinism gate set the boundaries. The compiler does not hide packages. Any package can import any other package. Authors follow the stated boundaries.

2.1 The tree lists the top level and the subpackages you open first

The tree lists the top level and the subpackages you open first. Test files (`*_test.go`) sit beside the code the test files test throughout the repository.

```
.
├── core/          Deterministic heart: Event/Action plain data, StateMachine, Env
├── broker/        Kafka broker state machine (pure, deterministic; core.Machine)
├── cp/            Control-plane service: HTTP handler + Store implementations
│   ├── applier/     Log-backed CP state machine over an ordered KV (decision 49)
│   │   └── pebblekv/  applier's Pebble-backed KV
│   ├── cpclient/    Broker-side HTTP client for the control plane
│   ├── cpobs/       Prometheus collectors for manicp (incl. sequencer metrics)
│   ├── cptest/      Shared conformance suites every Store must pass
│   ├── dynamo/      DynamoDB transaction Store + in-process Sequencer
│   │   └── fakedb/    In-memory DynamoDB fake used by dynamo tests
│   ├── logcp/       Log-backed CP service (DynamoDB log → applier; design A)
│   │   └── logcptest/   Test helpers for logcp
│   ├── logsm/       Prototype log state machine (not wired into manicp)
│   └── memstore/    In-memory Store: the spec behind a mutex
├── sim/           Deterministic simulator kernel (one goroutine, one counted RNG)
│   ├── echo/        Milestone-1 toy broker/workload smoke system
│   ├── engines/     Simulated outside world: cpengine/, logengine/,
│   │                netsim/, objstore/ (+ internal/ test support)
│   └── kafka/       Sans-I/O Kafka clients, oracle, invariants for sim runs
├── sys/           Production seam: narrow interfaces over real I/O
│   ├── awsobs/      AWS observability helpers
│   ├── httppool/    HTTP connection pooling helpers for CP clients
│   ├── prod/        Production wiring: clock, Env, executor, TCP Net, metrics
│   ├── runinfo/     Run id, log-level parsing, startup manifest
│   └── s3store/     S3 ObjectStore adapter (C3-checked against veritas)
```

```
├── kwire/          Pure Kafka wire layer: framing + codec over franz-go kmsg
├── format/         Segment object byte layout and segment-key naming (pure)
├── veritas/        Executable specs (oracles) of the outside world
│   ├── veritastest/   Shared lockstep checks for spec implementations
│   └── vsys/          Production-facing adapters over the specs (test fakes)
├── api/
│   └── openapi/    OpenAPI 3.1 spec of the CP HTTP API (+ cmd/specgen, json2yaml/)
├── cmd/            The six binaries (§2.4) — thin main packages over the above
├── it/             Integration tests: real brokers + real franz-go (kgo) clients
├── tools/
│   └── detcheck/   The determinism analyser (§2.3) + allow.txt allow-list
├── scripts/        audit-dep.sh, build-omb.sh, ci-determinism.sh, it-up/down.sh
├── deploy/         docker-compose.yml (test backends); aws/ (Terraform + bench)
├── docs/           Design record: decisions.md, lessons.md, runbooks (§2.6)
├── build/          Generated/local artefacts — do not edit (§2.5)
├── results/        Benchmark experiment outputs — do not edit (§2.5)
├── Makefile        make build is the full gate (§2.2)
├── crap-baseline.json  Checked-in CRAP ratchet baseline for cmd/crapscore
└── go.mod / go.sum     Single module: manifester
```

Directories named after a thing are libraries. The `broker/`, `format/`, and `kwire/` directories are examples. The `cmd/` directory holds argument parsing and wiring. The `sys/` package and the `sim/` package execute the same state machines. The `sys/` package uses real sockets and S3. The `sim/` package uses a seed and replays a run.

2.2 The deterministic packages sit at the centre and the edge packages hold the nondeterminism

The repository has layers. The centre reproduces a run bit for bit for a given seed. Each layer outward adds more of the real world. The outer layers add clocks, goroutines, sockets, and cloud SDKs. The outer layers lose reproducibility.

```
flowchart TD
    core["core/<br/>Events, Actions, StateMachine, Env"]
    format["format/<br/>segment bytes &amp;amp; keys"]
    kwire["kwire/<br/>Kafka framing &amp;amp; codec (kmsg)"]
    veritas["veritas/<br/>executable specs / oracles"]
    broker["broker/<br/>Kafka broker state machine"]
    sim["sim/ + sim/echo, sim/kafka, sim/engines<br/>deterministic simulator"]
    cp["cp/ + stores<br/>control-plane service"]
    sys["sys/ + sys/prod, sys/s3store<br/>production runtime"]
    cmd["cmd/*<br/>six binaries"]
    it["it/<br/>integration tests (kgo)"]
```

```
    format --> core
    kwire --> core
    veritas --> core
    broker --> core
    broker --> format
    broker --> kwire
    sim --> core
    sim --> broker
    sim --> veritas
    cp --> core
    sys --> core
    sys --> broker
    cmd --> broker
    cmd --> cp
    cmd --> sim
    cmd --> sys
    it --> sys
    it --> broker
```

Each arrow points from the importing package to the imported package. Five rules account for most arrows and most missing arrows.

- `core/` imports almost nothing. The `doc.go` file in `core/` is the contract. The package defines plain-data `Event` values and `Action` values. The package defines the `StateMachine` contract. The package defines the `Env`. The `Env` admits the permitted nondeterminism. The permitted nondeterminism covers randomness, “buggify” fault injection, and assertion recording. The package uses no `time`, `math/rand`, `crypto/rand`, `os`, `net`, `sync`, or SDK. The package uses no goroutines, channels, or `select`. Map iteration never decides behaviour. Today `core` imports a short allow-list of standard packages. The list includes `fmt` and `strconv`.
- Pure libraries sit on `core` alone. The `format/` package defines segment layout and `SegmentKey`. The `kwire/` package defines framing and codec. The `kwire/` package has the one audited third-party import, `github.com/twmb/franz-go/pkg/kmsg`. The `veritas/` package holds sequential, fault-free specs. The specs cover `ObjectStore`, `ControlPlane`, and one Kafka `Partition`. These libraries depend on `core` and the standard library. The libraries never depend on `sys/`, on `cp/`, or on a cloud SDK. The `broker/` package composes the pure libraries. The `broker/` package imports `core`, `format`, and `kwire`. The package imports nothing that performs input or output.
- The same machine runs in both executors. The `broker.Broker` type is a `core.Machine`. The `sys.Run` function in `sys/run.go` drives the machine in production from goroutines, sockets, and timers. The `sim/` kernel drives the same code from one goroutine, one counted RNG, and one global queue (`sim/doc.go`). The simulator never implements `sys` interfaces. Production never imports the simulator. The two executors meet at the `core` Event and Action vocabulary in `sys/sys.go`.
- The control plane is an edge, not the core. The `cp/doc.go` file describes `cp/` as “not deterministic (goroutines, `net/http`)”. The `cp/` package sits outside the determinism gate. The `veritas.ControlPlane` spec is the executable spec for the control plane. The `cp/cptest` suite

checks every `Store` operation against the spec, operation for operation. The stores are `cp/memstore`, `cp/dynamo`, and `cp/logcp` through `cp/applier`.

- The `cmd/` directory and the `it/` directory may import anything. The two directories form the outer layer. Flag parsing, AWS SDK setup, and Prometheus live in the outer layer. The real `kgo` client lives in `it/` only. An SDK import belongs in an edge package. The deterministic packages `core/`, `broker/`, `format/`, `kwire/`, and `sim/` hold no SDK import.

The two gates check imports and record third-party audits

The `tools/detcheck` tool is a `go/analysis` checker in `tools/detcheck/detcheck.go`. The command `go run ./tools/detcheck` runs the checker. The checker checks imports against an exact-path allow-list. The tool embeds the list from `tools/detcheck/allow.txt`. The list covers standard library basics, `manifester/...`, and the single audited import `github.com/twmb/franz-go/pkg/kmsg`. The checker reports the following items.

- Imports that the list does not name.
- `go` statements, channel types, channel sends, channel receives, and `select`.
- Ranging over a map. Go randomises that order.
- Map iteration that reflect builds.
- Mutable package-level variables. Error sentinels are the exception.

A same-line or previous-line `//detcheck:ok <reason>` comment suppresses a finding. The reason is mandatory. The checker does not check test files.

The `scripts/ci-determinism.sh` script runs detcheck in CI. The `build/ci/determinism.workflow.yml` file wires the script into CI with `go vet` and `go test ./...`. The package list is `./core/... ./sim/... ./format/... ./kwire ./broker/... ./veritas ./veritas/veritastest`. The script then makes two more checks. The script verifies audit records. Every third-party module that those packages touch needs a reviewed record at `docs/dep-audits/<module>@<version>.md` for the exact version in `go.mod`. A version bump breaks the gate. A new audit with `scripts/audit-dep.sh` restores the gate. The script also runs each `TestDeterminismCanary`. Every seed runs twice. The two runs produce identical results. A third-party dependency in the deterministic packages needs a record in `docs/dep-audits/` and the exact path in `allow.txt`.

The `make build` command is the everyday gate in the `Makefile`. The command runs `gofmt` checking. The command runs `go vet ./...`. The command runs `go build ./...`. The command runs `go test -coverprofile` into `build/cover.out`. The command runs the CRAP gate in `cmd/crapscore` (section 2.4) with threshold `CRAP_THRESHOLD` (default 30) against `crap-baseline.json`. The `make crap-baseline` command regenerates that baseline, and the diff carries the same weight as a code diff. The `make crap-strict` command lists all offenders.

2.3 The two executors live in `sim/` and `sys/`

Later chapters move between simulation and production. The deterministic executor is `sim/`. The files `sim/kernel.go`, `sim/runner.go`, and `sim/run.go` provide `NewRand`, `NewScheduler`, `NewTimeline`, `NewNodeInbox`, `NewCostModel`, `NewRunner`, and `RunSeed`. The `sim/engines/` directory simulates the object store, the control plane, the log, and the network. The `sim/kafka/` directory holds Kafka clients with no

input or output, the oracle, and invariants. The `sim/echo/` directory is the small milestone-1 smoke system. The directory is the first simulator code you meet.

The production executor is `sys/`. The `sys/sys.go` file defines the narrow interfaces `Clock`, `ObjectStore`, `ControlPlane`, `Conns`, `Classifier`, and `Sink`. The `sys/run.go` file holds the single-goroutine event loop, `sys.Run`. The `sys/prod/` directory supplies the clock, the `Env`, the executor, TCP networking, and fact-to-Prometheus wiring. The `cmd/manibroker` binary and the `it/` directory share that wiring. The `sys/s3store/` directory is the S3 adapter. Neither executor puts the executor types into `core`. Code at the edge classifies SDK and HTTP errors into plain `core.ErrCode` and `core.Outcome` values through `sys.Classifier`.

2.4 Each of the six binaries in `cmd/` has the same shape

Every binary has the same shape. The `main()` function has a few lines. A testable `run(...)` function does the work. The function returns an exit code. Code 0 means success. Code 1 means runtime failure. Code 2 means usage errors or flag errors. The standard `flag` package parses flags into a struct. The header comment in each `main.go` file gives a one-paragraph description of the program.

`cmd/manibroker` runs one Kafka broker

The `manibroker` binary runs a single Kafka broker over S3 for the data plane and over the control plane for offset assignment. The `cmd/manibroker/main.go` file describes the binary as a program that “runs one Kafka broker over S3 and the control plane.” The binary wires `broker` into `sys/prod`. The `broker` package supplies the state machine. The `sys/prod` package supplies the executor, TCP, and metrics. The wiring uses `cp/cpclient` and `sys/s3store`. The binary takes the following flags.

- `-listen` (default `127.0.0.1:9092`) and `-advertise` name the listen address and the advertised address.
- `-broker-id` and `-node-id` name the broker and the node.
- `-cp-url` (default `http://127.0.0.1:8080`) names the control plane. A comma-separated list in `-cp-url` fails over for the log-backed control plane.
- `-objects s3|mem` selects the object store. The S3 flags are `-s3-endpoint`, `-s3-bucket`, `-s3-prefix`, `-s3-region`, `-s3-path-style`, and `-s3-create-bucket`.
- The segment tuning flags are `-segment-bytes` (default `16MiB`), `-segment-window`, `-max-segment-partitions`, and `-max-segment-batches`.
- The buffer flags are `-max-buffered` and `-resume-buffered`.
- The timeout flags are `-obj-timeout`, `-commit-deadline`, and `-stop-grace`.
- `-metrics-addr` (default `:9100`) names the metrics address.
- `-log-level` and `-debug` name the log level and the debug switch.
- `-run-id` and `-manifest-out` name the run id and the manifest path.

In a local stack, `manibroker` starts second, after a control plane exists for `-cp-url`.

`cmd/manicp` serves the control plane over HTTP

The `manicp` binary “serves the control plane over HTTP” (`cmd/manicp/main.go`). The binary serves `cp.NewHandler` over your choice of `Store`. The `-store` flag takes the following values.

- `mem` is the default (`cp/memstore`).
- `dynamo` selects the DynamoDB transaction store. The value turns `-sequencer` on by default. The store also takes `-seq-pipeline`, `-seq-max-group-items`, and `-seq-cache-keys`. A stack with this store has one `manicp` process, and every broker points `-cp-url` at that process.
- `dynamolog` selects the log-backed control plane of decisions 48-53. The flags for the value live in `cmd/manicp/logstore.go`. The flags include `-table`, `-ckpt-bucket`, `-node-name`, `-rank`, `-takeover-after`, and `-pebble-dir`.

The common flags follow.

- `-addr` (default `:8080`) names the HTTP address.
- `-dynamo-endpoint`, `-region`, `-table-prefix` (default `manifester_`), and `-create-tables` name the DynamoDB endpoint, the region, the table prefix, and the table creation switch.
- `-metrics-addr` (default `:9101`) names the metrics address.
- `-log-level`, `-run-id`, and `-manifest-out` name the log level, the run id, and the manifest path.

In a local stack, `manicp` starts first. The binary serves the HTTP contract in `api/openapi/controlplane.yaml`.

`cmd/manisim` runs a scenario campaign through the deterministic simulator

The `manisim` binary “runs a scenario campaign … through the deterministic simulator” (`cmd/manisim/main.go`). The binary is not a deterministic package. The binary reads flags and the wall clock. The binary shards independent seeds over goroutines in `cmd/manisim/merge.go`. The binary takes the following flags.

- `-scenario` takes `kafka` (the default), `kafka-log`, or `echo`.
- `-seeds` (default 100) and `-start` (default 1) set the seed count and the first seed.
- `-seed N` replays one seed and prints the log for that seed.
- `-v` prints one line per seed.
- `-j` sets parallel shards (default `runtime.NumCPU()`).

Exit code 1 has four causes.

- A seed fails.
- A run breaks an `Always` invariant.
- A run shows nondeterminism.
- A campaign of at least `coverageMin` (100) seeds never satisfies a `Sometimes` property.

The command `go run ./cmd/manisim -seed 42` replays seed 42.

`cmd/manibench` generates produce load for `manibroker`

The `manibench` binary is a produce-only load generator for `manibroker`. The binary follows the WarpStream CLI benchmark-producer (`cmd/manibench/main.go`). The binary has one subcommand,

`benchmark-producer`. The command `manibench benchmark-producer -topic T` requires `-topic T`. The subcommand takes the following flags.

- `-brokers` and `-num-clients` name the brokers and the client count. Each client is an independent `kgo` client. The client sets `acks=all`. The client disables idempotence.
- `-record-size`, `-duration` (default 30s), and `-interval` set the record size, the run length, and the report interval.
- `-partitions` selects the partitioner. Value 0 selects the kgo sticky partitioner. Value N selects round-robin.
- `-max-inflight`, `-buffered-records`, and `-compression` (`none|gzip|snappy|lz4|zstd`) set the in-flight limit, the buffer size, and the compression.
- `-cp-url` with `-create-partitions N` creates the topic first through `POST /v1/create-topic`.

The binary prints one line per interval. The binary prints a summary at the end or on SIGINT. The binary is not part of the determinism gate.

`cmd/crapscore` computes the CRAP score for every Go function

The `crapscore` binary computes the CRAP (Change Risk Anti-Patterns) score of every Go function. The binary uses cyclomatic complexity and a coverprofile. The binary fails when a function is too crappy (`cmd/crapscore/main.go`). The binary uses the standard library, and the dependency policy requires that limit. The `Makefile` drives the binary. The `-profile` flag is required. The other flags follow.

- `-threshold` takes the default from `CRAP_THRESHOLD`, else 30.
- `-baseline` takes the default `crap-baseline.json`.
- `-merge-profile`, `-tolerance`, `-top`, `-exclude`, and `-json` set the merge profile, the tolerance, the report size, the exclusions, and the JSON output.
- `-strict` ignores the baseline.
- `-update-baseline` rewrites the baseline.

The `make crap` command and the `make crap-strict` command run the binary. The `docs/crap-gate.md` file describes the gate.

`cmd/ddbprobe` probes DynamoDB load

The `ddbprobe` binary is a small DynamoDB load tool with one purpose. The binary runs the commit-path experiments X1, X3, and X4 (`cmd/ddbprobe/main.go`). The results sit in `docs/x1-x3-x4-results.md`. The binary runs on an EC2 instance in the region of the table. The binary prints one JSON summary. The binary takes the following flags.

- `-mode` takes `put|txn|contend|query|batch`.
- `-table` is required.
- `-region` takes the default `ap-south-2`.
- `-rate`, `-k` (max in flight), `-writers`, `-size`, and `-gsize` set the rate, the in-flight limit, the writer count, and the record sizes.
- `-per`, `-parts`, and `-dur` (default 2m) set the period values, the partition count, and the run length.
- Query mode adds `-window`, `-qkind`, `-gap`, and `-start`.

The DynamoDB numbers in the results file come from this tool.

> A seventh `main` sits outside `cmd/`. The `api/openapi/cmd/specgen` program renders `api/openapi/controlplane.json` as `controlplane.yaml` (`go run ./api/openapi/cmd/specgen`). The `-check` flag verifies only. The program is a code-generation utility. The program is not a deployable service.

2.5 `build/` and `results/` hold outputs, and two checked-in files come from generators

The `build/` directory holds local build artefacts. The directory holds no source. The `make test` command writes `build/cover.out`. The `crapscore` binary reads that coverprofile. The `make test` command also generates a `build/.gitignore` file that contains `*`. The directory ignores itself. The `build/kafka/` directory holds a downloaded Kafka tarball. The `build/omb/` directory holds the OpenMessaging Benchmark checkout and tarball. The `scripts/build-omb.sh` script builds the tarball. The script uses Docker and a pinned Maven image. The output is `build/omb/openmessaging-benchmark.tar.gz`. A deletion of `build/` loses no source. The scripts and the `Makefile` recreate the files that the scripts and the `Makefile` need.

The `results/` directory holds recorded benchmark experiment outputs in `r1/`, `w1/-w4/`, `x1x3x4/`, and more. The `docs/*-results.md` write-ups refer to these outputs. The directory is read-only evidence. A file in `results/` does not state current behaviour. A new benchmark run states current behaviour.

Some source directories hold generated files. The `api/openapi/controlplane.yaml` file is the main case. The `cmd/specgen` program generates the YAML file from `controlplane.json`. The JSON file is the source of truth. A test fails if the two files drift. The JSON file carries the edit, and regeneration produces the YAML file. The `crap-baseline.json` file changes through `make crap-baseline`. The diff on that file carries the same weight as a code diff.

Unexpected changes in `git status` come from three places.

- The `build/` directory holds build artefacts.
- The `results/` directory holds benchmark outputs.
- Regenerated spec and baseline files hold generator output.

2.6 `docs/` holds the decision log, the lessons, the design notes, and the runbooks

The `docs/` directory holds the project record. The directory is flat. The `docs/decisions.md` file is the numbered decision log. Package comments cite the log as “decision 41” and “decision I”. A comment that cites a number points at that row first. The `docs/lessons.md` file holds the lessons behind the determinism rules. The directory also holds design notes and experiment write-ups. The files are `docs/log-cp-service.md`, `docs/sim-log-engine.md`, `docs/commit-path-redesign.md`, and `docs/benchmarks.md`. The directory holds runbooks in `docs/aws-benchmark-runbook.md`. The directory holds audit records in `docs/dep-audits/`. This book lives in `docs/onboarding/`. The API contract lives with the code in `api/openapi/README.md` and `api/openapi/controlplane.yaml`. The code and the `doc.go` files state the current behaviour when a doc and the code disagree. The doc fix lands in the same change as the code fix.

2.7 The table maps a task to the package, the file, or the command that holds the work

If you want to…	Go to…
Understand the Event and Action vocabulary that everything speaks	`core/` (`core/doc.go`, `core/event.go`, `core/action.go`, `core/machine.go`)
Change what a Kafka produce or fetch does	`broker/` (`broker/broker.go`, `broker/produce.go`, `broker/fetch.go`)
Change segment sealing, buffering, or flow-control limits	`broker/config.go`, flags in `cmd/manibroker/main.go`
Change the segment object byte format or segment keys	`format/format.go`, `format/key.go`
Add or fix a Kafka wire encoding	`kwire/` (`kwire/codec.go`, `kwire/framer.go`, `kwire/supported.go`)
Change control-plane HTTP routes or wire types	`cp/handler.go`, `cp/wire.go`, then sync `api/openapi/controlplane.json`
Add a control-plane storage backend	A backend implements `cp.Store` (`cp/store.go`), passes `cp/cptest`, and follows `cp/memstore/` as the model
Debug offset assignment or commit contention (DynamoDB)	`cp/dynamo/` (`commit.go`, `seq.go`). The design sits in `cp/doc.go`
Work on the log-backed control plane	`cp/logcp/`, `cp/applier/`, flags in `cmd/manicp/logstore.go`
See the “obviously correct” spec an implementation must match	`veritas/` (`veritas/controlplane.go`, `veritas/objectstore.go`, `veritas/partition.go`)
Reproduce a simulation failure from a seed	`go run ./cmd/manisim -seed <N>`. The kernel sits in `sim/runner.go` and `sim/kernel.go`
Add a fault, engine, or invariant to the simulator	`sim/engines/`, `sim/kafka/invariants.go`, `sim/roles.go`
Learn the simulator on something small first	`sim/echo/` (toy broker, workload, invariants)
Change production networking, executor, or metrics wiring	`sys/prod/` (`node.go`, `executor.go`, `net.go`, `metrics.go`)
Change how S3 errors become acks	`sys/s3store/classify.go`, `sys/sys.go` (`Classifier`)
Run real clients against a real broker end-to-end	`it/` (`it/fullstack_test.go`, `it/hermetic_test.go`), `scripts/it-up.sh`
Start a local broker + control plane by hand	`go run ./cmd/manicp` then `go run ./cmd/manibroker` (§2.4)
Load-test a broker	`go run ./cmd/manibench benchmark-producer -topic T`
Add a dependency to `core/` or `sim/`	`scripts/audit-dep.sh`, record in `docs/dep-audits/`, allow-list in `tools/detcheck/allow.txt`
Know why the code is shaped this way	`docs/decisions.md` (cite the row), `docs/lessons.md`
Deploy or benchmark on AWS	`deploy/aws/` (Terraform, bench), `docs/aws-benchmark-runbook.md`
Find why the build failed on code quality	`make crap-strict`, `docs/crap-gate.md`, `cmd/crapscore/`

Key takeaways

- The module has one deterministic centre. The `core/` package is deterministic. The package has few imports. The `tools/detcheck` tool and the `scripts/ci-determinism.sh` script enforce the import rules.
- The same `broker` state machine runs under two executors. The `sim/` package uses a seed and replays a run. The `sys/` package and `sys/prod` use real input and output. The executors meet at `core` Events and Actions.
- The `veritas/` package is the executable spec. Tests check `cp/` stores and `sys/` adapters against the spec. The spec states the intended behaviour.
- Six binaries do six jobs. The `manicp` binary runs the control plane. The `manibroker` binary runs the broker. The `manisim` binary runs the simulator. The `manibench` binary generates load. The `crapscore` binary runs the quality gate. The `ddbprobe` binary probes DynamoDB.
- The `build/` and `results/` directories hold outputs. The `controlplane.yaml` file and the `crap-baseline.json` file are generated files. Generators produce the two files, and no hand edit survives regeneration.

Checkpoint questions

1. You need to add a helper that reads the wall clock to `core/`. What stops you, and where should the code go instead?

Answer: The `core/doc.go` file bans `time`. The file also bans goroutines, channels, `os`, `net`, `sync`, and SDKs. Nondeterminism enters through `core.Env`. The clock read belongs in the executor. In production, `sys/` and `sys/prod` do the read. The two packages stamp `Event.At` and `Wall` at inbox dequeue. In simulation, the `sim/` kernel and the engines do the read. The `tools/detcheck` tool in `scripts/ci-determinism.sh` rejects a `time` import in `core/`. The import is absent from `tools/detcheck/allow.txt`.

1. A colleague proposes importing `github.com/prometheus/client_golang` from `broker/` to add a counter. Why is that the wrong layer, and what does the codebase do instead?

Answer: The `broker/` package is in the determinism gate. Prometheus is absent from `tools/detcheck/allow.txt` (decision 36 in `docs/decisions.md`). The broker emits durations as plain `core.Fact` values. Examples are `segment_put_done` and `segment_committed`. The `sys/prod.Metrics.Observe` method turns facts into Prometheus collectors at the edge. The `cmd/manibroker -metrics-addr` flag serves the collectors.

1. Which binary do you start first in a local stack, with which flag choosing its storage, and which binary points at it with `-cp-url`?

Answer: The `cmd/manicp` binary starts first. Brokers need a control plane at startup. The `-store` flag selects `mem` (default), `dynamo` (with `-sequencer`), or `dynamolog`. Then `cmd/manibroker` points at the control plane with `-cp-url` (default `http://127.0.0.1:8080`).

1. `git status` shows changes under `build/` and a modified `api/openapi/controlplane.yaml` after you edited the JSON spec. Which of these do you commit, and how were they produced?

Answer: The commit holds the YAML file and the JSON edit. The commit holds no file from `build/`. The `build/` directory holds artefacts that the directory ignores. The artefacts include `build/cover.out` and the OMB and Kafka downloads. The `go run ./api/openapi/cmd/specgen` command generates `controlplane.yaml` from `controlplane.json`. The JSON file is the source of truth. A test fails if the two files drift. Regeneration produces the YAML file, and the commit holds both files together.

1. A simulation run reports seed 1234 failing. Give the exact replay command and name the packages where the kernel, the Kafka invariants, and the production executor live.

Answer: The command `go run ./cmd/manisim -seed 1234` replays the seed. The flags `-scenario echo` and `kafka-log` apply when the campaign used those scenarios. The kernel is `sim/` (`sim/kernel.go`, `sim/runner.go`). Kafka invariants are in `sim/kafka/invariants.go`. The production executor runs the same machine outside the simulator. The `sys.Run` function in `sys/run.go` drives `sys/prod`.

Try it

1. Print the map. Run `go list ./...` and compare it against §2.1. Then run `go run ./tools/detcheck ./core/... ./sim/... ./format/... ./kwire ./broker/...`. Read one rule in `tools/detcheck/allow.txt` alongside `core/doc.go`.
2. Start the small stack. Run `go run ./cmd/manicp -store mem` in one terminal. Run `go run ./cmd/manibroker -objects mem` in another. Then create load with `go run ./cmd/manibench benchmark-producer -topic t -cp-url http://127.0.0.1:8080 -create-partitions 3 -duration 10s`. Stop both processes. Note which process owned which port (`:8080`, `:9092`, `:9100`, `:9101`).
3. Replay a run. Run `go run ./cmd/manisim -scenario echo -seeds 5 -v`. Then run `go run ./cmd/manisim -seed 1`. Find the invariant names in `sim/echo/invariants.go` that the run checked.

Further reading

- ../decisions.md. The numbered decision log that comments cite in the code.
- ../lessons.md. The lessons behind the determinism rules.
- ../crap-gate.md. How `cmd/crapscore` and `crap-baseline.json` work.
- ../log-cp-service.md. The log-backed control plane behind `manicp -store dynamolog`.
- ../benchmarks.md. Benchmark methodology for `manibench` and `ddbprobe` results.
- ../../api/openapi/README.md. The control-plane HTTP API contract.
- ../../core/doc.go. The deterministic core rules.
- ../../cp/doc.go. The control-plane, DynamoDB, and Sequencer design.
- ../../sim/doc.go. Simulator contracts and lane rules.
- ../../veritas/doc.go. The executable specs and conformance checks C1-C4.

Chapter 3. The deterministic core takes events and returns actions

Chapter 2 shows the system from outside. You see a Kafka broker, an object store, and a control plane (CP). The CP assigns offsets. This chapter explains why `core/` sits apart from that work.

> All decisions live in a pure function. All effects live outside it. The two sides speak only in plain data.

The pure function is `Step(Event) []Action` (`core/machine.go`). You give `core.Machine` one `Event`. The machine changes its own state and returns a slice of `Action` values. The machine does no work in the outside world.

A driver does the outside work for the machine. The production runtime (`sys/run.go`) is one driver. The simulator (`sim/`) is another driver. Each driver feeds an event in, runs the returned actions, and feeds the results back as new events.

The split exists so that you can replay an execution. If a machine reads a clock, opens a file, or talks to the network, the run depends on inputs that no one recorded. You cannot replay the run from a seed. If a machine takes only events and returns only actions, you control every input. The same seed and the same event sequence give the same execution. You can run a failing invariant again, step by step, until you see the step that breaks the invariant.

This design costs you work. You must name each input as an `Event` and each output as an `Action`. You must write a driver that runs the actions and reports the results. The cost is worth the gain. A bug that you can replay is a bug that you can fix and then prove fixed. A bug that depends on a clock read or a network result may not occur again on demand.

3.1 The core holds the contracts and leaves the work to other packages

`core/` holds contracts and terms. The package does not hold the main logic. The table below lists the files and the contract that each file defines.

File	Defines
`core/doc.go`	The package rules in one comment block
`core/types.go`	Time, IDs, `ErrCode`, `Outcome`, shared scalars
`core/event.go`	`Event` / `EventBody`, every possible input
`core/action.go`	`Action` / `IOAction`, every possible output
`core/machine.go`	`StateMachine`, `Hashable`, `Machine`, `Factory`
`core/env.go`	`Env = Rand + Buggify + Assertions`
`core/fact.go`	`Fact`, structured and append-only observations
`core/inbox.go`	Event `Class`, `Queue`, `InboxPolicy` contract

File	Defines
`core/policy.go`	`ControlFirst`, the one inbox policy
`core/kafka.go`	`Framer` / `Codec`, Kafka wire work as pure functions
`core/controlplane.go`	`CPOp` / `CPResp`, with invariants attached

You start with the tests. `core/core_test.go` pins the classification table and the frozen `ErrCode` spellings. `core/policy_test.go` pins inbox priority with a slice-backed queue. Neither test needs a broker, a clock, or a network. The tests need no clock or network because `Step` takes time and results as data. That fact shows the split at work.

3.2 The Step contract lets one machine run under two drivers

```
// core/machine.go
type StateMachine interface{ Step(Event) []Action }
type Hashable interface{ StateHash() uint64 }
type Machine interface{ StateMachine; Hashable } // hashing is not optional
type Factory func(env Env, self NodeRef) Machine
```

The contract has four clauses, and the clauses follow.

1. `Step` is the only place where state changes. If a value comes from outside the machine, the value re-enters as an event. The machine sees no value that does not re-enter as an event.
2. `Step` never blocks and `Step` never does `I/O`. The machine changes private state and returns requests (`core/machine.go:3–6`). The driver runs the requests. Section 3.11 covers the driver.
3. Hashing is mandatory. The simulator folds `StateHash()` into a trace hash after every step. You can check the rule that the same seed gives the same hash sequence. The broker version (`broker/hash.go:42`) folds each field that changes behaviour. The broker folds the fields in a fixed order. The fields are counters, queue lengths, per-connection slot state, and segment and op state. The hash omits timing fields that come from event times and that behaviour does not read. One example is `sealedAt` (`broker/segment.go:68–73`).
4. The runtime rebuilds machines and the runtime does not resume machines. `Factory` runs on first boot and after every crash. `Factory` uses a fresh `Incarnation` each time (`core/machine.go:23–25`). No memory content survives a crash. Recovery reads data from the CP and the object store.

Clause 2 explains why the runtime can swap the driver without a change to the machine. If `Step` did `I/O` itself, the production path and the simulation path would run different code. The test would not test the code that runs in production. Because `Step` returns requests, both drivers run the same machine. The drivers differ. The decisions do not differ.

`core/event.go:14–20` states a fifth clause. A completion answers an `I/O` action. If a completion has an unknown `ReqID`, the machine ignores the completion. If a completion has an already-finished `ReqID`, the machine ignores the completion. If a completion has a wrong-kind `ReqID`, the machine ignores the completion. The state does not change and the hash does not change.

Retries explain the fifth clause. A retry can produce a duplicate completion. A completion can arrive after the machine gave up on the request. If such a completion changed state, the result would depend on the order in which outside work finished. That order is not part of the recorded event sequence in the form the machine sees. Replay would lose its fixed result.

The broker handlers scan for a segment or an op in the expected state with that `ReqID`. The handlers return `nil` when the handlers find no match (`broker/broker.go:216–242`). `TestIgnoresStrayCompletions` (`broker/broker_test.go:517`) asserts a bit-identical hash after stray `CPDone`, `ObjPutDone`, `ObjGetDone`, `SendDone`, and `TimerFired` events. If you add a machine, you write the same test for that machine.

3.3 Plain data and closed sets keep every input visible

Events and actions use plain data. The data types are numbers, strings, booleans, byte slices, and nested plain structs. The data does not include functions, channels, runtime pointers, or Go `error` values. Errors use the `ErrCode` enum "so events stay comparable and hashable" (`core/types.go:57`).

Plain data matters because the driver and the simulator must record, compare, and hash what passes between the two sides. A function value or a channel hides work and state. You cannot hash the hidden work or compare the hidden state. Plain data makes the whole input visible in one place.

The event set and the action set are closed sets. Unexported marker methods seal the sets. The methods are `isEvent()` and `isAction()` (`core/event.go:11`, `core/action.go:8`). Only `core` can add a member. A new `EventBody` also needs a row in `ClassOf` (section 3.11). `ClassOf` panics on an unclassified type. The code calls that case "a programming error" (`core/inbox.go:31–33`).

A closed set costs you a central change for each new kind of input or output. An open set would let a package add a kind without a change to `core`. The code accepts the central cost. A closed set lets a reader list every input and every output, and the simulator can check that list. That check has more value here than local ease of extension.

Two rules follow from the closed sets, and the rules follow.

- Byte slices do not change after a machine sends the byte slices (`core/doc.go:9`). If you change a `Send` payload or an `ObjPut` payload after you send the payload, you can change data that code has hashed or stored.
- Names do not change after code hashes the names. `ErrCode.String()` spellings "are never changed" (`core/types.go:90`). `Fact` names are "frozen, past-tense constants, never derived from Go type names. Renaming one re-baselines the golden hash" (`core/fact.go:5–9`). Enums are append-only. `ErrCode` keeps a retired slot (`core/types.go:71`) so later numbers do not move.

3.4 Time arrives on the event so that replay controls time

The core has no clock, and the absence is the point. If a machine reads a clock inside `Step`, two runs of the same event sequence read two clock values. The runs can take two branches. No seed can fix the branch, because no one recorded the clock read.

Each event carries two stamps instead (`core/event.go:5–9`). `At Time` holds node-local monotonic nanoseconds. Code uses `At` for deadlines, leases, batch windows, and backoffs. `Wall WallTime` holds UTC nanoseconds. Code uses `Wall` for stored values such as Kafka `LogAppendTime` and credential expiry. The runtime and the simulator stamp both fields at dequeue. The stamp order keeps `At` from decreasing in Step order. The production loop reads `b.Clock.Mono()/Wall()` at that point (`sys/run.go:40–43`).

A machine treats the current event `At` value as the current time. The broker caches `b.now = ev.At` at the start of `Step` (`broker/broker.go:89`). The broker derives segment ages, deadlines, and fact timings from stamps in events that the broker has seen. If a machine needs a wake-up, the machine requests the wake-up with `SetTimer`. The machine then waits for `TimerFired`. `core/env.go:2–3` states the rule for `Env`. “Time is NOT here: it arrives on each Event”.

The other choice is a clock that a test can replace. A fake clock still lets code read time at a point that the event sequence does not show. The reader must find each read to know the input. An event stamp puts the time in the one value that the machine already takes. The reader sees the time input without a search.

The list below gives the main scalars from `core/types.go`, and the list follows.

- `ID [16]byte` comes from the seeded stream. The code states “drawn from the seeded stream (never `uuid.New`)” (`core/types.go:23`). Replay cannot see unseeded randomness.
- `NodeRef{Node, Inc}` names one boot of a node. A restarted node is a new machine with empty memory.
- `ReqID` links an action to a completion. The code states “unique within one incarnation only; it must never reach a durable name (object keys come from `Env.NewID()`)” (`core/types.go:30–33`).
- `ErrCode` has rules for each value. `ErrNotFound` and `ErrConflict` report the result without change. Engines inject only `ErrTimeout`, `ErrUnavailable`, `ErrIO`, and `ErrConnClosed`. `ErrCorrupt` comes only from the core decoder (`core/types.go:60–63`).
- `Outcome` states what a completion proves. The values are `NotApplied`, `MaybeApplied`, and `Applied` (`core/types.go:80–88`). Retry rules use `Outcome`. A lost response gives `ErrTimeout+MaybeApplied`. A lost response does not give silence. Code retries an operation only when the operation is idempotent.

3.5 Env names each source of nondeterminism that replay must control

```
// core/env.go
type Env interface{ Rand; Buggify; Assertions }
```

`Factory` injects `Env` once. `Env` lists each source of nondeterminism that a simulation must control for replay. Before you use a part of `Env`, you check the part against both implementations.

Random draws, fault choices, and assertion records can change a run. If code takes the inputs from a hidden source, the simulator cannot set the inputs or record the inputs. `Env` puts the three sources behind one interface. The simulator gives the machine a seeded form. Production gives the machine a live form. The machine code stays the same.

`Rand` is defined at `core/env.go:11–18`. `Rand` provides `Uint64`, `Intn(n) ∈ [0,n)`, and `NewID`. In sim, all machines of a seed share one run stream (`sim/assert.go:127–133`). Draw order forms part of the replayed execution. Production seeds ChaCha8 from `crypto/rand` (`sys/prod/env.go:22–31`). Production uses the same shapes and production does not promise replay. The broker draws `NewID()` for segment keys (`broker/segment.go:166`). The broker draws `Intn` for commit-retry jitter (`broker/segment.go:348`). Replay requires a record of each draw. Replay does not require few draws.

`Buggify` is defined at `core/env.go:20–34`. `Buggify` provides FoundationDB-style fault injection. `Maybe(site)` covers paths that occur with low probability and stay valid. A run activates a site first. The default probability after activation is p=0.25 (`sim/assert.go:104,115`). `MaybeP(site, p)` takes a probability from the caller. `Fault(site, p)` makes an operation fail. The code states that `Fault` “never fires once the simulator has entered recovery mode”. Sim gates `Fault` on the chaos phase (`sim/assert.go:121`). `Site` is a stable string. Code must not put values in a `Site` string. Production returns `false` in each case (`sys/prod/env.go:51–53`).

> Caveat. A grep check verifies this point. No `broker/` code calls `Maybe`, `MaybeP`, or `Fault` today. Sim engines fail, delay, or drop completions around the broker. The broker receives the results as `Err*` completions and timer expiries. Production gives the broker the same event forms. Sim implements Buggify. Other engines use fault contexts. A new broker site needs a stable name. A new broker site must not include values. A new broker site must return `false` in production.

`Assertions` is defined at `core/env.go:45–57`. `Assertions` provides `Always`, `AlwaysOrUnreachable`, `Sometimes`, `Reachable`, and `Unreachable`. Each method records a result and continues. No method panics. Campaign rules differ by method, and the rules follow. `Always` must hold each time code reaches the call. A campaign must reach `Always` at least once. `Sometimes` and `Reachable` must occur at least once across a campaign. `AlwaysOrUnreachable` limits only executions that reach the call. Sim records a violation and runs on. Sim records the first violation per property per seed (`sim/assert.go:30–72`). Prod logs a violation and continues (`sys/prod/env.go:55–77`). Prod treats `Sometimes` and `Reachable` as no-ops. The broker calls record facts that you can check in code. One call is `Reachable("broker: segment sealed by timer")` (`broker/segment.go:133`). One call is `Unreachable("broker: commit key reused with a different request")` (`broker/segment.go:251`). One call is `AlwaysOrUnreachable(shape, "broker: commit answers every batch in order")` (`broker/segment.go:237`).

3.6 The event and action tables list the whole contract

The two tables in this section are reference apparatus. The explanation in the sections before this section points at the tables. You read the tables to check which inputs and outputs exist.

The list is complete as of this chapter. The source lists are `core/event.go:114–126` and `core/action.go:110–124`. If you find a diff against those lists, you have a bug in this chapter or an unreviewed core change.

Events (`EventBody`, wrapped in `Event{At, Wall, Body}`)

Event	Class	Answers / source	Meaning
`Started{Self}`	Control	first event of every boot	Rebuild from CP + store; nothing survived
`Stop{Deadline}`	Control	runtime shutdown	Stop admitting, finish/hand off, `Exit` by deadline
`ConnOpened{Conn, Peer}`	Data	listener	New connection; `Peer` informational only
`BytesReceived{Conn, Data}`	Data	reader	Arbitrary TCP chunk; framing is the core's job
`ConnClosed{Conn, Abort}`	Data	reader/network	EOF (graceful) or reset (abort)
`SendDone{Req, Conn, Err}`	Control	`Send`	`Err != OK`. Bytes may be partially sent
`DialDone{Req, Conn, Err}`	Control	`Dial`	`Conn` valid only if `Err == OK`
`TimerFired{ID, Gen}`	Control	`SetTimer`	Ignored if `Gen` is stale for `ID`
`ObjPutDone{Req, Key, Err, Outcome}`	Control	`ObjPut`	Upload verdict
`ObjGetDone{Req, Data, Err}`	Control	`ObjGet`	Download verdict + bytes
`ObjDeleteDone{Req, Err, Outcome}`	Control	`ObjDelete`	Delete verdict
`ObjListDone{Req, Keys, Err}`	Control	`ObjList`	`Keys` sorted
`CPDone{Req, Resp, Err, Outcome}`	Control	`CPCall`	`Resp` nil unless `Err == OK`

The seven `*Done` bodies implement `Completion` (`ReqID()`). The other bodies are primary inputs. A machine does not need a case for each event. The broker has no case for `SendDone`, `DialDone`, `ObjDeleteDone`, or `ObjListDone` (`broker/broker.go:90–111`). Those events fall through without a state change. Section 3.2 states the rule.

Actions (one slice per `Step`)

Action	IO?	Completed by	Meaning
`SetTimer{ID, Gen, After}`	no	`TimerFired{ID, Gen}`	Wake after a node-local duration
`CancelTimer{ID}`	no	none	Optimisation only; correctness relies on `Gen`
`Send{Req, Conn, Data}`	yes	`SendDone`	Bytes, ordered with earlier `Send`s on the conn
`CloseConn{Conn, Abort}`	no	none	FIN after queued bytes, or RST
`PauseReads{Conn}` / `ResumeReads{Conn}`	no	none	Admission control; received bytes still process

Action	IO?	Completed by	Meaning
`Dial{Req, Addr}`	yes	`DialDone`	Outbound conn (sim workloads; brokers use `CPCall`)
`ObjPut{Req, Key, Data}`	yes	`ObjPutDone`	Key unique per upload ⇒ `MaybeApplied` PUT safe to retry
`ObjGet{Req, Ref}`	yes	`ObjGetDone`	Fetch a `SegmentRef` byte range
`ObjDelete{Req, Key}`	yes	`ObjDeleteDone`	Delete object
`ObjList{Req, Prefix}`	yes	`ObjListDone`	List keys under prefix
`CPCall{Req, Op}`	yes	`CPDone`	One control-plane op (§3.10)
`Emit{Fact}`	no	none	Append a fact (section 3.8)
`Trip{Cause}`	no	none	Kill this incarnation; memory untrusted
`Exit{}`	no	none	Last action of a graceful shutdown

`core/action.go` states the ordering rules for one returned slice. Actions in one slice start in slice order. `I/O` actions can complete in any order (`core/action.go:5–7`). An `IOAction` gets one completion. The code states the completion rule in full. “a live incarnation always gets that completion: a lost response surfaces as `ErrTimeout+MaybeApplied`, never as silence; completions addressed to a dead incarnation are dropped” (`core/action.go:10–13`). Runtimes must keep that rule. `Trip` and `Exit` are each “the last action of its Step” (`core/action.go:94–100`). `Trip` returns `ErrTripped` in prod for the supervisor. `Exit` does not count as a fault.

Timers use generations because a cancel can race with a firing. If correctness rested on the cancel, the race would decide the result. The race sits outside the machine, so the machine cannot record the race as an input. `CancelTimer` is only an optimisation. Each firing carries `Gen` and the machine ignores a stale generation. The code describes timers as “matched by generation, not cancelled” (`core/event.go:70–71`). The broker raises one counter for each arming (`broker/broker.go:79–82`). The broker checks `s.gen != gen` before the broker acts (`broker/segment.go:357`). The runtime can cancel the host timer (`sys/run.go:113–117`). The model does not include a cancel and fire race as a state input.

3.7 One Step in the broker shows the split in code

```
func (b *Broker) Step(ev core.Event) []core.Action {
    if b.exited { return nil }
    b.now = ev.At                       // time comes from the event, §3.4
    var acts []core.Action
    switch e := ev.Body.(type) {      // closed set: only these cases exist
    case core.Started:   b.started = true
    case core.Stop:      acts = b.stop()
    case core.ConnOpened:    acts = b.onOpen(e)
```

```
    case core.BytesReceived: acts = b.onBytes(e)
    case core.ConnClosed:    acts = b.onClosed(e)
    case core.TimerFired:    acts = b.onTimer(e)
    case core.ObjPutDone:    acts = b.onPut(e)
    case core.ObjGetDone:    acts = b.onGet(e)
    case core.CPDone:        acts = b.onCP(e)
    }
    return b.finish(acts)              // drain ready replies, flow control, maybe Exit
}
```

`Broker.Step` (`broker/broker.go:85–120`) shows four points, and the points follow. The switch has no `default` case with side effects. Unhandled bodies fall through without a state change. `SendDone` and `DialDone` in section 3.6 are examples. The common `finish` call runs after the switch. `finish` does not create work for an event that the machine ignored. Each handler returns actions and no handler performs the actions. The response drain in `finish` appends `Emit` and `Send` actions to the slice. Handlers check the correlation before the handlers act. `onPut` and `onCP` scan segments for `s.req == e.Req && s.state == segPut/segCommit` (`broker/broker.go:216–242`). The scan makes the stray-completion rule part of the handler code.

The handlers use no lock, no channel, no goroutine, no clock read, and no map range. The handlers need none of those controls. The runtime gives one thread to `Step` at a time and the runtime states “this goroutine alone owns the machine” (`sys/run.go:24`). One owner at a time removes the need for a lock inside the machine.

If you add a machine, you follow the same form. You cache `ev.At`. You dispatch on the closed `EventBody` set. You change only your own fields. You match completions by `ReqID` and by expected state. You return actions. You fold state that changes behaviour into `StateHash`. If a design needs a background goroutine or a `time.Sleep`, you use a timer event instead. Section 3.12 gives an example.

A background goroutine would break the form. The goroutine would run between steps and the goroutine could change state or read a clock between steps. The event sequence would not show that work. A timer event keeps the wake-up in the event sequence, where the simulator can order the wake-up and record the wake-up.

3.8 A machine returns facts because a log line is an effect

A machine cannot log from `Step`. A log call writes output, and the write is an effect inside the decision step. Log calls also have no fixed order that code can hash. A machine returns `Emit{Fact}` instead. The driver writes the fact, and the machine keeps the decision in data. The code follows.

```
// core/fact.go
type Fact struct{ Name string; Fields []Field } // Name: frozen past-tense constant
```

Field values use only `I64/U64/Str/Bool/Bytes` (`core/fact.go:20–35`). Facts use values that code can hash and compare. Prod treats `Emit` as tracing. Sim invariants read the timeline (`core/action.go:107`).

One ordering rule controls ack checks. The code states the rule in full. “an ack fact is emitted in the same Step, immediately before the `Send` that acks” (`core/fact.go:7–9`). The broker queues `produce_acked` on the request slot at commit time (`broker/segment.go:370–385`). `drain` emits queued facts before the `Send` for the request (`broker/conn.go:187–191`). If an ack fact sits at another point, the check for the “acked ⇒ committed” invariant loses the fact order that the check needs. `BugEarlyAck` in section 3.14 shows the failure.

3.9 The bans on clocks, goroutines, and map order protect replay

Each ban in this section removes one input that a seed cannot control. A clock read gives a value that depends on the moment of the read. A goroutine gives an order that depends on how the Go runtime schedules the goroutine. A map range gives an order that Go changes between runs. Any one of the three inputs can send two runs of the same seed down two paths. The bans trade ease for control, and the trade is right here. Code inside a machine is short and runs often. A recorded input costs little at each step. A run that no one can replay costs a long search each time a test fails.

`core/doc.go` states the rules. `tools/detcheck` and `scripts/ci-determinism.sh` enforce the rules over `core/ sim/ format/ kwire broker/ veritas` and more packages. The rules follow.

1. Deterministic packages do not import `time`, `math/rand`, `crypto/rand`, `os`, `net`, `sync`, or SDK packages. The check uses an exact-path allow-list (`tools/detcheck/allow.txt`). Kafka types come from the franz-go `kmsg` package. The `kgo` client “is never used here” (`core/kafka.go:1–3`).
2. Deterministic packages do not use goroutines, channels, or `select`. Detcheck reports each use. Concurrency stays outside the machine. The machine runs one `Step` for one event at a time.
3. Map iteration does not decide behaviour. Go changes map iteration order between runs. Detcheck flags `range` over maps and `reflect` map iteration. The broker keeps slice state. The state holds `conns`, `segs`, `ops`, and `topics []core.TopicInfo` (`broker/broker.go:46–58`). Code scans the slices in insertion order.
4. Deterministic packages do not use package-level mutable vars. Global state can carry data between sim runs in one process.
5. Machines use plain data only. Byte slices do not change after a machine sends the byte slices (section 3.3).

The slice in rule 3 is the other choice to a map. A slice has an order that code sets and code can hash. The scan costs more than a map lookup in some cases. The code pays the scan cost because the order stays fixed across runs. A sorted map key list would also fix the order. The code in the broker uses slices, and a new machine follows that form.

If a reported use is correct, a code comment can allow the one use. The form is `//detcheck:ok <reason>` on the same line or the previous line. The reason is required (`tools/detcheck/detcheck.go:13–17`). State after n steps is a function of the seed and the event sequence. `TestStateHash` requires bit-identical hash sequences across two runs (`broker/broker_test.go:482–503`) for that reason. CI runs each seed twice.

3.10 Wire work and control plane work stay inside the same contract

Kafka wire handling uses `core/kafka.go`. The code states that wire handling "runs inside the deterministic path (decision A)". `Framer` puts length-prefixed frames back together from chunks. Each connection has one `Framer`. The `Framer` forms part of machine state and the `Framer` uses plain data. `Codec` decodes requests and encodes responses in both directions. Chunk boundaries cannot change behaviour because the partial frame is state. The broker hashes `Buffered()` and a running receive hash (`broker/hash.go:62–64`). `TestStateHash` shows that partial frames with different bytes hash to different values.

If wire work sat outside the machine, the driver would decide where a frame starts and ends. Two drivers could split the same bytes in two ways. The machine would see two inputs for one client request. The code keeps the `Framer` in machine state so that the chunk split cannot change the decision.

The control plane uses `core/controlplane.go`. The file defines three ops and three responses. The ops are `CommitSegment`, `Lookup`, and `DescribeTopics`. Both sets are closed sets. The Produce trace in section 3.12 uses the rules in this section. Brokers hold no registration, epoch, or fencing state. Brokers do not use idempotent producers. The CP assigns offsets and the broker does not propose an offset (`core/controlplane.go:7–11`). `CommitSegment` assigns offsets for a multi-partition segment in one operation. The operation assigns all batches or no batches. Offsets stay contiguous per partition. The CP does not reuse offsets (`core/controlplane.go:16–24`). `CommitSegment` must be idempotent per `SegmentKey`. A replay with the same key and the same request returns the original ranges. The same key with a different request returns `ErrConflict`. The code calls this case "the only use of `ErrConflict`" (`core/controlplane.go:25–28`).

3.11 One policy picks the next event so that Go cannot pick the order

Machines do not pick the next event with `select`. If a machine uses `select`, Go picks one ready case without a fixed order. That choice adds scheduling order from Go to the model, and a seed cannot control the choice. The next event then depends on the runtime and not on the recorded input.

Each event has a class instead (`core/inbox.go:8–18`). `Control` covers completions, timers, and boot. `Control` has strict priority. The state machine caps in-flight work, and that cap bounds Control traffic. `Data` covers connection bytes. `Data` traffic has a bound and `PauseReads` gives backpressure. `ClassOf` (`core/inbox.go:22–35`) holds a fixed table. `ConnClosed` has class `Data` so "a client's EOF can never overtake its last request."

Strict priority can starve `Data` in a design without bounds. The design here has bounds on both queues, and the machine caps in-flight work. The bound is the reason that strict priority is safe here. Without the bound, a fair policy would be the safer choice.

The policy function is pure and stateless. The code states "the SAME code in production and simulation" (`core/inbox.go:45–56`). The code follows.

```
// core/policy.go — ControlFirst.Next(control, data Queue)
if control.Len() > 0 { return control.Pop() }
```

```
if data.Len() > 0 { return data.Pop() }
return nil, false // caller sleeps until either queue is non-empty
```

Production uses `sys.Inbox` for the queues. `sys.Inbox` uses a mutex and slices (`sys/inbox.go`). Sim and tests use a plain slice (`core/policy_test.go:5-15`). Both forms use the same interface, the same policy, and the same step order. The runtime can sleep only while both queues are empty. The rule is "Go's `select` must never decide priority." Bounded in-flight work bounds control traffic. Data events run after the machine processes the results of prior work. Starvation does not occur under the bound.

3.12 One Produce request shows each decision as a step

The trace shows one batch to topic `t`, partition 0. The trace shows events and actions only. That limit is the point of the trace. You can read each decision without work from the driver between the decisions. Chapter 4 covers encoding. Chapter 5 covers segment format. Broker behaviour comes from `broker/produce.go`, `broker/segment.go`, and `broker/conn.go`.

```
sequenceDiagram
    participant C as Client (TCP)
    participant R as Runtime / Sim kernel
    participant B as Broker (core.Machine)
    participant S as Object store
    participant P as Control plane
    C->>R: TCP bytes (Produce request)
    R->>B: Step(Event{At, Wall, BytesReceived}) [Data]
    Note over B: frame, decode, validate, append to open segment
    B-->>R: [SetTimer{timerFlush, Gen=g, After=BatchWindow}]
    R->>B: Step(Event{At', Wall', TimerFired{timerFlush, g}}) [Control]
    Note over B: first batch's window expired — seal
    B-->>R: [CancelTimer, Emit(segment_sealed), ObjPut{Req=r1, Key=seg/…, Data}]
    R->>S: execute ObjPut r1
    S-->>R: ObjPutDone{Req=r1, Err=OK, Outcome=Applied}
    R->>B: Step(Event{…, ObjPutDone r1}) [Control]
    B-->>R: [Emit(segment_put_done), CPCall{Req=r2, CommitSegment{Key, Batches}}]
    R->>P: execute CPCall r2
    P-->>R: CPDone{Req=r2, Resp=CommitSegmentResp{Assigned}, Err=OK}
    R->>B: Step(Event{…, CPDone r2}) [Control]
    Note over B: take Base from Assigned; queue produce_acked; drain response
    B-->>R: [Emit(segment_committed), Emit(produce_acked), Send{Req=r3, Conn, Data}]
    R->>C: TCP bytes (Produce response)
    S-->>R: SendDone{Req=r3, Err=OK} (later)
    R->>B: Step(Event{…, SendDone r3}) [Control — no case, ignored]
```

The points below check the trace against the contracts, and the points follow.

- No step does `I/O`. You can swap the driver between the prod executor and a sim engine, and the steps stay the same.
- The ack waits for the CP. Step 1 admits the batch. The broker answers after an OK `ObjPutDone` and an OK `CPDone`. The answer uses `Assigned[i].Base` and the broker does not guess the base.
- Retries use normal steps. A transient `ObjPutDone` leads to `SetTimer{Gen=g'}`. The matching `TimerFired` issues the same `ObjPut` under the same key (`broker/segment.go:315–328`). A transient `CPDone` or a `MaybeApplied CPDone` uses the same `SegmentKey`. The broker caps jittered backoff at `CommitDeadline` (`broker/segment.go:330–354`). Section 3.10 states the idempotency rule that allows this retry. Only the head segment commits. CP commits occur in seal order. A partition receives offsets for the batches of the partition in admission order (`broker/segment.go:202–209`).
- Facts follow a fixed order. The order is `segment_sealed`, `segment_put_done`, `segment_committed`, and `produce_acked`. The `produce_acked` fact comes before the `Send` for the fact.
- The final `SendDone` has no handler case. The event does not change state.

A seal can occur inside the `BytesReceived` step. Size caps, partition caps, and batch caps can cause the seal (`broker/segment.go:88–118`). That step then includes `ObjPut` in the returned slice of that step. The simulator tests different interleavings of the same machine. The invariants must hold in each interleaving.

3.13 The invariants state what replay must preserve

1. The same seed and script give the same `StateHash` sequence and the same fact timeline.
2. State changes only in `Step`. State changes occur in dequeue order. `At` never decreases.
3. An unknown, finished, or wrong-kind `ReqID` does not change the hash. A stale `TimerGen` does not change the hash.
4. A live incarnation gets one completion for each `IOAction`. A lost response gives `ErrTimeout+MaybeApplied`. A lost response does not give silence. A dead incarnation gets no completion.
5. Actions start in slice order. Completions can arrive in any order. Correctness does not depend on completion order. Correctness does not depend on `CancelTimer`.
6. The broker emits `produce_acked` in the same step as its `Send`. The fact comes before the `Send`. The broker emits the fact only after a successful `CommitSegment`.
7. Offsets stay contiguous. The CP does not reuse offsets. One commit assigns offsets in one operation. One `SegmentKey` gives idempotent commit. `ErrConflict` means the same key with a different request. No other case uses `ErrConflict`.
8. Control events run before data events. Both queues have bounds. `ControlFirst` decides priority. `select` does not decide priority.
9. Event sets, action sets, enum numbers, enum spellings, fact names, and fact field order are append-only. A change needs a stated hash re-baseline.
10. A step uses only the event, machine state, and `Env` draws. A reader checks the inputs in that order.

3.14 Mutants test whether an invariant catches a wrong machine

`core/` has no bug flags. `core/` has `Buggify` (section 3.5). The broker has a second mechanism named `Bug*`. The type is `type Bug uint8` inside `Config` (`broker/config.go:6–37`). The code marks the type with a comment. “TEST ONLY: the zero value is the real broker and production code must never set it.” A mutant selects one wrong behaviour at construction. Code selects the behaviour through `broker.New(cfg)` and `cfg.Bug`. A failing seed reproduces bit-for-bit. `StateHash` includes the `Bug` value first (`broker/hash.go:44`). Honest traces and mutant traces do not collide. The table below lists the nine mutants.

Mutant	Wrong behaviour	Invariant that catches it
`BugEarlyAck`	Ack after PUT, before `CommitSegment` (guessed offset, `broker/segment.go:193`)	acked range is committed in the CP
`BugFreshKeyOnRetry`	Retry lost commit under a fresh key	a stored batch is committed at most once
`BugNoPartitionCap` / `BugNoBatchCap`	Ignore segment caps	sealed segment spans/holds at most the cap
`BugWrongOffset`	Fetch patches base offset + 1	consumer sees every offset in order, none skipped
`BugServeCorrupt`	Skip CRC check on fetch	consumer never receives a corrupt batch
`BugSkipProduceCRC`	Admit batch without CRC (`broker/produce.go:75`)	committed batch is intact
`BugTimerFromLastBatch`	Flush window restarts per batch	segment sealed within first batch’s window
`BugSealOffByOne`	Size rule one batch late	overshoot ≤ last batch

`TestMutantsAreCaught` enforces the mutant rule (`sim/kafka/kafka_test.go:92–135`). Each mutant must fail within 20 seeds. The expected bound is 8 seeds or fewer. The invariant that names the mutant must catch the mutant. If no invariant catches a mutant, the mutant shows a missing invariant. The test does not pass such a mutant. If you add a broker rule, you add a mutant and an invariant in the same change. `sim/engines/logengine` keeps a second set under the same rule. The set holds `BugWriterOffsets`, `BugTrimDelete`, `BugPerReplicaWindow`, and `BugHoleSkip`.

Buggify and mutants change different code, and you keep the two apart. Buggify changes the world around a correct machine, and a buggify test checks the response of the machine to faults. A mutant changes the machine, and a mutant test checks that an invariant catches wrong machine behaviour. If a test mixes the two kinds of change, the test can inject a fault without an invariant that checks the result.

Key takeaways

- `Step(Event) []Action` defines the split in the code. Machines make decisions without `I/O`. Drivers run the effects. Machines and drivers pass plain data.
- Time is data on the event. You use `At` for deadlines. You use `Wall` for stored UTC values. Randomness, faults, and assertions enter through `Env`. Other inputs must already be state.

- Code and CI enforce determinism. The code uses closed sets, frozen names, and frozen numbers. Deterministic packages do not use `I/O`, goroutines, or map iteration for behaviour. Detcheck checks the rules in CI.
- Priority is a pure function. `ControlFirst` gives the priority rule. Timers use generation matching. Stray completions cannot change state.
- Facts give an ordered record. The broker emits `produce_acked` before its `Send`. Mutants check that invariants catch wrong fact order and wrong ack order.

Checkpoint

1. A completion arrives for an already-finished `ReqID`. What must happen to `StateHash`, and where is that written? **Answer:** Nothing changes. The machine ignores the completion. The hash does not change. The contract is at `core/event.go:14–20` and `core/machine.go:3–6`. `TestIgnoresStrayCompletions` covers the case (`broker/broker_test.go:517`).
2. Why does `CancelTimer` exist if correctness never relies on it? **Answer:** `CancelTimer` gives an optimisation only (`core/action.go:27`). The call ends the host timer at an earlier time. Correctness uses generation matching. The machine ignores a `TimerFired` with a stale `Gen` (`core/event.go:70–71`). A lost cancel-vs-fire race does not change the result.
3. Your machine needs a random retry delay. Which interface do you use, and what must you never do? **Answer:** Use `Env.Rand` through `Intn` or `Uint64` (`core/env.go:11–18`). `Env.Rand` gives one seeded stream. The broker commit jitter uses that stream (`broker/segment.go:348`). Do not import a clock source or a random source into a deterministic package. The detcheck allow-list does not include those packages (`tools/detcheck/allow.txt`).
4. Why is `ConnClosed` class `Data`, not `Control`? **Answer:** Events that must stay in order share a queue. A client EOF cannot pass the last request from that client (`core/inbox.go:20–21`). Control priority covers results of work on the node.
5. You rename fact `"segment_committed"` to `"segmentCommit"`. What breaks? **Answer:** Fact names are frozen past-tense inputs to the trace hash. Code does not derive fact names from Go type names (`core/fact.go:5–9`). The rename changes the golden hash. The rename breaks comparison with recorded traces. A change needs a stated and reviewed re-baseline. Append-only enums use the same path (`core/types.go:57`).

Try it

```
go test ./core/ -count=1 -v                                    # the core contracts
go test ./broker/ -run 'TestStateHash|TestIgnoresStrayCompletions' -count=1
grep -n 'func ClassOf' -A 14 core/inbox.go                      # the fixed class table
go run ./tools/detcheck ./core/... ./broker/...                 # determinism gate, as CI
go test ./sim/kafka/ -run TestMutantsAreCaught -count=1 -v      # mutants vs invariants
grep -rn 'env\.Maybe\|env\.Fault\|\.MaybeP(' broker/ --include='*.go'  # verify §3.5's caveat: no hits
```

Further reading

- `core/doc.go`, `core/event.go`, and `core/action.go` in full. Each file takes two screens. Read each file in full.
- `broker/broker.go`, `broker/segment.go`, and `broker/conn.go`. The files give the section 3.12 trace in source order.
- `sys/run.go` and `sys/inbox.go`. The files give the production driver, dispatch, host timers, and queue storage.
- `sim/assert.go`. The file gives the simulator `Env`, seeded `Rand`, per-site `Buggify`, and record-and-continue assertions.
- `docs/lessons.md` (lessons 3, 7, 10-12), `docs/decisions.md` (A, G, H, I, 41-44), and `docs/flow-control.md` sections 2-3. The files state why each rule exists.

Chapter 4: the broker acks a produce only after S3 stores the segment and the control plane commits the segment

You meet one rule first because the whole chapter follows from the rule. The broker acks a produce only after S3 stores the segment and the control plane commits the segment. The rule decides the produce path. The rule also decides the fetch path. You pay for the rule with delay and with memory. You gain an ack that names a committed offset.

The broker speaks Produce, Fetch, Metadata, and ListOffsets to clients. The broker stores record batches in S3. The broker asks the control plane to assign offsets. The broker owns no durable log.

`broker.New(cfg)` returns a `core.Factory` in `broker/broker.go`. The simulator and `sys.Run` drive the same broker code in production. One `Step(ev core.Event) []core.Action` call changes broker state. The broker holds no goroutines. The broker reads no clocks. The broker makes no direct S3 calls. The broker makes no direct HTTP calls. Side effects leave the broker as `core.Action` values. Completion events bring the results back to the broker. The sentence the broker uploads means `Step` returns a `core.ObjPut` action.

4.1 The package map puts the produce, seal, commit, and fetch code in separate files

File	What lives there
`broker/broker.go`	`Broker` state, `Step` dispatch, timers, stop, backpressure (`flow`)
`broker/produce.go`	Produce admission and batch validation (`admit`, `checkBatch`)
`broker/segment.go`	Open segment, seal rules, PUT then commit, ack and release
`broker/fetch.go`	Fetch, Lookup handling, ranged GET verification, offset patching
`broker/metadata.go`	Metadata and ListOffsets responses
`broker/ops.go`	`op` retry machinery, request dispatch, topic cache
`broker/conn.go`	Per-connection slots, in-order responses, connection lifecycle
`broker/config.go`	`Config`, `DefaultConfig`, Kafka error codes, test-only `Bug` mutants
`broker/hash.go`	FNV-1a `StateHash` over everything that influences behaviour

`format/format.go` defines the segment object layout. The layout has a `MSEG` header, an index, and payloads. `format/key.go` defines segment keys. A segment key has the form `seg/<32 lower-case hex>` (decision 43). `kwire/batch.go` parses record-batch headers, checks the CRC, and provides `PatchBatch`. `core/controlplane.go` defines the control plane contract. The contract covers `CommitSegment`, `Lookup`, and `DescribeTopics`. `core/action.go` defines `ObjPut`, `ObjGet`, `CPCall`, `Send`, `PauseReads`, and other actions.

Brokers hold no local state that another broker lacks (decisions 41-44 in `docs/decisions.md`). The system has no broker registration. The system has no epoch. The system has no fencing. The system has no idempotent producers. The broker never proposes an offset. Any broker can serve any partition. Metadata says the same (§4.7).

4.2 The open segment gathers batches before the broker seals one object

Incoming produce batches do not become objects one at a time. The batches gather in a single broker-wide buffer. The buffer is the open segment, `openSegment` in `broker/segment.go`.

```
type openSegment struct {
    batches []*pbatch             // admitted batches, in admission order
    bytes   int                   // sum of len(batch data)
    parts   []core.TopicPartition // distinct partitions present
    gen     core.TimerGen         // flush-timer generation; 0 = none armed
    since   core.Time             // when the first batch arrived
}
```

One open segment mixes topics and partitions. One sealed segment is one object and one control plane commit. The sealed segment may touch many partitions. The control plane assigns each batch an offset in one operation for one segment. The control plane call is `core.CommitSegment` in `core/controlplane.go`. A `pbatch` is one admitted batch on the way through the broker. The `pbatch` points at the request `slot`. The `pbatch` also points at the partition entry inside the Produce response. The broker fills in the response later. The broker fills in one batch at a time as commits land.

The word segment needs one meaning in this file. A segment is the unit that the broker seals, S3 stores, and the control plane commits. Before seal, the segment is open. After seal, the segment is one object under one key.

You read admission as the start of the produce path:

- `produce` in `broker/produce.go` walks every topic and partition of the request.
- `admit` first rejects `acks` other than 1 or -1 with `INVALID_REQUIRED_ACKS` (21).
- `acks=0` never reaches `admit`.
- `dispatch` in `broker/ops.go` marks the slot silent and closes the connection for `acks=0`.
- `admit` next rejects a transactional produce with `INVALID_REQUEST` (42).
- `admit` next rejects an unknown topic or partition with `UNKNOWN_TOPIC_OR_PARTITION` (3), and the topic cache decides the case (§4.7).
- `admit` next rejects a payload that is not exactly one record batch with `CORRUPT_MESSAGE` (2), and `kwire.OneBatch` makes the check.
- `admit` next rejects anything that `checkBatch` rejects (§4.6), and `checkBatch` makes compression-aware header checks.

A rejected partition gets an answer at once with the error code. An accepted batch raises the `pending` count of the slot by one. The batch joins the open segment through `addBatch`. The broker does not answer the batch yet. Validation errors get a fast answer. Success waits until commit (§4.4).

4.3 A seal rule ends the wait for more batches

`addBatch` in `broker/segment.go` appends the batch. `addBatch` adds the batch length to `open.bytes`. `addBatch` adds the batch length to broker-wide `bufBytes` (§4.8). `addBatch` records the partition in `open.parts` if the partition is new. `addBatch` then checks the seal rules. Decision 35 sets the rules. Decisions 39 and 52 set the caps again from the costs. `broker/config.go` holds the values.

The broker checks the seal rules in order:

- The broker checks the size rule first, and the broker seals with reason `size` when `open.bytes >= MaxSegmentBytes`.
- The broker checks the partition cap next, and the broker seals with reason `partitions` when the distinct partitions reach `MaxSegmentPartitions`.
- The broker checks the batch cap next, and the broker seals with reason `batches` when the batches reach `MaxSegmentBatches`.
- The broker checks the timer next, and the broker seals with reason `timer` when `BatchWindow` passes after the first batch.
- The broker checks shutdown last, and `stop()` seals with reason `stop` when the broker handles `core.Stop`.

`broker.Config` gives the defaults that you use in the common case. `MaxSegmentBytes` defaults to 16 MiB. `MaxSegmentPartitions` defaults to 4096. `MaxSegmentBatches` defaults to 256. `BatchWindow` defaults to 200 ms.

The detail below changes how you read the caps:

- The batch that reaches the size cap stays in the segment, and a segment overshoots 16 MiB by at most that batch.
- A single batch larger than the cap seals alone, and `broker: oversize batch sealed alone` is an expected reachability fact.
- `TestOversizeBatchSealedAlone` and `TestSizeRuleSealsAtExactlyTheCap` in `broker/flush_test.go` cover the size rule.
- The first batch arms the timer once, and the broker never moves the timer.
- `addBatch` sets `open.since` and a `timerFlush` timer only when the segment goes from empty to one batch.
- Later batches do not extend the 200 ms window, and a steady flow still seals at most 200 ms after the first batch.
- The mutant `BugTimerFromLastBatch` proves that tests catch a window that moves with each batch.
- The caps limit work for the control plane, and a commit costs the control plane about one item for each distinct partition.
- A partition costs about 20 bytes in a log entry, and the long comment on `MaxSegmentPartitions` in `broker/config.go` and `cp/doc.go` gives the cost.
- The value 4096 suits the log-based control plane store, and the DynamoDB transaction store needs `-max-segment-partitions 96`.
- The DynamoDB transaction store refuses wider segments with `ErrTooLarge`, and the broker never retries `ErrTooLarge` (§4.4).

- Sealing cancels the flush timer, emits a `segment_sealed` fact, resets the open segment, and hands the batches to `sealBatches`.
- The fact carries reason, bytes, batches, partitions, age, inflight, and buffered values.

The broker ignores a `TimerFired` when the `Gen` no longer matches `open.gen`. You see the check in `onTimer` in `broker/broker.go`. A timer that fires just after a size seal cannot seal the next segment early.

When the broker seals a segment, `sealBatches` encodes all batches into one object with `format.Encode` in `format/format.go`. The object has a 20-byte `MSEG` header. The object has one index entry for each batch. Each entry holds topic, partition, record count, byte offset and length, and a per-batch CRC32C. The object then holds the batch bytes without changes. The batch bytes sit one after another. Each `pbatch` remembers `core.SegmentRef{Key, Off, Len}` from the index. The control plane later stores the reference. Fetch later reads the reference. The object key is `format.SegmentKey(b.env.NewID())`. The key is `seg/` plus 32 hex digits of a random 128-bit ID. The injected `core.Env` draws the ID. The broker never draws the ID from a request ID (decisions 10 and 43). The broker chooses the key before the first upload attempt. The key never changes. Each retry retries the same object and the same commit.

4.4 The broker acks only after S3 stores the segment and the control plane commits the segment

The broker has no offset to give you before the control plane answers. S3 stores bytes but S3 assigns no offsets. The control plane assigns offsets but the control plane stores no batch bytes. An ack before both steps finish names an offset that no fetch can trust. An ack after the S3 write alone still lacks an assigned offset. An ack after the commit alone risks a committed range that points at bytes S3 never stored. The order removes both gaps.

`acks=1` and `acks=-1` get the same treatment. Both values mean committed. The broker has no weaker ack mode.

```
sequenceDiagram
    participant C as Kafka client
    participant B as Broker (Step)
    participant S as Object store
    participant CP as Control plane
    C->>B: Produce (acks=1/-1)
    B->>B: admit + checkBatch per partition
    Note over B: batch joins open segment<br/>slot pending++, bufBytes += len
    alt size / partition / batch cap reached
        B->>B: seal immediately
    else first batch of segment
        B->>B: arm flush timer (BatchWindow, fixed)
        B-->>B: TimerFired(timerFlush)
        B->>B: seal (reason=timer)
    end
    B->>S: ObjPut{seg/<id>, format.Encode(batches)}
```

```
    S-->>B: ObjPutDone OK
    Note over B: pump(): only head segment commits,<br/>strictly in seal order
    B->>CP: CPCall CommitSegment{SegmentKey, Batches{TP, Records, Loc}}
    CP-->>B: CPDone OK, Assigned[] ranges in request order
    B->>B: ack each batch: BaseOffset = Assigned[i].Base
    B->>C: ProduceResponse (produce_acked fact emitted<br/>immediately before the Send)
```

You follow a common produce in this order:

1. You send a Produce request with `acks=1` or `acks=-1`.
2. The broker runs `admit` and `checkBatch` for each partition.
3. The broker adds each accepted batch to the open segment.
4. The broker raises the `pending` count of the slot by one for each accepted batch.
5. The broker adds the batch length to `bufBytes`.
6. The broker seals the segment when a seal rule fires in §4.3.
7. The broker returns `core.ObjPut` with the segment key and the encoded object.
8. S3 stores the object and returns `ObjPutDone` with an OK result.
9. The broker marks the segment `segPutDone` and calls `pump()` in `broker/segment.go`.
10. The broker starts `CommitSegment` only for the first segment in `b.segs`.
11. The broker starts the commit only when the first segment is `segPutDone`.
12. The control plane assigns one range for each batch and returns `Assigned` in request order.
13. The broker checks that the control plane answers each batch in order for the right partitions.
14. The broker sets `BaseOffset` from `core.BatchRange.Base` for each batch.
15. The broker queues a `produce_acked` fact on the slot for each batch.
16. The broker counts the slot down through `finishPart`.
17. `drain` in `broker/conn.go` emits the facts just before the `core.Send` when the slot reaches zero.

Decision 12 covers the fact order. `TestFactImmediatelyBeforeSend` covers the fact order in a test.

The broker commits segments in seal order. PUT calls for later segments may run while an earlier segment waits. PUT calls may finish in a different order. Only the first segment in `b.segs` starts a commit. A later segment waits in `segPutDone`. The wait keeps control plane offset assignment in broker admission order for a partition. The `segment_committed` fact reports `put_ns + wait_ns + commit_ns = total_ns`. The `wait_ns` value gives the time a segment waits behind earlier commits (decision 36, `broker/timing_test.go`).

If the control plane gives any other answer, the broker raises `AlwaysOrUnreachable`. The broker does not guess a base offset. The `guess` code exists only for the `BugEarlyAck` mutant.

The broker never acks early in real code. Mutation tests check the rule. The test-only `BugEarlyAck` mutant in `broker/config.go` acks batches right after the PUT. The mutant uses offsets that the broker guesses. `TestEarlyAckMutant` in `broker/broker_test.go` must catch the mutant. The conformance suite in `TestConformanceBrokerCatchesMutants` must catch the mutant.

You pay for the order in three concrete ways:

- You wait for the 200 ms `BatchWindow` when no cap seals the segment first.

- You wait for S3 and then for the control plane, and the control plane wait grows when an earlier segment waits ahead of your segment.
- The broker holds your bytes in `bufBytes` until the ack or the error, and `bufBytes` can pause reads at 64 MiB (§4.8).

My view is that the price is right for this system. An ack that follows both steps gives you an offset that a fetch can resolve through the control plane to bytes that S3 stores. A cheaper ack saves time on the common path. The cheaper ack then leaves each fetch to face a range without bytes or an offset without a range. The broker takes the delay once at write time, and readers need no extra rule for such gaps.

If a step fails, every retry resends the same request under the same segment key. S3 treats a replayed PUT under the same key as the same write. The control plane treats a replayed `CommitSegment` as the same commit. A replayed `CommitSegment` returns the ranges that the control plane assigned the first time. The same key with a different request is `ErrConflict` in `core/controlplane.go`.

The retry rules below follow from the same order:

- `ObjPut` gets `MaxAttempts` (6) tries, with backoff `RetryBase` 20 ms doubling to `RetryMax` 640 ms, and the batches then fail with `REQUEST_TIMED_OUT` (7).
- `CommitSegment` gets no count limit, and the broker retries the commit until `CommitDeadline` (120 s from the first attempt).
- Commit backoff doubles to `CommitRetryMax` (2 s), and jitter in `[d/2, d]` from `Env.Intn` picks the exact delay.
- The broker never retries `ErrInvalid` at any stage, and the batches fail with `UNKNOWN_SERVER_ERROR` (-1).
- The broker never retries `ErrTooLarge` on a commit, and `ErrTooLarge` raises an `Unreachable` assertion because the broker and the control plane break the contract.
- The broker never retries `ErrConflict` on a commit, and `ErrConflict` raises an `Unreachable` assertion for the same reason.

Decision 47 sets the commit policy. A control plane restart must not fail each in-flight produce. The segment waits through the outage. The segment bytes stay in `bufBytes` during the wait. Backpressure keeps working during the wait (§4.8). Only the 120 s deadline fails the segment. The deadline stays under the `delivery.timeout.ms` value that producers set. PUT retries and op retries keep the count cap in `retry` and `retryOp`. After the deadline or the attempt cap passes, a `MaybeApplied` write may still land later (decision 15). The client sees a timeout. At-least-once delivery already allows the duplicate that a late write can create (decision 41).

When a segment fails, `failSegment` answers each unanswered batch with the error code. `endSegment` releases the `bufBytes` for the segment. `endSegment` removes the segment from `b.segs`. `endSegment` pumps the commit for the next segment. Success and failure both call `endSegment`. The commit queue cannot stall behind a failed first segment.

```
stateDiagram-v2
    [*] --> Open: first batch admitted<br/>(timer armed)
    Open --> Open: more batches admitted
    Open --> segPut: seal (size / partitions /<br/>batches / timer / stop)
```

```
    segPut --> segPutBackoff: ObjPutDone error
    segPutBackoff --> segPut: retry timer (≤ MaxAttempts)
    segPut --> segPutDone: ObjPutDone OK
    segPut --> Failed: ErrInvalid, or attempts exhausted
    segPutDone --> segCommit: pump(): segment is head
    segCommit --> segCommitBackoff: transient CPDone error<br/>or MaybeApplied
    segCommitBackoff --> segCommit: backoff + jitter,<br/>same key, until CommitDeadline
    segCommit --> Committed: CPDone OK → ack all batches
    segCommit --> Failed: ErrInvalid / ErrConflict /<br/>ErrTooLarge / deadline passed
    Committed --> [*]: endSegment: bufBytes released,<br/>next head pumped
    Failed --> [*]: endSegment: batches answered<br/>with error, bufBytes released
```

The `segState` values in `broker/segment.go` are `segPut`, `segPutBackoff`, `segPutDone`, `segCommit`, and `segCommitBackoff`. The open state is the separate `openSegment` buffer. The broker removes committed segments from `b.segs`. The broker removes failed segments from `b.segs`. The broker keeps no terminal state for a finished segment.

4.5 Fetch reverses the produce order through the control plane and then S3

Fetch reads committed data (decision 30). The fetch path starts where the produce rule leaves the data. The control plane holds the committed offset and the byte range. S3 holds the batch bytes. The segment stores the record-batch bytes that the producer wrote. The batch bytes include the `baseOffset` value that the producer wrote. No segment byte holds the committed offset. The broker rewrites the first bytes of each batch on the fetch path. `broker/fetch.go` serves each partition on its own. The broker answers at once. The broker ignores `MinBytes` and `MaxWait` (decision 29). The broker never grants incremental fetch sessions. A request with `SessionEpoch > 0` gets `FETCH_SESSION_ID_NOT_FOUND` (70).

```
sequenceDiagram
    participant C as Kafka client
    participant B as Broker (Step)
    participant CP as Control plane
    participant S as Object store
    C->>B: Fetch (per-partition FetchOffset)
    B->>CP: CPCall Lookup{TP, From, MaxBytes}
    CP-->>B: LookupResp{Ranges[] in offset order, End}
    Note over B: pick ranges fitting PartitionMaxBytes<br/>(always at least one); set HWM = End
    loop one ranged GET at a time (op.next)
        B->>S: ObjGet{Ref = BatchRange.Loc (Key, Off, Len)}
        S-->>B: ObjGetDone (batch bytes)
        B->>B: ParseBatch: CRC ok, len == Loc.Len,<br/>NumRecords == Range.Count
        B->>B: PatchBatch(data, base = Range.Base, epoch 0)
```

```
    end
    B->>C: FetchResponse (RecordBatches = patched bytes)
```

You follow a common fetch in this order:

1. You send a Fetch request with a `FetchOffset` for a partition.
2. The broker starts one `opFetch` for the partition in `broker/ops.go`.
3. The broker sends `core.Lookup{TP, From: FetchOffset, MaxBytes}` to the control plane.
4. The control plane returns ranges in offset order and returns `End` in `LookupResp`.
5. The broker sets `HighWatermark` and `LastStableOffset` to `LookupResp.End`.
6. The broker takes ranges until the next range would pass the partition limit.
7. The broker always takes at least one range, and a batch larger than the limit still goes out whole.
8. The broker issues one `core.ObjGet` for each range in sequence through `o.next`.
9. S3 returns the batch bytes for the byte range in `BatchRange.Loc`.
10. The broker parses the bytes with `kwire.ParseBatch`.
11. The broker checks that the byte length equals `Loc.Len`.
12. The broker checks that the header record count equals `Range.Count`.
13. The broker checks the batch CRC32C over the range bytes.
14. `kwire.PatchBatch(data, base, 0)` rewrites bytes 0-7 to `int64(rg.Base)` and bytes 12-15 to epoch 0.
15. The broker places the patched bytes in the Fetch response.

The broker uses `PartitionMaxBytes` from the request as `MaxBytes` for Lookup. The broker uses `DefaultFetchBytes` (1 MiB) when the request asks for no limit. The broker answers an unknown partition in the response with `UNKNOWN_TOPIC_OR_PARTITION`. The broker answers a negative offset in the response with `OFFSET_OUT_OF_RANGE`. The system has no transactions. The system has no retention. `HighWatermark` and `LastStableOffset` therefore match. A `From` value above `End` gets `OFFSET_OUT_OF_RANGE`. A `From` value equal to `End` gets a successful empty answer.

You read the byte range from `BatchRange.Loc`. The reference holds the segment key and the exact byte offset and length from seal time. The fetch path never reads a whole object. The fetch path never reads the segment header. The fetch path never reads the segment index.

If a GET returns bad bytes, the broker treats the GET as corrupt. The broker retries the GET. S3 checksums cover whole objects. S3 checksums do not cover byte ranges (decision 27). The batch CRC32C covers the range bytes. The broker never serves bytes that fail the checks. `BugServeCorrupt` is the mutant for the rule. If a committed segment returns `ErrNotFound`, the broker raises an `Unreachable` assertion. The broker fails the partition with `KAFKA_STORAGE_ERROR` (56). If retries end with corrupt data, the broker fails the partition with `KAFKA_STORAGE_ERROR` (56) through `exhaustCode` in `broker/ops.go`. Other GET failures and control plane failures retry up to `MaxAttempts`. The failures then end with `REQUEST_TIMED_OUT` (7).

The patch is safe for the CRC. The CRC covers bytes 21 onward. Bytes 0-7 hold `baseOffset`. Bytes 12-15 hold `partitionLeaderEpoch`. Both fields sit outside the CRC. A patched batch still passes client CRC checks (decision 30). `BugWrongOffset` patches `base + 1`. The mutant proves that `TestProduceFetchRoundTrip` checks real offsets.

The broker applies the per-partition limit at Lookup time. The broker applies the request-wide `MaxBytes` last in `finalize` in `broker/conn.go`. Partitions stay in order. After the running total would pass the limit, later partitions get empty `RecordBatches`. The broker never cuts the first non-empty partition.

The broker issues one GET for each range. The broker issues the GET calls for one partition in sequence. The `op` state machine holds `opCP`, `opGet`, and `opBackoff` states. Different partitions run at the same time as independent ops on the same slot.

4.6 The broker stores compressed batches as bytes the broker never reads

The broker never decompresses a batch. The produce rule needs the stored bytes to stay exact. Fetch patches only the two fields outside the CRC. Decompression would add work and would add a codec library to the deterministic packages. `checkBatch` in `broker/produce.go` validates one batch from the header alone. `checkBatch` does not read the payload for validation.

The checks give you the following results:

- The magic value must be 2 and the length must match the payload, or the broker returns `UNSUPPORTED_FOR_MESSAGE_FORMAT` (43).
- The CRC32C over the batch bytes must match, or the broker returns `CORRUPT_MESSAGE` (2).
- The codec id in attribute bits 0-2 must be 0-4, and the ids name none, gzip, snappy, lz4, and zstd.
- An id above 4 with a valid CRC gets `UNSUPPORTED_COMPRESSION_TYPE` (76).
- The batch must not be transactional and must not be a control batch, or the broker returns `INVALID_REQUEST` (42).
- `ProducerID < 0` must hold, because the system refuses idempotent producers (decision 41).
- `NumRecords > 0` must hold and `LastOffsetDelta == NumRecords - 1` must hold, or the broker returns `CORRUPT_MESSAGE` (2).

The batch CRC covers each byte after the CRC field. The batch CRC covers compressed records. The batch CRC excludes `baseOffset`. The batch CRC excludes the leader epoch. Fetch rewrites the two excluded fields. A gzip batch follows the full path without broker decompression. The broker admits the batch. S3 stores the batch bytes without changes in the segment. The control plane commits the batch by header record count. Fetch returns the batch bytes and patches the two fields. The consumer decompresses the batch. No codec library enters the deterministic packages (decision I). The broker does not check that the payload decompresses to `NumRecords` records. A producer can set a false record count in the header. Consumers of the producer then receive data with the false count. Offsets use the header count. `broker/compress_test.go` covers mixed-codec segments stored without changes. `broker/compress_test.go` covers the rejection table. `broker/compress_test.go` covers fetch limits in stored compressed bytes.

The section covers decision 45.

4.7 Metadata names the answering broker as the leader for each partition

Metadata has no leader election (decision 29, `broker/metadata.go`). The answering broker lists only the answering broker. The broker takes `BrokerID`, `Host`, and `Port` from `Config`. The broker names the broker as leader with epoch 0. The broker names the broker as the sole replica. The broker names the broker as the sole ISR of each partition of each topic that the control plane reports. The broker does not create topics. An unknown topic gets `UNKNOWN_TOPIC_OR_PARTITION` (3). A request by topic ID gets `UNKNOWN_TOPIC_ID` (100). The broker does not support topic IDs. A `nil` topic list means all topics. The broker answers the request from the `DescribeTopics` result.

The broker caches topic shapes in `b.topics` in `broker/ops.go`. `knows(tp)` reads the cache and writes nothing. Produce uses the cache. Fetch uses the cache. ListOffsets uses the cache. If the cache lacks a name, dispatch stores the request in an `opRefresh` op. Dispatch does not fail the request at the miss. The broker issues `DescribeTopics` for the missing names. The broker merges the answer into the cache. Partition counts only grow in the merge, and the merge uses `max`. The broker then runs the stored request once through `resume`. The partition gets an error only when the topic stays unknown after the refresh. Metadata requests use the same op machinery as `opMeta`. If a control plane call fails after retries, the broker marks each requested topic with `LEADER_NOT_AVAILABLE` (5).

ListOffsets in `broker/metadata.go` supports two timestamps. Timestamp -2 means earliest and always gives offset 0. Version 1 has no retention. Timestamp -1 means latest and gives `LookupResp.End`. The broker fetches the latest value with a past-the-end `Lookup{From: 1<<63 - 1, MaxBytes: 1}`. Any other timestamp gets `UNSUPPORTED_FOR_MESSAGE_FORMAT` (43).

4.8 Backpressure holds the bytes that the ack rule keeps in the broker

The ack rule keeps produce bytes in the broker until S3 and the control plane finish. `bufBytes` counts produce bytes that the broker admits and does not yet answer. `addBatch` raises `bufBytes`. `endSegment` lowers `bufBytes` on commit and on failure. The count covers the open segment. The count covers PUT calls in flight. The count covers segments that wait behind the first commit. The count covers a segment that waits through a control plane outage inside `CommitDeadline`. At the end of each `Step`, `finish` runs `flow` in `broker/broker.go`.

The flow rules give you the following results:

- `bufBytes >= MaxBufferedBytes` turns pressure on, and the 64 MiB default sets the mark.
- `flow` emits `backpressure_on` and sends `PauseReads` on each client connection.
- `bufBytes <= ResumeBufferedBytes` turns pressure off, and the 32 MiB default sets the mark.
- `flow` emits `backpressure_off` with the paused duration and sends `ResumeReads` on all connections.
- The gap between 64 MiB and 32 MiB stops pause and resume from switching back and forth in quick succession.
- Reads stay paused while `stopping` holds, and the value of `bufBytes` does not change the rule.

The broker does not drop admitted work. Pausing reads stops new bytes. The broker seals each admitted batch. S3 stores the segment. The control plane commits the segment. The broker answers the batch. A batch can fail under the retry budget. One request may pass the pause mark. Admission runs before `flow`. One frame can reach `MaxFrame` (100 MiB). `bufBytes` can pass the pause mark by at most one in-

flight request for each connection. The cap bounds steady buffered bytes. The cap is not a hard ceiling. Production holds two or three copies of buffered data. The copies are the frame, the admitted batch, and the encoded segment. `cmd/manibroker` sizes the broker cap from machine memory for this reason in `cmd/manibroker/tuning.go` (decision 35).

Retries form part of flow control. An op stops retrying when the client connection closes. `retryOp` checks `o.s.conn.closed`. Fetch ops give up after `MaxAttempts`. Metadata ops give up after `MaxAttempts`. The ops use the codes in §4.5 and §4.7. The ops do not retry without limit for a caller that has left.

4.9 The Config table lists each knob and the default

The table lists all fields of `broker.Config` (`broker/config.go`). Zero numeric and string fields take the `DefaultConfig` value through `withDefaults`. `BrokerID` is the exception. The broker takes `BrokerID` as given. Production flags live in `cmd/manibroker/main.go`.

Field	Default	Meaning
`BrokerID`	1	Node ID advertised in Metadata
`Host`	`localhost`	Advertised host
`Port`	9092	Advertised port
`MaxFrame`	100 MiB	Largest request frame; a larger one closes the connection (decision 45)
`BatchWindow`	200 ms	Flush timer, armed by the first batch of a segment
`MaxSegmentBytes`	16 MiB	Seal when the open segment holds this many batch bytes
`MaxSegmentPartitions`	4096	Seal at this many distinct partitions; use 96 for the DynamoDB transaction CP
`MaxSegmentBatches`	256	Seal at this many batches (bounds CP commit bookkeeping)
`MaxAttempts`	6	Tries per PUT, op (Lookup/GET/Metadata), before giving up
`RetryBase`	20 ms	First retry backoff; doubles each try
`RetryMax`	640 ms	Backoff ceiling for PUT/op retries
`CommitDeadline`	120 s	How long one segment's commit is retried, from its first attempt (decision 47)
`CommitRetryMax`	2 s	Backoff ceiling for commit retries (jittered `[d/2, d]`)
`MaxBufferedBytes`	64 MiB	Pause all client reads at or above this many unacked produce bytes
`ResumeBufferedBytes`	32 MiB	Resume reads at or below this; forced to `MaxBufferedBytes/2` if unset or above the max
`DefaultFetchBytes`	1 MiB	Per-partition fetch limit when a Fetch asks for none (`<= 0`)

Field	Default	Meaning
`Bug`	`BugNone` (0)	Test-only mutant selector; production must never set it

4.10 Slots keep responses in request order on each connection

Kafka clients send requests before earlier responses return. Clients expect responses in request order. `broker/conn.go` models each connection as a queue of `slot` values. One slot holds one decoded request. One slot holds the response under construction. One slot holds a `pending` count of unfinished parts. One slot holds facts to emit with the response. Handlers in `handle` and `dispatch` in `broker/ops.go` run at once and start async ops. `drain` sends only the first slot. `drain` sends the slot only when the `pending` count reaches zero. `drain` sends slots in queue order. A slow Fetch cannot pass a fast Metadata behind the Fetch. `TestPipelinedResponsesInOrder` covers the rule.

The connection rules give you the following results:

- `onOpen` registers the connection with a `kwire.Framer` bounded by `MaxFrame`, and the broker refuses a known ID.
- The broker refuses a new connection with `CloseConn{Abort: true}` while `stopping` holds.
- `onBytes` feeds the running receive hasher, and the hasher forms part of `StateHash`.
- `onBytes` pushes bytes through the framer and dispatches each complete frame.
- A framing error or a decode error marks the connection `closing`, and the broker reads no new requests from the connection.
- Queued responses still drain from a `closing` connection.
- An unsupported API closes the connection without a response through `s.silent, s.conn.closing = true` in `dispatch`.
- ApiVersions is the exception, and the broker answers ApiVersions for versions that the codec cannot fully decode through `kwire.ApiVersionsResponse`.
- An `acks=0` produce takes the same silent path, and the broker closes the connection after the queue drains.
- `drain` emits `CloseConn{Abort}` when a `closing` queue empties or `abort` holds, and `drain` then drops the connection.
- The broker drops the connection on peer close in `onClosed` and discards answers for the connection.
- `drain` skips connections where `c.closed` holds, and in-flight ops read `conn.closed` and retire in place of retrying.

If you send `core.Stop`, the broker sets `stopping`. The broker seals the open segment with reason `stop`. The broker pauses all reads. The broker refuses new connections. New requests close the connection for the request. `maybeExit` waits until no ops remain. `maybeExit` waits until no sealed segments remain. `maybeExit` waits until no open batches remain. `maybeExit` waits until no queued slots remain. The broker then closes each connection. The broker emits `broker_exited`. The broker returns `core.Exit`. `TestGracefulStop` covers the path.

`broker/hash.go` computes the FNV-1a `StateHash`. The hash folds in the config `Bug` value, counters, topics, connections and slots, the open segment, sealed segments, ops, `bufBytes`, and pressure. The

broker writes the hash code. The broker does not use `hash/fnv`, and `hash/fnv` sits outside the deterministic allow-list. The hash excludes pure timing fields. The fields are `sealedAt`, `putDoneAt`, and `pressureAt`. The timing fields feed facts. The timing fields never change behaviour.

Key takeaways

- One open segment exists for each broker. The open segment mixes partitions.
- The broker seals the open segment at 16 MiB, 4096 partitions, 256 batches, a fixed 200 ms window from the first batch, or shutdown.
- The produce order is `ObjPut`, then control plane `CommitSegment`, then ack. The broker has no offsets that the broker owns.
- An early ack is impossible outside the `BugEarlyAck` test mutant.
- The broker issues commits in seal order and commits the first segment only. The broker retries a commit under the same segment key until the 120 s deadline passes, and PUT calls retry 6 times.
- Fetch runs Lookup, then one ranged GET for each committed batch, then CRC, length, and count checks, then `PatchBatch(baseOffset = committed Base, epoch 0)`.
- The broker validates compression from the batch header and CRC only. The codec ids name gzip, snappy, lz4, and zstd.
- S3 stores compressed batches as opaque bytes, and the broker never decompresses the batches.
- Backpressure is one pair of marks over unacked produce bytes. The broker pauses reads at 64 MiB and resumes at 32 MiB through `PauseReads` and `ResumeReads`, and the broker never drops admitted work.
- Slots for each connection keep responses in request order. The broker emits each `produce_acked` fact just before the `Send` for the fact.

Checkpoint

1. A produce with acks=-1 is PUT to S3 successfully, then the CP is down for 30 s. What does the client see, and when? — You see nothing at first. You get no ack and no error. The segment sits in `segCommitBackoff`. The broker retries the same `CommitSegment` under the same key. Backoff carries jitter and a cap at `CommitRetryMax` (2 s). The 30 s outage sits inside `CommitDeadline` (120 s). The commit lands when the control plane returns. The control plane assigns offsets. The broker then acks you. The segment bytes count in `bufBytes` during the wait. Backpressure may pause reads during the wait. After 120 s, the batches fail with `REQUEST_TIMED_OUT`.

2. Why can't a later segment commit before an earlier one, even if its PUT finished first? — `pump()` in `broker/segment.go` starts the commit of `b.segs[0]` only. `pump()` starts the commit only when `b.segs[0]` is `segPutDone`. A later segment that uploads early waits in `segPutDone`. The wait is the `wait_ns` value on `segment_committed`. First-only commits keep control plane offset assignment in broker admission order for each partition.

3. A client fetches offset 100 and the CP returns three ranges. The second GET returns bytes whose header says 40 records but the range says 41. What happens? — `onFetchGet` in `broker/fetch.go` treats the bytes as a corrupt GET. The broker checks `ParseBatch` output against `Loc.Len` and `Range.Count`. The op emits `corruption_detected`. The op retries the ranged GET. S3 checksums do not

cover byte ranges. The batch CRC covers the range bytes. The broker never patches the bad bytes. The broker never sends the bad bytes. If retries end with the corrupt flag set, the partition fails with `KAFKA_STORAGE_ERROR` (56).

4. Why is patching `baseOffset` on fetch safe for the batch CRC, and why does that make compressed batches servable without decompression? — The v2 batch CRC32C covers bytes 21 to the end. `baseOffset` (0-7) and `partitionLeaderEpoch` (12-15) lie outside the range (decision 30, `kwire.PatchBatch`). Compression sits inside the covered region. The broker never modifies compressed bytes. The broker validates the CRC over the compressed bytes. S3 stores the bytes without changes. The broker serves the bytes without changes. The broker rewrites the two uncovered fields only. The consumer decompresses the batch (decision 45).

5. A single 20 MiB batch arrives at an empty broker with default config. Describe its path and the segment it creates. — The answer assumes a valid header. `admit` accepts the batch. `addBatch` appends the batch as the first batch. The broker arms the 200 ms timer. The size check then fires at once. The batch size is 20 MiB. The cap is 16 MiB. The broker seals the segment alone with reason `size`. A segment overshoots the cap by at most the last batch. The batch is the segment here. The broker cancels the timer. S3 stores the object under a fresh `seg/<id>` key. The control plane commits the segment. The broker acks you with the base offset that the control plane assigns.

Try it

The tasks below use tests that the broker package already holds:

- Run the round trip and seal-rule tests and read the facts that the tests assert.

```
go test ./broker/ -run 'TestProduceFetchRoundTrip|TestTimerRuleFromFirstBatch|
        TestSizeRuleSealsAtExactlyTheCap|TestTwoSegmentsInFlightCommitInOrder' -count=1 -v
```

- Use mutation to test the never ack early claim, and read how each `Bug` value wires into `broker/segment.go` and `broker/fetch.go`.

```
go test ./broker/ -run 'TestEarlyAckMutant|TestWrongOffsetMutant|
        TestConformanceBrokerCatchesMutants' -count=1 -v
```

- If you change `BatchWindow` and `MaxSegmentBytes` in a test on `broker/flush_test.go` and `broker/harness_test.go`, you predict each `segment_sealed` reason before you run the test.
- If you trace one fetch in `broker/fetch.go`, you follow a range from `core.LookupResp.Ranges` through `core.ObjGet{Ref: rg.Loc}` to `kwire.PatchBatch`, and you confirm that no whole-object GET exists on the path.
- If you run the whole package as CI runs the package, you use `go test ./broker/ -count=1`.

Further reading

The sources below give you the decisions and the code behind this chapter:

- `docs/decisions.md` records decisions 29 (leaderless metadata), 30 (fetch rewrites offsets), 35 (segment flush rule and caps), 41-44 (no idempotent producers, CP-assigned offsets, flat segment keys, stateless brokers), 45 (compression as opaque bytes), 47 (commit deadline), and 52 (partition cap).
- `broker/config.go` holds the long comment on `MaxSegmentPartitions`. The comment covers how the caps couple the broker to each CP store. `cp/doc.go` describes the CP side.
- `format/format.go` defines the `MSEG` object layout and the per-batch CRCs. Ranged GETs use those CRCs for verification.
- `core/controlplane.go` defines the `CommitSegment` and `Lookup` contract. The contract covers idempotence per segment key.
- `docs/flow-control.md` gives the design background for `PauseReads` and hysteresis. It covers why admission control lives inside `Step`.
- `broker/flush_test.go`, `broker/compress_test.go`, and `broker/timing_test.go` hold executable versions of §4.3, §4.6, and the segment timing facts.

Chapter 5. The Kafka wire format and the segment format

Manifester uses two byte formats. `kwire/` handles the Kafka protocol. Clients send length-prefixed frames that contain record batches. The broker answers with length-prefixed frames. `format/` handles the segment format. Manifester packs sealed batches from many partitions into one segment object. The broker stores the segment object under a `seg/<random>` key. Readers fetch parts of the segment object by byte range.

Both packages are pure and deterministic. The packages use no clock. The packages use no randomness. The packages do no input or output. Callers pass in timestamps and IDs. The same code runs in the broker state machine and in the simulated clients. A byte that a test produces matches a byte that a real client accepts. The sources are `kwire/doc.go`, `kwire/codec.go`, `kwire/framer.go`, `kwire/batch.go`, `kwire/supported.go`, `format/format.go`, and `format/key.go`. The call sites are in `broker/produce.go`, `broker/fetch.go`, and `broker/segment.go`.

5.1 Each file in `kwire` and `format` owns one task

Package	File	Responsibility
`kwire`	`kwire/framer.go`	Reassemble TCP streams into Kafka frames
`kwire`	`kwire/codec.go`	Decode requests and encode responses for the broker. Decode responses and encode requests for sim clients
`kwire`	`kwire/supported.go`	API keys and versions the broker advertises and serves
`kwire`	`kwire/batch.go`	Record batch v2 build, parse, verify, split, and patch
`format`	`format/format.go`	Segment object layout with header, index, and payloads
`format`	`format/key.go`	Segment keys in the form `seg/<32 lowercase hex>`

`kwire` wraps franz-go's `kmsg` (`github.com/twmb/franz-go/pkg/kmsg`). The `kmsg` package supplies per-API message types. Manifester computes the batch CRC, the batch lengths, and the batch timestamps. `kmsg` writes the values that Manifester computes. `kwire/doc.go` and `docs/dep-audits/github.com/twmb/franz-go/pkg/kmsg@v1.14.0.md` describe the split.

5.2 `kwire.Framer` reassembles a TCP stream into frames

Kafka frames have a 4-byte big-endian length. The payload follows the length. The payload has the number of bytes that the length names. TCP does not keep the frame boundaries. `Framer` reassembles the frames from the TCP stream.

```
stream bytes (arbitrary Push chunking)
+----------+-------------+----------+-------------+---
| len = N  | payload N B | len = M  | payload M B | ...
|  u32 BE  |             |  u32 BE  |             |
+----------+-------------+----------+-------------+---
      \________ frame 1 ______/ \_____ frame 2 ___/
```

`Framer.Push(data)` appends to an internal buffer. It returns every complete frame in order. The returned frames do not include the length prefix. Incomplete tails stay in the buffer. `Buffered()` reports the count of held bytes.

A length `<= 0` yields `ErrBadFrame`. A length above the `max` given to `NewFramer(max)` yields `ErrBadFrame`. `Push` still returns frames that it completed before the bad length. It returns those frames with the error. `ErrBadFrame` does not recover. Every later `Push` returns the same error. The framer discards the buffer. The connection ends after `ErrBadFrame`.

Each frame is a copy (`append([]byte(nil), ...)`). A frame is not a view into the framer buffer. A later `Push` cannot mutate a frame that a caller still decodes. Section 5.5 states the aliasing rules.

One `Framer` serves one connection. The `core.Framer` interface is in `core/kafka.go`.

5.3 `kwire.Codec` decodes requests and encodes responses

`kwire.Codec` is a stateless value. The value implements `core.Codec`. `kwire/doc.go` states the rule that pairs each encode function with a decode function.

> `Encode*` functions return a complete frame **with** the 4-byte length prefix, ready to write. `Framer` strips the prefix, so `Decode*` functions take a frame **without** it.

A request on the wire has this layout.

```
+----------+------------------+-----------------+-----------------+
|  length  | api key | version | correlation id | client id ...   |
|  u32 BE  |  i16 BE |  i16 BE |     i32 BE     | i16 len + bytes |
+----------+------------------+-----------------+-----------------+
 \_ EncodeRequest adds this /   \_______ DecodeRequest input ______/
   Framer strips it again
```

`DecodeRequest` parses the fixed header without `kmsg`. The header has a minimum of 10 bytes. The function parses the header before the function parses the body with `kmsg`. The fields are the api key `i16`, version `i16`, and correlation id `i32`. A nullable client-id follows. The client-id has an `i16` length. A length of `-1` means the client-id is absent. A length below `-1` is malformed.

`DecodeRequest` then takes one path from the list below.

- An unsupported key or version gives a parsed header and an `*UnsupportedError` (see §5.4). The code does not parse the body.
- For flexible APIs, `skipTags` walks the header tag section and discards the header tags. `kmsg` handles trailing body tags.
- For a supported key and version, the code parses the body with `kmsg`'s `ReadFrom`. The code does not use `UnsafeReadFrom`.

`EncodeResponse` writes the correlation id. The function writes a single `0x00` header-tag byte for flexible APIs. The function then writes the body. The `prefix` helper reserves 4 bytes for the length. The `seal` helper fills the 4 bytes and seals the frame. `EncodeRequest` does the same work for sim clients. The function encodes the client id as a non-null string. `DecodeResponse` checks the correlation id against the request header. The function returns `ErrMalformed` on a mismatch. All malformed input in this file wraps `ErrMalformed`. Callers match the error with `errors.Is`.

ApiVersions has one special case. Kafka answers an ApiVersions request that Kafka cannot serve at version 0 with error 35. `respVersion` encodes a response for an unserved version as v0. ApiVersions uses response header v0. Response header v0 has no tag byte. A flexible v3 request also gets response header v0 in this case.

5.4 `kwire/supported.go` serves five API keys

The broker serves five keys. The table below lists the ranges from `Supported()` in `kwire/supported.go`. `ApiVersionsResponse` advertises the ranges.

API key	Name	Min	Max	Notes
0	Produce	3	9	`>= 3` so batches are record batch v2
1	Fetch	4	12	below v13, which switches to topic IDs
2	ListOffsets	1	7	
3	Metadata	1	12	
18	ApiVersions	0	3	answered at v0 when request version unsupported

`DecodeRequest` rejects any other key or version with an `*UnsupportedError`. The type has this shape.

```
type UnsupportedError struct {
    Key, Version int16
    UnknownKey   bool // key is not served at all (versus version out of range)
}
```

`UnknownKey: true` means the key is absent from `Supported()`. InitProducerID (key 22) is one example. The broker does not serve InitProducerID because the broker does not support idempotent producers. `TestInitProducerIDNotSupported` in `kwire/codec_test.go` checks that result. The test checks that the broker does not advertise the key. The test checks that `DecodeRequest` does not decode the key. A client that needs InitProducerID fails at startup. `UnknownKey: false` means the broker serves the key. The version is out of range in this case. Produce v2, Fetch v13, ApiVersions v99, and a negative version are examples.

`DecodeRequest` returns the parsed `core.RequestHeader` with the error. The broker uses the header to send a v0 ApiVersions response. `ApiVersionsResponse` builds the response. The response carries `ErrCodeUnsupportedVersion` (35). This response path does not require the broker to close the connection during the handshake.

5.5 Decoded data aliases the input frame

Decoded `kmsg` messages alias the input frame. Record keys and values point into the bytes that the caller passes to `ReadFrom`. Produce partition `Records` point into the frame bytes. Fetch record batches point into the frame bytes. The code makes no defensive copy. The rules follow.

- A frame from `Framer.Push` is a fresh slice. The framer does not reuse it. The framer does not mutate it. Aliased decoded data stays valid while the caller keeps the frame alive and does not mutate it.
- Callers use `kmsg` through `ReadFrom` only. They do not use `UnsafeReadFrom`.
- `kwire` computes every value that `kmsg` writes verbatim. Those values are the batch CRC-32C, lengths, and caller-supplied timestamps. The code uses no clock. The same inputs give the same frame. The codec tests assert byte-identical re-encoding.

`ParseBatch` and `format.Decode` follow the same aliasing rule. `Batch.records` aliases the input to `ParseBatch`. `Batch.Payload()` aliases the input buffer. Decoded record keys and values alias the input buffer. `SplitBatches` returns capacity-capped subslices (`b[:n:n]`). An `append` to a subslice cannot write over the next batch. `format.Segment.Batches[i].Data` aliases the object that the caller passes to `format.Decode`. `PatchBatch` copies the batch before the patch writes to the batch (see §5.7). The caller copies bytes that must outlive the source frame. The caller copies bytes that must outlive the source segment object.

5.6 `kwire/batch.go` uses record batch format version 2 only

Manifester uses batch format version 2 only (magic `2`). Produce v3 is the first Produce version with record batch version 2. A version 2 batch has a 61-byte header and a records section. The layout follows.

```
offset  size  field
------  ----  ------------------------------------------------
 0       8    baseOffset            } rewritten by PatchBatch
 8       4    length (= total - 12) }
12       4    partitionLeaderEpoch  } rewritten by PatchBatch
16       1    magic (= 2)
```

```
17      4    CRC-32C  ------------------------------+
21      2    attributes            |  CRC covers  |
23      4    lastOffsetDelta       |  bytes 21    |
27      8    firstTimestamp        |  to the end  |
35      8    maxTimestamp          |              |
43      8    producerId            |              |
51      2    producerEpoch         |              |
53      4    baseSequence          |              |
57      4    numRecords            |              |
61      ...  records (or compressed records payload) ------+
```

`ParseBatch` verifies one batch. The function does not decompress the batch. The function runs the checks in the order that the list below gives.

- A batch with fewer than 17 bytes gives `ErrCorrupt`. The code cannot read the magic byte from a batch with fewer than 17 bytes.
- A batch with a byte 16 other than `2` gives `ErrUnsupportedBatch`. The code rejects magic 0 and magic 1 from the message-set era. The code rejects a newer magic. The code does not convert a batch with another magic.
- A batch with fewer than 61 bytes gives `ErrCorrupt`. A batch with `length != len(b) - 12` gives `ErrCorrupt`. A batch with a length below the 49-byte minimum for an empty records section gives `ErrCorrupt`.
- The CRC-32C (Castagnoli, `crc32.MakeTable(crc32.Castagnoli)`) over `b[21:]` must equal `b[17:21]`. A mismatch gives `ErrCorrupt`. `ParseBatchUnverified` skips the CRC-32C check. A broker mutant uses `ParseBatchUnverified` to model a missing CRC-32C check. Real code uses `ParseBatch`.
- A codec id (attributes bits 0-2) above 4 with a valid CRC gives `ErrUnknownCodec`. The check order matters. A batch with invalid codec bits and a bad CRC gives `ErrCorrupt`. Tests recompute the CRC with `refix` to reach the codec check.
- `kmsg.RecordBatch.ReadFrom` parses the batch structure again. `countFits` checks the record count. An uncompressed payload of `n` bytes holds at most `n` records. A compressed payload can expand to any size. `countFits` checks that the count is not negative. `countFits` checks that the payload is not empty for a positive count.

Error	Meaning
`ErrCorrupt`	structure or CRC validation failed
`ErrUnsupportedBatch`	magic != 2, or `Records()` on a compressed batch
`ErrUnknownCodec`	valid CRC, but compression codec id > 4
`ErrTruncated`	`SplitBatches` hit a partial trailing batch
`ErrNotSingle`	a Produce payload did not hold exactly one batch

`assembleBatch` builds each batch. The function takes the header from caller-supplied `BatchParams`. The params hold the leader epoch, the attributes, the producer id, the producer epoch, the base sequence, and the timestamps. The code uses no clock. The code sets `magic = 2`. The code sets `lastOffsetDelta = numRecords - 1`. The code computes the length. The code backpatches the CRC.

`BuildBatch` builds an uncompressed batch with base offset 0. `BuildBatch` numbers records by index (`OffsetDelta = i`). `BuildBatch` rejects a non-zero compression attribute. `BuildOpaqueBatch` wraps a payload that a caller encodes. The payload may be compressed. `BuildOpaqueBatch` does not read the payload content. The CRC must be valid. The payload does not need to decompress. Tests and the simulator use `BuildOpaqueBatch` for a payload from a real client. `SplitBatches` reads concatenated batches by length. `SplitBatches` checks structure only. `SplitBatches` does not check the CRC. `SplitBatches` returns complete batches. `SplitBatches` returns `ErrTruncated` for a partial tail. `OneBatch` requires one batch. `broker/produce.go` requires one batch per Produce partition.

The broker stores compressed batches as opaque bytes

Codec ids are the low three attribute bits. The table below lists each codec id, the codec name, and the constant.

Codec id	Name	Constant
0	none	`kwire.CodecNone`
1	gzip	`kwire.CodecGzip`
2	snappy	`kwire.CodecSnappy`
3	lz4	`kwire.CodecLZ4`
4	zstd	`kwire.CodecZstd`

The broker stores compressed payloads as opaque bytes. The broker serves the payloads as opaque bytes. The broker does not decompress the payloads (`docs/decisions.md`, decision 45). The CRC-32C covers the compressed bytes. A reader checks integrity from the CRC-32C without a decode of the payload. Offsets come from the header (`baseOffset + lastOffsetDelta`). Offset assignment does not need the records. `Batch.Records()` refuses compressed batches with `ErrUnsupportedBatch`. `Batch.Payload()` returns the stored records section for the storage path. `DecodeRecords` serves callers that hold a decompressed payload. The broker rejects a codec id > 4 at Produce time. The broker maps a codec id > 4 to Kafka's UNSUPPORTED_COMPRESSION_TYPE. The deterministic core uses no codec library. Clients compress a batch before the clients send the batch. Clients decompress a batch after the clients receive the batch.

The code exposes other attribute bits. The code does not interpret the other attribute bits. `IsTransactional()` reads bit `0x10`. `IsControl()` reads bit `0x20`. `broker/checkBatch` rejects both bits. The function rejects `ProducerID >= 0` for idempotent producers. The function rejects a non-positive record count. The function rejects `LastOffsetDelta != NumRecords - 1`. The function reads each value from the batch header.

5.7 Fetch patches `baseOffset` and `partitionLeaderEpoch` outside the CRC range

The broker stores a batch in the form that the producer sends. The stored `baseOffset` holds the value that the client writes. Clients write 0 by convention. The log assigns a different offset to the batch.

The CRC-32C covers bytes `21..end`. `baseOffset` at bytes 0-7 sits outside the covered range. The length at bytes 8-11 sits outside the covered range. `partitionLeaderEpoch` at bytes 12-15 sits outside the

covered range. `PatchBatch(b, baseOffset, leaderEpoch)` copies the batch (`append([]byte(nil), b...)`). The copy does not mutate the stored bytes. The function returns an input shorter than 16 bytes as an unchanged copy. The function rewrites bytes 0-7 and bytes 12-15. The rewrite leaves the CRC-32C valid. `ParseBatch` on the patched batch succeeds. The patch also succeeds on a compressed batch. The patch leaves the compressed payload untouched.

In `broker/fetch.go` (`onFetchGet`), the code verifies each ranged object-store read with `kwire.ParseBatch`. The code checks the read against the committed record count. The code appends the read to the response as `kwire.PatchBatch(e.Data, base, 0)`. The `base` value is the committed base offset for the batch from the lookup. The consumer receives the producer bytes with two header fields changed. Kafka rewrites the two header fields on append.

5.8 `format/format.go` puts the index before the payloads and has no footer

One segment object holds batches for many topic-partitions. `broker/segment.go` (`sealBatches`) seals the segment object. `core.CommitSegment` commits the segment object. The segment object has a header and an index. The segment object has no footer. The index sits at the front of the body. Payloads follow the index with no gaps. The segment object ends at the last payload byte. All integers use big-endian byte order.

```
object = header || body

header (20 bytes)
+--------+---------+--------+------------+--------------------+
| "MSEG" | version | flags  | body len   | body CRC-32C       |
|  0..3  | 4..5 u16| 6..7   | 8..15 u64  | 16..19 u32         |
|        |   = 1   | u16 =0 |            | over whole body    |
+--------+---------+--------+------------+--------------------+

body
+----------+---------------------------+----------------------+
| count    | index entries (x count)   | payloads, back to    |
| u32      | variable, see below       | back, in index order |
+----------+---------------------------+----------------------+

index entry (30 + len(topic) bytes)
+---------------+---------+-----------+---------+--------+--------+-----------+
| topic len u16 | topic   | partition | records | off    | len    | batch     |
|               | bytes   | i32       | u32     | u64 *  | u64    | CRC-32C   |
+---------------+---------+-----------+---------+--------+--------+-----------+
  * off is relative to body start on the wire; Entry values returned by
    Encode/Decode are absolute in the object (HeaderSize + off).
```

`Encode(batches)` returns one segment object and one `Entry` per batch. The function returns the `Entry` values in batch order. The encoding is canonical. One batch list has one encoding. The function rejects each case below with `ErrInvalid`.

- an empty list
- an empty topic
- a topic over 65535 bytes
- a negative partition
- an empty payload

The version is 1. The flags are 0. Payload offsets have no gaps. `Encode` writes the body CRC-32C into header bytes 16-19.

`Decode(obj)` validates the whole segment object. The function does not panic. Each failure below matches `ErrCorrupt`.

- truncation
- bad magic
- version != 1
- non-zero flags
- a body-length mismatch
- a body CRC mismatch
- a malformed index
- a non-contiguous payload range
- an out-of-range payload range
- trailing body bytes
- a per-batch CRC mismatch

`format.Code` maps these cases to `core` codes (`ErrCorrupt`, `ErrInvalid`, `OK`).

Three CRC-32C values cover the batch, the entry, and the body

The batch carries Kafka's CRC-32C over the batch records section. The segment body carries one CRC-32C over the index and the payloads. Each index entry carries a CRC-32C of the entry payload. A range read needs the per-entry CRC-32C. A reader fetches one batch with `Entry.Ref(key)`. The ref is a `core.SegmentRef` with a key, an offset, and a length. The reader does not receive the whole body. The reader cannot check the body CRC-32C from a range read. `VerifyBatch(entry, bytes)` checks the fetched length. The function checks the per-entry CRC-32C. The function returns the bytes on success. The broker Fetch path runs `kwire.ParseBatch` on ranged bytes too (see §5.7). `VerifyBatch` is the format-level check. Readers such as `sim/echo/broker.go` use `VerifyBatch`. The readers do not need Kafka-level parsing.

5.9 `format.SegmentKey` names each segment with 32 lowercase hex digits

`SegmentKey(id)` names segment objects. The form follows.

```
seg/<128-bit id as 32 lowercase hex digits>
seg/ab000000000000000000000000000000     <- SegmentKey(core.ID{0xAB})
```

The id is a random 128-bit value. The broker draws the id from `Env.NewID()`. The broker does not draw the id from a request id. The broker draws the key once at seal time in `broker/segment.go`. A retry of one commit reuses the same key. The key stays unique across nodes and boots. The namespace is flat. The spelling is canonical. A key has one `/`. A key has 32 lowercase hex digits. A key has no node component. A key has no incarnation component. `ParseSegmentKey` inverts `SegmentKey`. `ParseSegmentKey` rejects upper-case hex. The function rejects wrong lengths. The function rejects extra slashes. The function rejects the old nested `seg/<node>/<inc>/<id>` spelling. A case-insensitive store cannot alias two segment keys to one name.

5.10 One batch moves from Produce to Fetch through one segment

```
producer --Produce frame--> Framer.Push -> Codec.DecodeRequest
  -> kwire.OneBatch + ParseBatch (magic, CRC, codec, header checks)
  -> batches buffered, sealed: format.Encode(batches)
  -> object PUT under format.SegmentKey(env.NewID())
  -> CommitSegment(entries[i].Ref(key)) records each batch's range

consumer --Fetch frame--> Lookup returns committed ranges
  -> one ranged GET per batch -> kwire.ParseBatch verifies
  -> kwire.PatchBatch(data, committedBaseOffset, 0)
  -> Codec.EncodeResponse -> length-prefixed frame -> consumer
```

The broker does not decompress records. The broker does not re-encode records. The broker does not renumber records. The stored bytes are the producer bytes. Two header fields outside the CRC range change in the Fetch response.

Key takeaways

- `Encode*` returns frames with the length prefix. `Framer` strips the prefix. `Decode*` consumes frames without the prefix. `ErrBadFrame` does not recover. The connection ends after `ErrBadFrame`.
- The broker serves five APIs. The ranges are Produce 3-9, Fetch 4-12, ListOffsets 1-7, Metadata 1-12, and ApiVersions 0-3. An `*UnsupportedError` marks an unknown key or an out-of-range version. `DecodeRequest` returns the header with the error. The broker answers with the header.
- Batches use magic v2 only. CRC-32C covers bytes 21..end. `baseOffset` and `partitionLeaderEpoch` sit outside the covered range. Fetch patches offsets with `PatchBatch` on a copy. The patch leaves the CRC-32C valid for the consumer check.
- The broker stores compressed batches as opaque bytes. The broker serves compressed batches as opaque bytes. The codecs are gzip, snappy, lz4, and zstd with ids 1-4. The broker reads the codec id from the batch header. A codec id above 4 with a valid CRC gives `ErrUnknownCodec`. The broker does not decompress a batch.

- Decoded `kmsg` data aliases the input frame. Framer frames are fresh slices. `ReadFrom` is the one parse entry point. The caller copies data that must outlive the source buffer.
- A segment has a 20-byte `MSEG` header. The body holds an index and then payloads. The body has no footer. The segment has three CRC-32C values. The three values cover the batch, each entry, and the whole body. A whole-object decode checks the body CRC-32C. A single-batch range read checks the per-entry CRC-32C. Segment keys are flat and canonical. The form is `seg/<32 lowercase hex>`.

Checkpoint

1. A client sends Produce v2. What does `DecodeRequest` return, and why not simply fail? — The parsed `core.RequestHeader` plus an `*UnsupportedError` with `UnknownKey: false`. The broker serves Produce. It does not serve Produce at v2. The range is 3-9. The header lets the broker answer or close with the right correlation id. The code does not parse the body.

2. Why can Fetch rewrite a stored batch's base offset without invalidating its CRC? — The v2 CRC-32C covers bytes 21 to the end. `baseOffset` (0-7) and `partitionLeaderEpoch` (12-15) sit before byte 21. They sit outside the covered range. `PatchBatch` copies the batch. It rewrites those fields. `ParseBatch` on the result verifies.

3. `Batch.Records()` errors on a gzip batch, yet the broker stores and serves it. Why is that consistent? — The broker does not need the records. The batch CRC-32C covers the compressed bytes. The CRC-32C gives the integrity check. Offsets come from the header (`baseOffset + lastOffsetDelta`). The broker stores compressed batches as opaque bytes. The broker serves compressed batches as opaque bytes (decision 45). Clients decompress the batches. `Records()` serves uncompressed batches. `DecodeRecords` serves decompressed payloads.

4. The object has a body CRC and each batch a Kafka CRC. Why does every index entry carry a third CRC? — A ranged read of one batch (`Entry.Ref`) does not fetch the whole body. A reader cannot check the body CRC from that read. Format-layer readers do not need Kafka parsing. The per-entry CRC-32C lets `VerifyBatch` check the length from a range GET. The per-entry CRC-32C lets `VerifyBatch` check the CRC from that GET. `VerifyBatch` returns the bytes after the length check and the CRC check pass.

5. You decode a Produce request, keep a record value, then push more bytes through the same `Framer`. Is the value still valid? — Yes. `Framer.Push` returns fresh slices. The framer does not reuse a returned slice. The framer does not mutate a returned slice. Parsed `kmsg` data aliases the frame. The value stays valid while the caller holds the frame. A mutation of the frame changes the value. The caller copies data that must outlive the source buffer. `PatchBatch` copies the batch before the patch.

Try it

```
go test ./kwire/ ./format/
go test -run 'TestPatchBatch|TestOpaqueCompressedBatch' ./kwire/ -v
```

```
go test -run 'TestUnsupported|TestInitProducerIDNotSupported' ./kwire/ -v
go test -run 'TestFlipEveryByte|TestParseRejects' ./format/ -v
```

`TestFlipEveryByte` flips every bit of a segment object. The test requires `ErrCorrupt` for each flipped object. `TestPatchBatch` shows that a patched batch passes `ParseBatch`. Run `go test -run TestCompression ./it/ -v` for the compression test. The compression test is in `it/compression_test.go`.

Further reading

- `kwire/doc.go` states the aliasing rules and the framing rules in canonical form.
- The tests in `kwire/batch_test.go`, `kwire/codec_test.go`, and `kwire/framer_test.go` encode each wire layout as executable checks. The tests in `format/format_test.go` and `format/key_test.go` encode each segment layout and each rejection rule as executable checks.
- `broker/produce.go` (`checkBatch`), `broker/fetch.go` (`onFetchGet`), and `broker/segment.go` (`sealBatches`) are the call sites that consume the two formats.
- `docs/dep-audits/github.com/twmb/franz-go/pkg/kmsg@v1.14.0.md` describes the `kmsg` wrapper.
- `docs/decisions.md` records decision 30 (Fetch patches committed offsets), decision 43 (`seg/<random>` keys), decision 45 (compression as opaque bytes), and decision 41 (no idempotent producers).

Chapter 6: the replica assigns the offset when the replica applies the log

You meet two DynamoDB stores in this chapter. The old store lives in `cp/dynamo`. The log store lives in `cp/logcp` and uses the applier in `cp/applier`. The two stores share the contract in `cp/store.go`, the HTTP layer in `cp/handler.go`, and the run-row format. The two stores share little else.

The old store and the log store solve the same problem in opposite orders. The old store reads the partition end, computes offsets, and defends the computed offsets with a transaction. The log store writes raw commands to a log slot first. The replica then applies the log in slot order, and the applier assigns offsets during that apply step. The log order is the order that matters. A broker uploads a segment object to S3 and asks one store to commit the segment. The store assigns every batch in the segment a contiguous range of offsets in one atomic step, or the store assigns no offsets.

> Old store and log store. Read this first.

Generation	Packages	Status
The old store	`cp/dynamo` with `Store` and `Sequencer`	The old store stays in the tree for rollback. One `TransactWriteItems` handles each commit. A compare-and-set on the run item provides safety. Decisions 39, 42, and 46 record the old design.
The prototype	`cp/logsm`	The prototype has no wiring into any binary. The seeded model proved design A. The prototype applies the log through `veritas.ControlPlane`.
The log store, design A	`cp/logcp` and `cp/applier`, with `cp/applier/pebblekv`	The log store is the current production store. A DynamoDB log holds raw commands. Every node applies the log in slot order through the Pebble applier. Decisions 48-53 and `docs/log-cp-service.md` record the log design.

1. The `cp/` packages split the old store from the log store

```
cp/
  doc.go          package doc: the old DynamoDB design + Sequencer, in full
  store.go        the cp.Store interface and cp.ErrUnavailable
  wire.go         JSON wire types and core <-> wire conversions
  handler.go      cp.NewHandler: POST /v1/{commit-segment,lookup,describe-topics,create-topic}
  memstore/       cp.Store over veritas.ControlPlane behind a mutex (dev/tests)
  dynamo/         OLD: Store (one transaction per commit) + Sequencer (group commit)
    runs.go         packed run encoding, shared with the log CP's indexer
```

```
    fakedb/         in-memory DynamoDB for tests
  logsm/          PROTOTYPE (decision 48): log of commands + Replica + Committer over veritas
  logcp/          NEW: the production log-backed service (this chapter's second half)
    dynlog.go       DynLog: the log as DynamoDB items LOG/<slot>
    committer.go    the commit queue, K flushers, slot allocation and retry
    replica.go      tailing, buffering, hole filling, apply position
    apply.go        applying contiguous slots through the applier
    role.go         advisory active/standby rules
    fresh.go        shared tail checks for fresh reads
    checkpoint.go   snapshots to S3, the CKPT pointer, log trimming
    index.go        background writer of the old R# rows (decision 50)
    service.go      Service: cp.Store + loops + Start/Close lifecycle
    handler.go      role-aware HTTP edge (standbys refuse writes with 503)
    format.go       slot entry codec and versioning (decision 53)
  applier/        NEW: the production state machine on an embedded KV (decision 49)
    pebblekv/       the only Pebble-importing package; NoSync commits
  cpclient/       broker-side HTTP client (sys.ControlPlane) with sticky failover
  cpobs/          Prometheus metrics + per-call logging at the edge (decisions 37-38)
  cptest/         conformance: every store vs veritas, op for op (see §8)
cmd/manicp/       the binary: main.go (mem/dynamo wiring) + logstore.go (dynamolog wiring)
```

The old store and the log store are not deterministic code. Each store uses goroutines, the wall clock, and `net/http`. `scripts/ci-determinism.sh` excludes both stores. `cp/doc.go` records the exclusion. `veritas.ControlPlane` is the reference implementation under decision 21. The reference implementation is a small in-memory spec. `cp/cptest` checks each store against the reference implementation.

2. `cmd/manicp` serves the log store in production and the old store for rollback

The flag `-store` in `cmd/manicp/main.go:parseFlags` accepts three values. The table below lists the three values.

`-store`	Backend	Served by
`mem` as the binary default	`memstore.New()`	`cp.NewHandler(st)`
`dynamo`	`dynamo.Store`, wrapped by `dynamo.Sequencer` unless you pass `-sequencer=false`	`cp.NewHandler(st)`
`dynamolog`	`logcp.Service` with a DynamoDB log, Pebble, and S3 checkpoints	`svc.Handler()`, the role-aware wrapper in `cp/logcp/handler.go`

`openHandler` sends `dynamolog` to `openLog` in `cmd/manicp/logstore.go`. `openHandler` sends the other values to `openStore`, and `openStore` calls `cp.NewHandler`. Two cases matter when you run the binary. The list below states the two cases.

- The binary default is `mem`, and the `mem` store starts empty each time. You do not test either production store when you run `manicp` with no flags.

- Production serves `dynamolog`, and `docs/log-cp-service.md` records the `dynamolog` wiring from stage 4. The Terraform default for the benchmark is `manicp -store dynamolog`. `deploy/aws/terraform/variables.tf` makes `log` the default `cp_store`, and `log` maps to `-store dynamolog`. Terraform documents `dynamo` as the rollback store. The old store uses one host for rollback. Brokers paired with the old store use `-max-segment-partitions 96`.

`openLog` shows the startup order of the log store. `openLog` builds the `DynLog` and opens Pebble in a wiped directory. Pebble state does not carry across a restart. `svc.Start(ctx)` restores the checkpoint and replays the log. `main.go:serve` opens the HTTP port after replay ends. A restoring node refuses connections on the HTTP port. Brokers use another URL from the broker URL list during restore. Section 8 covers the broker behaviour.

3. Both stores keep the same `cp.Store` contract

`cp/store.go` defines four methods. Each method behaves exactly like `veritas.ControlPlane`, op for op.

```
type Store interface {
    CommitSegment(ctx, core.CommitSegment) (core.CommitSegmentResp, core.ErrCode, error)
    Lookup(ctx, core.Lookup) (core.LookupResp, core.ErrCode, error)
    DescribeTopics(ctx, core.DescribeTopics) (core.DescribeTopicsResp, error)
    CreateTopic(ctx, name string, partitions int32) (core.ErrCode, error)
}
```

The contract keeps two error channels apart. A business error is a non-OK `core.ErrCode` with a nil Go error. The handler returns the business error as HTTP 200 with `{"err":"ErrConflict"}`. A Go error covers infrastructure failure. `cp.ErrUnavailable` gives 503, a context error gives 503, and another Go error gives 500. `cp/wire.go` holds the JSON structs and the conversions between `core` types and wire types. The structs include `CommitReq{segment_key, batches[]}` and `Range{topic, partition, base, count, seg_key, seg_off, seg_len}`. `api/openapi/controlplane.yaml` describes the same API. A test keeps the OpenAPI file and `cp/wire.go` in sync.

The broker never proposes an offset to either store. `core.CommitSegment` in `core/controlplane.go` carries only `{SegmentKey, Batches{TP, Records, Loc}}`. The request states where the bytes are. The request does not state where the bytes belong. Decision 42 sets this rule, and the rule makes the retry strategies in this chapter safe.

4. Three rules explain both stores

4.1 The replica assigns offsets at apply time, and no code assigns offsets before apply time

A batch receives `Base` from the end of the partition at the moment the commit applies. The applier reads the end of the newest run for the partition. The applier then adds the records of earlier batches for the same partition in the same request, in request order. No offset counter exists in either store. No high-watermark item exists in either store. No reservation exists in either store. Decision 42 records the rule.

Two results follow from the rule. The list below states the two results.

- A commit that loses a race has assigned nothing, so the broker can retry the commit without creating a hole. The retry burns no offsets.
- Two commits applied in any order produce contiguous offsets for each partition, and the two stores never reuse an offset. Concurrency control only picks an order and makes the chosen order atomic.

Each store picks a different mechanism to pick the order. The old store defends computed offsets with a transaction. The log store writes commands to a slot and lets the slot order become the apply order. You pay for the log order with replay and checkpoint work. You gain a commit path where a lost race costs a new slot and nothing else. That trade favours the log store at production scale, because the old store pays for the same race with fresh reads and a fresh transaction. The old store stays simpler to reason about. The log store stays safer to run with more than one node ready to write.

4.2 One run stores the contiguous batches that one commit assigns to one partition

The batches that one commit assigns to one partition are contiguous. The stores save the batches as one run. A run holds a base offset, a total count, and a packed per-batch list. The packed list holds a format byte, a table of distinct segment keys, and one entry for each batch. Each entry holds a key index, a record count, a byte offset, and a byte length. The code uses varints in `cp/dynamo/runs.go`, mirrored in `cp/applier/keys.go:encodeRun`. The packing costs about 6 B for each batch.

`Lookup{TP, From, MaxBytes}` finds the run that holds `From`. The lookup selects the last run with `base <= From`. The lookup expands that run and the runs after that run into `core.BatchRange` values. The lookup drops batches that sit wholly below `From`. The lookup cuts the answer on `MaxBytes` for each batch. `LookupResp.End` comes from the newest run at `base+cnt`. The end is 0 when no run exists. ListOffsets at latest and Fetch both use the end.

4.3 One segment key identifies one commit in both stores

Decision 3 states the idempotency rule, and `core/controlplane.go` restates the rule. A replay of a committed key with an identical request returns the ranges that the first commit assigned. The same key with a different request returns `ErrConflict`. No other code path uses `ErrConflict`. The broker retry loop depends on the rule. Decision 47 sets the broker retry until `-commit-deadline`, and the default deadline is 120 s.

A commit with a lost answer raises `MaybeApplied` in the broker. The broker resends the same request under the same `seg/<128-bit random id>` key. Decision 43 sets the key form. The resend must never assign offsets twice.

The old store and the log store remember a key for different lengths of time. You can see the difference as a user of either store. The old store keeps the `G#` item forever. The log store keeps the key for a deterministic window of `WindowSlots`, and `WindowSlots` is 131,072 slots. Section 7.4 covers the window. The same key commits again as a new segment after the window, and the records duplicate. The system accepts the duplicate because delivery is at-least-once. Decisions 41, 48, and 53 record the acceptance.

5. The old store computes offsets first and defends the computed offsets with one transaction

Read `cp/doc.go` in full before you touch the old store. The file is the design doc for the old store. The old store uses one DynamoDB table named `<prefix>cp`. The table uses string keys `pk` and `sk` and runs on demand.

5.1 The old store keeps three item kinds in one table

The table below lists the item kinds in the old store.

pk	sk	Attributes	Meaning
`TOPICS`	`<topic>`	`parts`	The partition count. `CreateTopic` and growth raise the count. Neither operation lowers the count, because the merge keeps the maximum.
`G#<segkey>`	`-`	`blob` with JSON that holds the request batches and the full answer	The idempotency key. Presence means the segment has committed. A replay returns the stored `blob`. `maxBlob` bounds the blob at 300 KB, and 300 KB holds about 1000 batches.
`R#<topic>#<p>`	`<base,%020d>`	`base`, `cnt`, and `runs` in packed form	One run for each pair of partition and segment commit. The newest run at `base+cnt` gives the partition end. `maxRun` bounds the run item at 200 KB.

The old store has no node item and no producer state. The old store has no offset counter and no high-watermark item. Decisions 41-44 removed the first three categories. Decision 42 derives the partition end and removes the fourth category.

5.2 The old store reads the ends, computes offsets, and writes one transaction

`cp/dynamo/commit.go` runs the old commit path, and `cp/doc.go` summarises the path. The sequence below gives the steps in order.

1. The old store reads with at most 16 reads in flight. One consistent `BatchGetItem` reads `G#<segkey>` and the `TOPICS` items. One consistent descending `Query Limit 1` reads the newest run for each distinct partition and gives the partition end.
2. The old store checks `G#` first. A present `G#` item replays the stored answer, and a different request returns `ErrConflict`. A malformed request reads `G#` only, and the malformed request can still replay or conflict. The old store returns `ErrInvalid` when the malformed request does neither.
3. The old store assigns bases from the ends that step 1 read, and the old store assigns the bases in request order.

4. The old store writes one `TransactWriteItems` call. The transaction puts `G#` with `attribute_not_exists(pk)`. The transaction puts one `R#` item at the computed base for each partition with `attribute_not_exists(sk)`. The transaction updates each grown `TOPICS` item when the item is absent or `parts < new`.

The put on the run item is the compare-and-set for the old store. Another commit may write to a partition after step 1. An item then sits at the computed base in step 4, and DynamoDB cancels the transaction. The old store re-reads only the partitions that lost. The old store retries with jittered backoff for 20 attempts and then returns `ErrUnavailable` with 503. The broker resends the same key after 503, so the broker loses nothing.

A commit with `P` partitions and no retry costs `P + 2` calls. The 100-item transaction limit in `dynamo.MaxTxItems` gives the refusal rule `1 + P + grown topics ≤ 100`. Brokers paired with the old store must run `-max-segment-partitions 96`. Decisions 32 and 52 record the limit and the broker flag.

The history matters here. The old store computed offsets before the write, so the old store had to defend the computed offsets. A lost race made the computed offsets stale. The old store paid a fresh read and a fresh computation for each lost race. That cost forced the next change.

5.3 The Sequencer groups old-store commits in one process and reads offsets from memory

Decision 46 exists because the compare-and-set in section 5.2 cancelled 387 of 705 transactions in the measured run. Six brokers each ran a separate manicp against shared partitions in that run. `dynamo.Sequencer` wraps `cp.Store` and owns sequencing in one process. `manicp -store dynamo` serves the Sequencer by default, and `-sequencer` defaults to true.

The Sequencer works as follows. The list below states the four parts.

- One loop goroutine takes queued `CommitSegment` requests in order and forms groups. One `TransactWriteItems` call handles many segments in one group. The transaction holds 1 `G#` item for each segment, 1 run item for each distinct partition, and 1 `Update` for each grown topic. The transaction holds at most 100 items and at most 1 MiB of `G#` blobs. The Sequencer adds no artificial wait. Groups form from the requests that arrive while a transaction runs.
- The Sequencer reads offsets from in-memory partition ends. The Sequencer loads each end with the same descending Query on first touch. The Sequencer also holds topic counts in memory. A bounded LRU holds recent segment keys for replay without a read, and the default LRU size is 4096.
- The Sequencer moves retries inside the process. Per-item cancellation reasons tell the Sequencer what to refresh. One cancellation refreshes one partition, one topic, or one `G#` item. An outcome that may have applied drops the cached state for the group and reads every `G#` item first. A 5xx response, a timeout, or a lost answer counts as an outcome that may have applied. The group answers the caller from storage after a lost answer. The group does not assign offsets twice. The Sequencer allows 40 attempts in 8 s and then returns 503.
- Pipelining in `SeqOptions.Pipeline` allows groups on disjoint partitions and topics to run at the same time. The default pipeline depth is 4. A group reserves the partitions and topics for the group. The

> Sequencer never issues a transaction with a base that chains after an in-flight base on the same partition. A failed transaction cannot leave a hole.

Correctness does not depend on one Sequencer. The compare-and-set in section 5.2 stays in place when you run two Sequencers. A second Sequencer costs retries, and a stale Sequencer costs retries. One manicp process is a performance rule for the old store. The log store changes the performance rule, and section 7.5 covers the new form. The constraint is now plain to you. The old store needs one process to own the in-memory ends before the old store can group commits without constant cancellations.

6. The prototype proved the log design before the log store used the design in production

Design A first ran in `cp/logsm`. The `doc.go` file in `cp/logsm` states that no binary and no default path uses the prototype. The prototype has the same basic shape as the log store. The list below states the three parts.

- The log holds raw commands, and `logsm.Log` provides `Append` at a slot if the slot is empty, `Read`, and `CheckTail`. `MemLog` and a DynamoDB `DynLog` implement the log. Each command is a `CommitSegment`, a `CreateTopic`, or a no-op. The log never holds computed bases.
- Each `Replica` applies the log in slot order through a `Machine`, and the default `Machine` is `veritas.ControlPlane`. The replica derives identical offsets, replays, and conflicts on every node. Replica state is a function of the applied prefix of the log.
- A `Committer` appends with up to K appends in flight. The `Committer` answers only after the replica on the same node applies the winning slot through `Await`.

Two ideas carry from the prototype into the log store. Offsets exist only after a replica applies the log. A writer that loses a slot has computed no offsets, so the writer can re-append the same commands later. A hole is a slot that a dead writer allocated and did not fill. The replica fills a hole with a no-op after `HoleAfter`. The log does not block behind one missing slot.

7. The log store records commands first and assigns offsets at apply time

Brokers send every commit to one appender node in the log store. The appender groups commands into slots of a DynamoDB log. The appender writes one conditional `PutItem` for each slot. Every node applies the slots in order through `cp/applier` on Pebble. The appender applies slots, and each hot standby applies slots. A node answers a commit only after the node applies the slot. Decisions 48 and 51 record the flow.

7.1 The `DynLog` slot holds commands and holds no bases

`cp/logcp/dynlog.go` implements the `Log` interface over the same `<prefix>cp` table. The default comes from `-table <table-prefix>cp`. A slot is one item in the table. The table below lists the items that the log store writes.

pk	sk	Attributes	Meaning
`LOG`	`<slot,%020d>`	`d` as B for the entry, `v` for the format version, `w` for the writer, `id` for the append ID, `ts` for writer milliseconds as an advisory value, `n` for the command count, and `ttl`	One slot holds up to 256 grouped commands or one no-op. A no-op serves as a heartbeat or a hole fill. The entry plus 256 B of overhead must fit in `MaxEntryBytes` at 350 KB. A single larger command returns `ErrTooLarge` and never enters the queue.
`CKPT`	`-`	`ck_slot`, `ck_key`, `ck_size`, `ck_sum` as sha-256, and `ck_ts`	The checkpoint pointer. A conditional put moves the pointer forward only. The writer moves the pointer after the S3 object is complete. The object is `ckpt/<slot,%020d>` in the checkpoint bucket.
`IDX`	`-`	`ix_slot` and `ix_ts`	The index watermark. Slots below the watermark have `R#` rows in the table. Section 7.6 covers the indexer. A conditional put moves the watermark forward only, as with `CKPT`.
`TRIM`	`-`	`tr_below`	The point that trimming has reached. A restarted node reads `tr_below` and does not rescan from slot 0.
`TOPICS` and `R#<topic>#<p>`	As in section 5.1	As in section 5.1	The log service does not read these rows. The background indexer writes these rows so the old store can take over on rollback. Section 7.6 covers the indexer.

`cp/logcp/format.go:MarshalEntry` writes the entry itself. The entry holds a version byte, a rank, a writer, an append ID, a timestamp, and a command count. Each command follows in the `cp/logsm` codec with a kind, a key table, and batches. The log holds commands. The log never holds bases.

Versioning follows decision 53. A reader meets a version above `logcp.Version` at some slot. The reader stops with `Blocked`, returns 503, and sets `/healthz` to 503. A reader that cannot decode a payload also stops. The reader does not skip a slot and does not produce different state. An upgrade takes two deploys. All nodes read v+1 first. Writers then switch to v+1.

Trimming needs care from you. Seeded tests found a bug in trimming. Trimmed slots receive one-byte tombstones. The log store does not delete the trimmed slots. `TrimBelow` controls the trim, and `TrimDelete` defaults to true. A lagging node holds a stale slot number and tries to append into a deleted slot. The append would win an `attribute_not_exists` put against a deleted slot and would rewrite history. The same append fails against a tombstone. The node learns `ErrTrimmed` and must restart and restore from the checkpoint. The `ttl` attribute is only a backstop. The default `ttl` is 7 days through `-log-ttl`.

7.2 One if-absent slot write replaces a lease and a fencing epoch

The old store uses `attribute_not_exists(sk)` on the run item. The log store uses `attribute_not_exists(pk)` on the slot item. `DynLog.Append` makes one `PutItem` call with `ReturnValuesOnConditionCheckFailure: ALL_OLD`. The loser learns the winner entry from the failure response. The loser needs no extra read.

You can now see why the log store needs no lease and no fencing epoch. A lease gives one writer the right to write for a period of time. A fencing epoch gives each new lease a higher number, and storage rejects writes from an older number. The slot write needs neither mechanism. DynamoDB checks the absence of the slot at the instant of the write. One slot accepts exactly one entry, and a later writer cannot overwrite the entry in that slot. A stale appender cannot claim a slot that another appender already filled. A stale appender cannot create a second entry for the same slot. Safety rests on the conditional write for one slot. Safety does not rest on a belief about which node holds a lease.

Two nodes may both believe that each node is the appender. The two nodes race on the same slot, and one node wins. The loser re-appends the same entry at a later slot. No offset came from a local view in either node. A lost slot costs no offset work. Decision 51 records that no lease is needed for safety.

Rank, heartbeats, and takeover timers make one node the usual appender. Sticky single-appender routing keeps the log at one `PutItem` for each commit. The routing avoids the contention in decision 46. Decision 48 and experiment X4 record the routing. The role machinery saves writes. The role machinery does not carry safety.

7.3 The appender answers the caller only after the replica applies the slot

`cp/logcp/committer.go` and `service.go` hold the appender path. The list below states the four parts.

- `submit` admits the command. `submit` refuses with 503-class errors when the service has stopped, the service is `Blocked`, the node is not active, or the queue is full. `QueueMax` limits the queue to 4096 commands and 64 MiB. `submit` encodes the command, checks the single-command size guard, and queues the command in FIFO order.
- K flusher goroutines take everything in the queue as one slot entry. The flag `-log-k` sets K, and the default is 4 appends in flight. Each flusher takes at most 256 commands for one slot. Each flusher reserves a slot-rate token before the flusher writes. The flag `-slot-rate` sets the rate, and the default is 200 slots per second. The rate paces writes to the one hot `LOG` partition key. A lone commit goes at once. Groups form from load, as groups form in the Sequencer.
- `appendGroup` allocates the next slot from `max(high, applied)`. `appendGroup` appends the entry, and a lost slot shows the winner entry to the loser. The loser retries the same entry at a later slot. An unknown outcome retries the same slot until the condition answer reveals the owner of the slot. `appendGroup` matches the owner by append ID.
- The appender does not answer the caller. The replica applies the slot, and the answer arrives at that moment. `replica.deliverLocked` matches the entry ID to the waiters for the entry. `replica.deliverLocked` hands the waiters the `core.CPDone` values from the applier in `cp/logcp/apply.go`. A commit ack means the slot is in the log and the replica has applied the slot. The node that applies the slot sends the answer.

7.4 The replica assigns offsets when the replica applies slots in order

Every node runs the same replica loop in `replica.go`, `apply.go`, and `service.go:runLoop`. The replica reads the log with consistent `Read` calls. The replica buffers out-of-order slots above a hole. The replica fills a hole with a no-op after the hole stays empty for `HoleAfter`. `HoleAfter` is 250 ms, and later slots must wait behind the hole before the replica fills the hole. The replica applies contiguous slots in batches through `applier.CommitBatch`. A batch holds at most 256 slots. A batch does not cross a `TrimStride` boundary. Each batch uses one Pebble transaction. Each command sees writes from commands before the command in the same batch.

`cp/applier` is the production state machine under decision 49. The applier has the same semantics as `veritas`. The applier uses an ordered KV in place of in-memory maps. Veritas keeps every run without limit. Veritas grows by about 6 GB each hour at the target rate.

The layout in `applier/doc.go` and `keys.go` uses `tp` for one topic-partition key. The key `tp` is a uvarint topic length, the topic, and a 4-byte big-endian partition. The table below lists the applier keys.

Key	Value	Meaning
`T <name>`	The partition count	The topic registry. Growth keeps the maximum count.
`E <tp>`	The end offset as a uvarint	The next offset for the partition. The value stores the end of the newest run.
`R <tp> <base,8BE>`	A packed run	One run for each pair of partition and segment commit. The packing matches section 4.2.
`K <segment-key>`	The slot, a request digest with 16 B of SHA-256 over the batches, and the assigned bases	The idempotency record. A digest match replays the stored bases. A digest mismatch returns `ErrConflict`.
`S <slot,8BE> <segment-key>`	A marker	The window index. `Trim` uses the index to find and drop the `K` records of old slots.

`assign` in `applier.go` implements section 4.1. For each batch, `Base` is the current `E` for the partition. `assign` reads `E` once for each partition in each transaction. `assign` then adds `Records` to `E`. `assign` writes one `R` record for each partition and grows topics. `commitTx` checks `K` before `assign` runs. The first slot wins. A replay rebuilds the ranges from the replayed request and the stored bases. A replay changes no offset.

Replicas cut the idempotency window by slot number. The apply batch starts at a multiple of `TrimStride`, and `TrimStride` is 1024. The batch start is at or past `WindowSlots`, and `WindowSlots` is 131,072. `Trim(start - WindowSlots)` then runs before the batch. The cut is a function of the slot number. Every replica forgets the same keys at the same point of the log.

A clock-based cut would give you a different system. Replicas that trimmed keys 2× `CommitDeadline` ago by separate clocks would answer the same replay in different ways. The replicas would produce different state. Finding 1 in `docs/log-cp-service.md` and decision 53 record the clock problem. The window lasts at least 655 s at the 200 slots per second ceiling. The window lasts at least 2× the broker commit deadline plus a checkpoint interval. A broker retry arrives inside the window in the normal case.

7.5 Roles choose the usual appender, and the slot write keeps a wrong role safe

Roles are advisory in `cp/logcp/role.go`. The slot check in section 7.2 provides safety. A wrong role belief costs retries. A wrong role belief does not break correctness. The rules work as follows. The list below states the four rules.

- Rank 0 starts active, and higher ranks start as standbys. A standby reads and applies slots as the slots arrive. Pebble on the standby holds current state. The standby serves fresh reads. The standby refuses `commit-segment` and `create-topic` at the HTTP edge with `503`, `Retry-After: 1`, and `X-CP-Role: standby` in `cp/logcp/handler.go`. The handler refuses the request before the handler reads the body.
- The active node appends a no-op heartbeat slot after an idle period of `Heartbeat`. `Heartbeat` is 1 s. The heartbeat makes the liveness of the appender visible in the log itself.
- A standby promotes the standby node after the standby sees no slot from another writer for `TakeoverAfter`. You set `TakeoverAfter` with `manicp -takeover-after`, and the default is 5 s. You keep `TakeoverAfter` at several times `-heartbeat`. You keep `TakeoverAfter` far below the 120 s broker commit deadline. Reads on the standby must keep succeeding during the period. A node with failing reads does not promote. The node cannot tell node death from a network partition.
- An active node steps down when a writer of lower rank appends at or after the slot where the active node promoted. The lower-rank writer is the preferred node back in service. Tests cover split brain. Both nodes append in the split-brain test, and the if-absent slot picks one entry for each slot. The states stay identical.

Standbys serve fresh reads before any failover. `Lookup`, `DescribeTopics`, and `End` call `fresh()` in `fresh.go`. One consistent descending `CheckTail` with `Limit 1` covers every read that arrives inside a `TailBatch` window. `TailBatch` is 10 ms. The node waits until `applied >= tail`. The node then answers from Pebble. Any node can serve a read that reflects prior writes. Standbys can scale reads in the log store.

7.6 Checkpoints shorten replay, and the indexer keeps rollback to the old store open

The log is the record that decides state in the log store. Pebble holds a copy that the node rebuilds from the log. `pebblekv` commits with `pebble.NoSync`. `cmd/manicp` wipes the Pebble directory at every start. Checkpoints in `checkpoint.go` make restart faster. The sequence below gives the steps in order.

1. The active node snapshots the applier at a consistent slot every `CkptEvery`. `CkptEvery` is 5 min, and the node needs at least `CkptMinSlots` at 1000 new slots. `SnapshotAt` takes the store view and reads `applied` inside the view. `SnapshotAt` streams the `APLR1` format with a magic value, a varint slot, and length-prefixed key and value entries in key order. The stream includes the `K` and `S` window records. Replays of recent keys still answer after a restart.
2. The node writes the object to S3 as `ckpt/<slot>` through the injected `sys.ObjectStore`. `logstore.go` injects `sys/s3store`, and `-ckpt-bucket` names the bucket. An empty bucket name means no checkpoints and no trim. The node then moves the `CKPT` pointer item with a forward-only conditional put. Readers see the old checkpoint or the new checkpoint. Readers never see a partial checkpoint. The node keeps the pointed object and one older object.
3. The node trims the log below `min(ckpt - KeepSlots, index watermark)`. `KeepSlots` defaults to `WindowSlots`. The node writes tombstones as section 7.1 describes.

4. A starting node reads the `CKPT` pointer in `Start`. The node gets the S3 object and verifies the SHA-256 sum. The node runs `applier.Restore`, calls `startAt(slot)`, and replays the log to the tail. The node serves after replay ends. A damaged object falls back to replay from slot 0 when the log still starts at slot 0. A damaged object returns `ErrNoCheckpoint` when the log no longer starts at slot 0. A graceful shutdown writes a final checkpoint first. `-final-checkpoint` controls the final checkpoint, and `logstore.go:stopLog` allows a 20 s budget.

The indexer in `index.go` writes the `R#` and `TOPICS` rows of the old store in the background. Decision 50 records the indexer. The active node runs the indexer every `IndexEvery`, and `IndexEvery` is 10 s. The indexer moves `IDX.ix_slot` after the indexer writes the rows. The indexer plan is a pure function of the log plus `applier.Assigned`. Rows from the indexer match the bytes that the old store would have written through `dynamo.RunRow`. The indexer writes rows for each partition in offset order. The table holds a gap-free prefix of rows.

No part of the log service reads the indexer rows. The log store answers `Lookup` from Pebble. The rows support rollback to `-store dynamo`. You quiesce commits and wait for `manicp_log_index_lag_slots` = 0. You then switch stores. The log store does not write `G#` keys. A key from before rollback replays as a new commit after rollback. At-least-once delivery permits the duplicate records. Decision 41 records the delivery rule.

8. The broker retries with one segment key across the old store and the log store

`cpclient.New(baseURL)` accepts a comma-separated URL list under decision 51. Calls go to the URL that last worked. The client stores the working URL in `cur` as an atomic index. A connection failure triggers failover to the next URL inside the same call. A 503 response also triggers failover. A standby node, a starting node, or a blocked node returns 503. The client tries each URL at most once in `shouldFailover`.

Failover is safe for commits because section 4.3 gives one segment key to one commit. The retry carries the same segment key to the next node. Business errors come back as `*cpclient.Error`. An HTTP 200 response with `err` carries a business error. Transport failures come back as `*CallError{Op, Sent, Status}`. `Sent` records whether the client obtained a connection. The broker uses `Sent` to classify `MaybeApplied` and to decide whether a resend could duplicate records. The broker cannot duplicate offsets in either case. `Sent` drives error classification, and `Sent` does not drive safety. Deadlines come from the context of each call. The broker commit-deadline loop under decision 47 sits above the client.

`cp/memstore` supports development and tests. The `mem` store puts `veritas.ControlPlane` behind a mutex. `cp.NewHandler` serves the `mem` store. A behaviour differs between `mem` and another store in a test. The other store has the bug in that case. `cp/cptest` enforces the rule. Conformance tests in `conform_test.go` run every store through the same op sequences against veritas. The test set covers memstore, dynamo on `fakedb`, the `logsm` store, and the log store over HTTP. The test set includes the seeded X2 tests for `logcp` with crashes, takeovers, split brain, and trim. `docs/log-cp-service.md` describes the X2 tests.

9. The diagrams show commit and failover in slot order

9.1 `CommitSegment` on the log store follows the current path

```
sequenceDiagram
    participant B as Broker (cpclient)
    participant H as manicp active (handler)
    participant Q as Commit queue + K flushers
    participant D as DynamoDB LOG/<slot>
    participant R as Replica apply loop
    participant A as Applier (Pebble)
    B->>H: POST /v1/commit-segment {segment_key, batches}
    H->>Q: submit: admit (active? queue full?) + size guard
    Q->>Q: group queued commands (<=256), reserve slot-rate token
    Q->>D: PutItem LOG/<n> if attribute_not_exists(pk)
    alt slot won
        D-->>Q: ok
    else slot lost (another writer / hole filler)
        D-->>Q: ConditionalCheckFailed + winner entry (ALL_OLD)
        Q->>D: PutItem same entry at LOG/<n+1> ... (no offsets computed yet)
    end
    D-->>R: tail Read / observed append: slot n now contiguous
    R->>A: CommitBatch: check K<segkey> -> assign bases from E<tp>, write R, E, K, S
    A-->>R: assigned ranges per command
    R-->>H: deliver by entry ID: applied >= n
    H-->>B: 200 {assigned: [{base, count, ...}]}
    Note over B: Retry with same key at any point -> K record replays original ranges; different request
        -> ErrConflict
```

The old store replaces the middle steps in the diagram. The old store reads the partition ends. The old store runs one `TransactWriteItems` call with the `G#` put, the run compare-and-set, and the topic updates. DynamoDB cancels the transaction in a lost race. The old store re-reads the losing partitions and retries. Section 5.2 gives the old sequence. The broker contract stays the same across the old store and the log store.

9.2 A standby takes over after the active node dies

```
sequenceDiagram
    participant B as Broker (cpclient URL list a,b)
    participant P as CP rank 0 (active)
    participant S as CP rank 1 (standby)
    participant D as DynamoDB LOG
    Note over S: tailing + applying continuously (warm Pebble); reads served fresh
    P->>D: heartbeat / commit slots ... then P dies
    B->>P: POST /v1/commit-segment
```

```
    P--xB: connection refused / 503
    B->>S: same call, same segment key (failover within the call)
    S-->>B: 503 X-CP-Role: standby (until promotion)
    Note over S: no slot of another writer for TakeoverAfter (5s), own reads OK -> promote
    B->>S: retry (broker commit-deadline loop, same key)
    S->>D: PutItem LOG/<tail> if-absent (wins; or loses to a late P slot and re-appends later)
    S->>S: apply slot through applier
    S-->>B: 200 {assigned}
    Note over P,S: If P revives and appends: both race per slot (if-absent arbitrates);<br/>S steps down
        only when a lower-rank writer appends after its promotion (failback).
```

Entries from the dead node landed in the log before the node died. The node that applies an entry sends the answer for the entry. Entries from the dead node did not land in the log. Entries that did not land received no offsets. The broker resends an entry that did not land under the same key. The resend commits as a new entry.

A standby discovers that trimming passed the standby position. The standby receives `ErrTrimmed` and stops with 503. A standby meets an unknown slot version. The standby enters `Blocked` and stops with 503. The standby does not guess in either case. A restart restores the standby from the newest checkpoint.

Key takeaways

The list below gives the points to keep from this chapter.

- The old store and the log store assign offsets at commit-apply time from the newest run for the partition. The broker never proposes a base. Decision 42 records the rule. A lost race costs a retry in either store, and a lost race costs no offset.
- Idempotency is per segment key in the old store and the log store. An identical replay returns the original ranges. A different request returns `ErrConflict`. The old store remembers keys without limit in `G#`. The log store remembers keys for a deterministic `WindowSlots` window.
- A run is the storage unit in the old store, the log store, and the applier. One run holds the contiguous batches that one commit assigns to one partition in packed form.
- The old store uses `attribute_not_exists` on the run item inside one transaction for safety. The Sequencer uses the same compare-and-set under one sequencing loop. The log store uses `attribute_not_exists` on the log slot for safety. Exactly one writer is a performance rule in the three designs, and exactly one writer is never a correctness rule.
- `manicp` defaults to `mem`, and Terraform serves `-store dynamolog` in production. The log store uses one appender, and hot standbys read the same log. Nodes rebuild Pebble state from S3 checkpoints and log replay.
- `cp/logsm` is the prototype, `veritas` is the reference implementation, and `cp/applier` is the production state machine. `cp/cptest` checks each store against `veritas`.

Checkpoint

1. A broker resends a `CommitSegment` after a timeout and gets different offsets than the first attempt would have given. Which invariant broke, and where is it enforced in the log store? — Idempotency per segment key in section 4.3 broke. The first attempt never applied in one case. The resend is then the first commit, and any offsets from the resend are correct. The first attempt applied in the other case. The `K <segment-key>` record in `cp/applier` must replay the stored bases in the second case. `commitTx` checks `K` before the applier assigns offsets. Different offsets mean the log store forgot the key before the resend. Section 7.4 covers the window cut. You check `WindowSlots` against the broker `-commit-deadline`. The resend may differ from the first request in a third case. A digest mismatch gives `ErrConflict` in the third case, and a digest mismatch gives no new offsets.

2. Why can the log store's appender lose a slot race and re-append the same entry later, while the old store had to re-read partition ends before retrying? — The old store computed bases from the partition ends that the old store read. The old store defended the computed bases with the run-item compare-and-set. A lost race meant the computed bases were stale. The old store re-read the ends and recomputed the bases. Section 5.2 gives the old sequence. The log entry contains no bases. A replica derives offsets only when the replica applies the log in slot order. Sections 7.3 and 7.4 give the log sequence. A lost slot changes only the position of the commands in the order. Any order yields contiguous offsets under section 4.1.

3. Two `manicp -store dynamolog` nodes both think they are active for a minute. What breaks, and what does not? — Correctness holds for the log store. Both nodes race for each slot on `attribute_not_exists(pk)`. The loser observes the winner entry and re-appends the entry later. All replicas apply the same slot sequence into identical Pebble state. Sections 7.2 and 7.5 cover the race. Performance degrades, and latency degrades. The costs are extra lost slots, extra retries, and extra role changes when the lower-rank node appends again. Roles, ranks, and heartbeats make the race rare. Roles, ranks, and heartbeats do not make the race safe.

4. A standby is restarted after being down for a day. Walk through what it needs before it may serve, and two ways that can fail permanently. — `Start` in `cmd/manicp/logstore.go:openLog` wipes Pebble and reads the `CKPT` pointer. The node gets the S3 object and verifies the SHA-256 sum. The node restores the object with the window records included. The node replays the log from the checkpoint slot to the tail. The HTTP port opens only after replay ends. Sections 2 and 7.6 give the startup sequence. Two failures can be permanent. The checkpoint object is unusable and the log no longer starts at slot 0, so the node returns `ErrNoCheckpoint`. The node meets a slot in a newer format version, or the node meets a tombstone where the node expected history and receives `ErrTrimmed`. Both failures stop the node with 503. The node does not skip a slot and does not produce different state. Sections 7.1 and 7.6 cover the two failures.

5. You must roll production back from `dynamolog` to `dynamo`. What do you wait for, what flag must the brokers get, and what quietly changes about replays? — You quiesce commits and wait until `manicp_log_index_lag_slots` = 0. The background indexer has then written every `R#` row that the old store will read. Section 7.6 covers the indexer. You stop the log nodes and start the old manicp. You give brokers `-max-segment-partitions 96` for the 100-item transaction rule in the old store. Section 5.2 gives the transaction rule. One change is easy for you to miss. The log store never wrote `G#` items. A replay of a segment key from before rollback commits again as a new segment after rollback. The replay duplicates

the records from the segment. At-least-once delivery permits the duplicate records under decision 41. Operators must accept the duplicate records before rollback, and `docs/log-cp-service.md` records the acceptance.

Try it

```
# Terminal 1: the current production store, fully local
# (needs DynamoDB + S3 stand-ins: scripts/it-up.sh starts ExtendDB + SeaweedFS)
go run ./cmd/manicp -store dynamolog -create-tables \
  -dynamo-endpoint http://127.0.0.1:18080 \
  -ckpt-bucket test -s3-endpoint http://127.0.0.1:8333 -s3-path-style

# Terminal 2: the old store, for contrast (note: exactly one, sequencer on)
go run ./cmd/manicp -store dynamo -create-tables \
  -dynamo-endpoint http://127.0.0.1:18080 -addr :8081

# Either one: create a topic, commit the same segment twice, compare.
curl -s localhost:8080/v1/create-topic -d '{"topic":"t","partitions":3}'
curl -s localhost:8080/v1/commit-segment -d '{"segment_key":"seg/0123456789abcdef0123456789abcdef",
  "batches":[{"topic":"t","partition":0,"records":10,"seg_key":"seg/
        0123456789abcdef0123456789abcdef","seg_off":0,"seg_len":100}]}'
# …repeat the commit verbatim (same ranges back), then change "records" (ErrConflict).
go test ./cp/cptest/ -run TestConformance -count=1   # every store vs veritas
```

You see the difference with `dynamolog` when you add a second node. You give the second node `-rank 1`, a different `-addr`, and a different `-node-name`. You commit against the second node and expect `503` with `X-CP-Role: standby`. You kill the rank-0 node. You time the delay until the standby promotes and the retry with the same key succeeds.

Further reading

The list below gives the files to read next.

- `cp/doc.go` covers the old DynamoDB design and the Sequencer. You read `cp/doc.go` to understand section 5.
- `docs/log-cp-service.md` is the reference for the log store. The file lists flags, gives a failure-case table, and reports verification results. The file covers the two findings that changed the design. The findings are the deterministic window and tombstone trim.
- `docs/decisions.md` rows 42-53 cover offsets from runs, no fencing, the Sequencer, design A, the applier, hot standby, and versioning.
- `core/controlplane.go` defines the contract that the old store and the log store implement.
- `docs/design-a-prototype.md` explains the choice of a log from experiments X1-X4. `docs/applier-bakeoff.md` explains the choice of Pebble over bbolt. The two files give the evidence behind decisions 48-49.

Chapter 7. One goroutine owns the broker state in production

Chapters 1 to 6 stay inside the deterministic world. `core/` defines Events and Actions as plain data. `broker/` is a pure state machine. `sim/` schedules every completion on a simulated clock. None of that code opens a socket, reads a wall clock, or starts a goroutine. This chapter explains the one layer that does real input and output. That layer is `sys/` and `cmd/manibroker/`.

You keep the machine pure for one reason. One goroutine owns the broker state, so the machine sees one event order and needs no lock for that state. The loop goroutine calls `Step`, and no other goroutine calls `Step`. The cost is strict. The loop goroutine must never wait for a disk, a network, or S3. Real input and output belongs in `sys/prod` adapters behind the `sys` interfaces. The executor does that work on other goroutines, and the work returns as one completion in the inbox.

7.1 The `sys/` layer holds every real input and output

`sys/sys.go` opens with a description of the layer. The layer gives you narrow interfaces over real input and output. The production Executor calls the interfaces from goroutines. SDK types and HTTP types never pass an adapter. The simulator does not implement the interfaces. The simulator implements `sim.Engine` over the same Action and Event vocabulary.

Path	Role
`sys/sys.go`	Interfaces. `Clock`, `ObjectStore`, `ControlPlane`, `Conns`, `Classifier`, `Sink`, `Executor`, and the `Bundle` carrying them
`sys/run.go`	`sys.Run`, the single loop goroutine that owns one machine incarnation
`sys/inbox.go`	`Inbox`, the two queues (control, data) behind `Sink`
`sys/signal.go`	`ShutdownOnSignal` turns SIGTERM/SIGINT into a closed channel
`sys/prod/`	Production runtime. `Executor`, `Net` (TCP), `Clock`, `Env`, `Metrics`, `SegmentLog`, memory sizing
`sys/s3store/`	`sys.ObjectStore` over AWS SDK v2 S3, plus its `Classifier`
`sys/httppool/`	HTTP pool sizing shared by the S3 and CP clients
`sys/awsobs/`	Smithy middleware that measures every AWS SDK HTTP attempt
`sys/runinfo/`	Run identity. Logger, `build_info`, run manifest, `/metrics` + `/healthz` mux
`cmd/manibroker/`	The broker binary. Flags (`main.go`), tuning (`tuning.go`), size parsing (`units.go`)

The package doc for `sys/prod/` calls the package “nondeterministic by design and is not in the determinism gate.” The gate protects `core/`, `broker/`, and `sim/`. This chapter covers the code outside that gate.

7.2 One loop goroutine owns the machine in `sys.Run`

`sys.Run(ctx, b Bundle, exec Executor, f core.Factory, self core.NodeRef)` in `sys/run.go` drives the machine in production. The simulator kernel drives the same machine in tests. Two drivers run one machine, and the drivers differ only outside the machine.

The single owner buys you a plain order of events. The loop goroutine dequeues one event, stamps the event, and calls `Step` once. The executor never calls `Step`, and a connection reader never calls `Step`. You can compare the prod event order with a sim event order because both drivers use `core.ControlFirst`. The cost sits in `sys.Run`. The loop goroutine must keep moving, so `sys.Run` gives every blocking call to the executor or to a timer.

`sys.Run` works in this order:

1. The loop goroutine refuses a nil `Bundle.Inbox`. A private inbox stays unreachable for the Executor and for connection readers. Completions vanish in a private inbox.
2. The loop goroutine builds the machine with `f(b.Env, self)`. The loop goroutine delivers the first event with `step(core.Started{Self: self})`.
3. The loop goroutine checks `ctx.Err()` on each turn. The loop goroutine also checks the graceful-stop kill deadline before the loop goroutine picks work. The code states the reason in plain words. A busy inbox cannot starve cancel, and a busy inbox cannot starve the deadline. `select` never decides priority. The pure `core.ControlFirst` policy in `core/policy.go` picks the control head if the control queue is non-empty. The policy picks the data head if the control queue is empty. The loop goroutine sleeps on the inbox wake channel, `ctx.Done()`, shutdown, or the kill timer if both queues are empty.
4. The loop goroutine stamps the event at dequeue and only at dequeue. The stamp is `core.Event{At: b.Clock.Mono(), Wall: b.Clock.Wall(), Body: body}`. `sys.Clock` is “read only by the runtime, at inbox dequeue, to stamp Event.At/Wall.” A fact duration is a difference of event times. That rule makes metrics identical in sim and prod (§7.6).
5. `m.Step(event)` returns `[]core.Action`. `runner.dispatch` dispatches the actions in slice order.

You read the return value as the end of one incarnation:

`sys.Run` returns	Meaning
`nil`	Machine emitted `core.Exit` (graceful stop completed)
`ErrTripped` (wrapped)	Machine emitted `core.Trip`. Its memory can no longer be trusted. The supervisor should restart the process
`ErrStopped`	`ctx` cancelled (`Node.Kill`)
`ErrKilled`	`Bundle.Shutdown` fired, `StopGrace` elapsed, machine still had not emitted `Exit`

On every return, `Run` closes the inbox with `defer in.Close()`. That close is the incarnation boundary. Late completions drop at the close. Late completions never block. Late completions never reach the next incarnation.

`dispatch` keeps non-input and non-output actions on the loop goroutine

`runner.dispatch` in `sys/run.go` handles non-I/O actions in the loop goroutine. `runner.dispatch` gives each `core.IOAction` to the Executor.

Action	Runtime behaviour
`core.IOAction` (Send, Dial, Obj*, CPCall)	`exec.Execute(a)` (§7.3)
`core.Emit`	`b.Facts(a.Fact)` if a facts sink is wired (§7.6)
`core.SetTimer`	`time.AfterFunc(a.After, …)` → `DeliverControl(core.TimerFired{ID, Gen})`. Replacing stops the old timer. The machine ignores stale generations
`core.CancelTimer`	Stops and deletes the `time.Timer`
`core.CloseConn`, `PauseReads`, and `ResumeReads`	Called directly on `b.Conns` (in prod, `*prod.Net`)
`core.Trip`	Returns `ErrTripped`. Actions before it in the slice were already started
`core.Exit`	Returns `nil`
anything else	`panic("sys: unknown action …")`. This is a programming error, not data

7.3 The executor moves real input and output off the loop goroutine

The `sys.Executor` contract in `sys/sys.go` has three rules. The executor “runs I/O only.” The executor gives you “exactly one Completion per IOAction, with the same ReqID, delivered to the Sink and never on the caller’s goroutine.” “Execute MUST NOT block: it hands work to goroutines/queues.” In this chapter, the executor means `sys/prod.Executor` in `sys/prod/executor.go`. The executor implements the contract in this way:

Action	Becomes	Completion
`core.ObjPut`, `ObjGet`, `ObjDelete`, `ObjList`	`go func() { e.deliver(e.object(a)) }()`. One goroutine per action. `context.WithTimeout` of `ObjTimeout + len/ObjTransferRate`. Then `objs.Put/Get/Delete/List`	`ObjPutDone` / `ObjGetDone` / `ObjDeleteDone` / `ObjListDone{Req: a.Req, …}`
`core.CPCall`	`go func() { e.deliver(e.control(a)) }()`. `cp.Call(ctx, a.Op)` under `CPTimeout` (default 10 s)	`CPDone{Req, Resp, Outcome}`
`core.Send`	Not a fresh goroutine. `conns.WriteAsync(conn, data, done)` queues onto that connection’s single ordered writer goroutine (§7.5). Only if queueing fails is `done` run via `go done(err)`	`SendDone{Req, Conn, Err}` from the writer goroutine, once bytes left

Action	Becomes	Completion
`core.Dial`	Goroutine delivering `DialDone{Err: core.ErrInvalid}` immediately. “brokers reach the CP through CPCall,” so prod has no raw dials. Dial exists for sim workloads	`DialDone`

Errors become data before the core sees the errors. On failure the executor calls `e.class.Classify(a, err)`. The call returns plain `(core.ErrCode, core.Outcome)` values. `prod.Classifier` in `sys/prod/classify.go` routes by action type. The Classifier sends CPCall to the CP classifier. The Classifier sends each object action to the object classifier. `sys/s3store/classify.go` maps SlowDown, 503, and 429 to `ErrUnavailable` + `NotApplied`. The Classifier maps a PUT timeout to `ErrTimeout` + `MaybeApplied`. The PUT may have landed before the timeout. The Classifier maps a GET failure to `NotApplied` in all cases. “Getting Outcome right here is what makes acks honest.” The broker may ack a produce only after a commit with an outcome of `Applied`.

`Executor.deliver` checks `sink.Closed()` first. `Executor.deliver` drops the completion if the sink has closed. A goroutine that finishes after `sys.Run` returns cannot block on a dead inbox.

The object-store timeout takes size into account. The formula is `objTimeout(n) = ObjTimeout + n × (1s / ObjTransferRate)`. The defaults are 30 s and 2 MiB/s. A 16 MiB segment PUT gets about 38 s. A flat 30 s limit does not end a 16 MiB PUT on a slow link at 30 s.

7.4 Control completions run before new data in the two-queue inbox

`sys.Inbox` in `sys/inbox.go` gives the node two queues. `core.ClassOf` in `core/inbox.go` classifies each item.

Control covers everything the node caused itself. The control items are all `*Done` completions, `TimerFired`, `Started`, and `Stop`. `DeliverControl` never blocks. The queue is an unbounded slice. The slice is “bounded by construction: the machine caps in-flight work.” The queue has nothing to drop.

Data covers `ConnOpened`, `BytesReceived`, and `ConnClosed` from connection readers. `DeliverData` first takes a slot from a semaphore channel of capacity `dataCap`. The prod default is `DataInbox = 1024`. `DeliverData` blocks when the semaphore has no slot. `ConnClosed` is Data. The classification means a client end of file can never pass the last request from that client.

`ControlFirst` drains control before data. Under a produce flood, completions free buffer space and generate acks. Completions run before the loop goroutine admits new bytes. Prod and sim run the same `core.InboxPolicy` code. `core/inbox.go` states the rule in one sentence. “Go’s select must never decide priority.” The flow-control decision sits in `docs/flow-control.md` §3.

Backpressure reaches the client in three steps

Three mechanisms apply in sequence:

1. The broker sets the limit. At `MaxBufferedBytes` of unacked produce bytes the broker machine emits `backpressure_on`. The broker machine also emits a `PauseReads` action for every open connection.

The broker machine resumes at `ResumeBufferedBytes`. The default resume value is half of the max value. This rule sits in `broker/broker.go` and `docs/flow-control.md` §2.

2. The network applies the pause. `prod.Net.PauseReads` parks that connection reader goroutine on a `resume` channel in `sys/prod/net.go`. The socket buffer fills after the reader parks. TCP window pressure then reaches the client. The loop goroutine still processes bytes that the reader received before the pause.
3. The inbox limits deliveries. Readers can still outrun the loop goroutine after the pause. `DeliverData` then blocks on the 1024-slot semaphore. The `sys.Sink` doc calls the semaphore a safety net and states that real backpressure is PauseReads.

The buffer cap is `clamp(memory/16, 128MiB, 1GiB)`

The `-max-buffered` flag help says `auto = clamp(memory/16, 128MiB, 1GiB)`. `prod.AutoBuffered` in `sys/prod/memory.go` implements that rule.

```
func AutoBuffered(limit uint64) (max, resume int) {
    m := uint64(MinBufferedBytes) // 128 << 20
    if limit != 0 {
        m = min(max64(limit/16, MinBufferedBytes), MaxBufferedCap) // 1 << 30
    }
    return int(m), int(m / 2)
}
```

Unknown memory gives 128 MiB. A known limit gives `limit/16` clamped into [128 MiB, 1 GiB]. Resume is half of max. A 1 GiB container gives 128 MiB at the floor. An 8 GiB container gives 512 MiB. A 64 GiB container gives 1 GiB at the cap. The comment gives the reason for the 1/16 share. The broker holds about 2-3 copies of buffered record data. The copies are the request frame, the admitted batch, and the encoded segment. A 1/16 share of RAM keeps the total use near a fifth of memory.

`prod.MemoryLimit()` finds `limit`. `prod.MemoryLimit()` prefers the container limit over the host total. `prod.MemoryLimit()` checks cgroup v2 at `/sys/fs/cgroup/memory.max`. `prod.MemoryLimit()` checks cgroup v1 at `memory.limit_in_bytes`. `prod.MemoryLimit()` checks `/proc/meminfo` `MemTotal`. `prod.MemoryLimit()` then uses the OS fallback. The fallback is `sysctl hw.memsize` on Darwin. Other systems report “unknown.” The code rejects cgroup `max` as no limit. The code also rejects the ~2^{63} “unlimited” values as no limit. `cmd/manibroker/tuning.go` wires the result into the broker config. `auto` calls `AutoBuffered(mem)`. An explicit size overrides the auto value. Resume is half of an explicit size. An explicit `-resume-buffered` must be positive and ≤ max.

7.5 Real adapters sit under the `sys` interfaces

One reader and one writer serve each TCP connection

One accept goroutine delivers `ConnOpened` to the data inbox. The accept goroutine then starts one reader goroutine and one writer goroutine for each connection. Readers read 64 KiB chunks. Readers call `DeliverData(BytesReceived)`. Writers drain a queue for one connection in call order. Each write has a 30 s

deadline. Writers report each write through `SendDone`. A graceful close queues a FIN item. The close then drains the peer for a 2 s linger. Unread input cannot turn the close into a RST that removes final acks. An abort sets linger 0 and resets the connection. `OpenConns()` feeds the `manibroker_open_connections` gauge. You find this code in `sys/prod/net.go`.

The S3 store lets the broker own retries

The store uses AWS SDK v2 over the pooled transport. The store sets `aws.NopRetryer{}`. The broker owns retries, and the store does not retry. The machine sees a retry as a new Action with backoff. The SDK does not hide a retry from the machine. The store prepends the prefix to every key. `List` strips the prefix from each key. `Get` issues a `Range` header from `core.SegmentRef{Off, Len}`. `Get` translates a range at the end of the object into an empty result. `Get` does not return an error for that range. `Delete` sends a HEAD request first. Deleting an absent key returns `ErrNotFound`. The store passes conformance check C3 against `veritas`. The simulator fake passes the same check. You find this code in `sys/s3store/store.go`.

The HTTP pool keeps 256 idle connections for each host

Go's default keeps 2 idle connections per host. A caught-up consumer fleet and hundreds of in-flight S3 requests would dial fresh connections at every burst. Those connections would stay in TIME_WAIT. The comment cites one measurement. "16k sockets in six seconds were measured." The pool sets `MaxIdleConnsPerHost = 256`. The pool sets `MaxConnsPerHost = 1024`. The pool sets `IdleConnTimeout = 60s`. The pool sets `ForceAttemptHTTP2 = false`. The S3 client uses the pool through `httppool.AWS()`. The CP client uses the same pool. You find this code in `sys/httppool/httppool.go`.

Smithy middleware records each AWS attempt

Smithy middleware on the S3 client records one histogram observation for each HTTP attempt. SDK retries stay visible in the attempt data. The middleware records one histogram observation for each operation. The middleware also records retry counters, request-byte counters, API-error counters, and throttle counters. `manibroker` wires the counters as `manibroker_s3_*`. The CP store wires the counters as `manicp_dynamo_*`. Every method accepts a nil `*Recorder`. A nil recorder records no metrics and adds no cost. You find this code in `sys/awsobs/awsobs.go`.

The prod clock and the prod env remove chance from the machine

`Clock.Mono` counts from clock creation. `Clock.Mono` uses the Go monotonic reading. `Wall` is UTC UnixNano. `prod.Env` is the production `core.Env`. ChaCha8 gets a seed from `crypto/rand`. `Buggify` does no work in prod. `Maybe` returns false in all cases, and `Fault` returns false in all cases. Assertions log an Error line and do not stop the process. In prod a violated invariant becomes telemetry. In sim the same call fails the run. You find this code in `sys/prod/clock.go` and `env.go`.

7.6 Metrics are facts, so sim and prod report the same durations

The broker core imports no Prometheus. The broker core imports no logger. The broker core imports no metrics type. The broker core emits `core.Emit{Fact}` actions. Decision 36 uses the name "metrics-are-

facts” for this rule. A `Fact` has a stable name. A `Fact` has ordered plain-data fields. Fact durations are differences of event times. The runtime stamps those event times at dequeue (§7.2). An invariant in sim reads the same values that prod reports.

`sys.Run` turns `Emit` into `Bundle.Facts(f)`. In `manibroker`, `nodeConfig` in `cmd/manibroker/main.go` sets that function. The function calls `prod.Metrics.Observe` if `-metrics-addr` is non-empty. The function also calls `prod.SegmentLog.Observe`. `prod.SegmentLog.Observe` runs in all cases. The same facts feed two consumers.

`sys/prod/metrics.go` aggregates facts into a private Prometheus registry. The registry uses the namespace `manibroker`.

Fact	Collectors
`segment_sealed`	`segments_sealed_total{reason}`, `segment_sealed_batch_bytes_total`, `segment_open_age_seconds`, `segments_in_flight_at_seal`, `segment_partitions`, gauges `segments_in_flight`, `buffered_bytes`
`segment_put_done`	`segment_put_seconds`, `segment_put_attempts_total`
`segment_committed`	`segment_commit_seconds`, `segment_queue_wait_seconds`, `segment_seal_to_ack_seconds`, `segments_committed_total`, `segment_committed_bytes_total`
`segment_failed`	`segments_failed_total`, `segment_seal_to_ack_seconds`
`backpressure_on` and `backpressure_off`	`reads_paused` (gauge), `reads_paused_seconds_total`

`cp_call_seconds{op}` also reports CP calls. `cpclient.Client.Observe` supplies that data through a separate wire. The registry includes Go collectors and process collectors. `sys/runinfo` adds `manibroker_build_info`. The labels are version, go_version, git_sha, and run_id. `sys/runinfo` adds `start_time_seconds` and `uptime_seconds`. The single loop goroutine calls `Observe`. Collectors accept concurrent scrapes. `serveMetrics` serves the registry on `-metrics-addr`. The default is `:9100`. `runinfo.Mux` serves the registry. `runinfo.Mux` also answers `/healthz`.

`sys/prod/seglog.go` writes one structured log line for each segment lifecycle event. The events are `segment_sealed`, `segment_put_done`, `segment_committed`, `segment_failed`, and `backpressure_on/off`. The `segment_sealed` fact carries no key. `SegmentLog` pairs that fact with the next `ObjPut`. `SegmentLog` sees `ObjPut` through `NodeConfig.OnIO`. `NodeConfig.OnIO` is a `tapExecutor` wrapper. The wrapper shows every action to the hook on the loop goroutine before the wrapper executes the action. The `-log-level` flag help gives you one instruction. “raise -log-level to warn to keep only failures”

Run identity ties the three outputs together. `sys/runinfo.New` builds an `Info`. The run id comes from `-run-id`. The run id comes from `$MANIFESTER_RUN_ID` if `-run-id` is empty. The code generates `run-<utc>-<hex>` if both values are empty. Version data and git data come from build info or from `-ldflags -X`. `NewLogger` stamps `run_id`, `service`, `version`, and `git_sha` on every JSON log line. `runinfo.Manifest` and `Write` can save every effective flag at startup. `runinfo.Manifest` and `Write` can also save resolved tuning at startup. The flag is `-manifest-out`. A manifest write failure never stops startup.

7.7 Graceful shutdown gives the machine 10 s to emit `Exit`

Graceful shutdown follows these steps:

1. `main` wires `sys.ShutdownOnSignal(syscall.SIGTERM, os.Interrupt)` in `sys/signal.go`. The first signal closes the channel. Later signals get the default disposition.
2. `prod.Node` watches the channel in `stopOn`. `prod.Node` calls `Node.Stop()`. `Node.Stop()` closes the node internal `stop` channel one time. `Node.Stop()` uses `sync.Once` for that close. That channel is `Bundle.Shutdown`.
3. `sys.Run` delivers `core.Stop{Deadline: Mono + StopGrace}` to the control queue. In-flight produce can finish and receive answers. `sys.Run` arms a real `time.Timer` for `StopGrace`. The default is 10 s. The flag is `-stop-grace`.
4. The machine emits `Exit` before the deadline in a clean stop. `Run` returns nil. `Node.run` gives `Net.Shutdown` a `FlushGrace`. The default is 5 s. Writers flush the last answers before the code resets connections.
5. The deadline can fire first. `Run` then returns `ErrKilled`. The process skips flush grace. The code resets remaining connections. That error reaches `serve` in `manibroker` through `n.Wait()`. The process exits 1.
6. `Kill()` cancels the context. `Run` returns `ErrStopped` on the next turn. The code resets connections at once. Tests use `Kill()` to simulate crashes.

Exit codes come from `cmd/manibroker/main.go`. A flag parse error gives 2. `-h` gives 0. Any `serve` error gives 1. `ErrTripped` and `ErrKilled` count as `serve` errors. The process writes one `slog.Error("manibroker", …)` line for a `serve` error. A clean stop gives 0 after `manibroker stopped`.

7.8 `cmd/manibroker` wires the runtime, then starts `sys.Run`

`serve()` in `cmd/manibroker/main.go` runs in a fixed order:

1. `serve()` parses flags.
2. `serve()` builds `runinfo.Info` and the JSON logger.
3. `serve()` builds `nodeConfig`.
4. `nodeConfig` builds the CP client, applies tuning, and sets the facts chain.
5. `serve()` calls `openObjects`, and `openObjects` returns S3 storage or mem storage.
6. `serve()` starts the metrics listener.
7. `serve()` binds the Kafka listener and resolves the advertised address.
8. `serve()` calls `prod.Start`.
9. `serve()` writes the optional run manifest.
10. `serve()` calls `n.Wait()`.

`openObjects` has one branch. If you set `-objects mem`, `openObjects` uses the in-process store from `veritas/vsys` with `prod.MemClassifier`. The flag help describes that mode as “single broker, lost on exit.” If you set `-objects s3`, `openObjects` builds `s3store.Store`. `openObjects` attaches the `awsobs` recorder when metrics are on. `openObjects` can call `EnsureBucket`. `prod.Start` in `sys/prod/node.go` validates the listener, the store, and the CP. `prod.Start` creates `sys.NewInbox(cfg.DataInbox)`. `prod.Start` creates `NewNet`. `prod.Start` creates `NewExecutor`. `prod.Start` wraps the executor with the `OnIO` tap if the config

sets a tap. `prod.Start` builds the `sys.Bundle`. `prod.Start` runs `sys.Run(ctx, b, ex, broker.New(bc), self)` on a separate goroutine.

Tuning knobs (`tuning.go`, `units.go`)

`parseSize` in `cmd/manibroker/units.go` parses sizes. `parseSize` accepts plain bytes. `parseSize` accepts binary `KiB/MiB/GiB/TiB`. `parseSize` accepts decimal `KB/MB/GB/TB`. `parseSize` accepts an optional fraction such as `1.5GiB`. `parseSize` accepts an optional space. `parseSize` ignores case. `parseSize` rejects values at or above 2^{62}. Durations use Go `time.Duration` flag strings.

Flag	Default	Resolved to	Effect
`-segment-bytes`	`16MiB`	`Broker.MaxSegmentBytes`	Seal the open segment at this many batch bytes
`-segment-window`	`200ms` (`broker.DefaultConfig().BatchWindow`)	`Broker.BatchWindow`	Seal this long after the segment's first batch
`-max-segment-partitions`	`4096`	`Broker.MaxSegmentPartitions`	Seal at this many distinct partitions (DynamoDB transaction store needs ≤ 96)
`-max-segment-batches`	`256`	`Broker.MaxSegmentBatches`	Seal at this many batches. This bounds the CP segment record (~300 B/batch)
`-max-buffered`	`auto`	`Broker.MaxBufferedBytes`	Pause client reads above this much unacked produce data. `auto` = `clamp(memory/16, 128MiB, 1GiB)` (§7.4)
`-resume-buffered`	half of max	`Broker.ResumeBufferedBytes`	Resume reads below this. It must be positive and ≤ max
`-commit-deadline`	`120s`	`Broker.CommitDeadline`	Retry a segment's CP commit (capped exp. backoff + jitter) this long, then fail its produces. Keep it under producers' `delivery.timeout.ms`
`-obj-timeout`	`30s`	`ExecConfig.ObjTimeout`	Base object-call timeout. Body size adds time at 2 MiB/s
`-stop-grace`	`10s`	`Bundle.StopGrace`	Time the machine gets after SIGTERM before `ErrKilled`

Prod also uses fixed defaults with no flags. The defaults are plain values in code:

- The CP timeout is 10 s.
- The data inbox is 1024.
- Flush grace is 5 s.
- The read chunk is 64 KiB.

- The write timeout is 30 s.
- Close linger is 2 s.

`sys/prod/{executor,node,net}.go` sets these values in `withDefaults`.

The remaining flags set addresses and identity. You set the listen address with `-listen` (`127.0.0.1:9092`). You set the advertised address with `-advertise`. You set the broker number with `-broker-id` (1). You set the node name with `-node-id` (default `broker-<id>`). You set the CP address with `-cp-url`, and the flag takes a comma-separated failover list. The S3 flags are `-s3-endpoint/-bucket/-prefix/-region/-path-style/-create-bucket`. The other flags are `-metrics-addr` (`:9100`, empty disables), `-log-level`, `-debug`, `-run-id`, and `-manifest-out`. The process logs every effective value one time at startup. The log line is `manibroker tuning`. The line includes `memory_bytes` and `memory_source`. The manifest captures every effective value.

7.9 Two drivers run one machine, and the split tells you where a bug lives

Prod and sim run the same `broker` machine. Both drivers take the same Actions. Both drivers use the same `core.ControlFirst` policy. The drivers run in different outside worlds. The simulator does not implement `sys.Executor`. Each external system has one `sim.Engine`. The engines live in `sim/engines/{objstore,cpengine,logengine,netsim}`. Each engine routes by `Handles(a)`. Each engine answers through scheduled kernel entries.

You gain replay in this split. The sim driver controls time, faults, and randomness, so you can repeat one run from one seed. The prod driver controls none of those inputs. The prod driver instead classifies real errors, enforces real timeouts, and applies real backpressure. The cost is a second driver to keep honest. A timeout error, a pool limit, and a pause wire live only in `sys/`, and a sim run does not test those parts.

Dimension	Production (`sys/prod.Executor`)	Simulator (`sim.Engine`)
Interface	`Execute(a core.IOAction)`, sync return, must not block	`Execute(ctx, from, a)`, schedules entries. “never completes anything synchronously”
Concurrency	One goroutine for each object action and each CP action. One writer goroutine per connection. One loop goroutine owns the machine	One goroutine total (kernel). Ordering comes from lanes and scheduled times
Time	`prod.Clock`, monotonic + UTC wall, read only at dequeue	Simulated `core.Time`, advanced by scheduler. Per-node clock skew is a fault knob
Completion path	Goroutine → `Classifier` (error → `ErrCode` + `Outcome`) → `DeliverControl` → stamped `Event`	`EngineCtx.Deliver(to, at, lane, body)` → kernel entry → stamped `Event`
Ordering	FIFO only where physics gives it. Per-connection writer queue. Control-before-data policy	FIFO guaranteed per lane key (`EngineCtx.Lane`). Distinct keys never share a lane
Faults	Whatever S3, TCP, and the CP do. The runtime classifies faults after the fact	Drawn from `CountedRand` at issue time: 503s, timeouts, lost responses, outages, corruption (`sim/engine.go` fault families)

Dimension	Production (`sys/prod.Executor`)	Simulator (`sim.Engine`)
Randomness and Env	`prod.Env`, crypto-seeded, buggify inert, assertions log-and-continue	Seeded `CountedRand`. Every draw counted in trace hash. Assertions fail the run
State of the world	Real bucket, real CP, real sockets. This state survives the process	Engine-owned maps folded into trace hash after every step (`core.Hashable`)
Crash	`Node.Kill` cancels ctx. Inbox close drops late completions	`OnCrash`. Applied effects stand. Only entries addressed to the dead node dropped. A PUT already on the wire can land after the broker dies
Observability	Facts → Prometheus + segment logs (this chapter)	Facts → timeline checked by invariants

A bug reproduced in sim with a seed is a bug in the machine or the vocabulary. The bug is not in the code in this chapter. A bug that happens only in prod lives in classification, timeouts, pool sizes, or backpressure wiring. You put the fix in `sys/`.

Key takeaways

- `sys.Run` is the only caller of `Step` in production. One goroutine runs `sys.Run`. `ControlFirst` picks each event. The runtime stamps `At/Wall` at dequeue. The runtime dispatches actions in slice order. The runtime closes the inbox on return.
- `prod.Executor` turns each object Action and each CP Action into a goroutine under a size-aware timeout. `prod.Executor` turns each `Send` into an entry on the connection ordered writer queue. One `Completion` with the same ReqID comes back. The completion never arrives on the caller goroutine.
- Errors never cross into the core as Go `error` values. A `Classifier` reduces each error to `ErrCode` + `Outcome`. The outcome is `Applied`, `NotApplied`, or `MaybeApplied`. Acks use that outcome.
- Backpressure uses a broker limit, a network pause, and an inbox limit. The broker limit is `MaxBufferedBytes`. The broker limit emits `PauseReads` with resume at `ResumeBufferedBytes`. The network parks readers. The inbox holds 1024 data slots. `-max-buffered auto` is `clamp(memory/16, 128MiB, 1GiB)`. Resume is half of max.
- Metrics are facts. `Emit` goes to `Bundle.Facts`. `prod.Metrics.Observe` writes Prometheus data. `SegmentLog` writes one line per segment. Durations come from event times. Sim and prod use the same duration rule.
- Shutdown follows a protocol. SIGTERM produces a `Stop` event with a deadline. The machine emits `Exit` before the deadline in a clean stop. The machine returns `ErrKilled` at `StopGrace` (10 s) after the deadline. The supervisor sees a non-zero exit after `ErrKilled`.

Checkpoint

1. An `ObjPut` goroutine finishes 200 ms after `sys.Run` returned `ErrTripped`. What happens to its `ObjPutDone`, and at which two points is it dropped?

Answer: The `ObjPutDone` drops at two points. `prod.Executor.deliver` checks `sink.Closed()` and returns without delivering. `sys.Inbox.DeliverControl` can also drop it. The inbox is closed at that point because

`Run` uses `defer in.Close()`. The completion cannot block. It cannot reach the next incarnation. See `sys/prod/executor.go` (`deliver`) and `sys/inbox.go` (`DeliverControl`, `Close`).

1. A broker runs in a container limited to 4 GiB with no tuning flags. What are `MaxBufferedBytes` and `ResumeBufferedBytes`, and where does the 4 GiB number come from?

Answer: 4 GiB divided by 16 gives 256 MiB. 256 MiB sits inside the clamp. `MaxBufferedBytes` is 256 MiB. Resume is 128 MiB. `prod.AutoBuffered` returns `m, m/2`. `MemoryLimit` reads the cgroup limit first. It reads `/sys/fs/cgroup/memory.max` on cgroup v2. It prefers that value over host `/proc/meminfo`. The startup line `manibroker tuning` logs the source as `memory_source`. See `sys/prod/memory.go` and `cmd/manibroker/tuning.go`.

1. Which component decides that a client reader parks, which one actually parks it, and which queue is only a safety net?

Answer: The broker machine sets the park condition. At `MaxBufferedBytes` of unacked produce it emits `backpressure_on` and `PauseReads` actions. Resume uses `ResumeBufferedBytes`. `prod.Net` parks the connection reader goroutine on its `resume` channel (`sys/prod/net.go`). The data inbox 1024-slot semaphore in `DeliverData` (`sys/inbox.go`) is the safety net. It blocks reader deliveries if readers outrun the loop after the pause.

1. A segment's S3 PUT times out. What `(ErrCode, Outcome)` pair can the machine see, why that outcome, and who owns the retry?

Answer: The pair is `ErrTimeout` with `MaybeApplied` for a PUT. DELETE uses the same pair. The request may have landed before the caller gave up (`s3store.Classifier`, `sys/s3store/classify.go`). The SDK does not retry. `s3store.New` sets `aws.NopRetryer{}` because "the broker owns retries." The machine re-issues the PUT as a new Action. Segment keys are unique per upload, so a second PUT is safe. GET and LIST failures carry the same codes. GET and LIST failures are `NotApplied` in all cases.

1. SIGTERM arrives while produces are in flight. Trace the stop, and name what makes it end as `ErrKilled` instead of a clean exit.

Answer: `sys.ShutdownOnSignal` closes the shutdown channel. `Node.Stop` closes `Bundle.Shutdown` one time. `sys.Run` delivers `core.Stop{Deadline: Mono + StopGrace}` on the control queue. The default `StopGrace` is 10 s. `sys.Run` arms a kill timer. The machine finishes in-flight work and emits `Exit`. `Run` returns nil. `Net.Shutdown` gets 5 s `FlushGrace` to flush final acks. The stop ends as `ErrKilled` if the machine has not emitted `Exit` when the timer fires. `Run` checks the timer on every loop turn. A busy inbox cannot block that check. The process exits 1 through `n.Wait()` in `serve`. See `sys/run.go`, `sys/signal.go`, and `sys/prod/node.go`.

Try it

1. Run the smallest local stack with no Docker and no AWS. Use one terminal for `go run ./cmd/manicp -store mem` (CP `:8080`, metrics `:9101`). Use another terminal for `go run ./cmd/manibroker -objects mem` (Kafka `:9092`, metrics `:9100`). Run `curl -s 127.0.0.1:9100/metrics | grep -E 'build_info|buffered_bytes'`. Run `curl -s 127.0.0.1:9100/healthz`. Press Ctrl-C in

the broker terminal. Watch for `manibroker stopped` and exit code 0. Check the `manibroker tuning` line for `memory_bytes` / `memory_source` on your machine.

2. Exercise the clamp. Restart with `-max-buffered 512MiB -manifest-out /tmp/manifest.json`. Compare `max_buffered` / `resume_buffered` in the tuning log line and manifest against the auto values from step 1.
3. Run the broker against a real S3 backend. `./scripts/it-up.sh` starts SeaweedFS S3 on `127.0.0.1:8333` and the DynamoDB-compatible backend on `:18080` from `deploy/docker-compose.yml`. `./scripts/it-down.sh` stops them. Run the broker with `-objects s3 -s3-endpoint http://127.0.0.1:8333 -s3-bucket manifester -s3-path-style -s3-create-bucket` against the in-memory CP. Watch `manibroker_s3_attempt_seconds*` on `:9100/metrics` under load from `go run ./cmd/manibench benchmark-producer -topic t -cp-url http://127.0.0.1:8080 -create-partitions 3 -duration 10s`.
4. Break classification on purpose. Run `docker compose -f deploy/docker-compose.yml stop seaweedfs` during a run. Read the broker log for `segment_failed`. Compare that line with the `manibroker_s3_*` error counters when it restarts.

Further reading

You can read the code and the design notes in this order:

- `../../sys/sys.go` and `../../sys/run.go`. Read the interface and the loop in full.
- `../flow-control.md`. Read the design behind the two queues, `ControlFirst`, and `PauseReads` (§2-§3).
- `../lessons.md`. Read lessons 0, 1, 6, and 10 on the same vocabulary, errors as data, connection faults, and facts.
- `../decisions.md`. Read decision 35 on buffer sizing, decision 36 on metrics-are-facts, and the CP item limits behind `-max-segment-partitions`.
- `../observability.md`. Read how operators use the metrics and segment logs to judge a run.
- `../../sys/prod/metrics.go` and `../../sys/prod/seglog.go`. Read every fact consumer.
- `../../sim/engine.go`. Read the `Engine`, fault-family, and latency vocabulary behind the contrast in §7.9.
- `../aws-benchmark-runbook.md`. Read the same flags and metrics at benchmark scale.

One seed replays a whole cluster

You give the simulator one seed. The simulator gives you one whole cluster history. The history has crashes. The history has partitions. The history has lost responses and corrupt reads. You run the seed again and you get the same history, step by step.

That property is the point of the design. A seed fails in a campaign at night. You replay the seed on your machine in the morning. You see the same steps in the same order. You do not guess which order the cluster took.

The price is real, and this chapter takes a view on the price. Production code gives up direct input and output. Production code gives up direct clocks and direct randomness. The simulator gives up parallel run time and runs in one goroutine. The trade is worth the price for this project. Order decides the hard bugs here. A crash at one step and a crash at the next step can give you two different clusters. If you cannot replay the step, you cannot prove the fix.

You find the design brief in `loop.md` at the repo root. You find the implementation in `sim/`. You find the engines in `sim/engines/`. You find two scenarios in `sim/kafka/` and `sim/echo/`. You find the driver in `cmd/manisim/`. If you read one file first, you read `loop.md`.

The contract in `loop.md` makes one history per seed

`loop.md` states the deal in fifteen instructions. Six instructions carry the rest:

- One event loop runs each broker. A broker is a single-threaded state machine. The broker does no input and no output. The broker takes events and returns actions. The code defines `core.Machine` (`core/machine.go`) as `Step(Event) []Action` plus a mandatory `StateHash() uint64`. State changes only inside `Step`. A completion with an unknown request ID leaves state unchanged. A completion with a stale request ID leaves state unchanged.
- Events and actions are plain data. Events and actions hold no channels. Events and actions hold no funcs. Events and actions hold no pointers into shared state. Code can log, compare, hash, and replay each event and each action. Each input and output is asynchronous. An action goes out. A `...Done{req, result, err}` event comes back. A request ID joins the action and the event. Timers are actions too. `core.SetTimer` goes out. `core.TimerFired` comes back.
- `core.Env` (`core/env.go`) injects each source of nondeterminism. `core.Env` holds `Rand` (`Uint64`, `Intn`, `NewID`), `Buggify`, and `Assertions`. Time is not in `Env`. Time arrives on each `core.Event` (`At`, `Wall`).
- The stores hold all durable state. A broker rebuilds from the object store and the control plane after a crash. Memory is disposable. The phrase stateless broker means that rule here.
- The simulator is one goroutine. The simulator uses a priority queue ordered by `(time, rngTie, seq)`. The simulator uses one seeded RNG. The simulator uses a virtual clock. Instruction 11 sets this order. The tie sorts before `seq`, or the tie decides nothing.
- Instruction 12 requires the fakes to inject faults. Instruction 13 checks invariants after every step. Instruction 14 hashes state into a running trace hash and runs each seed twice. Instruction 15 gives one command for a failure.

`loop.md` gives you the shape. The code makes three choices sharper than the shape. The code uses a counted SplitMix64 stream instead of `*rand.Rand`. The code uses lanes as an explicit ordering domain. The code uses a three-phase run model. When `loop.md` and the code disagree, you trust the code. `loop.md` §15 writes `go test ./sim -seed=81237`. The real reproduction command is `go run ./cmd/manisim -seed …` (§8.11).

The kernel in `sim/` holds no Kafka, S3, or TCP code

Package `sim` declares contracts only (`sim/doc.go`). The package declares one goroutine. The package declares one counted RNG. The package declares one global queue. The package declares an `Engine` for every external system. The package declares the `Process`, `Workload`, and `Invariant` roles (`sim/roles.go`). The kernel (`sim/kernel.go`, ~900 lines) implements the contracts.

The table gives you the map, and the map follows the package:

File	What it owns
`sim/doc.go`	Package contract. The lane reservation rule
`sim/scheduler.go`	`Scheduler`, `Entry`, `Lane`, `Time` types
`sim/schedimpl.go`	The heap. Order `(At, Tie, Seq)`. Per-lane FIFO
`sim/rand.go`	`CountedRand` contract
`sim/randimpl.go`	SplitMix64, `Derive`, `TwoSpeed.Sample`
`sim/engine.go`	`Engine`, `EngineCtx`, all fault-knob structs
`sim/kernel.go`	`RunSeed`, boot, crash, restart, the main loop
`sim/run.go`	`Config`, `SeedConfig`, `Swarm`, `Phases`, results
`sim/runner.go`	`Runner.Run` (campaigns) and `Runner.Replay`
`sim/roles.go`	`Process`, `Workload`, `Invariant`, `View`, `History`
`sim/timeline.go`	`MemTimeline`. The append-only fact log
`sim/view.go`	The read-only `View` invariants receive
`sim/inboximpl.go`	Node inboxes (control, data) and the cost model
`sim/cost.go`	`CostModel` and `CostParams` contracts
`sim/hash.go`, `sim/tags.go`	FNV-1a trace hasher. Frozen type tags
`sim/assert.go`	Assertion bookkeeping and `Buggify`

The kernel routes each action to one engine. The engine answers `Handles` for the action. If no engine handles an action, the seed fails with `FailProtocol/unrouted_action`. Engines schedule completions back into the queue. The same kernel drives the toy echo scenario and the real broker. The kernel holds no system detail, so the kernel needs no change when the system changes.

The queue order `(At, Tie, Seq)` decides what the seed means

`sim/schedimpl.go` is a binary min-heap. The `less` function orders entries first by `At`. The `less` function orders entries next by `Tie`. The `less` function orders entries last by `Seq`. `Seq` is assignment order. Two rules make that order the right order.

- `Schedule` draws one RNG value per call. `Schedule` draws `e.Tie = rng.Uint64()`. `Schedule` draws even when code later overrides the tie. The draw count never depends on lane history. Each draw shifts all later draws, so the count stays independent of lane history.
- Each entry carries a `Lane` (`sim/scheduler.go`). A lane is an ordering domain, and a lane keeps one meaning in this chapter. If an entry joins a lane with a pending entry at the same `At`, the entry inherits that pending entry's tie (`laneGroup`). The pair falls through to `Seq`. `Seq` is call order, so the lane stays FIFO. The drawn ties order entries across lanes at the same instant. The seeded stream sets that cross-lane order. Without inheritance, two chunks of one TCP stream could land at the same time and reorder. Without the random tie, scheduling order would set the cross-lane order. The seeded stream gives each seed a new cross-lane order.

You schedule into the past and the scheduler clamps the entry to `Now()`. `Cancel` and `DropNode` mark heap items as dead. The kernel discards dead items at the top of the heap. Virtual time advances in `Pop`. Virtual time moves to the next entry. An idle simulated hour costs zero steps.

Lanes keep one stream in order and keep engine lanes below `1<<62`

One rule in `sim/doc.go` and `sim/kernel.go` does more work than the rule gets credit for:

- Engine lanes are `1, 2, 3, …`. The kernel allocates each lane in first-use order. Engines never construct a `Lane`. Engines ask `EngineCtx.Lane(key)` with keys named `"<engine>/<whatever>"`. Netsim uses `net/<pair>/<dir>`. Fakeio uses `obj/<node>`. Allocation draws no randomness. First use decides the number. One key maps to one lane for one seed. Distinct keys never collide.
- Lanes at or above `laneReserved = 1<<62` belong to the kernel. The kernel lanes cover `laneSim` and per-node lanes `nodeLane(n, k) = laneReserved + 1 + idx<<8 + k`. `laneSim` covers phase changes, crashes, restarts, stops, and quiesce ticks. Per-node lanes use timers with `k=0` and wakeups with `k=1`. `EngineCtx.Lane` panics if engine space ever reaches `1<<62`. An engine invents a lane constant and the engine breaks the FIFO guarantee. The engine can also break another engine's lane.

Read the bound as a contract, not as a detail. Engine lanes live below `1<<62`. Kernel lanes live at `1<<62` and above. The two sets never meet.

One counted stream makes each draw part of the history

`sim/randimpl.go` is a SplitMix64 stream with a draw counter (`Calls()`). The kernel builds the stream with `sim.NewRand`. All draws come from the one stream. All draws follow event order. Draws cover scheduler tie-breaks. Draws cover engine latencies and fault coins. Draws cover attrition rolls. Draws cover workload choices through `core.Env`. Draws cover incarnation IDs (`NewID`, two draws). `SeedResult.RandCalls` is a coordinate. Two runs of one seed consume different draw counts and the two runs have diverged. You see the divergence before you compare a hash.

`Derive(label)` returns an independent stream. The stream seeds from `mix64(seed ^ mix64(fnv1a(label)))`. `Derive` consumes nothing from the parent stream. Two consumers rely on that rule:

- `Swarm` draws per-seed knob selection from `Derive("swarm")` in `newRun`. You edit a swarm menu and the edit reshuffles no run stream.
- `Buggify` activation draws from `Derive("buggify/"+site)` (§8.7). You add a site and the site shifts no other draw.

Nothing else invents randomness. The code uses no `math/rand` global. The code uses no map iteration with unknown order. Engines sort with explicit code. Engines index with explicit code. The code uses no wall clock. Node clocks are `sched.Now()` plus a per-node skew. The kernel draws the skew at setup. The wall clock is the scripted `wallEpoch` (2023-11-14) plus virtual time.

You see the draw-count rule in many places. `Chance(p)` draws nothing when `p <= 0`. `Between(lo, hi)` draws nothing when `hi <= lo`. `TwoSpeed.Sample` skips the range draw when `Max <= Min`. An extra draw is a determinism bug. The extra draw changes each later draw. `Fingerprint()` probes stream state without a draw. You use `Fingerprint()` to find the step where two runs diverge.

The kernel loop takes one step at a time and charges for the step

In this chapter, a step is one entry the kernel pops and the work the kernel does for that entry. `RunSeed(cfg, seed)` (`sim/kernel.go`) builds fresh engines for each seed. `RunSeed` builds fresh invariants, processes, and workloads for each seed. `RunSeed` boots processes before workloads. Broker refs reach workloads before the loop starts. `RunSeed` then runs `loop()`.

```
for !halt:
    e := sched.Pop()                  # advances virtual Now to e.At
    foldEntry(e)                      # trace-hash the entry itself
    process(e)                        # deliver / engine / crash / phase …
    drain pending attrition rolls     # crash dice after steps & landings
    foldState()                       # trace-hash every StateHash
    for inv in invariants: inv.Observe(view)
```

The kernel dispatches each entry by kind. `EntryDeliver` pushes onto a node's inbox and pumps the inbox. `EntryEngineInternal` calls an engine's `OnInternal`. `EntryCrash`, `EntryRestart`, `EntryStop`, `EntryPhase`, and `EntryQuiesce` go to the simulator itself. `MaxSteps` (default 10,000,000) fails a seed with `FailMaxSteps`. A timer loop at one instant cannot hang CI.

The kernel does not step a node on delivery. Deliveries land in a two-class inbox (`sim/inboximpl.go`). The inbox holds a control class and a data class. `core.ClassOf` routes each delivery. `pump` reads the next item from `core.ControlFirst`. `pump` calls `Step` once. `pump` dispatches returned actions in slice order. `pump` then charges busy-time from the `CostModel` (`sim/cost.go`). The cost model uses `PerEvent`. The cost model uses `PerByte` for `BytesReceived`. The cost model uses seeded GC-like pause episodes. A busy node gets a wakeup entry at `BusyUntil`. The wakeup models a production loop goroutine with finite

speed. Backpressure becomes seed-visible behaviour. Queueing becomes seed-visible behaviour. The kernel folds inbox lengths into the trace hash. The kernel folds `busy` into the trace hash.

The run stops new damage before the run judges recovery

Each run has three phases (`sim/run.go: Phases`). The Kafka preset uses 30s chaos, 10s settling, and 5s settle (`sim/kafka/config.go`).

- In PhaseChaos, the run injects faults. Faults come from engine fault coins. Faults come from attrition. Faults come from buggify `Fault` sites.
- In PhaseSettling, the kernel calls `EnterRecovery` on every engine. Engines generate no new faults. Plans in flight land as drawn. The rule preserves damage that exists. The phase judges liveness.
- In PhaseSettle, the run drains remaining events to `tEnd`. The run then calls each invariant's `AtEnd` once.

The kernel schedules phase changes on `laneSim`. The Kafka workload is `PhaseAware`. A `PhaseAware` workload gets an `OnPhase` call. The kernel treats that call like a step. The kernel traces the call. The kernel dispatches the call. The kernel hashes the call.

A crash throws the node away and the stores rebuild the node

`end()` in `sim/kernel.go` is the crash path. A hard crash takes this path. `Trip` takes this path. The graceful-stop deadline kill takes this path. The path has four parts, and the parts run in order:

- The kernel calls `sched.DropNode(ref)` and drops each pending entry for that incarnation. The kernel counts the entries as `dropped_by_crash`. A delivery to a dead incarnation remains and the kernel drops the delivery as stale at delivery time.
- The kernel calls `OnCrash(ref)` on every engine. Most engines do nothing in that call. Engine-internal entries have no node address. Engine-internal entries survive and can land after the broker dies. An object-store PUT apply step is one such entry. A control-plane commit is another such entry. An effect on the wire stands. Netsim is the exception. Netsim retires the dead node's connection ends and notifies peers.
- The kernel discards the machine, the inbox, and the timers. The kernel draws a recovery delay from `Attrition.RecoveryDelay`. An `EntryRestart` boots a new incarnation. The new incarnation gets a fresh incarnation ID from `rng.NewID()`. The new incarnation gets a fresh inbox. The kernel calls the factory again (`core.Factory(env, self)`). The broker rebuilds state from the object-store spec and the control-plane spec. The specs hold the durable state.
- The kernel pushes `core.Started{Self}` as a control-class event. The push makes `Started` the first `Step` of the new incarnation. If `Started` is not first, the kernel fails the seed with `started_first`.

Attrition (`kernel.go: attrition`) rolls `CrashProb` after each node step. Attrition rolls `CrashProb` after each engine landing. Attrition rolls during chaos. Attrition caps dead processes at `MaxDead` at one time. Attrition picks a hard crash or a graceful `core.Stop`. Attrition picks by `CrashWeight` and `GracefulWeight`. A graceful stop arms a kill at `now + stopGrace` (5s). A node `Exit`s first and the node drops the node kill entry. `Config.Scripted` adds crashes, stops, and restarts at fixed times. You use scripted faults when a test needs a fault at an exact point. A captured failure becomes a permanent test in that form.

Faults draw in a fixed order, so one fault cannot move another fault

Faults come in two layers. The layers draw at different places, and both layers keep the draw order fixed.

Engine fault menus (`sim/engine.go`) are per-system knobs. The menus cover `ObjectStoreFaults`, `ControlPlaneFaults`, and `NetworkFaults`. Each menu has `TwoSpeed` latencies and probabilities. `TwoSpeed` gives a fast range and a rare slow tail. The shared store core, `sim/engines/internal/fakeio`, draws each coin at issue time in `Execute`. The shared store core draws each delay at issue time in `Execute`. The core draws in fixed order. The order is outage roll, 503, timeout, lost-response, apply latency, respond latency, and extra entropy. Draws happen in issue order. Draw order does not depend on landing order. The plan is one of four plans:

- `Unavailable`. A 503 gives this plan. An outage window gives this plan. The op is `NotApplied`. The engine delivers the error at respond time. Outages are correlated windows. The engine rolls an outage once. Each op fails until the window ends.
- `Timeout`. The caller gets `ErrTimeout/MaybeApplied` at the caller deadline. Code draws whether the op applies at landing. `OnInternal` uses a 0.5 coin for that draw. The caller cannot learn the result from the timeout alone.
- `Lost`. The op applies. The response never arrives. The caller cannot tell this case from a timeout. The case tests idempotency of retries.
- `Normal`. The op applies at `T1`. The engine responds at `T2`. The caller dies and the effect stands. The engine tells nobody.

Object-store code adds one case. `GetCorruptProb` flips one bit in the copy a GET returns. The stored object stays intact. The case tests the broker checksum path. Netsim adds the netsim fault families (§8.8).

Buggify (FoundationDB-style, `core/env.go`, implemented in `sim/assert.go`) is cooperative fault injection inside production code. Code calls `env.Buggify.Maybe(site)`, `MaybeP(site, p)`, or `Fault(site, p)`. Each stable `core.Site` gets activation for one run. Activation has probability 0.5 from the site derived stream. An active site fires with default probability 0.25 per call. `Fault` never fires after chaos. In production, the implementation returns false. The sites cost a predictable branch. The sites cost zero behaviour. You never interpolate values into a site string. A site string is a stable identity.

The swarm is `Swarm` in `sim/run.go`. Kafka defines the Kafka swarm in `sim/kafka/config.go`. The swarm re-rolls the menus for each seed from the derived swarm stream. The stream switches each fault knob off with probability 0.4. The stream scales each fault knob by ×0.5, ×1, ×2, or ×4 (capped at 0.4). Segment sizes come from profiles. Caps come from profiles. Profiles are `tiny`, `accumulate`, `defaults`, `wide`, and `mixed`. One seed gives one set of fault choices. A campaign runs many sets of fault choices. `Replay` re-derives the same configuration from the seed alone. A seed is a complete bug report for that reason.

The engines supply the world, and the kernel supplies the order

All engines satisfy `sim.Engine` (`sim/engine.go`). The interface gives `Handles`, `Execute`, `OnInternal`, `OnCrash`, `EnterRecovery`, and `StateHash`. `Execute` never returns a completion in the same call. Each

result is a scheduled entry. An engine with ground truth also implements `sim.ObjectTruth` or `sim.CPTruth`. The invariant `View` gets that truth through a type assertion (`sim/view.go`).

- `sim/engines/objstore` is a fake S3. The state core is the spec `veritas.ObjectStore`. The engine wraps the spec in `fakeio.Core` and adds the corrupt-GET hook. `Object` and `EachObject` (sorted keys) give `ObjectTruth`.
- `sim/engines/cpengine` is a fake control plane over `veritas.ControlPlane`. The engine adds `MaxCommitItems`. A `CommitSegment` needs more bounded-transaction items than the limit and the engine returns a refusal. The engine returns `ErrTooLarge` before the spec runs. The refusal changes nothing. The engine also gives `CPTruth` (`Committed`, `HighWatermark`).
- `sim/engines/logengine` is the log-backed control plane (`docs/sim-log-engine.md`). The engine uses an append-only slot log. The engine writes slots with if-absent puts. Two applying replicas run the log. One replica is the active writer. One replica is a hot standby. The log keeps `K` appends in flight. The log re-appends lost slots at a later time. The log fills holes with no-ops after `HoleAfter`. Slot numbers fence takeover and failback. The engine uses checkpoints. Trimming leaves tombstones. Replicas can stall, crash, and restart from a checkpoint. A slot-based idempotency window applies. The engine is an independent implementation. A reference replay of the log is the engine `CPTruth`. Code checks replica digests against that replay. `sim.LogParams.Bug` selects mutants. Mutants include writer-chosen offsets. Mutants include delete-instead-of-tombstone. A named property expects to catch each mutant.
- `sim/engines/netsim` gives TCP-like byte streams. A connection has two ends and two directions. Each direction has one lane. Each direction has monotone arrival times. Bytes never reorder. Chunks draw different latencies and bytes still never reorder. Sends chunk to `1..PartialIOMax` bytes. `SendDone` means accepted. `SendDone` does not mean delivered. The engine models half-close, abort, and pause and resume. Pause and resume backpressure is invisible to senders. The engine models per-pair permanent extra latency. The engine models clock skew. Fault families are partition, clog, random close, and black hole. A partition freezes in-flight bytes. The bytes deliver in order at heal. A black hole drops one direction without notice. Sends still succeed.

`sim/engines/conformance_test.go` and `sim/engines/internal/enginetest` hold engines to the engine contracts. The tests hold engines to the contracts independent of any scenario.

Echo teaches the shape, and Kafka tests the real broker

`sim/echo` is the teaching scenario. The scenario has a minimal broker. A PUT stores one batch as a segment and then commits the segment. A GET looks the batch up and reads the batch back (`sim/echo/broker.go`). The scenario has four simple clients and four invariants. `AckedIsCommitted` checks one rule on the step a PUT is acked. The offset is committed at that step. The segment exists at that step. The segment decodes to the acked payload. `ReadYourAck` is another invariant. The mutants (`mutEarlyAck`, `mutAckOnMaybe`, `mutBadRead`) prove the invariants can fail. The scenario stays small on purpose.

`sim/kafka` is the real scenario. The scenario runs the production `broker` package as processes (3 brokers). Kafka clients over `kwire` and `kmsg` drive the scenario (`sim/kafka/workload.go`). The clients do input and output only through actions. The workload is a deterministic client model. A real client library

cannot run inside the simulator. One request outstanding per client gives max-in-flight 1 per (producer, partition). That limit makes acked-offset ordering checkable.

- Producers (`sim/kafka/producer.go`) are at-least-once and not idempotent, with `maxAttempts = 3`. Record values encode record identity (`c<client>.b<batch>.r<record>|xxxx…`), so the oracle can match each committed record. Ambiguous error codes mark a batch as ambiguous. The codes are timeout, storage, unknown, and coordinator-not-available. A retry may commit the batch again. Duplicates are legal once per send. The workload mixes in deliberate bad batches. One batch in 40 claims a producer ID. The broker must reject that batch with `INVALID_REQUEST`. One batch in 40 is CRC-damaged after checksumming. The broker must reject that batch with `CORRUPT_MESSAGE`.
- Consumers (`sim/kafka/consumer.go`) list earliest offsets. Consumers then fetch all partitions from offset 0. Consumers track the next expected offset per partition. In `PhaseSettle`, consumers drain. Consumers stop at the first fetch that finds the end of every partition.
- History (`sim/roles.go`) is the client append-only op log. History records `Invoke` and `Return` with outcomes. `OpDefinite` puts the effect inside [invoke, return]. `OpIndeterminate` records a `MaybeApplied` result. The effect may take effect at any later time. The effect remains a candidate. The case is the Jepsen `:info` case. `OpAbandoned` is the third outcome. The workload records ops. Invariants judge the ops.

The oracle checks each ack at the step the ack lands

`sim/kafka/oracle.go` is one invariant. The oracle owns the reconstructed logs. For each partition, the oracle rebuilds the committed record sequence. The oracle rebuilds the sequence from `View.CP()` ranges. The oracle rebuilds the sequence from `View.Objects()` segment contents. The oracle skips steps where neither `Gen()` moved. On every step, the oracle checks several rules:

- An acked range is committed in the control plane at the step the range was acked.
- Each committed record came from a produced batch.
- Records keep batch order.
- Records stay in the partition where the client produced the records.
- No committed record came from a batch with a definite rejection.
- A consumer never receives a corrupt batch.
- A consumer never receives an out-of-range batch.
- A consumer never reads past the high watermark.
- A consumer resumes where the consumer last read ended.
- A consumer sees each offset in order with the committed value.

At end (`AtEnd`), the oracle checks more rules. Acked records sit at the acked offsets. A client has acked batches that increase per partition. Abandoned batches were never committed. Each consumer caught up with the final log.

`sim/kafka/invariants.go` adds `liveness` and `faultCoverage`. `liveness` requires no op pending at end. An op still runs at `livenessGrace` = 6s into Settling. That op must end acked or successful. A late-landing chaos plan can excuse a late op. Segment-cap invariants are per-seed invariants from `InvariantsFor` (`sim/kafka/config.go`). Segment-timing invariants are per-seed invariants from `InvariantsFor`. Both kinds depend on the swarmed broker configuration.

Assertions record a failure, and the run keeps going

Invariants report through `core.Assertions` (`sim/assert.go`). Methods are `Always`, `AlwaysOrUnreachable`, `Sometimes`, `Reachable`, and `Unreachable`. The methods use stable `core.Prop` strings. Code never interpolates values into those strings. A violated safety assertion records a `FailViolation`. The kernel records the first violation per property per seed. The run continues. One seed can show many bugs. `Runner.Run` aggregates results. Violated safety properties land in `Report.Always`. Coverage properties with no satisfaction land in `Report.Coverage`. `Config.ExpectReachable` declares paths the campaign must execute at least once. Kafka lists each fault family and each broker path. Examples are `"obj 503 injected"`, `"cp lost response injected"`, `"net: partition injected"`, and `"broker: segment sealed by batch cap"`. The campaign fails on any failing seed. The campaign fails on any `Always` violation. The campaign fails on any nondeterminism. The campaign fails on any coverage gap in a campaign of ≥100 seeds (`coverageMin` in `cmd/manisim/main.go`). A passing run never injects a partition and the run does not pass.

The trace hash turns one changed step into one failed seed

After each popped entry, the kernel folds data into a single FNV-1a hash (`sim/hash.go`). The code writes the hash by hand. `hash/fnv` is off the deterministic allow-list. The kernel folds the entry (`At`, kind, lane, target, body). The kernel folds each node step (name, event, actions). The kernel then runs `foldState`. `foldState` folds each live node cached `StateHash`. `foldState` folds inbox lengths. `foldState` folds busy-until values. `foldState` folds each engine `StateHash`.

Types fold by frozen numeric tags (`sim/tags.go`). Events use 1-99. Actions use 101-199. CP ops use 201-299. CP responses use 301-399. Field values use 401-499. The code never uses `%T`. The code never uses reflection. You rename a Go type and the rename moves no hash. A type has no tag and the code panics. The panic stops the run. Tags are append-only. You change one tag and the change re-baselines each golden trace. The kernel caches `StateHash` results. The kernel recomputes a result after the kernel calls into that machine. The kernel recomputes a result after the kernel calls into that engine. `Config.VerifyHashes` can recompute every step. `Config.VerifyHashes` fails the seed if state changed outside a kernel call. The check catches a machine that a goroutine mutated in secret. The check catches a machine that a closure mutated in secret.

The canary is a re-run check. `Runner.Run` re-runs every `CheckDeterminism`-th seed. Kafka uses every 10th seed. The re-run flips `Log`. The re-run requires identical `(TraceHash, RandCalls, Steps)`. Logging must not change the hash. The kernel never folds log text. A mismatch is `FailNondeterminism`. The mismatch appears in `Report.Nondeterminism`. `RunSeed` runs one seed. `SeedResult` pins `(Steps, RandCalls, TraceHash, Stats)`. Golden files compare that tuple. `TestDeterminismCanary` exists in both `sim/` and `sim/kafka/`. `scripts/ci-determinism.sh` runs `TestDeterminismCanary`.

Determinism is worth the price for this project

Determinism costs you three things you can name. You write brokers as state machines, and state machines take more design than direct calls. You draw each random choice from one stream, and one

extra draw breaks the history. You hash state after each step, and the hash work runs even on passing seeds.

You get one thing back, and the thing pays for the three costs. You get a failure you can run again. A seed of 81237 fails once and you hold the whole order, the whole fault set, and the whole workload in one integer. You fix the broker. You run the seed again. The seed passes and the pass means the same history now ends clean. A test a reader cannot run again gives the reader a claim. A seed gives the reader a run.

You run campaigns first, and you replay one seed after a failure

The driver is `cmd/manisim/main.go`. The flags (`parseOptions`) are the whole interface. The driver has no hidden config. The commands in this section are apparatus. You use the commands after you hold the model from the sections before this section.

```
# Campaign: defaults -seeds 100 -start 1 -scenario kafka -j <NumCPU>
go run ./cmd/manisim

# A bigger, verbose campaign over the log-backed control plane.
go run ./cmd/manisim -scenario kafka-log -seeds 1000 -start 5000 -v

# The echo smoke scenario, single shard.
go run ./cmd/manisim -scenario echo -seeds 20 -j 1
```

Scenarios are `kafka` (default), `kafka-log` (`kafka.ConfigLog`), and `echo`. Exit code is 1 on any failure. Failure covers a failing seed. Failure covers a violated `Always`. Failure covers nondeterminism. Failure covers a coverage gap in a ≥100-seed run. Exit code is 2 on usage errors. Campaigns shard seeds over `-j` goroutines in contiguous blocks (`cmd/manisim/merge.go: runSharded`). A seed result cannot depend on the shard that ran the seed. Each seed builds a world for that seed. Shard reports merge in shard order by the same rules as one `Run` (`mergeReports`). Sharding is safe for that reason.

Each failing seed prints a seed reproduction line. The form is this line:

```
FAIL seed 81237: violation (kafka: acked range is committed in the control plane) step=… at=…: …
  repro: go run ./cmd/manisim -scenario kafka -seed 81237
```

If a campaign gives you a failing seed, you replay the seed in four steps:

1. Copy the repro line the campaign prints for the seed.
2. Run the repro line on your machine.
3. Read the step lines `Runner.Replay` prints for the seed.
4. Compare the replay `hash=` with the campaign `hash=` for the seed.

`-seed <n>` selects replay mode when `-seed` ≥ 0. `Runner.Replay` re-runs that one seed with `HaltOnFirst` and full logging. `Runner.Replay` prints one line per step. Each line gives `step`, virtual time, and text. `Runner.Replay` then prints the seed summary and the failure. The summary gives `steps`, `rand`, `hash`,

`crashes`, `trips`, and `exits`. For a passing seed, `HaltOnFirst` changes no trace hash. For a passing seed, logging changes no trace hash. The replayed `hash=%016x` must match the campaign `-v` line for that seed. A mismatch means nondeterminism. A mismatch does not mean a flaky test.

You keep failing seeds. A seed and a scenario form a regression test. The test keeps the same input over time. `sim/kafka/c4_faults_test.go`, `sim/kafka/logmutants_test.go`, and the mutant tests use this pattern. The tests include scripted faults pinned at fixed times.

Key takeaways

- One seed gives one complete history. Queue order draws from a single counted SplitMix64 stream (`sim/randimpl.go`). Fault coins draw from the stream. Latencies draw from the stream. Crashes draw from the stream. Workload choices draw from the stream. Draws follow event order. `Derive` side-streams keep swarm menus from reshuffling the run. `Derive` side-streams keep buggify activation from reshuffling the run.
- The global queue pops by `(At, Tie, Seq)`. Entries in one lane at one instant inherit a tie. Entries in one lane stay FIFO. Lanes ≥ `1<<62` are kernel-private. Engine lanes come from `EngineCtx.Lane(key)`.
- A crash discards state and rebuilds state. Pending deliveries to the dead incarnation drop. Engine effects on the wire stand. The factory reboots the process from the object-store spec and the control-plane spec. `Started` is the first step of the new incarnation.
- Invariants run after every step against ground truth. Invariants run against `View.Objects()` and `View.CP()`. Invariants run against the fact timeline. Invariants run against the op history. The Kafka oracle rule is “acked ⇒ committed, at that offset, with that value”. The oracle checks the rule on every step. The oracle does not wait for the end.
- The trace hash uses frozen tags. The trace hash uses each `StateHash` on every step. The re-run canary makes nondeterminism a test failure. `go run ./cmd/manisim -scenario kafka -seed <n>` gives the full reproduction command.

Checkpoint

1. Two entries are scheduled for the same virtual time, one on lane A and one on lane B, and later a third entry joins lane A at that time. What decides each pair’s order, and why does the third entry not get its own random tie? — Across lanes, the seeded `Tie` from `Schedule` decides. Order is `(At, Tie, Seq)` in `sim/schedimpl.go`. The third entry inherits lane A pending tie for that instant. The pair in lane A falls through to `Seq`. `Seq` is call order. The lane stays FIFO. The third entry draws a tie too. `Schedule` draws once per call. The draw count stays independent of lane history.

2. Why would adding a new `Buggify` site, or a new swarm menu entry, be a determinism bug if activation drew from the run stream, and how does the code avoid it? — Each draw from the single run stream shifts all later draws. An unrelated edit would reshuffle each seed history. An unrelated edit would re-baseline each golden trace. Swarm choices draw from `rng.Derive("swarm")`. Buggify activation draws from `Derive("buggify/"+site)` (`sim/kernel.go`, `sim/assert.go`). `Derive` derives from the seed and the label alone. `Derive` consumes no draw from the parent stream.

3. A broker issues an object PUT and crashes before the PUT's apply entry lands. After restart, can the object exist even though the broker never saw `ObjPutDone`? Which code makes that so? — Yes. `DropNode` drops entries addressed to the dead incarnation. `DropNode` does not drop engine-internal entries. The PUT landing is an engine-internal entry in `sim/engines/internal/fakeio`. `OnCrash` is a no-op there. The plan applies at `T1`. The effect stands. No code tells the broker. The rebuilt broker must discover the object from the store. Durable state can live in the stores alone for this reason.

4. A campaign passes every seed but never injects a network partition. Does `manisim` call that success? — Not at campaign scale. `"net: partition injected"` is in Kafka's `ExpectReachable` list (`sim/kafka/invariants.go`). If no run reaches the path, the path appears in `Report.Coverage`. `cmd/manisim` fails campaigns of ≥100 seeds (`coverageMin`) with any coverage gap. Below that size, the run reports the gap but does not fail.

5. You replay a failing seed and the printed trace hash differs from the campaign run's hash for the same seed, before the failure step. What have you found, and what triple does the runner itself compare to catch it? — You have found nondeterminism in the run. A hidden draw can add nondeterminism. Map iteration can add nondeterminism. A goroutine can add nondeterminism. A wall clock can add nondeterminism. Replay must reproduce the campaign in full. `HaltOnFirst` cannot change the hash. Logging cannot change the hash. `Runner.Run` catches this case. `Runner.Run` re-runs every `CheckDeterminism`-th seed with `Log` flipped. `Runner.Run` compares `(TraceHash, RandCalls, Steps)` (`sim/runner.go`). A mismatch is `FailNondeterminism`.

Try it

- You run the smoke scenario. You use `go run ./cmd/manisim -scenario echo -seeds 20 -v`. You note each seed `hash=`, `rand=`, and `steps=`. You run the scenario again. You confirm each line is identical.
- You replay one seed with the full log. You use `go run ./cmd/manisim -scenario echo -seed 3`. You find a crash (`sim_crashed`) in the log and the restart after the crash. You confirm the final `hash=` matches the `-v` campaign line for that seed.
- You run a short Kafka campaign. You use `go run ./cmd/manisim -scenario kafka -seeds 30 -v`. You replay any seed and check the hash again. You try `-j 1`. You confirm the per-seed lines do not change. Wall time can change.
- You break determinism on purpose in a scratch copy. You draw from the run RNG inside a `StateHash` method. Or you range over a map in an engine. You run `go test ./sim/kafka/ -run TestDeterminismCanary -count=1`. The canary catches a case a normal test run can miss.
- You pick any seed and replay the seed against the log control plane. You use `go run ./cmd/manisim -scenario kafka-log -seed 3`. The scenario changes the world for the same seed. The hash changes. The procedure does not change.

Further reading

- `loop.md` gives the original fifteen-instruction design brief. You reread §11-§15 after the code. You spot where the implementation differs from the brief.

- `docs/lessons.md` gives the numbered lessons that code comments cite. Topics are single seeded stream, record-and-continue, phases, and swarm.
- `docs/decisions.md` gives the decision rows that `sim/` references. Topics cover lanes, tie order, fault families, and the log control plane.
- `docs/sim-log-engine.md` and `docs/log-cp-service.md` describe the log-backed control plane. `sim/engines/logengine` simulates that plane. The `kafka-log` scenario exercises that plane.
- `docs/flow-control.md` §4.1 gives the production behaviour that the cost model and two-class inbox in `sim/inboximpl.go` imitate.
- Tests give worked examples. `sim/kernel_test.go` covers crash, restart, lanes, and the canary. Other examples are `sim/kafka/kafka_test.go`, `sim/kafka/c4_faults_test.go`, `sim/echo/echo_test.go`, and `sim/engines/conformance_test.go`.

Chapter 9. Verification, veritas specs, conformance, and quality gates

`manifester` runs operations at the same time and adds faults during the runs. The spec in `veritas/` states the result that the outside world must see for each operation. Tests compare the implementation with the spec. Conformance tests, deterministic simulation, and integration tests run the comparison. A determinism gate and a CRAP gate add static checks. Read `veritas/doc.go` first. `veritas/doc.go` states the contract for most of this chapter.

9.1 Four parts check the implementation against the spec

The design has four parts:

- Executable specs in `veritas/` state the required behavior. The specs are sequential and fault-free. The specs hold plain data. A reader can check each spec line by line (`veritas/doc.go`, decision 21).
- Lockstep conformance in `veritas/veritastest/` compares two runs. The test runs the same random sequence against the implementation and the spec. The test compares every result. The test checks invariants after every step. The test shrinks a failure to a replayable example.
- Mutants are broken variants that each check must reject. A test that passes correct code and broken code proves nothing.
- Static gates check the source and the test coverage. `tools/detcheck` and `scripts/ci-determinism.sh` keep deterministic packages deterministic. `cmd/crapscore` reports complex functions that have low test coverage.

The layers repeat checks with different methods. One layer uses a sequential spec. Another layer uses Kafka wire requests. Another layer uses a simulated faulty world. Another layer uses a real client. Each layer can detect a bug that another layer missed.

9.2 The veritas specs state the required behavior

Package `veritas` holds three specs. `ObjectStore` and `ControlPlane` answer `core.IOAction` with `core.Completion` through `Exec`. All three specs have a `StateHash`. No code iterates a map to decide behaviour. Where code iterates a map, the code sorts the keys first (`sortedKeys` in `veritas/hash.go`, with its `//detcheck:ok` reason).

`Partition` states the rules for one Kafka partition (`veritas/partition.go`)

`Partition` is an append-only log of `[][]byte`:

- Offsets are dense from 0. The offset of a record is the index of the record.
- `Append(batch)` returns the offset of the first record in the batch. The offset is the prior high watermark. An empty batch changes nothing.
- The log commits everything the log appends. `HighWatermark()` is `core.Offset(len(recs))`.

- `Read(from, max)` treats `max <= 0` as a request for all records. `from < 0` or `from > HighWatermark()` gives `ok == false` (`OFFSET_OUT_OF_RANGE`). `from == HighWatermark()` is valid and empty.
- `StateHash()` folds the count first. `StateHash()` then folds every record in order.

The spec has no concurrency, no input or output, and no failure mode. When the broker disagrees with `Partition`, the broker code must show why the spec is wrong.

`ObjectStore` states the rules for stored objects (`veritas/objectstore.go`)

`ObjectStore` is a key to bytes map with sorted keys alongside. `Exec` handles `ObjPut`, `ObjGet`, `ObjDelete`, and `ObjList`:

- `Put` rejects an empty key with `ErrInvalid` and changes nothing. For any other key, `Put` stores a copy and replaces any object at that key. `Put` returns `OK` and `Applied`.
- `Get` returns `ErrNotFound` for an absent key. `Get` reads `[Off, Off+Len)`. `Len <= 0` reads to the end. `Off < 0`, `Off > size`, or `Off+Len > size` gives `ErrInvalid`. The request is malformed, and the key is not missing. `Off == size` with `Len <= 0` gives empty data.
- `Delete` returns `ErrNotFound` and `NotApplied` for an absent key. For a present key, `Delete` removes the object and returns `OK` and `Applied`.
- `List` returns prefix keys in sorted order. `List` returns `nil` if no key matches. `List` never fails.

The spec has no faults. The spec outcomes are `Applied` or `NotApplied`, and the outcomes are never `MaybeApplied`. Production code must resolve an uncertain write before a test can compare the write. This requirement is intentional.

`ControlPlane` states the rules for committed segments (`veritas/controlplane.go`)

`ControlPlane` models the control plane as a broker sees the control plane (`core.CPOp`). `ControlPlane` stores committed segments by key. `ControlPlane` stores per-partition runs (`core.BatchRange`). `ControlPlane` stores topic partition counts.

The following rules state how `ControlPlane` records and reads the runs:

- The log of a partition is a set of runs `[Base, Base+Count)`. The end of the log is the `Base+Count` of the newest run. The end is 0 when the log is empty. No separate counter can drift from the runs.
- `CommitSegment` is idempotent by key. The same key with the same batches returns the original ranges again, with `OK` and `Applied`. The same key with different batches gives `ErrConflict`.
- An invalid commit records nothing, and another commit can reuse the key. Invalid commits have an empty key, empty batches, `Records <= 0`, an empty topic, or a negative partition. Each case gives `ErrInvalid`.
- A valid commit assigns each batch a `Base`. The `Base` is the current end of the partition for the batch. The commit assigns bases in request order. The commit grows the end after each batch. Runs stay contiguous per partition. The commit covers all partitions in one step, and the commit is all or nothing.
- A commit creates a topic, and a topic grows to cover the partition. Real stores guard this step with compare-and-set on the run base and retry. The sequential spec does not need that guard.

- `Lookup` returns `ErrInvalid` for `From < 0`. For any other `From`, `Lookup` returns runs whose last offset is `>= From`, in order. `Lookup` also returns the end offset. `MaxBytes > 0` stops after the batch that reaches `MaxBytes`. `Lookup` returns at least one batch if any batch matches. An unknown partition returns an empty result with end 0.
- `DescribeTopics` returns topics in sorted order. An empty request means all topics. `DescribeTopics` omits unknown names. `CreateTopic` never shrinks a topic. `CreateTopic` rejects an empty name and `partitions <= 0`.

`veritas/snapshot.go` adds two read-only functions. `Ends()` returns the partitions that have a run and the ends of the partitions, in topic then partition order. `SegmentKeys()` returns the segment keys in sorted order. Tests use both functions for checkpoints. Neither function changes state.

`StateHash` detects state drift (`veritas/hash.go`)

`StateHash` uses FNV-1a over a canonical, length-prefixed byte stream. Distinct states cannot collide by concatenation. `ObjectStore` and `ControlPlane` cache folded prefix states. A change at sorted index `i` truncates the cache to `i+1`. A put near the end costs little. `veritas/hashinc_test.go` (`TestIncrementalStateHashMatchesFull`) checks the incremental hash against a fold from scratch. The test verifies the optimisation against the simple version. Conformance compares full completions first. The hash detects drift that no read has exposed.

The specs serve as fakes in `veritas/vsys/`

`veritas/vsys/vsys.go` adapts the specs to the production interfaces in `sys/`. `vsys.ObjectStore` implements `sys.ObjectStore`. `vsys.ControlPlane` implements `sys.ControlPlane`. The adapters return spec error codes as `*vsys.Error` for a `sys.Classifier`. `vsys` is not deterministic, unlike `veritas`. `vsys` uses `context` and a serialising `sync.Mutex`. `scripts/ci-determinism.sh` includes `./veritas` and `./veritas/veritastest`, but the script excludes `veritas/vsys`. A production `Executor` can run against the specs with no network. Compile-time assertions check that the adapters satisfy the `sys` interfaces.

9.3 `veritastest` runs the implementation and the spec in lockstep

`veritas/veritastest/doc.go` describes the method. The test runs random sequences against the implementation and the spec. The test compares the results. The test checks invariants after every step. The test shrinks a failing sequence and prints the sequence as a Go literal. A developer can paste the Go literal into a test. The core type is `Check[S, Op]` in `veritas/veritastest/lockstep.go`:

Field	Role
`New`	Fresh implementation paired with a fresh spec, both empty
`Gen`	Uses the supplied `*Rand` for the i-th operation
`Step`	Apply one op to both. An error marks any difference
`Invariant`	Optional, after every `Step`. A panic counts as failure
`Shrink` and `ShrinkAll`	Simpler versions of one op and the whole sequence
`Format`	Render an op as a Go expression for the report

Field	Role
`Ops`, `MaxShrinkRuns`	Sequence length and shrink budget (0 = 5000)

Determinism comes from the code structure. `veritastest.Rand` (`veritas/veritastest/rand.go`) is SplitMix64 with Lemire's unbiased `Intn`. `veritastest.Rand` imports `math/bits` and nothing else. `Options` maps test flags to seeds. `Seeds` means seeds 1..N. `Seed >= 0` means a single seed. `Stop` means a wall-clock campaign. `Replay` renders the replay command. Shrinking drops chunks first, in halves, quarters, then singles. Shrinking then tries the first failing `Shrink` candidate for each op. Shrinking then tries `ShrinkAll`. Shrinking tries candidates in fixed order until no candidate makes progress. Any failure counts. A shrunk sequence may expose a different bug, and that bug still counts as a failure. A `Failure` report prints the seed and the sequence as a `[]Op{...}` literal. A developer can paste that literal into a regression test. A developer can paste that literal into a regression test for the bug. `TestObjectStoreCheckShrinksPlantedBug` (`veritas/objectstore_test.go`) is an example. A store ignores `Off` when `Len <= 0`. The test shrinks the failure to one put and one get.

9.4 C1, C2, C3, and C4 compare four pairs

`veritas/doc.go` states the four checks:

Check	Under test	Against	Lives in
C1	Real broker, no faults	One `veritas.Partition` per (topic, partition)	`broker/conformance_test.go`
C2	Each sim engine, faults off	Its spec, same completions, same `StateHash` per op	`sim/engines/conformance_test.go`
C3	`sys` adapters (S3, CP HTTP client)	These specs	With the adapters. The only check that executes `sys`
C4	Kafka oracle	`veritas.Partition` on its own	`sim/kafka/c4_test.go`

C1 uses `TestConformanceBroker`. The test drives the real broker state machine. The test uses a fault-free test setup over `veritas.ObjectStore` and `ControlPlane`. The test builds requests with `kwire`. C1 compares every ack, fetch, and `ListOffsets` response with the spec. For produce, C1 compares the error and the base offset with the high watermark from the spec. The broker refuses idempotent entries (decision 41). For fetch, C1 compares the watermark, contiguity, and per-record values with `Partition.Read`. C1 checks the record CRC with `kwire.ParseBatch`. For `ListOffsets`, the latest offset equals the watermark, and the earliest offset is 0. The invariant requires no violations from the test setup and no stray responses. `corruptGets=true` flips a byte in every third `ObjGet`. Fetch must detect the corruption, and fetch must not serve the corrupted bytes. `TestConformanceBrokerSmallSegments` reduces `MaxSegmentBytes`, `MaxSegmentBatches`, and `MaxSegmentPartitions`. Each seal rule fires in short runs. The sequences have 80 ops. The mix is about 60 percent produce, 25 percent fetch, and 15 percent list. The test runs 100 seeds.

C2 sets fault probabilities to zero. Each engine must return the same completions as the spec for the engine. The cached hash of each engine must track `StateHash` after every operation. `TestConformanceObjStore` compares `sim/engines/objstore` with `veritas.ObjectStore`.

`TestConformanceCPEngine` compares `cpengine` with `veritas.ControlPlane`. Both tests run 200 seeds at 120 ops each. `TestConformanceLogEngine` compares `logengine` with `ControlPlane` on completions. The tick chain for `logengine` never drains, so each op runs 100 ms of virtual time. The test has no spec hash to compare. C2 runs with faults off. C2 proves that an engine implements the spec when nothing goes wrong. The invariant layer of the simulator checks correctness with faults on. C2 does not check that case.

C3 is the one check that runs `sys` code. C3 tests the S3 and CP-HTTP adapters against the specs. The tests live with the adapters. The fakes need no Docker. The fakes are `cp/dynamo/fakedb`, a fake S3 for `sys/s3store`, and `httptest` for `cp/cpclient` (see `docs/crap-gate.md`). The tests run under `go test ./...`. A change to an adapter must keep the C3 test for the adapter passing.

C4 keeps two implementations separate. The oracle in `sim/kafka` never imports `veritas` (decision 21). The external test package may import `veritas`, because `detcheck` skips `_test.go` files. `TestC4OracleDoesNotImportVeritas` enforces the split. The test parses non-test sources and fails on any `veritas` import. `TestC4AcceptsCorrectHistories` runs 200 seeds. The test covers incremental checks and one-shot checks. The test requires the log that the oracle reconstructs to equal `veritas.Partition` with no differences. `TestC4RejectsEveryFault` requires the test to report each fault in `sim/kafka/c4_faults_test.go` over enough applicable seeds. The faults include duplicated records, dropped acked ranges, and off-by-one acks. `veritas/doc.go` summarises the set as 28 faults and fenced acks.

9.5 Each test layer detects a different class of bug

The table lists each layer, the code that the layer runs, and the bugs that the layer detects:

Layer	What runs	Detects
Unit (`go test ./veritas/ ./kwire/ ./broker/`)	Spec rules, codecs, single components	Wrong rule, offset, CRC
Lockstep (C1, C2)	Random sequences against the spec	Divergence between two implementations of one contract
Simulation (`go test ./sim/...`)	Faulty worlds, invariants, C4 oracle	Lost, duplicated, and reordered records under faults
Integration, hermetic (`go test ./it/`)	Real brokers + real `kgo`, in-process backends	Wiring, wire protocol, client-visible behaviour
Integration, full-stack (`scripts/it-up.sh`)	Same flows on DynamoDB and S3 in Docker	Backend behaviour no fake reproduces
Static gates	`detcheck`, audits, CRAP	Nondeterminism, unreviewed deps, risky untested code

The specs have unit tests of their own. `veritas/partition_test.go`, `objectstore_test.go`, and `controlplane_test.go` test each rule. `TestObjectStoreVsSortedSlice` checks the store against a separate sorted-slice model. `TestControlPlaneInvariants` checks contiguous offsets. `TestControlPlaneInvariants` checks that the end equals the committed counts. `TestControlPlaneInvariants` checks CAS by base. `TestControlPlaneInvariants` checks idempotent

commit. A spec could share a wrong rule with the implementation. The spec tests reduce that risk, and C4 independence reduces that risk further.

9.6 Integration tests run real brokers and the real client (`it/`)

Package `it` (`it/doc.go`) drives real brokers with the real `franz-go` client. Package `it` uses `sys/prod` wiring, as in `cmd/manibroker`. Package `it` imports `kgo`, and no deterministic package imports `kgo`.

Hermetic tests live in `it/hermetic_test.go` and `it/stack_test.go`. The hermetic tests use loopback and nothing else. The CP uses `memstore` over `httptest`. The store is an in-memory store from `vsys.NewObjectStore`. Brokers come from `prod.Start` on `127.0.0.1:0`. The tests run under `go test ./...`. The tests cover round trips (`TestProduceConsumeRoundTrip`). The tests cover metadata and API versions. The tests cover InitProducerId, which the broker does not offer (decision 41). The tests cover unknown-topic produce and `ListOffsets`. The tests cover `TestKillBrokerMidProduce`, a graceful-stop flush, and the default 200 ms batch-window rule.

Full-stack tests live in `it/fullstack_test.go`. The full-stack tests run the same shared flows, `runRoundTrip` and `runKill`, over `cp/dynamo` and `sys/s3store`. The tests skip unless both endpoints are set. `scripts/it-up.sh` starts `deploy/docker-compose.yml`. The script runs ExtendDB on `127.0.0.1:18080` and SeaweedFS S3 on `127.0.0.1:8333`, on loopback and nothing else. The script prints `MANIFESTER_DYNAMO_ENDPOINT` and `MANIFESTER_S3_ENDPOINT`. `scripts/it-down.sh` stops the stack with `docker compose down -v`.

9.7 The determinism gate rejects nondeterministic code and unaudited dependencies

`tools/detcheck` (`tools/detcheck/detcheck.go`) is a `go/analysis` checker. `detcheck` reports imports that are not on the exact-path allow-list in `tools/detcheck/allow.txt`. The tool embeds that allow-list. `detcheck` reports `go` statements, channel types, channel sends, and channel receives. `detcheck` reports `select`. `detcheck` reports range over a map and reflect map iteration. `detcheck` reports package-level variables, because mutable globals share state between sim runs. Error sentinels and blank assertions are exceptions to that rule. Suppression needs a reason. Write `//detcheck:ok <reason>` on the same line or the previous line. A bare `//detcheck:ok` is a finding in itself. The checker does not check `_test.go` files, and C4 relies on that exemption. The allow-list uses exact paths. Adding a line requires an audit. A line may use `/...` for a whole audited subtree and for nothing else. The allow-list omits `time`, `math/rand`, `crypto/rand`, `os`, `net`, `sync`, `runtime`, `unsafe`, `reflect`, `maps`, `hash/maphash`, and `context`. The header of `allow.txt` lists the full set.

`scripts/ci-determinism.sh` runs the gate in three steps:

1. `detcheck` runs over `./core/... ./sim/... ./format/... ./kwire ./broker/... ./veritas ./veritas/veritastest`. `detcheck` also runs over every `toys3` package except `prod` and `cmd`. The script derives that list from `go list`, so a new pure package cannot pass without a check.
2. Audit records. Every third-party module that those packages depend on needs a record at `docs/dep-audits/<module>@<version>.md`. The version in the file name must match the version in `go.mod`. A version bump requires a new audit.

3. Determinism canary. Each package that implements `TestDeterminismCanary` (`sim/`, `toys3/sim/`) runs every seed twice. The two runs must match. The script reports a package without a canary as skipped.

To audit a dependency, use `scripts/audit-dep.sh` and `.claude/skills/audit-go-dependency/SKILL.md`. The template is in `docs/dep-audits/README.md`. Run the script in a temporary module. The script leaves the repo untouched. The script reports imports, transitive packages, `detcheck` findings, and hints. Review each finding for effects across runs. Check what grep cannot check. Check aliasing through three-index slices and `unsafe`. Check hidden timestamp or random defaults. Check the required Go version. Prove the result with a test. Encode and decode twice, in two processes. The outputs must match byte for byte and under `DeepEqual`. Then decide ACCEPTED, ACCEPTED WITH RULES, or REJECTED. If you accept the dependency, write the record with findings that name a file and a line. Add the exact paths to `allow.txt`. Run `scripts/ci-determinism.sh`. Production code in `sys/` needs no audit. Do not weaken `detcheck` to pass CI. Do not suppress a finding without a reason. Do not accept a dependency because "it's just a codec."

9.8 The CRAP gate rejects complex functions that lack coverage (`cmd/crapscore`)

`make build` is the full gate. `make build` runs `fmt-check`, `vet`, `compile`, and `test`. `make build` runs `crap` last (`Makefile`). CRAP (Savoia & Evans) scores change risk (`docs/crap-gate.md`, decision 40).

[(m) = (m)^2 (1 - (m))^3 + (m)]

At 100% coverage, CRAP equals complexity. At 0% coverage, CRAP is `comp² + comp`. Past a complexity of about 30, no function can score below the default threshold of 30. Complexity comes from `cmd/crapscore/analyze.go`. The count is `1 + if + for + range + non-default case + non-default select clause + && + ||`. The count includes nested function literals. Coverage comes from `build/cover.out`, which `make test` writes. A function in a file absent from the profile counts as 0%. A function with no profiled statements counts as 100%. The tool uses the standard library and nothing else (decision I).

The gate uses a baseline. `crap-baseline.json` records each function over the threshold and the CRAP of the function. The file is empty at present, so no function scores over 30. The gate fails when a new function scores over the threshold. The gate fails when a baselined function rises by more than the 0.5 tolerance. An improvement never fails the gate. `make crap-strict` ignores the baseline. `make crap-baseline` rewrites the baseline. Review the diff after any rewrite. A lower entry does not fail the gate. A new entry records accepted debt, and the change needs a reason for that debt. The threshold is 30, and the project did not set the threshold to 6. At full coverage, CRAP equals complexity. A threshold of 6 would have blocked 275 of 1248 functions when the project added the gate. A threshold that low could lead authors to split functions to lower the score. You can run `make build CRAP_THRESHOLD=15` to test a lower threshold. Skipped Docker tests mean the gate measures coverage from plain `go test ./...`. That result is stable anywhere. Backend packages use tests that need no Docker, so that coverage is available on any machine. You can merge integration coverage with `make crap CRAP_MERGE=build/it-cover.out`.

9.9 Each check must fail on a mutant

A check that has never failed is a hypothesis. The repo ships broken code to test each check:

- `broker.Config.Bug` lives in `broker/config.go`. `broker.Config.Bug` is test-only, and zero means the real broker. `BugEarlyAck` acks after PUT and before `CommitSegment`. `BugWrongOffset` adds 1 to the fetch base. `BugServeCorrupt` skips the fetch CRC. Other mutants change segment caps, timers, retry keys, and produce CRC checks.
- `TestConformanceBrokerCatchesMutants` is the C1 mutant test. The test fails if `BugWrongOffset` survives 20 seeds. The test fails if `BugServeCorrupt` survives 20 seeds under corrupt GETs.
- Direct tests live in `broker/broker_test.go`. `TestWrongOffsetMutant`, `TestEarlyAckMutant`, and `TestCorruptGetRetried` assert that a mutant misbehaves. The tests assert that the real broker does not misbehave. A pass rules out a mutant that never broke the code.
- `brokenStore` is a planted bug in `veritas/objectstore_test.go`. `nonIdempotent` is a planted bug in `veritas/controlplane_test.go`. `nonIdempotent` uses a fresh key for each commit, so a replay commits twice. Each planted bug must be found, and each failure must shrink to two ops.
- Each fault in `sim/kafka/c4_faults_test.go` must be rejected. `TestC4RejectsEveryFault` checks that rule.

The simulator records the first seed that detects each mutant. See `sim/kafka/kafka_test.go` and `sim/kafka/logmutants_test.go`. For new work, add a check and the mutant for the check in the same change.

9.10 Each bug has a test that detects the bug first

The table lists each bug, the first layer that the project expects to detect the bug, and the test for the bug:

Bug	First layer expected to detect it	Test
Fetch batch at base offset +1	C1 against `Partition`	`TestConformanceBrokerCatchesMutants`, `TestWrongOffsetMutant`
Corrupted object bytes served	C1 `corruptGets=true`, fetch CRC	`TestConformanceBroker`, `TestCorruptGetRetried`
Ack before `CommitSegment` lands	Broker unit test, CP held	`TestEarlyAckMutant`
Commit replay gets fresh offsets	`ControlPlane` lockstep	`TestControlPlaneCheckShrinksPlantedBug`
Range get ignores `Off`	`ObjectStore` lockstep	`TestObjectStoreCheckShrinksPlantedBug`
Engine completion or state drifts, faults off	C2 + `StateHash` per op	`TestConformanceObjStore`, `TestConformanceCPEngine`, and `TestConformanceLogEngine`
Incremental hash ≠ full fold	Spec unit test	`TestIncrementalStateHashMatchesFull`
Oracle accepts duplicate, dropped, or mis-offset records	C4 fault injection	`TestC4RejectsEveryFault`
Oracle rejects a correct history	C4 acceptance, 200 seeds	`TestC4AcceptsCorrectHistories`
Oracle imports `veritas`	Source-parsing guard	`TestC4OracleDoesNotImportVeritas`

Bug	First layer expected to detect it	Test
Nondeterministic construct or unlisted import	Static analysis	`detcheck` through `scripts/ci-determinism.sh`
Dependency bumped without re-audit	Audit-record check	`scripts/ci-determinism.sh` step 2
Same seed, two runs differ	Determinism canary	`TestDeterminismCanary`
Acked records lost when a broker dies	Hermetic integration	`TestKillBrokerMidProduce`
Real backend differs from fakes	Full-stack integration	`TestFullStackRoundTrip` and `KillBroker`
Complex function loses coverage	CRAP baseline check	`make crap`

9.11 Commands

Command	What it does
`make build`	Full gate, gofmt, vet, build, test with coverprofile, CRAP last
`make test`	`go test -coverprofile=build/cover.out ./...`
`go test ./veritas/ ./veritas/veritastest/`	Spec units, planted-bug shrink tests, test code self-tests
`go test ./broker/ -run TestConformanceBroker -count=1`	C1 (+ small-segments and mutant variants)
`go test ./sim/engines/ -run TestConformance -count=1`	C2, objstore, cpengine, logengine vs specs
`go test ./sim/kafka/ -run C4 -count=1`	C4, accepts correct, rejects every fault
`go test ./it/ -run TestProduceConsumeRoundTrip -count=1`	Hermetic integration, no Docker
`eval "$(scripts/it-up.sh)" && go test ./it/ -run TestFullStack -count=1; scripts/it-down.sh`	Full-stack integration
`./scripts/ci-determinism.sh`	detcheck + audit records + canary
`scripts/audit-dep.sh <module>@<version>`	Start a dependency audit (repo untouched)
`make crap` / `make crap-strict` / `make crap-baseline`	CRAP gate / no-baseline / rewrite baseline (review the diff)

The determinism gate is separate from `make build`. See the `Makefile` header. You can add `./scripts/ci-determinism.sh` before `crap` in the `build` recipe. You can run the gate before you push changes that touch `core/`, `sim/`, or a dependency.

Key takeaways

- `veritas/` holds three sequential specs with no faults. Each spec has a `StateHash`. A reader can argue about each spec line by line.
- Conformance runs the implementation and the spec in lockstep. Conformance compares every completion, and C2 compares every hash. Conformance shrinks a failure to a Go literal that a developer can paste into a test.
- C1 covers the broker. C2 covers the sim engines. C3 covers the `sys` adapters. C4 covers the oracle. A source check keeps C4 from importing `veritas`.
- A check without a mutant is unproven. Add the check and the mutant in the same change.

- Determinism and change risk have gates. The gates are `detcheck`, the audit records, the canary, and the CRAP baseline.

Checkpoint

1. Why must the C4 oracle not import `veritas`, and what enforces it? — Shared code could make a wrong history look correct to both sides. Independence makes C4 a check on the oracle. Without independence, C4 would repeat the same code and agree with the same code. `TestC4OracleDoesNotImportVeritas` (`sim/kafka/c4_test.go`) parses non-test sources. The test fails on any `veritas` import. The external test package may import `veritas`, because `detcheck` skips `_test.go` files.

2. A fetch returns batches at offsets 5 then 7. Which check names that, and how? — C1 in `broker/conformance_test.go` detects the gap. Fetched batches must be contiguous. Each base must equal the next expected offset. The test compares values against `Partition.Read`, and the test compares the watermark against the spec. The gap fails even if no client-visible value differed.

3. A `sim/` dependency bump fails `ci-determinism.sh` step 2. What do you do, and never do? — Run a new audit. Run `scripts/audit-dep.sh <module>@<new-version>`. Follow `.claude/skills/audit-go-dependency/SKILL.md`. Write `docs/dep-audits/<module>@<new-version>.md` with findings that name a file and a line. Run the gate again. Do not weaken `detcheck`. Do not delete an allow-list check. Do not suppress a finding without `//detcheck:ok <reason>`.

4. `make crap` says a baselined function "got worse". Meaning and options? — The CRAP of that function rose by more than 0.5 above `crap-baseline.json`. The function has more branches, less coverage, or both. The preferred fixes are tests or a split of the function. Run `make crap-baseline` if you accept the debt, and give a reason for the diff in the PR.

5. Why does C2 run with faults off, and what covers faults-on? — C2 makes one claim. With faults at zero, completions and `StateHash` match the spec after every operation. Readers can interpret faulty runs after readers know that fault-free runs match. Simulator invariants cover behaviour with faults on. C4 covers faulty histories. Mutants record the first seed that detects each fault.

Try it

1. Run `go test ./veritas/ -run 'TestObjectStoreCheckShrinksPlantedBug|TestControlPlaneCheckShrinksPlantedBug' -v -count=1`. The shrunk sequences have two ops.
2. Run `go test ./broker/ -run TestConformanceBroker -count=1`. Run `go test ./sim/engines/ -run TestConformance -count=1`. Run `go test ./broker/ -run TestConformanceBrokerCatchesMutants -count=1`.
3. Run `./scripts/ci-determinism.sh`. Find a `//detcheck:ok` (start at `veritas/hash.go`) and judge whether its reason suffices.
4. Run `make crap-strict`. For the worst function, compute the coverage it needs to reach CRAP 30. Compute the complexity cut as the other option. Which fix would you choose for a PR?

5. Run `eval "$(scripts/it-up.sh)"`. Run `go test ./it/ -run TestFullStackRoundTrip -count=1`. Run `scripts/it-down.sh`. Compare what the full-stack run proves with what the hermetic round trip proves.

Further reading

- `veritas/doc.go`. `veritas/doc.go` states the four checks.
- `docs/crap-gate.md`. `docs/crap-gate.md` covers CRAP sources, the threshold rationale, and the baseline.
- `docs/dep-audits/README.md`. `docs/dep-audits/README.md` gives the audit template. `tools/detcheck/allow.txt` gives the allow-list.
- `.claude/skills/audit-go-dependency/SKILL.md`. `.claude/skills/audit-go-dependency/SKILL.md` gives the full audit workflow.
- `docs/decisions.md`. `docs/decisions.md` records decisions 21 (specs), 40 (CRAP), 41 (no idempotent producers), and I (audits).
- `docs/lessons.md` lesson 12. `docs/lessons.md` lesson 12 lists the determinism rules that `detcheck` enforces.
- Earlier chapters in `docs/onboarding/`. Earlier chapters cover the broker, the CP, and the simulator that these checks test.

Chapter 10: Deployment, benchmarking, and observability on AWS

Previous chapters ran `manibroker` and `manicp` as processes. This chapter answers one question. How do you put the system on AWS hosts that you can delete, run load at a stated rate, and record what the system did? The chapter keeps three concerns apart.

Concern	Code	Main document
Local test backends	`deploy/docker-compose.yml`, `scripts/it-up.sh`	`docs/decisions.md` row 24
AWS stack that you can delete	`deploy/aws/terraform/`	`deploy/aws/terraform/README.md`
Benchmarks	`deploy/aws/bench/`, `cmd/manibench`, `cmd/ddbprobe`	`docs/aws-benchmark-runbook.md`
Observability	`deploy/aws/observability/`, `/metrics` + JSON logs	`docs/observability.md`

Most numbers in this chapter come from short runs with one sample. This chapter names the source for each number. Read each number as a guide unless the source document says more. `docs/w4-log-store-results.md` opens by calling its 1-4 minute, no-warm-up stages "indicative, not a steady-state characterisation".

1. Local deployment runs test backends only

`deploy/docker-compose.yml` does not run manifester. The file runs two backends that use real services. The backends bind to loopback. `scripts/it-up.sh` and `it-down.sh` wrap Compose.

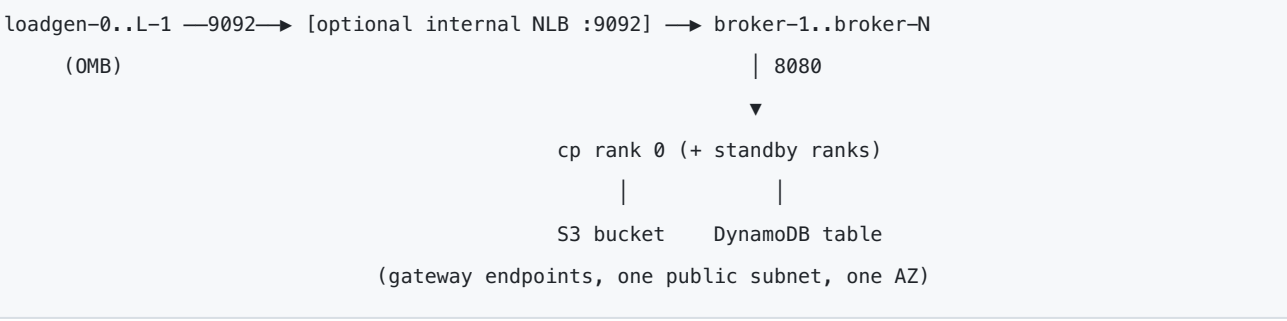
- `extenddb` (`extenddb/extenddb-dev:latest`) provides a DynamoDB-compatible endpoint over in-memory SQLite on `127.0.0.1:18080`. Each run starts empty. The endpoint uses AWS's example credentials as seed data.
- `seaweedfs` (`chrislusf/seaweedfs:latest`) provides an S3 gateway on `127.0.0.1:8333` for the object-store adapter. The CP does not use the gateway.

Local benchmarks in `docs/benchmarks.md` start `manicp` (`:8080`), `manibroker` (`:9092`), and `cmd/manibench` by hand. The benchmarks use setup A (`mem/mem`), setup B (ExtendDB and memory), and setup C (ExtendDB and SeaweedFS). That document warns that ExtendDB and SeaweedFS latencies do not predict real DynamoDB or S3 latencies. Use the ratios and the latency structure from setups A, B, and C. Do not use the absolute values.

2. AWS topology in `deploy/aws/terraform/` puts each part on its own hosts

The Terraform stack is a benchmark stack that you can delete. WarpStream published a comparable benchmark. The stack is not production. The reference is `deploy/aws/terraform/README.md`. Read

`main.tf`, `ec2.tf`, `nlb.tf`, `storage.tf`, `iam.tf`, `variables.tf`, and `outputs.tf` in that order. `main.tf` covers the VPC, the security groups, and the endpoints.



The list below gives the defaults from `variables.tf`:

- Brokers use `aws_instance.broker[0..N-1]`. The default `broker_count=6` accepts 1-12. Brokers use `m6in.4xlarge`. Each broker runs `manibroker` (`:9092`, metrics `:9100`), a 10 s `/metrics` snapshotter, and Alloy. The broker id equals index+1. Brokers have no registration and no fencing. Each broker advertises its own private IP. Each broker waits for CP `/healthz`. User-data limits the wait to 10 min.
- The control plane uses `aws_instance.cp` rank 0 on `m6in.xlarge`. The control plane also uses `aws_instance.cp_standby` with `cp_standby_count=1`. Each CP host runs `manicp` (`:8080`, metrics `:9101`). No broker runs `manicp`. `bench-ctl.sh check-cp` verifies that placement. The default `cp_store=log` makes rank 0 the preferred appender with `-store dynamolog`. A standby tails the log. A standby serves reads. A standby answers commits with 503 until takeover with `-takeover-after 5s`. The `cp_store=dynamo` setting is the rollback. The `cp_store=dynamo` setting uses the old single-sequencer transaction store on one host. The `cp_store=dynamo` setting has no standby. Terraform forces brokers to `-max-segment-partitions 96` under that setting because the transaction limit is 100 items.
- Load generators use `loadgen_count=2` on `m6in.8xlarge`. Load generators run Java 17. Load generators have Kafka 3.8 CLI in `/opt/kafka`. Load generators have OMB in `/opt/omb`. Load generators have the bench code in `/opt/bench`. `/etc/profile.d/bench.sh` exports `BOOTSTRAP`, `BOOTSTRAP_NLB`, `BROKER_IPS`, `LOADGEN_INDEX`, `RUN_ID`, `BUCKET`, `TABLE`, and `REGION`.
- The NLB uses `use_nlb=true`. The NLB is an internal TCP load balancer on `:9092`. The NLB uses instance targets. The NLB has cross-zone load balancing off. The NLB spreads connections. The NLB never spreads requests. A broker Metadata response names that broker alone. Clients stay on the bootstrap broker. Skew with few connections is expected. The `balanced` mode in section 5 reduces that skew. The NLB does not reduce that skew.
- Storage uses one S3 run bucket. The bucket uses `force_destroy`, SSE-S3, and whole-bucket request metrics. Objects expire after `bucket_expire_days=3`. Storage also uses one DynamoDB table named `manifester_<run_id>_cp`. The table uses `pk` and `sk`. The table uses on-demand capacity. The table sets `warm_throughput` to `30k/30k units/s`. Terraform creates the table. `manicp` runs without `-create-tables`. `manicp` lacks that permission. Gateway VPC endpoints reach both stores. Traffic stays off the internet.
- The stack limits access and cost with set rules. The stack uses VPC `10.42.0.0/16`. The stack uses one public subnet in one AZ. Access uses SSM Session Manager. The stack has no SSH. Hosts require IMDSv2. A budget alarm fires at 80% actual spend and at 100% forecast spend. Hosts end

themselves `max_runtime_minutes=360` after boot. Security groups admit CP `:8080` from the broker security group alone. Security groups admit broker `:9092` from the loadgen security group and from the subnet CIDR for NLB health checks. Loadgen ports admit traffic between loadgens. No metrics ports are open. Alloy scrapes loopback.

A CP host replacement changes the private IP of that host. Brokers carry that IP in user-data as `-cp-url`. Terraform replaces the brokers too. A replacement ends a run in progress. Self-termination ends EC2 instances alone. The NLB, the table, the bucket, and the budget keep billing until `terraform destroy`. Destroy uses `force_destroy` and deletes `results/`. Collect results first under section 6.

Bootstrap contract

```
cd deploy/aws/terraform
cp example.tfvars my.tfvars   # region, az, run_id, owner, budget_*
terraform init -upgrade       # provider ~> 6.13: warm_throughput needs 6.x
terraform plan  -var-file=my.tfvars
terraform apply -var-file=my.tfvars
$(terraform output -raw artifact_upload_command)   # from repo root
```

`scripts/build-and-upload.sh` builds `linux/amd64` `manibroker` and `manicp`. The script uploads `deploy/aws/observability/` without dashboards. The script uploads `deploy/aws/bench/`. The script uploads `build/omb/openmessaging-benchmark.tar.gz` if that file exists. The script writes the `READY` marker last. The marker holds the git sha. User-data proceeds once the marker exists. Key outputs are `kafka_bootstrap`, `kafka_bootstrap_nlb`, `cp_urls`, `bucket`, `dynamodb_table`, and `ssm_start_session_*`.

3. Three load tools run in three places and measure three things

The table below names each tool, the place where the tool runs, and the thing that the tool measures:

Tool	Runs where	Measures
`cmd/manibench`	Laptop, local only	franz-go producer load against one local broker
`cmd/ddbprobe`	EC2 host in the region of the table	Raw DynamoDB operations (X1, X3, and X4, §7)
OMB through `deploy/aws/bench/`	Loadgen hosts	Multi-broker benchmark that readers can compare with the WarpStream benchmark

`cmd/manibench benchmark-producer` uses N independent `kgo.Client` values. The command uses acks=all. The command disables idempotence per decision 41. The command buffers 5000 records per client. The command reports interval latency and summary latency. The runbook says “Never use `cmd/manibench` for the AWS run”. The tool runs on a single machine. That machine shares cores with the broker, the CP, and Docker.

`cmd/ddbprobe` tests DynamoDB operations in isolation. Modes are `put|txn|contend|query|batch`. The tool disables SDK retries. Throttling stays visible. The tool uses open-loop pacing at `-rate`. The tool keeps at

most `-k` requests in flight. The tool prints one JSON summary line per run. The tool tests commit-path assumptions. The tool must run in the region of the table.

Two bench flows share the AWS stack. Part B uses `run-benchmark.sh`. That script drives Kafka's `kafka-producer-perf-test.sh` through cells `c1_1k c8_1k c32_1k c1_100b`. Each cell runs 60 s with 3 reps. The `--smoke` flag runs a 20 s test that must pass before a longer run. Part B covers the case with one broker. Part A uses the OMB flow in the next section.

4. OMB needs four patches to run this broker

The unmodified OpenMessaging Benchmark cannot send load to this broker. The broker has no CreateTopics. The broker has no consumer groups. `scripts/build-omb.sh` builds a pinned OMB commit. The tarball is about 29 MB. Four patches live in `omb/patches/`:

- `0001-manifester-fixed-topic-assign.patch` maps each logical topic to one pre-created topic named `maniTopic`. Consumers `assign()` a static partition share. The patch has no assignment shuffle.
- `0002-kafka-driver-only.patch` drops non-Kafka drivers from the build.
- `0003-produce-only-skip-consumer-wait.patch` stops produce-only workloads from waiting on consumers that do not exist.
- `0004-affine-partitioner.patch` adds an opt-in mode. Producer k writes partitions p where `p % N == k % N`. Results from this mode are not comparable with WarpStream results. Label results "partition-affine producers".

`omb/driver-kafka-manifester.yaml.tmpl` uses WarpStream's published client settings. Settings are `linger.ms=25`, `batch.size=100000`, `acks=all`, `enable.idempotence=false` per decision 41, and `compression.type=@@COMPRESSION@@`. `omb/render-driver.sh` substitutes bootstrap, compression, topic, and flags. `run-omb.sh` calls the render script with `bash`. The S3 upload drops the exec bit. Runbook w2 notes record a bug from this drop. The bug ended a run after pre-flight without a clear error.

Workloads live in `omb/workloads/*.yaml`. Each workload uses 1 KiB messages, 288 partitions, 64 producers, and `randomBytesRatio 0.5`.

The table below gives the rate and the shape for each workload file:

File	Rate	Shape
`smoke-100k-produce-only.yaml`	100k messages per second = 97.7 MiB per second	3 min + 1 warm-up, must pass before a longer run
`ramp-{100,250,500,750,1000}k-produce-only.yaml`	100k-1M messages per second	5 min each + 1 warm-up
`vanity-1gibps-produce-only.yaml`	1M messages per second = 976.6 MiB per second	15 min + 2 warm-up, main stage
`vanity-1gibps.yaml`	1M messages per second + 64 consumers	Adds the Fetch path. The team runs this workload later. This workload has the least test history
`w3-*.yaml`, `w4-*.yaml`	100k-1M messages per second	Short (1-4 min) experiment stages. Indicative only

File	Rate	Shape
`later/*.yaml`	varies	Fan-out 3×, 256 producers, 1152 or 4608 partitions. `--include-later` only

With lz4 and half-random payloads, bytes committed to S3 equal about half the OMB payload rate. Use that ratio when you compare client throughput with broker throughput.

5. A benchmark run has seven steps in order

Part A of `docs/aws-benchmark-runbook.md` covers the OMB run. Part B uses the same steps with `run-benchmark.sh` and `broker_count=1`.

The run has seven steps in order:

1. Prepare the workstation. Create Grafana tokens and SSM params under `/manifester-bench/grafana/*` per Part B section 1 of the runbook. Run `scripts/build-omb.sh`. The first build takes about 5 min. You can run `python3 -I -m unittest discover -s deploy/aws/bench -p 'test_*.py'`.
2. Apply Terraform with `broker_count=6` and `loadgen_count=2`. Upload artifacts after apply. Apply early. S3 request metrics take about 15 min to show data.
3. Verify the stack. Run `bench-ctl.sh check-cp`. The check confirms one `manicp` per CP host. The check confirms no `manicp` on brokers. The check confirms that brokers reach the CP. Run `apply-dashboards.sh`. Confirm `up{run_id="<run>"}` in Grafana shows all brokers and `manicp`.
4. Create the topic and start workers. Run `bench-ctl.sh create-topic 288`. The command uses SSM to reach the CP host loopback `manicp` API. OMB cannot create topics. The security groups close loadgen to CP traffic by design. Run `bench-ctl.sh start-workers` next. The command runs `start-omb-workers.sh` on each loadgen. Workers serve HTTP on `:8080` and stats on `:9090`. The first half of workers produce records. The second half of workers consume records.
5. Run smoke first. Run `bench-ctl.sh run --stages smoke`. `run-omb.sh` runs on loadgen 0 in detached mode through `bench-ctl.sh run`. The script runs pre-flight checks before each stage. The checks take a new `metrics/cp/` snapshot from S3. The snapshot proves that the CP lives. Loadgens cannot reach the CP. The checks run ApiVersions probes and topic visibility probes on each broker. The checks run an lz4 probe. The probe uses `none` if brokers reject lz4. That choice prints a loud “not apples-to-apples” warning. Do not continue unless the run prints `SMOKE PASS`. Do not continue unless each item on the runbook checklist passes. The checklist covers per-broker Grafana traffic, `segments_failed` at 0, no throttles, and a CP queue that does not grow.
6. Run stages. Run `bench-ctl.sh run --stages smoke,ramp,vanity-produce` for a run that includes the ramp stages. The default is `smoke,vanity-produce`. The coordinator renders a driver per stage. The coordinator enforces `--max-minutes`. The default is 120. The limit controls cost. The coordinator aborts a stage on any 10 s OMB line that meets a limit. One limit is an error rate above 0.1% of publish rate. One limit is publish p99 above 10 s through `--abort-p99-ms`. One limit is consumer backlog at or above 100k with growth. An abort stops OMB. An abort writes `ABORT` with the reason. An abort uploads results. An abort skips later stages. Each broker host and each CP host pushes `/metrics` to S3 every 10 s. No metrics ports are open. The push copies server data from the hosts to S3.
7. Collect results. Run `collect-results.sh <bucket> <run_id> <region>` next. The script syncs `s3://<bucket>/results/<run_id>/`. The script writes `summary.md`. Run `terraform destroy` after

collection. Run the README post-destroy checklist after destroy. The checklist covers the NLB and the target groups. The NLB costs about $0.03 per hour as a leftover.

Two more commands change a run in progress. `bench-ctl.sh kill-appender 60` sends SIGKILL to rank-0 `manicp` in the middle of a stage. The experiment measures failover. Runbook A11 covers the experiment. `bench-ctl.sh broker-args -segment-window 1s` restarts brokers with new flags. `run-omb.sh --affine` enables patch 0004. `--bootstrap-mode random` shows unmodified-client skew.

6. The results directory starts with `summary.md`

```
results/<run>/
  loadgen/   cells.tsv + <cell>.r<rep>.c<client>.out     (run-benchmark.sh)
  omb/       stages.tsv, run.json, <stage>.json/.log, driver-<stage>.yaml, ABORT?
  metrics/   <broker_id>/<epoch>-broker.txt              every 10 s, per broker
             cp/<epoch>-manicp.txt                       rank 0 (summariser's CP input)
             cp-<rank>/<epoch>-manicp.txt                standby CP nodes (log store)
  markers/   kill-appender-<epoch>.txt                   (failover runs only)
  *-run_id.json manifests, build-git-sha.txt, terraform-outputs.json, summary.md
```

Start with `summary.md`. `deploy/aws/bench/summarize.py` uses the Python standard library alone. Tests live in `test_summarize.py`. Fixtures live in `deploy/aws/bench/testdata/`. The fixtures include a failover fixture. The script selects snapshots around each cell or stage. The script picks the snapshot at or before start. The script picks the snapshot at or after end. Averages are exact. The window can include up to one 10 s idle interval per side. For OMB the script reports target rate and achieved rate in messages per second and MiB per second. The script reports publish percentiles and end-to-end percentiles. The script reports backlog. The script reports per-broker deltas. The script reports skew. The script reports cost per GiB. Prices come from `SUMMARIZE_ARGS='--broker-hourly ...'`. The script prints an apples-to-apples checklist. Read the checklist before you quote any number. Failover runs use the `markers/` marker. Failover runs use `cp/` and `cp-<rank>/` snapshots. The script reconstructs the active CP node from those inputs.

Two known errors need a check by hand. After failover the active node numbers sit in `metrics/cp-1/`. `summarize.py` reads `metrics/cp/` alone. Reconcile the numbers by hand. In w4 a second `run-omb.sh` call overwrote `omb/stages.tsv` and `run.json`. The w4 `summary.md` covers the kill-test stage alone. The per-stage `<stage>.json` files hold the correct record in all runs.

7. Past runs report short stages, and most runs have one sample

The runs used short stages. Most runs have one sample. Do not quote a headline without the caveat column.

The table below gives the setup, the headline, and the caveat for each run:

Run	Setup (per source)	Headline	Caveat
Local (`docs/benchmarks.md`)	1 broker, M1 laptop, 30 s cells	The commit stage reaches its limit before the S3 PUT stage. PUT calls overlap	Pre-decisions 41-44. Docker backends
r1 (`docs/aws-benchmark-results-r1.md`)	1 broker AWS, cells ×3	Use the cell table as history only	The source says “not comparable” with current build
w2 (`docs/w2-bottleneck-verification.md`)	6 brokers, transaction store, 288 partitions	The DynamoDB transaction loop of the sequencer stayed about 100% busy at 500k. Each transaction took about 0.7 ms per item. The 250k stage finished below the limit, and that stage did not find the plateau	500k lasted about 45 s. A ramp, not steady state
w3 (`docs/w3-affinity-and-window-experiments.md`)	Same stack, 2 experiments	Affine producers held 500k and 750k, and disjoint partitions let groups overlap. A 1 s window did not help, and the 96-partition cap seals first	Affine producers are a benchmark-only setting, not comparable. Stages lasted minutes
w4 (`docs/w4-log-store-results.md`)	Log store, random clients	The 500k, 750k, and 1M stages passed produce-only with p99 below 1 s. Append p99 stayed about 8 ms. The kill-appender ack gap stayed about 8 s with zero errors	1-4 min, no warm-up, one sample each
X1, X3, and X4 (`docs/x1-x3-x4-results.md`)	`ddbprobe`, 1 EC2 host, throwaway tables	PutItem p50 stayed about 3 ms, and the result does not support the 15 ms pessimistic assumption. A hot key throttles at about 200 entries per second for items above 4 KB. A cold on-demand table throttles at about 4k WRU per second	45-120 s points, one sample, no p99.9 claimed

The w2, w3, and w4 sequence changed one thing at a time. The limit sat in one big transaction per commit. The limit then sat in transactions with a cost per partition touched. The limit then sat in one small PutItem per log slot. W4 found no component near saturation at 1M after the last move. The load generator was the closest limit in w4. Each step rests on one run. The source documents state that limit.

8. Observability puts `run_id` on each metric series and each log line

`docs/observability.md` is the reference.

The list below gives the metric, log, and manifest rules:

- `manibroker` serves metrics on `:9100`. `manicp` serves metrics on `:9101`. Paths are `/metrics` and `/healthz`. Metric names use the `manibroker_` and `manicp_` prefixes. The broker key metric decomposes seal-to-ack. Parts are open age, PUT, queue wait, and commit. An example name is `manibroker_segment_seal_to_ack_seconds`. The CP exposes `manicp_seq_*` under the transaction

store. The CP exposes `manicp_log_*` under the log store. The live family tells you the store. The other family is absent.
- Logs are JSON. Each line goes to stderr. Each line carries `time`, `level`, `msg`, `run_id`, `service`, `git_sha`, and `broker_id`. Filter on the stable `event` field. Values are `segment_sealed`, `segment_committed`, `cp_summary`, and `log_role`. The broker writes about 3 lines per segment. `cp_call` is DEBUG alone. A consume stage reaches tens of thousands of requests per second.
- Manifests come from `-manifest-out`. The flag writes one startup JSON per process. The JSON holds flags, resolved tuning, git sha, and EC2 identity. The target is a file or `s3://…/results/<run_id>/`. The 10 s snapshots and the manifest reconstruct a run after destroy.

Alloy sends host data to Grafana Cloud from `deploy/aws/observability/`

The stack has no self-hosted Prometheus. Grafana Alloy 1.10.1 runs on each host. `install-alloy.sh` installs Alloy. Credentials come from SSM Parameter Store. The repo holds no credential. The env file is `/etc/alloy/alloy.env` with mode 0600. `alloy/config.alloy` is common to all hosts. The common config covers remote-write, the embedded unix exporter, and journald to Loki. The common config sets the external label `run_id`. One role file covers each host type.

The list below gives the role file for each host type:

- `alloy/broker.alloy` scrapes `manibroker :9100`. The file adds the `broker` label. The file adds the embedded CloudWatch and YACE exporter for S3 request metrics.
- `alloy/cp.alloy` scrapes `manicp :9101`. The file adds the `cp_rank` label. `alloy/cpcloudwatch.alloy` covers DynamoDB CloudWatch. Rank 0 alone runs the CloudWatch file. The split avoids duplicate series.
- `alloy/loadgen.alloy` scrapes the WarpStream bench tool on `:8081` when that tool runs. OMB runs have no such tool. `up{job="warpstream_bench"}=0` is expected in OMB runs.

App scrapes run every 5 s. CloudWatch scrapes run every 60 s. Grafana Cloud free retention is 14 days. Export data that you need before the window ends.

Dashboards need a check against each hypothesis

Four generated dashboards live in `deploy/aws/observability/dashboards/`. Names are `overview`, `pipeline`, `aws`, and `hosts`. Edit `gen/gen_dashboards.py`. Run the generator. Run `apply-dashboards.sh`. Do not edit the JSON. Overview covers throughput and skew. Overview has one CP row per store. Pipeline covers PUT, queue, commit, and seal-to-ack. AWS covers broker-side SDK metrics and CloudWatch metrics. Hosts covers CPU, memory, NIC, and process data.

`docs/dashboard-vs-hypotheses.md` shows a method for dashboard use. The document decomposes the w4 narrative into ten hypotheses. The hypotheses are H1-H10. The document checks each hypothesis against the panel queries in the dashboards.

The list below groups the ten hypotheses by the part of the system that each group tests:

- H1-H3 concern the old transaction bottleneck. The audit confirms that the bottleneck is gone. Log append stays flat as entry size grows. Lost slots and hole fills are 0. Append p99 is about 8 ms.

- H4-H5 concern segment size and S3 PUT cost. Segments become size-bound above about 500k. S3 PUT latency grows with object size at about 10 ms per MiB. S3 PUT latency is not the throughput limit. The audit confirms H4-H5. Several printed p99 values are histogram bucket edges. Values are 254 ms and 318 ms. The values are not measured tails. Use the mean. Use the fraction above a bucket.
- H6-H7 concern index lag and the failover ack gap. Index lag is about 13 s at 1M. The failover ack gap is about 8 s. The audit confirms both effects. The panels in the dashboard read the wrong series for these effects. One panel reads standby lag. One panel uses 1 m windows. A 1 m window dilutes a 10 s stall. The effect held. The cited panel did not support the effect.
- H8-H9 concern load generator limits. The load generator is nearest saturation at 1M. Per-broker NIC use is about 81 MB per second as a mean. The audit confirms H8-H9.
- H10 concerns CloudWatch S3 series. All CloudWatch `aws_s3_*` series read 0 for the whole w4 run. The w2 run shows the same zeros. The document gives a likely cause. YACE uses a scrape window. S3 request metrics arrive with delay. `nil_to_zero` turns no data into 0. H10 stays open. Broker-side `manibroker_s3_*` series carry the same information.

The document lists 18 dashboard defects. Examples include dead transaction-store panels under the log store. Examples also include a role-change counter that `increase()` cannot see. Examples also include legends that use host IP in place of `cp_rank`. Use that list as the list of dashboard work that remains. A panel that draws data does not prove a claim. Check the query against the hypothesis.

9. Cost ends with destroy, and failure depends on the store

The Terraform README prices the default stack at an estimated $12 per hour in fixed costs in ap-south-2. Brokers cost about $7.03 per hour for 6 brokers. Load generators cost about $4.69 per hour for 2 loadgens. The CP costs about $0.23 per hour. The NLB costs about $0.03 per hour. Usage costs add S3 PUTs. DynamoDB write units form the largest usage cost at high load. The README worked example sets a 4-hour run at about $48 in fixed costs. The example sets the worst case at up to about $160 at a sustained 1 GiB per second. The w4 run lasted about 37 min. The w4 document estimates $10-13 in total for that run. The figures are list-price estimates from the documents. The figures are not bills. Set `budget_usd` and `budget_email` with true values. Respect `run-omb.sh --max-minutes`.

Failure behaviour differs by store. The runbook treats the stores in separate sections.

`cp_store=dynamo` is case A10. One CP is a single point of failure for availability. One CP is not a single point of failure for correctness. DynamoDB compare-and-set keeps the store correct. Brokers retry commits under the same key until `-commit-deadline`. The default deadline is 120 s. A CP restart of a few seconds fails no produce request. A restart of a few seconds delays acks. A longer gap fails in-flight produces with `REQUEST_TIMED_OUT`. Producers that retry can duplicate records. Duplicates arise when a lost commit reached the store.

`cp_store=log` is case A11. Loss of the standby is invisible to clients. Loss of rank 0 stalls acks. The standby promotes after `-takeover-after` at 5 s. Promotion needs a tail pass after the wait. W4 measured about 8 s from loss to ack flow. Brokers replay commits by segment key after promotion. Replay creates no duplicates. Replay loses no acked data. Loss of both CP nodes degrades the system to the A10

behaviour. Verify roles with `bench-ctl.sh cp-roles`. The command reads `X-CP-Role: active|standby`. Watch the Overview “Control plane (log store)” row during `kill-appender`.

Keep `terraform.tfstate` and `my.tfvars` until the post-destroy checklist prints empty results for each query. Queries cover tagged resources, instances, ENIs, volumes, the VPC, the bucket, the table, IAM, the budget, and the NLB family. Loss of state orphans billable resources. That failure stays invisible without the checklist.

Key takeaways

- Compose provides test backends alone. `deploy/aws/terraform/` defines a benchmark topology that you can delete. The topology uses 6 brokers, a ranked log-store CP with a hot standby, 2 OMB loadgens, and an optional NLB. SSM access, a budget alarm, and self-terminating hosts guard the stack.
- Three load tools have three jobs. `manibench` covers local runs. `ddbprobe` covers DynamoDB assumptions. Patched OMB covers AWS runs. The wrong tool invalidates comparisons. Affine producers invalidate comparisons.
- A run follows steps in order. Build OMB. Apply Terraform. Upload artifacts and write `READY` last. Verify the stack. Create the topic through the CP. Run smoke. Run stages under abort rules. Run `collect-results.sh`. Run destroy. Collect results before destroy. The bucket uses force-destroy. Objects expire in 3 days.
- Read `results/<run>/summary.md` first. Read the raw `<stage>.json` file when the summary and the stage file disagree. Read the apples-to-apples checklist before you quote a number.
- Each headline number comes from a short run. Most runs have one sample. Each source labels its numbers indicative. The w2 to w4 arc shows a bottleneck move. The arc is not a capacity promise.
- Observability uses Alloy to Grafana Cloud. Each series and log line carries `run_id`. S3 snapshots support offline reconstruction after destroy. `docs/dashboard-vs-hypotheses.md` sets the standard of proof with H1-H10. Verify each query. Distrust bucket-edge p99 values. Treat CloudWatch S3 series as broken until a fix closes H10.

Checkpoint

1. Why does topic creation go through SSM to the CP host instead of from a loadgen or OMB? — The broker has no CreateTopics and no consumer groups. OMB cannot create topics. The CP security group admits `:8080` from the broker security group alone. Loadgen to CP traffic is closed. The NLB forwards Kafka traffic alone. Loadgens have no `ssm:SendCommand` permission. `bench-ctl.sh create-topic` sends a POST to the `manicp` loopback API on the CP host through SSM. Patch 0001 maps each OMB logical topic onto that pre-created topic.

2. One broker carries twice another’s load. Is the NLB broken? — No. The internal NLB spreads connections with a flow hash. The NLB never spreads requests. A broker Metadata response names that broker alone. Clients stay on the bootstrap broker. Skew is expected with few connections. The bench code defaults to `--bootstrap-mode balanced`. The mode pins producer k to broker k % N. `--bootstrap-mode random` shows unmodified-client skew.

3. You destroyed the stack, then remembered you never collected results. What survives? — The stack deletes most data by design. `force_destroy` deletes `results/`. The deleted data covers snapshots, OMB JSON files, and manifests. Objects face expiry after 3 days in any case. Old offsets fail with "committed segment missing" after expiry. Grafana Cloud data survives. Free retention is 14 days. A local `results/<run>/` copy from an earlier `collect-results.sh` run survives. Write `summary.md` to local disk first. Destroy the stack second.

4. In w3, why did affine producers fix 500k but a 1 s segment window did not? — W2 gave the diagnosis. Each segment touched many of 288 partitions. Each sequencer transaction carried about 86 items. Each transaction took about 86 ms. Groups could not overlap. The single loop saturated. Affine producers cut the count to about 48 partitions and about 49 items per transaction. Each transaction took about 48 ms. Disjoint partition sets let up to 4 groups overlap. Loads at 500k and 750k held. A longer window changed no binding limit. Segments hit the 96-partition cap before the timer. Partition seals numbered 1161. Timer seals numbered 71. Transactions grew in size under the longer window. The stages ran for minutes. Each stage had one sample.

5. A panel shows S3 p99 = 254 ms at every load and CloudWatch S3 latency = 0. S3 problem? — Treat the panel as a measurement problem first. H5 and H10 in `docs/dashboard-vs-hypotheses.md` cover the two values. The 254 ms value is a histogram bucket top. About 93% of attempts fall in the 128-256 ms bucket. The bucket bounds the true p99. The bucket does not measure the true p99. Use the mean. Use the fraction above a bucket. The zeros come from the H10 defect. The exporter window and delayed S3 request metrics cause the defect. `nil_to_zero` renders missing data as 0. W2 and w4 show the zeros. Broker-side `manibroker_s3_*` series are trustworthy. The series show about 10 ms per MiB of scaling with object size. The scaling is expected. The scaling is not the throughput limit.

Try it

The list below gives five exercises that you can run without a full AWS benchmark:

- Run a local smoke test without AWS. Run `scripts/it-up.sh`. Start `manicp` and `manibroker` as in `docs/benchmarks.md`. Run `manibench benchmark-producer --topic bench --create-partitions 6 --num-clients 1 --duration 2s`. Run a 30 s run after the short run. Scrape `:9100` and `:9101` before the runs. Scrape the ports after the runs. Find `manibroker_segment_seal_to_ack_seconds`.
- Run the summariser test suite. Run `python3 -I -m unittest discover -s deploy/aws/bench -p 'test_*.py'`. Run `summarize.py` against fixtures in `deploy/aws/bench/testdata/`. Read the failover section in the output.
- Regenerate a dashboard. Change directory to `deploy/aws/observability/`. Run `python3 gen/gen_dashboards.py`. Check that `git diff` touches the JSON files that you intend to change. Run `apply-dashboards.sh --dry-run`.
- Audit one hypothesis. Pick H6 or H7 in `docs/dashboard-vs-hypotheses.md`. Write down the panel for each row. Write down the PromQL for each row. Write down one reason that a naive reading misleads for each row.
- Plan a stack. Do not apply the plan. Copy `example.tfvars` to `my.tfvars`. Run `terraform init -upgrade && terraform plan -var-file=my.tfvars`. Locate the NLB in the plan. Locate the standby CP in the plan. Locate DynamoDB warm throughput in the plan. Locate the bucket lifecycle in the plan. Note the resources that `destroy` deletes. Stop at the plan unless you own the budget.

Further reading

The list below gives the document to read for each part of the system:

- `deploy/aws/terraform/README.md`. The README covers variables, outputs, the artifact contract, cost, and the destroy and post-destroy checklist.
- `docs/aws-benchmark-runbook.md`. The runbook covers Part A with OMB, Part B with the single-broker smoke test, CP failure modes A10 and A11, and w2 and w3 notes.
- `docs/aws-benchmark-warpstream-comparison.md`. The document states the claims that numbers can and cannot support against WarpStream's published benchmark.
- `docs/benchmarks.md`. The document covers local `manibench` measurements. The document covers the commit-path analysis from those measurements.
- `docs/w2-bottleneck-verification.md`, `docs/w3-affinity-and-window-experiments.md`, `docs/w4-log-store-results.md`. These documents hold the AWS results behind section 7. Read the caveats before the tables.
- `docs/x1-x3-x4-results.md`. The document covers `ddbprobe` experiments. The experiments cover PutItem cost, the hot-key ceiling, and cold-table throttling.
- `docs/observability.md`. The document is the metric, log, and manifest reference. `deploy/aws/observability/README.md` covers the Alloy layout, SSM secrets, and the YACE rationale.
- `docs/dashboard-vs-hypotheses.md`. The document covers the H1-H10 audit. The document lists the dashboard defect backlog.

The design changed its mind in public, and two verbs tell you what still holds

You can understand the code without its history. You cannot change the code in safety without that history. You need to know which ideas the team tried, measured, superseded, or left as a proposal.

Two verbs run this file. Supersede and refine.

A later decision supersedes an earlier decision when the later decision removes the earlier one. You must not implement the earlier decision. A later decision refines an earlier decision when the later decision narrows the earlier one. The earlier decision stands except where the later decision narrows it.

Keeping superseded decisions in the repo is right. A superseded decision records the constraint that forced the turn. The new decision alone does not record that constraint. The cost is small. You check for “Superseded” and “Refines” before you quote a row.

`docs/` holds records with different authority. A decision is not a benchmark result. A proposal is not a plan. A dated audit is not a description of today’s code.

1. A later decision has more authority than the earlier text it names

If you need a rule, you read in this order:

- For a rule that must never change, read `docs/decisions.md` first. Read the topic doc after the decision. The topic docs include `docs/log-cp-service.md` and `docs/flow-control.md`.
- For a measured claim, read the results docs. Read the measured, modelled, and computed labels in `docs/surprises-audit.md`.
- For work that the team accepted but did not build, read `docs/TODO.md`. Read only `docs/TODO.md` for that question.
- For an idea only, check the headers. The headers say PROPOSAL, IDEAS, or “proposal only, nothing applied”. The headers appear in `docs/commit-path-redesign.md`, `docs/delos-logdrive-ideas.md`, and `docs/refactoring-proposal.md`.
- For AWS, treat the plan as intent. Treat the results as fact. Compare `docs/stage5-run-plan.md` with `docs/w4-log-store-results.md`.

A later decision has more authority than the earlier text it names. The later decision says “Supersedes”, “Refines”, or “Under 41-44”. The earlier text stays for its reasoning. Dates and `Status:` lines matter more than confident prose.

2. A row in `docs/decisions.md` holds only where no later row limits it

`docs/decisions.md` is one table. The table has rows numbered 1-53. The table also has lettered rows A, C, D, F-J, and S from the earliest sessions. Rows 7-22 came from building the predecessor system toys3. `docs/interface-changes-from-toys3.md` expands the evidence for rows 7-22.

You read a row in this way:

- The row is terse by design. The justification lives in the topic doc or commit that the row cites. Follow those references before you argue with the row.
- “Accepted” means the decision is current. Row 28 says “Superseded by 41 and 44”. Row 34 says “Superseded by 44”. You must not implement from row 28 or row 34.
- “Refines” means the earlier row stands except where the later row narrows it. Row 48 refines 46. Row 50 refines 39 and 42. Row 53 refines 48 and 51.
- Rows 35 and 39 carry notes that say “Under 41-44 …”. The notes redo old arithmetic after a contract change. The original numbers are historical.
- Some rows record staged acceptance. Row 52 sets the partition cap at 4096. Row 52 says that no hash changes in the stage that records it. Row 52 also says the work was “Done in stage 3”. Acceptance and landing are different events.

The ten decisions that set the current architecture

Read these ten rows in full. Together the ten rows describe the current architecture. Most other rows refine these ten rows.

#	Decision, in one line	Why the decision sets the architecture
I	Deterministic packages `core/` and `sim/` use an exact-path import allow-list, fail closed, and `tools/detcheck` enforces the list. A third-party module needs a record in `docs/dep-audits/`.	The simulator can act as the oracle because decision I holds. Every dependency question starts here.
21	`veritas/` holds executable specs. The specs double as test fakes. Conformance C1-C4 checks the specs. The oracle never imports veritas.	Two independent implementations agree. That agreement is the main correctness argument in the project.
35	The segment flush rule seals at 16 MiB or a 200 ms fixed window. Partition, batch, and item caps also seal a segment. `sys/prod` derives the buffer cap from memory at runtime.	Decision 35 sizes the whole produce path. Later rows 39 and 52 only move the caps.
41	The system has no idempotent producers. Delivery is at-least-once. Clients must disable idempotence. A retried commit cannot duplicate. A retried produce can duplicate.	Decision 41 supersedes the producer half of decision 28. A case for producer dedupe reopens decision 41.
42	The system has no high-watermark item. The CP derives the next offset for a partition from its newest run `[Base, Base+Count)`. The CP assigns offsets itself. Compare-and-set is a store detail.	The invariant “contiguous, never reused” rests on decision 42. The invariant does not rest on broker honesty.
44	The system has no broker fencing. Brokers are stateless. A zombie broker cannot harm the system through a commit because decision 42 assigns the offsets.	Decision 44 supersedes decision 34 and the epoch parts of decision 28. Decision 44 deletes `Register`, epochs, and the fencing `ErrFenced`.

#	Decision, in one line	Why the decision sets the architecture
45	The broker stores and serves compression as opaque bytes. The broker validates the header only. The header checks cover CRC and codec id 0-4. The broker never decompresses.	Decision 45 keeps codec libraries out of the deterministic packages. Decision I still holds.
48	Design A is the production design. A DynamoDB log holds raw `CommitSegment` commands. Replicas apply the commands deterministically. All commits go to one CP node. Sticky routing sends the commits. K=4 appends stay in flight.	Decision 48 is the current control plane. Decision 48 refines decision 46. Decision 48 replaces the per-partition run-item CAS with the if-absent write on the log slot.
49	The production state machine is not veritas. The production state machine is an embedded-KV applier in `cp/applier`. Pebble backs the applier. Lockstep tests check the applier against veritas. The tests use 100,000 seeds in `docs/applier-bakeoff.md`.	Decision 49 keeps the spec separate and the production store fast. Veritas keeps every run forever. Veritas would use about 6 GB per hour at the target rate. Veritas cannot act as the store for that reason.
53	Log entries start with a format-version byte. An undecodable entry stops a node. The node returns 503 and never skips the entry. The idempotency window is a function of slot number. `WindowSlots` is 131,072. The window is identical on all replicas. Trimming leaves tombstones.	Decision 53 gives the upgrade story. Decision 53 also explains why replicas cannot fork when the system forgets a key.

Other rows still matter. Row 3 sets a separate CP that assigns offsets. Row 3 also makes commit idempotent per segment key. Row 29 sets leaderless metadata. Row 46 sets a single sequencer. Decision 48 refines row 46. Row 51 sets a hot standby. Row 51 says the log slot is the fence. Row 51 says a lease is not the fence.

3. Measurements forced the large turns, and the dates show the order

The repository has a compressed recorded history. About three weeks of commits carry two systems. The commits carry toys3 first and manifester after toys3. The phases are approximate.

The timeline has eight dated steps:

When	What happened	Evidence
2026-10-05	The team built the core interface skeleton, the determinism gate, dependency-audit tooling, and Go 1.24.	commits `1f762c6`, `011144f`
2026-10-12-15	The team built toys3 end to end as a learning system. The system had a simulator, an oracle, veritas specs, and mutants. The team added a porcupine cross-check and then reverted the cross-check in `9cc93a1`.	toys3 commits and `docs/interface-changes-from-toys3.md`

When	What happened	Evidence
2026-10-16	Milestone 1 landed the simulator kernel. The Kafka scenario, the broker, and the CP service followed. The CP service had idempotent producers under decisions 28-31. The team imported decisions 7-22 from toys3.	`17ca3db`, `918eac0`, `176d541`, `af422bd`
2026-10-17-18	The team built production wiring in `sys/`, `cmd/`, and `it/`. The first AWS run r1 followed. Decision 35 set the segment flush rule. Decisions 36-38 set metrics-as-facts. Decision 39 set CP range compaction. Decision 40 set the CRAP gate. The team discussed the DDD proposal.	`44d41e5`, `b418b0b`, `d7e14a9`
2026-10-18	Decisions 41-44 made the stateless turn. Decisions 41-44 removed idempotent producers, the high-watermark item, and broker fencing. Decisions 41-44 marked 28 and 34 superseded. The team wrote the second edition of the refactoring proposal.	`04ca77d`, `22958b1`, `fe9afae`
2026-10-19	Decision 45 added compression. Decision 46 gave the control plane sequencing. The team chose single-CP-host deployment. The team wrote the surprises audit before the next benchmark.	`79f2b21`, `2f75286`, `1a11024`
2026-10-20	Runs w2 and w3 showed the transaction store reaching only about 240-275k msg per second. The commit-path redesign proposed design L. The Delos and LogDrive ideas proposed design A. The team prototyped A in `cp/logsm`. The team measured A in X1-X5. Decisions 48-53 adopted A. The bake-off chose Pebble. The team built the Log CP service in stage 2. The team built the sim log engine in stage 3.	`aab17c0`, `fcd6228`, `bd6b64d`, `5b2b90c`, `ae027ec`, `392d372`, `c301a20`
2026-10-21	The team wired the Log CP as `manicp -store dynamolog` in stage 4. The team wrote the stage-5 plan. Run w4 then executed and the team reported run w4. The team wrote the roadmap and the OSS-readiness doc. The team added sample DynamoDB rows.	`8bd349e`, `d4dd5d5`, `513bc67`, `c6da4c5`, `df0e176`

A measurement forced each large turn. The team built the idempotent version first. Decisions 41-44 came after that build. Run w2 came before decision 48. Several docs describe systems that existed for a day. If you cite a doc, check `git log -- docs/<file>` first.

4. `docs/lessons.md` gives the shortest reason for each odd contract

`docs/lessons.md` adapts the Moonpool Sim Book to this Go codebase. The Moonpool Sim Book draws on FoundationDB, TigerBeetle, and Antithesis practice. The lessons doc gives the shortest explanation of why the contracts look odd.

You will cite these sections most:

- Sections 0-1 cover Go scheduler limits. You cannot swap the scheduler in Go. The Action and Event vocabulary is the provider interface for that reason. The codebase uses sans-I/O state machines for that reason. Every completion carries an `Err` and an `Outcome`. The `Outcome` is `NotApplied`, `MaybeApplied`, or `Applied`. The code acks producers only on `Applied`. `CommitSegment` must be idempotent per segment key for that reason.
- Sections 2-3 cover time and random numbers. Two clocks sit on the event. The clocks are `At` and `Wall`. The core has no `Now()` call. One counted RNG stream exists. Derived streams serve the swarm knobs. Decision 17 superseded the "same stream" wording in §9.
- Sections 5-6 cover crashes and network faults. Crashes resolve pending results one by one. A landed PUT with a lost response stands. Network faults act at connection level. Kafka framing runs inside the deterministic path for that reason.
- Sections 8-11 and 13 cover runs and checks. The sections describe run phases. The phases are chaos, recovery, quiet tail, settle, and check. The sections describe swarm testing that can switch fault families fully off. The sections describe invariants that read a fact timeline. The sections describe control-before-data backpressure. `docs/flow-control.md` expands the backpressure part.

Section 4 is historical. Section 4 covers incarnations and fencing. Section 4 opens with "Moot since decisions 41-44". If broker membership returns, you may revisit section 4. Broker membership is a `TODO.md` item today. Do not design fencing from section 4 today.

5. An S-id names one claim in the audit, and the audit froze on 2026-10-19

The audit is `docs/surprises-audit.md`. The audit is dated 2026-10-19. The team wrote the audit against branch `toys3` at `4854ad7`. The team was still writing the decision-46 sequencer at that time. The audit ranks 19 surprises, S1-S19, by severity. The severity values are BLOCKER, HIGH, MEDIUM, LOW, and KNOWN. Each item has an evidence label. The labels are verified by experiment, verified by reading code, computed, and suspected. Experiments ran on local setups only. Experiments used fakedb and loopback. Section 4 of the audit lists what the audit could not check. The list covers real 97-item transaction latency, regional quotas, and hot-key behaviour.

When someone cites "S3", the person cites one specific claim. A test can prove the claim wrong.

The audit makes these claims:

- S1 says every segment overlaps every other segment at 288 partitions. The single sequencer can neither pipeline nor merge groups for that reason. The bound is one segment per transaction. The bound is about 430-600k msg per second. The bound is not the imagined group rate.

- S3 says Fetch never waits. One idle consumer made about 20.6k CP Lookups per second on a local setup. The audit predicted about 48k per second for 64 consumers. S3 is still open. See `TODO.md`.
- S6 says the code makes one ranged S3 GET per batch. The rate is about 6.5k GET per second at 1 GiB per second read. That rate sits above the per-prefix starting rate. S6 is still open.
- S7 says a 97-item commit costs about 240 WRU. The cost is about $59 per hour at the prices then used. Section 2 of `docs/commit-path-redesign.md` corrects the price basis.
- S8 says a CP outage over about 0.6 s failed every in-flight produce. The code used 6 retries and 20-320 ms backoff. Decision 47 fixed S8. Decision 47 changed the code to retry until a 120 s deadline.
- S13 says the system never deletes anything. The 3-day bucket lifecycle can delete `seg/` objects that the index still points to. S13 is still open. S13 is roadmap item 2.
- S19 is marked KNOWN. S19 says the system has no consumer groups. S19 says the system has no transactions, SASL, TLS, or retention. S19 lists those absences as known facts.

Status varies by id. The team fixed S2 and S4 within days. The fixes were sequencer wiring and `sys/httppool` at 256 idle connections under decision 47. Other items ended without a direct fix. The team declined the partition affinity that S1 proposed. See `docs/TODO.md`. The log control plane in decisions 48-53 then removed the problem. Run w4 measured that system. Pair an S-id with `docs/TODO.md` and the decisions at all times.

6. Design A superseded design L, and the three debate docs keep distinct jobs

This debate shows the layers of the library in one place. Learn to tell the three documents apart.

The three documents have distinct jobs:

- `docs/commit-path-redesign.md` is a proposal dated 2026-10-20. The proposal starts from w2. Run w2 showed transaction latency at about 8 + 0.7 ms per item. Run w2 showed one transaction in flight. The plateau was about 240k msg per second. The proposal rejects C and keeps B as fallback. The proposal recommends design L. Design L used log-slot CAS in a small transaction. Design L used `G#` items that held computed bases, an asynchronous index, and active-only `Lookup`. The team did not build L as proposed.
- `docs/delos-logdrive-ideas.md` holds ideas from the same day. The doc reads Delos from OSDI 2020 against design L. The doc reads LogDrive and Conflux from OSDI 2026 against design L. Idea A logs the commands and not the results. Every replica is a learner. Offsets exist only at apply time. Any replica can serve a fresh `Lookup`. The design needs no permanent `G#` items. Idea B moves the index out of DynamoDB into S3 checkpoints. Idea B is now the unscheduled S3-index item in `docs/TODO.md`. Ideas C and D cover the Loglet seam and striping. Ideas C and D are insurance. The team did not build ideas C and D.
- `docs/design-a-prototype.md` prototyped A in `cp/logsm`. Test X2 ran 100,000 seeded interleavings with 0 failures. Decisions 48-53 adopted A. Adoption carried the modifications in `docs/log-cp-service.md`. The modifications appear under “Findings that change earlier text”. The modifications are a deterministic slot-based idempotency window and tombstone trimming. Seeded tests found both points.

Parts of design L survive in the built system. The asynchronous `R#` index survives. The idea of a log slot as the CAS survives. If a paragraph in the redesign doc conflicts with decisions 48-53, decisions 48-53 have more authority. If a paragraph conflicts with `docs/log-cp-service.md`, `docs/log-cp-service.md` has more authority.

7. The current design docs describe stages 2-4 as built

`docs/log-cp-service.md` describes the log-backed control plane as built in stages 2-4. The service uses sticky routing to one appender. The service keeps K=4 appends in flight. A slot is one `LOG/<slot>` item. The service writes the item with `attribute_not_exists`. Replicas tail the log. Replicas apply entries through `cp/applier`. Pebble backs `cp/applier`. A replica answers only after the replica has applied its slot. Standbys serve fresh reads. Standbys answer commits with 503, `Retry-After`, and `X-CP-Role`. Checkpoints go to S3 at `ckpt/<slot>`. Trimming overwrites slots with tombstones. The `R#` index flushes every 10 s under decision 50. Nothing in the service reads the `R#` index yet. The index exists for rollback and a future S3 index. The “NOT done” list in that doc is dated. The list records no AWS run at writing time, no S3 index, and no streaming checkpoints. Stage 5 has since run as w4.

`docs/sim-log-engine.md` describes the matching simulator at `sim/engines/logengine`. Run the simulator with `go run ./cmd/manisim -scenario kafka-log`. The simulator models two replicas, holes, and hole-filling. The simulator models takeover, failback, tombstone trim, and checkpoint restore. The simulator scales slot counts down. A 45 s run crosses the scaled counts for that reason. Four mutants exist. The mutants are `BugWriterOffsets`, `BugTrimDelete`, `BugPerReplicaWindow`, and `BugHoleSkip`. Tests catch each mutant within 6 seeds. A run of 2,000 seeds and 50.7 M steps passes. The Limits section lists what the simulator does not model. The list covers DynamoDB throttling, a third replica, and the `R#` index. Tests in `cp/logcp` cover those points.

`docs/applier-bakeoff.md` holds the measurement behind decision 49. The bake-off compares the same applier on bbolt and Pebble. The bake-off uses 150-partition segments over 10-20k preloaded segments. Pebble ran 1,100-1,330 segments per second. Pebble used about 2.8x less memory. Pebble used half the disk. Bbolt ran 300-430 per second. The team removed bbolt. Use this doc as the model for arguing a dependency. The doc states the workload. The doc states a target above 430 per second. The doc reports results. The doc states dependency cost. The cost was 17 modules at edge only. A record sits at `docs/dep-audits/github.com/cockroachdb/pebble@v1.1.5.md`. The doc also gives lockstep evidence and a “Not done” list.

`docs/flow-control.md` is older and still current. The header says “design agreed, interfaces applied”. The doc sets two inboxes per node. The control inbox has strict priority. Bounds on the control inbox come from its construction. The data inbox is bounded. The data inbox keeps one ordered stream per connection. Throttling sits inside `Step`. Soft throttling uses `ThrottleTimeMs` and mute under KIP-219. Hard throttling pauses all reads. The design uses hysteresis. The design never drops an admitted request. Production and the simulator share the pure `core.InboxPolicy`. The doc also records a mutation check. The check swaps in one FIFO queue. The check requires seeds to go red.

8. Toys3 supplied the evidence, and the later proposals mostly stayed unbuilt

`docs/interface-changes-from-toys3.md` holds the evidence for decisions 7-22.

The evidence covers these decisions:

- Decision 7 covers `Trip`.
- Decision 8 covers `Stop` and `Exit`.
- Decision 9 says `Machine` includes `Hashable`.
- Decision 10 sets typed I/O.
- Decision 11 sets contract rules.
- Decision 12 says facts are hash inputs.
- Decision 13 says invariants see a `View`.
- Decision 14 says the system draws faults at issue time.
- Decision 15 separates history and the oracle.
- Decision 16 sets per-seed results and golden files.
- Decision 17 sets derived RNG streams.
- Decision 18 sets scheduler order `(At, Tie, Seq)`.
- Decisions 19-20 split `sys`.
- Decision 21 sets `veritas/`.
- Decision 22 sets `format/`.

The header says “accepted, not yet applied”. That statement was true when the team wrote the doc. The changes landed on 2026-10-16. The commits are `00241cb` - `1d147fa`. Read the doc for provenance and not for status. Section 6 rejects porcupine as checker of record. Porcupine verdicts depend on wall-clock time. Section 6 also bars the oracle from importing veritas.

`docs/ddd-refactoring-proposal.md` proposed a glossary and value objects. The value objects were `SegmentKey`, `OffsetRange`, and a `Verdict`. The `Verdict` would replace `Base == -1`. The proposal also proposed broker aggregate extraction. The proposal also proposed fact-name constants. The proposal also proposed a split of the overloaded `ErrFenced`. Decision 34 adopted only item 7, option A, on typed accessors. Decision 34 states plainly, “The rest of the DDD proposal is deferred.” Decisions 41-44 then removed fencing. Even item 7 lost its subject after decisions 41-44.

`docs/refactoring-proposal.md` is in its second edition. The second edition supersedes the open items in the DDD doc. The second edition measures the open items again. The broker had grown to 1,949 lines. Fact-name literals had spread to 18 files. The header says “proposal only, nothing applied”.

The deferred work comes from its waves and §7:

- Typed fact and metric names with contract tests sit under Q3 and Q4.
- Extraction of the segment commit pipeline behind a `commitSched` seam sits under S1.
- Shared `cmd` wiring and one effective-config description sit under Q5 and Q6.
- `core.Hasher` unification sits under Q7.
- Broker renames sit under Q2.
- Kafka error translation in one place sits under S2.

- Value objects sit under S4. This item sits furthest out.
- Non-serial commit scheduling sits under S3. This item also sits furthest out.

Section 5, “What NOT to do”, still binds. Section 5 bars a broker sub-package split. Section 5 bars a shared idempotence-rules package. Section 5 bars renaming facts or enum order. Section 5 bars Prometheus in `broker/`. The sequencing assumed “range compaction in flight”. That work landed as decision 39. If you revive an item, check the premises again first.

9. `docs/TODO.md` alone lists work that the team accepted but did not build

`docs/TODO.md` is the only doc whose job is unfinished work.

Current items in `docs/TODO.md` are these items:

- The consumer read path comes before any consume stage. The read path needs long-poll Fetch that honours `MinBytes` and `MaxWait`. The read path needs a Lookup cache. The read path needs an S3 read cache with coalescing. Consumers sit at the tail. This item turns surprises S3, S5, and S6 into a plan. This item is roadmap item 1.
- Metadata lists every live broker. This work needs new heartbeat-lease and `DescribeBrokers` ops. The lease feeds Metadata only. The lease fences nothing under decision 44.
- The index moves to S3 as a follow-up to decisions 48-50. Checkpoints and immutable S3 index files take over. The system trims the log behind the checkpoints and files. DynamoDB `R#` goes.
- Go 1.26 and current franz-go come in as planned upgrades. The upgrade needs a new audit of kmsg under decision J.

Idempotent producers are not planned. The doc says so in plain terms. The team declined per-broker partition ownership. Per-broker partition ownership would raise the sequencer limit. Per-broker partition ownership breaks leaderless metadata under decision 29.

Done items stay marked done. Compression is decision 45. Range compaction is decision 39. Check the markers. Do not rely on the headings alone.

10. Results supersede the plan that produced them, and the legal caution blocks the teardown as a source

`docs/stage5-run-plan.md` is dated 2026-10-21. The plan said “NOT executed” when the team wrote the plan. The plan set w4 at 500k, 750k, and 1M msg per second. The plan set random clients, 288 partitions, and produce only. The plan set a log CP plus hot standby. The plan also set a kill-the-appender stage at 500k. The stage had an ack-gap budget of 15 s. The plan records two choices from the product owner. The project dropped 4096 partitions. The project deferred consumers until the TODO read path exists. The run then happened. `docs/w4-log-store-results.md` reports about 998k msg per second for 4 minutes at p99 756 ms. The results doc reports an about 8 s ack gap on the kill. Stages ran 1-4 minutes with one sample each. Those numbers are indicative and not steady-state. Where the plan and the results differ, the results have more authority. The differences cover stage lengths and the missed “<10 s” index-lag bar.

`docs/roadmap-from-warpstream-teardown.md` is dated 2026-10-21. The roadmap ranks five takeaways.

The roadmap ranks the work in this order:

- Consumer read path comes first, at about 20 engineer-days. The work covers long-poll, Lookup cache, coalesced GETs, and chunk cache.
- Retention, orphan GC, and bucket preflight come second, at about 10-12 engineer-days. The system deletes nothing today. The 3-day lifecycle rule is a correctness hazard.
- Broker membership, AZ hints, and drain come third, at about 10-13 engineer-days. The work includes a 1M failover test. This item revisits decision 29.
- Consumer groups, offsets, and minimal admin APIs come fourth, at about 35-45 engineer-days. The work comes from the Kafka protocol spec.
- Compaction of per-partition slices comes fifth, at about 30 engineer-days after the S3 index. One partition-hour is about 40k small slices today.

TLS and SASL need about 10 engineer-days before any serious OSS announcement. Section 3 is an OSS-readiness checklist. The checklist lists BLOCKER items. The items are no root LICENSE and no root README. The checklist lists a secrets scan and a timeline. The timeline puts a quiet `v0.1.0-alpha.1` before any blog post. The timeline says, “If any gate slips, slip the date”.

The legal caution blocks any use of the teardown until a lawyer clears the use

Section 0 of the roadmap is not legal advice. This chapter is not legal advice either. You must not miss section 0. The teardown document sits at `docs/arun-warpstream-agent-reverse-engineering.md`. The teardown document is untracked and not committed. Someone produced the teardown by extracting protobuf descriptors from a vendor binary. Someone extracted symbol tables and strings from the vendor binary and ran the binary. The roadmap says that work is reverse engineering in the ordinary sense. The roadmap says nobody has read the applicable agent licence. Nobody has read the Confluent and IBM terms. The roadmap says the public-sources study in this project treats reverse engineering as presumptively prohibited. Shipping code visibly modelled on the teardown could create legal exposure. Publishing derived material could also create legal exposure. The exposure covers contract, trade secret, and credibility. Get legal advice before the project publishes anything derived from the teardown. Keep the teardown out of the OSS repo. Keep anything copied from the teardown out of the OSS repo. Keep the teardown and copied material out of git history. Implement the roadmap clean-room. Use public docs, the Kafka protocol spec, AWS guidance, and our own measurements. Add `docs/arun-*` to `.git/info/exclude`. Never run `git add -A` in this area.

11. Superseded decisions stay in the repo, and you must not follow the superseded text

Keep this list next to you for your first month. Each item was once the design. If you follow a superseded item now, you reproduce removed behaviour.

The superseded list is this list:

- Decisions 28 and 34 covered idempotent producers and broker and producer fencing. Decisions 41-44 superseded decisions 28 and 34. The current code has no `InitProducer`. The current code has no epochs. The current code has no `ErrFenced`.

- Decision 46 set a single `Sequencer` over the transaction store as the production design. Decision 48 refined decision 46 into the log appender. The group-commit measurements from decision 46 remain useful evidence.
- `docs/commit-path-redesign.md` offered design L as a build plan. Decisions 48-53 superseded design L with adopted design A. `docs/log-cp-service.md` also supersedes design L.
- `docs/design-a-prototype.md` says in its own status line that the prototype has "no `-store` kind, not wired". Stages 2-4 overtook that line. `manicp -store dynamolog` exists.
- `docs/interface-changes-from-toys3.md` says decisions 7-22 are "not yet applied". The decisions landed on 2026-10-16.
- `docs/refactoring-proposal.md` superseded the open items in `docs/ddd-refactoring-proposal.md`. Decision 34 deferred almost all of those items.
- `docs/lessons.md` §4 covers incarnation fencing. Decisions 41-44 made §4 moot. Decision 17 superseded the "same stream" wording in §9.
- The old DynamoDB transaction-store numbers are historical. This group covers decisions 32, 35, and 39 item arithmetic. This group covers the w2 and w3 plateaus of about 240-275k msg per second. The "~$43-59k/month" DynamoDB figures use the pre-cut $1.25/M WRU price. `docs/commit-path-redesign.md` §2 corrects this price.
- `docs/surprises-audit.md` is a 2026-10-19 snapshot. The audit is not a description of current code. The project fixed or dissolved several S-ids. See §5 above.
- Older benchmark docs use the `vanity` workload name. Older benchmark docs use "WarpStream parity" phrasing. Section 3 of the roadmap asks for dated, attributed, and nominative comparisons only. Section 3 bars "parity" claims. Section 3 bars "faster than" claims.
- Anything derived from the teardown is not a source to design from. See §10.

You keep the superseded text for the reason in the opening. A superseded decision shows the measurement that forced the turn. A reader who sees only the current decision cannot see that constraint.

12. The trust table tells you when to read each doc and how far to trust each doc

Doc	When to read it	Trust level
`docs/decisions.md`	Before you change any contract, cap, or dependency	Current. Check each row for Superseded and Refines.
`docs/lessons.md`	When a contract looks odd and you want the reason	Current, except §4 and §9 wording. Decision 41-44 made §4 moot. Decision 17 superseded §9 wording.
`docs/flow-control.md`	When you touch inboxes, throttling, or backpressure	Current. Interfaces applied.
`docs/log-cp-service.md`	When you work on `cp/logcp`, failover, checkpoints, or the index	Current for stages 2-4. Run w4 partly overtook the "NOT done" list.
`docs/sim-log-engine.md`	When you add sim coverage for the log CP	Current. Stage 3.
`docs/applier-bakeoff.md`	When you question Pebble or argue a dependency choice	Historical measurement. Decision 49 keeps the conclusion current.
`docs/TODO.md`	When you pick up new work or check whether an idea has a schedule	Current. Accepted and not scheduled.

Doc	When to read it	Trust level
`docs/stage5-run-plan.md`	When you need to understand what w4 was meant to prove	Historical plan. `docs/w4-log-store-results.md` superseded the plan as fact.
`docs/roadmap-from-warpstream-teardown.md`	When you plan what comes next or ask about open-sourcing	Current, dated 2026-10-21. Section 0 legal caution applies first.
`docs/surprises-audit.md`	Before an AWS run or consume stage, or when you cite S-ids	Historical snapshot, dated 2026-10-19. Verify status per id.
`docs/commit-path-redesign.md`	When you need to understand why the transaction store ended	Superseded proposal. The team did not build design L. Sections 1-2 numbers still count as evidence.
`docs/delos-logdrive-ideas.md`	When you trace where design A and the S3-index idea came from	Historical ideas. Decision 48 adopted idea A. `TODO.md` holds idea B. The team did not build ideas C and D.
`docs/interface-changes-from-toys3.md`	When you need provenance for decisions 7-22	Historical. The changes applied on 2026-10-16 and overtook the status line.
`docs/ddd-refactoring-proposal.md`	Only through the second edition	`docs/refactoring-proposal.md` superseded this proposal. Decision 34 deferred the rest.
`docs/refactoring-proposal.md`	When you plan a refactor or check what not to do	Proposal. The team applied nothing at writing. Premises need a new check.

Key takeaways

The key points are these points:

- Authority has layers. A decision has more authority than a topic doc. A topic doc has more authority than a proposal. A proposal has more authority than a dated audit and a dated plan. The verbs “supersede” and “refine” set the order inside the decisions. A results doc has more authority than the plan that produced the results doc.
- Decisions 41-45 and 48-53 set the current architecture. Brokers are stateless. Delivery is at-least-once. A log-backed control plane holds raw commands. Replicas apply those commands through a Pebble applier. Veritas is the independent oracle. Production code must agree with veritas. Production code must never become veritas.
- Most docs that look wrong are not mistakes. Each such doc records a measurement or a design that was true for one stage. `git log` and the `Status:` line tell you which stage you read.
- Open work lives in two places. `docs/TODO.md` holds accepted work. `docs/roadmap-from-warpstream-teardown.md` §2 holds ranked work. Any other document that proposes work is a proposal and not a commitment.
- The teardown is a legal risk and not a design source. Keep the teardown out of git. Keep the teardown out of the OSS repo. Build clean-room from public sources.

Checkpoint

1. A colleague proposes re-adding per-producer duplicate detection “like decision 28 says”. What is wrong? — Decisions 41 and 44 superseded decision 28. The system removed idempotent producers. The

system removed the 5-entry window, `Dup`, `ErrOutOfOrder`, and `InitProducer`. Delivery is at-least-once. `docs/TODO.md` lists idempotent producers as not planned. The proposal means reopening decision 41.

2. Design L in `docs/commit-path-redesign.md` recommends a small transaction holding `G#` items with computed bases. Is that the production CP? — No. Design L was a proposal. Decisions 48-53 adopted idea A from `docs/delos-logdrive-ideas.md` instead. The log holds raw commands. Every replica applies the commands. Every replica assigns offsets at apply time. The system has no permanent `G#` items. `docs/log-cp-service.md` describes the system as built.

3. What does "S3" mean in a review comment, and how do you check whether S3 still applies? — S3 is item 3 of `docs/surprises-audit.md`. The audit is dated 2026-10-19. Fetch never waits. Idle consumers generate tens of thousands of CP Lookups per second for that reason. The audit is a dated snapshot. Check status in `docs/TODO.md` under "Consumer read path" and check the decisions. S3 is still open.

4. Why is veritas not the production state machine, if veritas is the spec? — Decision 49 gives the reason. Veritas keeps every run forever. One batch costs about 257 B. Target rate costs about 6 GB per hour. Veritas inserts keys in O(n). The production applier keeps the same semantics behind a KV interface. The applier is `cp/applier`. Pebble backs the applier per `docs/applier-bakeoff.md`. Lockstep tests use 100,000 seeds. `cptest` conformance proves the applier against veritas.

5. You find a great idea in the WarpStream teardown doc. What must happen before you use the idea? — Treat the idea as legally tainted until a lawyer clears the idea. Section 0 of `docs/roadmap-from-warpstream-teardown.md` says someone reverse-engineered the teardown from a vendor binary. Section 0 says the licence terms are unread. Section 0 says reverse engineering is presumptively prohibited. Get legal advice. Keep the idea out of git and the OSS repo until clearance. Derive the idea again clean-room. Use public docs, the Kafka spec, AWS guidance, or our own measurements.

Try it

You test your reading in these ways:

- Run `git log --date=short --format='%ad %h %s' -- docs/decisions.md`. Read how rows 41-44 and 48-53 arrived. Search your favourite row for "Superseded" and "Refines" before you quote the row.
- Pick one S-id from `docs/surprises-audit.md`. Classify the S-id as fixed, dissolved, declined, or open. Use `docs/decisions.md`, `docs/TODO.md`, and `docs/w4-log-store-results.md` as your only evidence.
- Run `go run ./cmd/manisim -scenario kafka-log -seeds 300`. Match the properties in `docs/sim-log-engine.md` to the report. Find the four log mutants and the seed that first catches each mutant.
- Open `docs/refactoring-proposal.md` §7. Argue from the proof column in §7. Name which of the three priority items costs least to revive today. State which premise already changed. Range compaction landed as decision 39.
- Check `git status` for untracked files. Confirm that no file matching `docs/arun-*` has a staged state. Read §0 of the roadmap once more before any `git add`.

Further reading

Further reading covers these docs:

- Read `docs/decisions.md` rows I, 21, 41-45, and 48-53. The rows form the core that sets the architecture.
- Read `docs/design-a-prototype.md` for the prototype and the X2 and X5 measurements behind design A. The doc includes the staged production plan.
- Read `docs/w4-log-store-results.md` for what stage 5 measured. The doc includes the caveats. Pair the doc with `docs/w2-bottleneck-verification.md` and `docs/w3-affinity-and-window-experiments.md` for the picture before stage 5.
- Read `docs/x1-x3-x4-results.md` for the DynamoDB experiments behind decisions 48-52. The experiments cover PutItem and transaction use, hot-key headroom, and contention.
- Read `docs/observability.md` and `docs/benchmarks.md`. The docs cover evidence-grade logging and metrics. The docs cover the local limits behind decisions 36-38.
- Read `docs/dep-audits/` for what an accepted dependency record looks like under decision I. Start with the Pebble record from the bake-off.

Chapter 12. Newcomer playbook. Your first week, first PR, and where to change what

Run the tasks in order. Read background in Chapter 3, Chapter 6, Chapter 8, and Chapter 9. Follow the source when a command in this chapter disagrees with the source. Fix this chapter.

Check the prerequisites

Check each need before you start work.

Need	Why	Check
Go 1.24	`go.mod` says `go 1.24.0` (decision J says franz-go `kmsg` v1.14.0 requires it)	`go version`
Docker (with Compose)	Only for the Docker-backed integration tests in `it/`	`docker compose version`
AWS credentials	Only for the AWS benchmark under `deploy/aws/` and real DynamoDB and S3 stores	Read `docs/aws-benchmark-runbook.md`

1. Run `go version`.
2. Check the output names Go 1.24.
3. If you want the Docker-backed integration tests, run `docker compose version`.
4. Check the command prints a version.
5. If you want the AWS benchmark, read `docs/aws-benchmark-runbook.md`.

Run the normal loop with Go alone. Skip Docker and AWS for the normal loop.

Plan the first week

Complete one task on each day.

1. On day 1, run the build gate, the broker, and a simulator campaign.
2. On day 2, read `loop.md`, `core/event.go`, `core/action.go`, `core/machine.go`, and `broker/broker.go`.
3. Trace one produce from `BytesReceived` to the `produce_acked` fact.
4. On day 3, replay a failing seed.
5. Pick a `broker.Bug*` mutant in `broker/config.go`.
6. Find the test that catches the mutant.
7. Check `TestConformanceBrokerCatchesMutants` in `broker/conformance_test.go` and the mutants in `sim/kafka/`.
8. On day 4, pick a first issue.
9. Merge the issue with the smallest test that proves the change.
10. On day 5, ship the first PR.
11. If the change touched a contract, write or update one `docs/decisions.md` entry.

Run the build gate

Run the full gate before you ship a change.

```
make build          # fmt-check + vet + compile + test + crap (the full gate)
make fmt-check      # fail if gofmt -l . lists anything
make vet            # go vet ./...
make compile        # go build ./...
make test           # go test -coverprofile=build/cover.out ./...
make crap           # CRAP gate alone (runs make test first if no profile)
make crap-strict    # full offender list, ignoring crap-baseline.json
./scripts/ci-determinism.sh   # detcheck + dep audits + determinism canary
```

1. Run `make build`.
2. Check the command exits with code 0.
3. Run `./scripts/ci-determinism.sh` after `make build`.
4. Check the script prints `determinism gate passed`.

Read Chapter 9 for the gate design. The determinism gate runs apart from `make build`. The comment at the top of the `Makefile` states that split. `make build` ends with `crap` in `cmd/crapscore`.

Choose an option from the list below.

- If you want a threshold of 15, run `make build CRAP_THRESHOLD=15`.
- If you want extra coverage in the CRAP input, run `make crap CRAP_MERGE=build/it-cover.out`.
- Read `docs/crap-gate.md` for both settings.

Warning. Do not run `make crap-baseline` to turn a red gate green. The note in `crap-baseline.json` calls a rewrite of `crap-baseline.json` “deliberate, reviewable”.

1. Format each file before you commit the file.
2. Run `gofmt -w <files you touched>`.
3. Run `make fmt-check`.
4. Check `make fmt-check` exits with code 0.

Run the broker locally

Run the control plane, one broker, and one benchmark client. Read Chapter 6 for the control plane design.

Terminal 1 runs the control plane with the in-memory store. The `-store` default is `mem`. `cmd/manicp/main.go` defines the flags.

```
go run ./cmd/manicp                # listens on :8080, metrics on :9101
curl http://127.0.0.1:8080/healthz  # -> ok
```

1. Open terminal 1.

2. Run `go run ./cmd/manicp`.
3. Run `curl http://127.0.0.1:8080/healthz`.
4. Check the output is `ok`.

Terminal 2 runs one broker with the in-process object store. The `-objects` default is `s3`. The `mem` value selects a single-broker store. That store loses data on exit. `cmd/manibroker/main.go` defines the option.

```
go run ./cmd/manibroker -objects mem   # Kafka on 127.0.0.1:9092, metrics :9100
```

1. Open terminal 2.
2. Run `go run ./cmd/manibroker -objects mem`.
3. Check the broker listens on `127.0.0.1:9092`.

Terminal 3 creates a topic through the HTTP API of the control plane. `cp/handler.go` serves `POST /v1/create-topic`. `CreateTopicReq` in `cp/wire.go` defines the JSON fields. The benchmark client then loads the topic. The header of `cmd/manibench/main.go` gives its usage.

```
curl -X POST http://127.0.0.1:8080/v1/create-topic \
  -d '{"topic":"orders","partitions":3}'
go run ./cmd/manibench benchmark-producer -topic orders \
  -brokers 127.0.0.1:9092 -duration 10s
```

1. Open terminal 3.
2. Run the `curl` command.
3. Check the command exits with code 0.
4. Run the `manibench` command.
5. Check the client produces to `orders` for 10s.
6. Stop both processes with Ctrl-C or SIGTERM.
7. Check the broker finishes work in flight within `-stop-grace`.
8. Note the `-stop-grace` default is `10s`.

Choose a store from the list below.

- If you want the log-backed control plane, set `manicp -store` to `dynamolog`.
- If you want the transaction store, set `manicp -store` to `dynamo`.
- Keep `mem` for the local run.

`manicp -store` also accepts `dynamo` and `dynamolog`. The `dynamolog` value selects the log-backed control plane from decisions 48-53. `cmd/manicp/logstore.go` defines its extra flags. Read Chapter 6 for the store design.

Choose an object store from the list below.

- If you want a single local broker, keep `-objects mem`.

- If you want S3, set `-objects s3` and set `-s3-endpoint/-s3-bucket/-s3-region/-s3-path-style/-s3-create-bucket`.

1. Run `go run ./cmd/manicp -h`.
2. Check the `flag` package prints the flags for `manicp`.
3. Run `go run ./cmd/manibroker -h`.
4. Check the `flag` package prints the flags for `manibroker`.

Run a simulator campaign

Run the default campaign first. Read Chapter 8 for the simulator design.

```
go run ./cmd/manisim                       # 100 kafka seeds from seed 1 (-seeds, -start)
go run ./cmd/manisim -v                    # one line per seed: steps/rand/hash/crashes/exits/failures
go run ./cmd/manisim -seed 42              # replay one seed, full step log printed
go run ./cmd/manisim -scenario echo -seeds 20   # smoke scenario; also kafka-log
go run ./cmd/manisim -j 4 -seeds 1000      # shard seeds over 4 jobs (-j defaults to NumCPU)
```

1. Run `go run ./cmd/manisim`.
2. Check the command prints the seed summary line.
3. Check the command exits with code 0.
4. If you want one line per seed, run `go run ./cmd/manisim -v`.
5. If you want the smoke scenario, run `go run ./cmd/manisim -scenario echo -seeds 20`.
6. If you want four jobs, run `go run ./cmd/manisim -j 4 -seeds 1000`.

`cmd/manisim/main.go` defines the `kafka`, `kafka-log`, and `echo` scenarios. Exit code 1 means one of these conditions:

- A seed failed.
- The run violated an Always property.
- The run found nondeterminism.
- A campaign ran at least 100 seeds (`coverageMin`) and never satisfied a Sometimes property.

Run the integration tests

Run the hermetic test first. Run the full-stack test after the hermetic test passes.

```
go test ./it/ -run TestProduceConsumeRoundTrip -count=1 -v   # hermetic, no Docker
./scripts/it-up.sh        # docker compose up (deploy/docker-compose.yml); PRINTS export lines
# copy/paste the two printed lines into your shell:
export MANIFESTER_DYNAMO_ENDPOINT=http://127.0.0.1:18080
export MANIFESTER_S3_ENDPOINT=http://127.0.0.1:8333
```

```
go test ./it/ -run TestFullStackRoundTrip -count=1 -v        # now runs instead of skipping
./scripts/it-down.sh      # docker compose down -v
```

1. Run `go test ./it/ -run TestProduceConsumeRoundTrip -count=1 -v`.
2. Check the test passes.
3. Note the hermetic test needs no Docker.
4. Note the hermetic test uses in-process brokers over `cp/memstore`.
5. Note the hermetic test uses an in-memory object store.

If you want the full-stack test, start the Docker backends.

Warning. `scripts/it-up.sh` prints the `export` lines. The script does not set the lines in your shell.

1. Run `./scripts/it-up.sh`.
2. Copy the two printed lines into your shell.
3. Run the two printed lines in your shell.
4. Run `go test ./it/ -run TestFullStackRoundTrip -count=1 -v`.
5. Check the test runs and passes.
6. Check the test does not skip.
7. Run `./scripts/it-down.sh`.
8. Check the command exits with code 0.

Ports bind to loopback only. ExtendDB and DynamoDB use port 18080. SeaweedFS S3 uses port 8333. The full-stack tests in `it/fullstack_test.go` skip when their endpoint variables are unset. The hermetic tests in `it/hermetic_test.go` use in-process brokers over `cp/memstore`. Read `docs/crap-gate.md` for the CRAP gate and Docker-free coverage. Read Chapter 9 for the test design.

Replay a failing seed

Copy the repro line from the failure output. Read Chapter 8 for replay.

1. Find the repro line in the failure output.
2. Note `manisim` prints `repro: go run ./cmd/manisim -scenario kafka -seed <n>` under each failure.
3. Note `manisim` prints the same line for nondeterminism.
4. Run the repro line.
5. Check replay prints the per-step log with `Step`, `At`, and `Text`.
6. Check replay prints the seed summary.
7. Note the seed summary holds `steps/rand/hash/crashes/trips/exits`.
8. Check replay prints each `FAIL <kind> (<prop>) step=… at=…: <msg>` line after the summary.
9. Note the kinds are `violation`, `inconclusive`, `protocol`, `panic`, `max_steps`, and `nondeterminism` in `cmd/manisim/main.go`.
10. Find the failing step number in the output.
11. Read the fact timeline at that step.
12. Find the `Step(Event)` that went wrong.
13. Run the neighbouring seeds with `-start <n-2> -seeds 5 -v`.
14. Check whether the bug affects one seed or several seeds.

If two runs of one seed diverge, treat the divergence as nondeterminism.

1. Run `TestDeterminismCanary` in `sim/kernel_test.go`.
2. Run `TestDeterminismCanary` in `sim/kafka/kafka_test.go`.
3. Run `TestDeterminismCanary` in `sim/echo/echo_test.go`.
4. Look for a banned construct from the detcheck task below.
5. Look for an uncounted source of randomness.
6. Read lesson 3 in `docs/lessons.md`.
7. Note lesson 3 requires one counted RNG stream with `Calls()` and `Derive(label)`.

Read a trace

Compare hashes before you compare logs. Read Chapter 8 for the timeline.

1. Run `go run ./cmd/manisim -v`.
2. Find `hash=%016x` on one seed line.
3. Note that value is the trace hash.
4. Check equal seeds produce equal hashes.

Use the timeline from the list below.

- Call `Snapshot(name)` to return all facts with one name.
- Call `Since(name, &cursor)` to stream facts with one name.

The sim records facts on the append-only timeline in `sim/timeline.go`. Invariants read the timeline after each step. The sim emits an ack fact in the same `Step` as the ack. It emits the fact before the `Send` that acks.

Warning. Do not rename a fact without care.

`core/fact.go` defines a fact as `core.Fact{Name, Fields}`. Fact names use frozen past-tense constants such as `produce_acked` and `segment_committed`. The name and the field encoding feed the trace hash. A rename re-baselines every golden hash. Decision 12 covers that rule. Read Chapter 3 for facts.

Fix a detcheck failure

Run `./scripts/ci-determinism.sh` first. Read step 1 of the script output. Read Chapter 9 for the gate.

The gate runs `go run ./tools/detcheck` over the deterministic packages. The gate fails on these constructs:

- An import that does not appear on the exact-path allow-list in `tools/detcheck/allow.txt`.
- A `go` statement.
- Channels or `select`.
- `range` over a map.
- Reflect map iteration.

- A package-level variable.

1. Read the diagnostic as written.
2. Note the diagnostic names the construct and the file.
3. Match the fix to the construct.
4. Iterate sorted keys in place of ranging over a map.
5. Return an `Action` in place of spawning a goroutine.
6. Move wall-clock input and random input behind `core.Env`.
7. Note each event carries its own `At` and `Wall`.
8. Note the core has no `Now()`.
9. Read decision 2 for that rule.

Warning. Do not append a path to `allow.txt` to make CI pass.

If you prove a finding harmless, suppress the finding on one line.

1. Add `//detcheck:ok <reason>` on that line.
2. Write the reason.
3. Check detcheck accepts the line.
4. Note detcheck rejects an empty reason.

If the fix needs a new import, treat the import as a dependency change.

1. Follow the dependency recipe below.
2. Read the audit skill.
3. Note the audit skill says “Never weaken tools/detcheck…”.
4. Run `./scripts/ci-determinism.sh`.
5. Check the script prints `determinism gate passed`.

Fix a CRAP failure

Run `make build` first. Find the CRAP failure at the last step. Read Chapter 9 for the gate.

1. Run `make test` to refresh `build/cover.out`.
2. Run `make crap` after `make test`.
3. Check `make crap` names the offender.
4. Run `make crap-strict` to show every offender without the baseline.
5. Read the score with the formula in `docs/crap-gate.md`.
6. Note the formula is `CRAP = comp² × (1 − cov)³ + comp`.
7. Note the gate fails above 30 by default.
8. Lower one factor.
9. Split a function to lower complexity.
10. Add a test for the missing case to raise coverage.
11. Write the test for the behaviour.
12. Check `make crap` passes.

Warning. Do not split a function for the number alone. `docs/crap-gate.md` warns against a split that serves the number alone.

Warning. Run `make crap-baseline` after a reviewer agrees to the change. The command exists for a reviewed re-baselining.

1. After a reviewer agrees, run `make crap-baseline`.
2. Check `make build` passes.

Fix a conformance failure

Find the layer that failed first. Read Chapter 9 for the checks.

The conformance checks compare code against the `veritas/` executable spec in lockstep. Decision 21 covers that design. C1 checks the real broker against the spec. C2 checks the sim engines against the spec. C3 checks the `sys` adapters against the spec. C4 checks the oracle against the spec.

Choose the command for the layer that failed.

- If the broker failed, run `go test ./broker/ -run TestConformanceBroker -count=1`.
- If the spec failed, run `go test ./veritas/`.
- If the sim world failed, run `go test ./sim/kafka/ -run TestDeterminismCanary`.
- If the sim world failed, run the `c4_*` tests in `sim/kafka/`.

1. Run the command for the failed layer.
2. Check the command names the failing case.
3. Read the shrunk case first.
4. Note the lockstep runner in `veritas/veritastest` shrinks a failing op sequence.
5. Note the original campaign gives less signal than the shrunk case.
6. Decide which side has the bug.
7. Note the spec in `veritas/` is the reference for the checks.
8. Note the spec contains executable code and can contain a bug.
9. If the spec is wrong, fix the spec.
10. Say the spec fix in the PR.
11. Log a decision if the fix moved a contract.

Check the checker after you fix the bug.

1. Run `TestControlPlaneCheckShrinksPlantedBug`.
2. Run `TestConformanceBrokerCatchesMutants`.
3. Check each test passes.
4. Note a change that lets a mutant survive points at the bug in your change.

Follow a change recipe

Pick the row for your change. Touch the files in that row. Add the test in that row.

Change X	Touch Y	Add test Z
Add a Kafka API	Add the API to `kwire/supported.go`. `Supported()` supplies the data for ApiVersions. Add the dispatch case in `broker/ops.go` (`dispatch`). Add the handler in `broker/produce.go`, `broker/fetch.go`, or `broker/metadata.go`. Add the codec in `kwire/codec.go`.	Add a round-trip case in `kwire/codec_test.go`. Add a C1 case in `broker/conformance_test.go`. Add a hermetic case in `it/hermetic_test.go`. Add sim support in `sim/kafka/` if the workload must speak the API.
Add a config knob	Add the field and its doc comment in `broker/config.go` (`Config`, `DefaultConfig()`, `withDefaults()`). Add the flag in `cmd/manibroker/main.go` (`parseFlags`). Put tuning in `cmd/manibroker/tuning.go`. Add the value to `manifestTuning` if the runbook reads it.	Add coverage in `cmd/manibroker/tuning_test.go` or `cmd/manibroker/units_test.go`. Add a `broker` test. That test must prove the knob changes behaviour. A parse test alone does not prove it.
Add an event or action	Add the type to the closed sets in `core/event.go` and `core/action.go` (`isEvent()` and `isAction()`). Keep the `ReqID()` pairing between `IOAction` and `Completion`. Add handling in the `Step` method in `core/machine.go`. Add execution in `sys/` for production. Add execution in the `sim/` engine that runs the type.	Add a protocol test in `sim/kernel_test.go`. An unknown-ReqID completion must change nothing. Re-baseline golden trace hashes as the point of the change. Decision 11 makes hashed enums append-only.
Add a CP op	Add the contract to `core/controlplane.go` (`CPOp` and `CPResp`). Add the wire types in `cp/wire.go`. Add the route in `cp/handler.go`. Add the method to the `cp.Store` interface in `cp/store.go`. Add that method to `cp/memstore`, `cp/dynamo`, and `cp/logcp`. Follow the OpenAPI steps in the next row.	Add the lockstep case in `cp/store_test.go` across the stores. Add the spec tests in `api/openapi/spec_test.go`. Run them with `go test ./api/...`.
Add a CP op spec	Edit `api/openapi/controlplane.json`. That file is the source of truth. Add the wire type to `wireTypes` in `api/openapi/spec_test.go`. Regenerate with `go run ./api/openapi/cmd/specgen`. Check the result with `…/specgen -check`.	Add an `x-replay` example for the op. The spec test replays each `x-replay` example against `cp.NewHandler` over `cp/memstore`. An op without an example has no spec-test coverage.
Add a dependency	Follow `.claude/skills/audit-go-dependency/SKILL.md`. Run `scripts/audit-dep.sh <module>@<version>`. Write `docs/dep-audits/<module>@<version>.md` with the template in `docs/dep-audits/README.md`. Add the exact path to `tools/detcheck/allow.txt`. Run `go get` after those steps.	Run `./scripts/ci-determinism.sh`. It must pass. The script fails on a module in the dep tree of a deterministic package when that module lacks an audit record for the exact `go.mod` version.

Apply two rules to every recipe.

- Never let a broker propose offsets.
- Keep commits idempotent per `SegmentKey` under decisions 42-44.

If a recipe seems to need an exception, add a decision row first.

Note the system rejects idempotent producers under decision 41. Note `manibench` disables idempotence for the same reason. Read Chapter 3 for events and actions. Read Chapter 6 for CP ops.

After you add a dependency, run the check below.

1. Run `./scripts/ci-determinism.sh`.
2. Check the script prints `determinism gate passed`.

After you add a CP op spec, run the check below.

1. Run `go run ./api/openapi/cmd/specgen`.
2. Check the generator exits with code 0.
3. Run the generator check with `…/specgen -check`.
4. Check the check exits with code 0.

Check a deterministic package

Check the package set before you change code in a deterministic package. Read Chapter 3 and Chapter 9.

`scripts/ci-determinism.sh` gates a fixed package set. The script header gives the exact list. The set covers `core/`, `sim/`, `format/`, `kwire`, `broker/`, and `veritas` without its `vsys` adapters. The set also covers `veritas/veritastest` and the derived `toys3` set from the script. These rules apply in those packages:

- Decision I sets an allow-list in place of a ban-list. `tools/detcheck/allow.txt` names the exact imports. The stdlib set covers `bytes`, `encoding/binary`, `errors`, `fmt`, `hash/crc32`, `math`, `math/bits`, `slices`, `sort`, `strconv`, `strings`, and `unicode/utf8`. The list also names `manifester/…` paths and the audited `github.com/twmb/franz-go/pkg/kmsg` path.
- An absent import fails the gate in practice. The absent names are `time`, `math/rand`, `crypto/rand`, `os`, `net`, `sync`, `runtime`, `unsafe`, `reflect`, `maps`, `hash/maphash`, `context`, `io/fs`, and `syscall`. The constructs in the detcheck task fail the gate too. `kgo` appears in `it/` and `cmd/manibench`. It does not appear in a deterministic package.
- Events and actions hold plain data. Byte slices stay immutable after a send under `core/doc.go`. Randomness and time enter through `Env` and the stamps on each event.

1. Run `./scripts/ci-determinism.sh` after you change a deterministic package.
2. Check the script prints `determinism gate passed`.

Format the code

Format the code before you commit the code.

1. Run `gofmt -w <files you touched>`.
2. Run `make fmt-check`.
3. Check `make fmt-check` exits with code 0.

`gofmt` sets the format. `make fmt-check` runs as step 1 of the build, and a failure stops the build. CRAP gates complexity under the CRAP task. Taste does not gate complexity. Comments in this repo explain why a choice exists. They cite decisions. Reviewers reject a comment that restates the code.

Add a decision row

Add a numbered row in `docs/decisions.md` after one of these changes:

- You change a contract in `core/` or the CP wire format.
- You rename or add a fact.
- You change a store layout on disk or on the wire.
- You move a limit or a default that clients depend on.
- You accept a dependency.
- You supersede an earlier decision. State the supersession in the row. Decision 41 supersedes part of decision 28.

1. Write one row.
2. Give the reason for the change.
3. Give the evidence for the change.
4. Check the row names the reason, the evidence, and the change.

If you ask “is this big enough for a decision?”, the answer is yes.

Pick a first issue

Pick small and safe work for a first PR.

- Add a missing hermetic `it/` case for an existing API.
- Add a `veritas/` spec test for an edge case that the lockstep runner generates.
- Add coverage that lowers a baselined CRAP score.
- Add an example in `api/openapi`.
- Add a cross-reference in `docs/decisions.md`.
- Add a `kwire` codec round-trip case.

1. Pick one item from the list.
2. Open the recipe for that item.
3. Check the recipe names the files and the test.

Pair with a reviewer before you touch a dangerous area

Pair with a reviewer before you touch a dangerous area. Add a decision row before that work.

- Offset assignment and commit idempotency in `cp/` and `core/controlplane.go`.
- The scheduler and inbox priority policy in `sim/` and `core/policy.go`. Metastability risk sits in that policy. `docs/flow-control.md` covers that risk.
- The `format/` segment layout. That layout sets the bytes on disk.

- Flush caps and segment caps in `broker/config.go`. Those caps take their sizes from CP limits. Read the long comment in that file before you change a cap.
- An edit to `tools/detcheck/allow.txt`, `crap-baseline.json`, or a golden trace hash. Make that edit the point of the PR. Do not make that edit as a side effect of other work.

1. Name the dangerous area in the PR.
2. Name the reviewer in the PR.
3. Add the decision row.
4. Check the PR holds the area, the reviewer, and the row.

Ship the first PR

Ship one idea in one PR. Read Chapter 9 for the gates.

1. Create a branch.
2. Keep the diff to one idea.
3. Note this repo expects small PRs.
4. Note the commit history shows single-concern docs commits and code commits.
5. Add or update the test in the same package as the change.
6. Find the test in the recipe for your change.
7. Format the files you touched with `gofmt -w`.
8. Run `make build`.
9. Check `make build` exits with code 0.
10. Run `./scripts/ci-determinism.sh` if you touched a deterministic package.
11. Run the script if you are unsure.
12. Check the script prints `determinism gate passed`.
13. Note CI uses the same gate.
14. Add a `docs/decisions.md` row if you changed a contract.
15. State what changed in the PR description.
16. State which test proves the change.
17. State which seeds you ran for a change that touches the sim.
18. Note the output line from `go run ./cmd/manisim -seeds 100` gives that seed evidence.
19. Check the PR description holds the change, the test, and the seed evidence.

Prepare one review answer before you open the PR.

Warning. An answer that cites passing tests does not cover the review question.

1. Expect a review question about regression coverage.
2. Note the question is “what catches this if it regresses?”
3. Prepare an answer that names a mutant, a conformance case, or a canary.
4. Check the answer names one of those three items.

Complete the graduation checklist

Check each item. Review PRs in an area after you check each item in this list.

- [ ] `make build` and `./scripts/ci-determinism.sh` pass on my machine. I can explain what each step gates.
- [ ] I ran `manicp` and `manibroker -objects mem` on my machine. I created a topic through `/v1/create-topic`. I produced into that topic with `manibench`.
- [ ] I replayed a failing seed from its printed repro line. I found the failing `Step` from the step number.
- [ ] I can read a `manisim -v` line with steps, rand, hash, crashes, and exits. I can say what a hash change means.
- [ ] I fixed one detcheck failure, one CRAP failure, and one conformance failure on a branch. Each failure came from the repo or from a planted bug.
- [ ] I shipped one PR from the safe list with its test. I know which recipe my next change follows.
- [ ] I added or updated a `docs/decisions.md` row. I can defend its wording.
- [ ] I know which dangerous areas I should not touch alone yet.

Use the glossary

- Action. Output of `Step` in `core/action.go`. Each `IOAction` carries a `ReqID` and gets one `Completion`.
- Event. Input to `Step` in `core/event.go`. The sim stamps each event with node-local `At` and wall `Wall` on dequeue.
- Broker and `manibroker`. Kafka state machine in `broker/` as a pure `core.Machine`. Production binary in `cmd/manibroker`. Binary serves Kafka on `:9092`.
- Buggify. Seeded fault injection through `Env`. Production runs leave `Env` inert. `Sometimes` assertions pair with `Env` under lesson 7.
- Canary (determinism). `TestDeterminismCanary` runs each seed twice. Two runs must match on `(TraceHash, RandCalls, Steps)`.
- CommitSegment. CP op that assigns offsets to one uploaded segment. Op works as one atomic unit per `SegmentKey`. Decisions 3 and 42 cover that rule.
- Conformance (C1-C4). Lockstep checks of the broker, the sim, the `sys` adapters, and the oracle against the `veritas/` executable spec. Decision 21 covers that design.
- Control plane (CP) and `manicp`. HTTP service that assigns offsets. Code lives in `cp/` and `cmd/manicp`. Service serves on `:8080`. Stores are `memstore`, `cp/dynamo`, and `cp/logcp` (`dynamolog`).
- CRAP. Score `comp² × (1-cov)³ + comp` for one function. Gate sets the limit at 30 with a baseline. `docs/crap-gate.md` covers the gate.
- detcheck. Tool in `tools/detcheck`. Tool enforces an exact-path import allow-list and bans constructs that break determinism.
- Env. Type in `core/env.go`. Type supplies randomness, buggify, and assertions to the core.

- Fact and Timeline. Fact is an append-only named observation from `core/fact.go`. Sim records facts on the timeline in `sim/timeline.go`. Invariants read the timeline after each step.
- Hermetic test. Test in `it/` that runs in process with memstore and in-memory objects. Full-stack test needs Docker backends through `scripts/it-up.sh`.
- Incarnation. One boot of a node. System drops completions for a dead incarnation. Brokers hold no durable local state. No fencing applies under decisions 43-44.
- Inbox (Control and Data). Two queues for each node. Control has strict priority. System bounds Data. `core/policy.go` and decisions G and H cover that rule.
- Machine and `Step`. `core.StateMachine` defines the machine. `Step(Event) []Action` defines the step. Machine takes `Started` as its first Step.
- manisim. Binary in `cmd/manisim`. Binary runs seed campaigns and replays one seed on request.
- Outcome. `NotApplied`, `MaybeApplied`, and `Applied` name what a completion error proves under lesson 1. An idempotent op can retry a `MaybeApplied` outcome.
- Run. Batch range `[Base, Base+Count)` in a partition log with no gaps. CP derives the next offset from the newest run. Decision 42 covers that rule.
- Seed and TraceHash. Seed drives one sim run. Trace hash covers each step of that run. Equal seeds must produce equal hashes.
- Segment and SegmentKey. Broker buffers batches into a segment. Size, window, and caps seal a segment. `format/` encodes a segment. Broker uploads a segment as one object under `seg/<32 hex>` from `Env.NewID()`. Decision 43 covers that layout.
- Swarm. Per-seed random subset of faults and workload ops. Derived stream draws each subset. Draw lets a run reach extreme cases. Lesson 9 and decision 17 cover that design.
- Trip and Exit. `Trip` stops a node after a failure with untrusted memory and ends the incarnation. `Exit` ends an incarnation after `Stop` without a failure. Decisions 7-8 cover both cases.
- veritas. Package in `veritas/`. Package holds executable specs. Specs double as test fakes. Conformance checks compare code against those specs.

Key takeaways

- Run `make build` and `./scripts/ci-determinism.sh` as the trusted loop. Other commands run narrower parts of those two gates.
- A local run needs no Docker and no AWS. Run `manicp` with the mem store on `:8080`. Run `manibroker -objects mem` on `:9092`. Docker runs the full-stack tests in `it/`. AWS runs the benchmark.
- A sim failure gives a seed and a step number. Replay prints both values. Fact names and trace hashes stay frozen on purpose. Those values tell you how to read the failure.
- Deterministic packages import from an exact allow-list. A new dependency needs an audit. That audit runs `scripts/audit-dep.sh`. It writes an audit record. It follows the procedure in the audit skill.
- Each change recipe pairs the files to touch with the test that proves the change. A contract change needs a `docs/decisions.md` row.

Checkpoint

1. `make test` passes but `make build` fails at the last step. What is that step, and what two changes can fix it? — The CRAP gate in `cmd/crapscore` is that step. Lower the cyclomatic complexity of the

function. Raise its coverage with a real test. `make crap-baseline` needs agreement from a reviewer. A red build alone does not give that agreement.

2. A colleague says "just add `time` to `allow.txt`, the core needs a timestamp." Why not, and what instead? — The allow-list states the determinism contract under decisions I and 2. Deterministic packages get time from the `At` and `Wall` stamps on each event. Use those stamps. A new import needs the dependency-audit recipe. The recipe forbids a change to detcheck that exists to pass CI.

3. `manisim` reports `FAIL seed 77 … repro: go run ./cmd/manisim -scenario kafka -seed 77`. What do you run, and what three numbers do you compare across runs? — Run that repro line. Compare `steps`, `rand` for RNG calls, and `hash` for the trace hash. One seed must reproduce each number. A mismatch points to nondeterminism. It does not point to a logic bug.

4. You need a new CP operation. Name the files in order. — Start with `core/controlplane.go` for the op and response contract. Move then to `cp/wire.go`. Move then to the route in `cp/handler.go`. Move then to `cp.Store` in `cp/store.go` and each store. Move then to `api/openapi/controlplane.json` and `wireTypes` in `spec_test.go`. Regenerate with `go run ./api/openapi/cmd/specgen`. Cover the op with a store lockstep test. Cover the op with an `x-replay` example.

5. For the hermetic `it/` test, the full-stack `it/` test, `make build`, and the AWS benchmark, which one needs Docker, which one needs AWS, and which ones need neither? — The hermetic test needs neither. `make build` needs neither. The full-stack test needs Docker through the endpoints from `scripts/it-up.sh`. The benchmark under `deploy/aws/` needs AWS. `docs/aws-benchmark-runbook.md` covers that benchmark.

Further reading

- Read `loop.md` and `docs/lessons.md`. Those files give the `Step(Event) []Action` model and the 14 lessons behind the interfaces.
- Read `docs/decisions.md`. That file gives the contract history. Read decisions I, J, 21, and 40-45 before your second week.
- Read `docs/crap-gate.md`. That file gives the CRAP formula. It gives the threshold reason and the baseline policy.
- Read `docs/flow-control.md`. That file covers inbox priority and backpressure. It explains why control traffic does not queue behind data.
- Read `api/openapi/README.md` and `docs/log-cp-service.md`. Those files cover the CP spec workflow and the log-backed store from decisions 48-53.
- Read `docs/aws-benchmark-runbook.md`. That file covers the one part of the repo that needs AWS.
- Read `.claude/skills/audit-go-dependency/SKILL.md`. That file gives the dependency-audit procedure in the recipe.

Try it (a half-day hands-on lab)

Budget about four hours. Each part ends at an artifact you can check.

1. Start cold. Take 30 minutes. Run `make build` from a clean checkout. Run `./scripts/ci-determinism.sh` after that. Note the time each command takes. Name the step that catches a stray `import "time"` in `core/`. Prove that step on a scratch branch. Revert the branch after the proof. Check the branch holds no stray import.
2. Run the system. Take 45 minutes. Start `manicp` and `manibroker -objects mem` in the broker task. Create `orders` with 3 partitions. Run `manibench` for 10 s. Stop the broker in the middle of the run. Restart the broker. Write down what survived. Write down which store held what and why it survived. Check the notes name the surviving data and the store.
3. Break the system in simulation. Take 60 minutes. Enable one `broker.Bug*` mutant on your machine. Run `go test ./broker/ -run TestConformanceBrokerCatchesMutants`. Run `go run ./cmd/manisim -seeds 100 -v`. Record the first seed that catches the mutant. Replay that seed with `-seed`. Write three sentences. Those sentences name the `Step` and the invariant that caught the mutant. Revert the mutant after that. Check the three sentences name the `Step`, the invariant, and the seed.
4. Make one safe change. Take 60 minutes. Pick an item from the safe list. A hermetic `it/` case suits this part. Apply the recipe for that item. Run the loop for the first PR. Open the PR. Put the seed evidence or the test evidence in the PR description. Check the PR holds the evidence.
5. Leave a paper record. Take 15 minutes. Add the `docs/decisions.md` row if your change moved a contract. Write the reason in the PR if your change moved no contract. Record that reason in each PR. Check the row or the PR holds the reason.